

Voice TicTacToe

Generated by Doxygen 1.15.0

Chapter 1

PocketSphinx API Documentation

Author

David Huggins-Daines dhdaines@gmail.com

Version

5.0.4

Date

January 10, 2025

1.1 Introduction

This is the documentation for the PocketSphinx speech recognition engine. The main API calls are documented in [ps_decoder_t](#) and [ps_config_t](#). The organization of this document is not optimal due to the limitations of Doxygen, so if you know of a better tool for documenting object-oriented interfaces in C, please let me know.

1.2 Installation

To install from source, you will need a C compiler and a recent version of CMake. If you wish to use an integrated development environment, Visual Studio Code will automate most of this process for you once you have installed C++ and CMake support as described at <https://code.visualstudio.com/docs/languages/cpp>

The easiest way to program PocketSphinx is with the Python module. See <http://pocketsphinx.readthedocs.io/> for installation and usage instructions.

1.2.1 Unix-like systems

From the top-level source directory, use CMake to generate a build directory:

```
cmake -S . -B build
```

Now you can compile and run the tests, and install the code:

```
cmake --build build
cmake --build build --target check
cmake --build build --target install
```

By default CMake will try to install things in `/usr/local`, which you might not have access to. If you want to install somewhere else you need to set `CMAKE_INSTALL_PREFIX` *when running cmake for the first time*, for example:

```
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=$HOME/.local
```

1.2.2 Windows

On Windows, the process is similar, but you will need to tell CMake what build tool you are using with the `-G` option, and there are many of them. The build is known to work with `nmake` but it is easiest just to use Visual Studio Code, which should automatically detect and offer to run the build when you add the source directory to your list of directories. Once built, you will find the EXE files in `build\Debug` or `build\Release` depending on your build type.

1.2.3 Compilation options

By default, PocketSphinx does *not* build shared libraries, as there are not very many executables, and the library is quite smol. If you insist on building them, you can add `BUILD_SHARED_LIBS=ON` to the CMake configuration. This is done either in the CMake GUI, in Visual Studio Code, or with the `-D` option to the first CMake command-line above, e.g.:

```
cmake -S. -B build -DBUILD_SHARED_LIBS=ON
```

GStreamer support is not built by default, but can be enabled with `BUILD_GSTREAMER=ON`.

PocketSphinx uses a mixture of fixed and floating-point computation by default, but can be configured to use fixed-point (nearly) exclusively with `FIXED_POINT=ON`.

1.3 Using the Library

Minimally, to do speech recognition, you must first create a configuration, using `ps_config_t` and its associated functions. This configuration is then passed to `ps_init()` to initialize the decoder, which is returned as a `ps_decoder_t`. Note that you must ultimately release the configuration with `ps_config_free()` to avoid memory leaks.

At this point, you can start an "utterance" (a section of speech you wish to recognize) with `ps_start_utt()` and pass audio data to the decoder with `ps_process_raw()`. When finished, call `ps_end_utt()` to finalize recognition. The result can then be obtained with `ps_get_hyp()`. To get a detailed word segmentation, use `ps_seg_iter()`. To get the N-best results, use `ps_nbest()`.

When you no longer need the decoder, release its memory with `ps_free()`.

A concrete example can be found in `simple.c`.

You may, however, wish to do more interesting things like segmenting and recognizing speech from an audio stream. As described below, PocketSphinx will *not* handle the details of microphone input for you, because doing this in a reliable and portable way is outside the scope of a speech recognizer. In theory, `PortAudio` should work across many platforms. An example using it is in `live_portaudio.c`.

On Windows, an example of using the `Waveform Audio API` can be found in `live_win32.c`.

On GNU/Linux and some other platforms, audio might be handled by the PulseAudio library/server, in which case you can also use the technique in `live_pulseaudio.c`.

Finally, if you have `sox` on your platform, you can simply use the method shown in `live.c`.

1.4 Frequently Asked Questions

1.4.1 My code no longer compiles! Why?

Some APIs were intentionally broken by the 5.0.0 release. The most likely culprit here is the configuration API, where the old "options" which started with a `-` are now "parameters" which do not, and instead of a `cmd_ln_t` it is now a `ps_config_t`. There is no backward compatibility, you have to change your code manually. This is straightforward for the most part. For example, instead of writing:

```
cmdln = cmd_ln_init(NULL, "-samprate", "16000", NULL);
cmd_ln_set_int32_r(NULL, "-maxwfp", 40);
```

You should write:

```
config = ps_config_init(NULL);
ps_config_set_int(config, "samprate", 16000);
ps_config_set_int(config, "maxwfp", 40);
```

Another likely suspect is the `search module API` where the function names have been changed to be more intuitive. Wherever you had `ps_set_search` you can use `ps_activate_search()`, it is the same function. Likewise, anything that was `ps_set_*` is now `ps_add_*`, e.g. `ps_add_lm()`, `ps_add_fsg()`, `ps_add_keyphrase()`.

1.4.2 What does ERROR: "acmod.c, line NN: ..." mean?

In general you will get "Acoustic model definition is not specified" or "Folder does not contain acoustic model definition" errors if PocketSphinx cannot find a model. If you are trying to use the default module, perhaps you have not installed PocketSphinx. Unfortunately it is not designed to run "in-place", but you can get around this by setting the `POCKETSPHINX_PATH` environment variable, e.g.

```
cmake --build build
POCKETSPHINX_PATH=$PWD/model build/pocketsphinx single foo.wav
```

1.4.3 There is literally no output!

If by this you mean it doesn't spew copious logging output like it used to, you can solve this by passing `-loglevel INFO` on the command-line, or setting the `loglevel` parameter to "INFO", or calling `err_set_loglevel()` with `ERR_INFO`.

If you mean that you just don't have any recognition result, you may have forgotten to configure a dictionary. Or see [below](#) for other reasons the output could be blank.

1.4.4 Why doesn't my audio device work?

Because it's an audio device. They don't work, at least for things other than making annoying "beep boop" noises and playing Video Games. More generally, I cannot solve this problem for you, because every single computer, operating system, sound card, microphone, phase of the moon, and day of the week is different when it comes to recording audio. That's why I suggest you use SoX, because (a) it usually works, and (b) whoever wrote it seems to have retired long ago, so you can't bother them.

1.4.5 The recognized text is wrong.

That's not a question! But since this isn't Jeopardy, and my name is not Watson, I'll try to answer it anyway. Be aware that the answer depends on many things, first and foremost what you mean by "wrong".

If it *sounds* the same, e.g. "wreck a nice beach" when you said "recognize speech" then the issue is that the **language model** is not appropriate for the task, domain, dialect, or whatever it is you're trying to recognize. You may wish to consider writing a JSGF grammar and using it instead of the default language model (with the `jsgf` parameter). Or you can get an N-best list or word lattice and rescore it with a better language model, such as a recurrent neural network or a human being.

If it is total nonsense, or if it is just blank, or if it's the same word repeated, e.g. "a a a a a", then there is likely a problem with the input audio. The sampling rate could be wrong, or even if it's correct, you may have narrow-band data. Try to look at the spectrogram (Audacity can show you this) and see if it looks empty or flat below the frequency in the `upperf` parameter. Alternately it could just be very noisy. In particular, if the noise consists of other people talking, automatic speech recognition will nearly always fail.

1.4.6 Why don't you support (pick one or more: WFST, fMLLR, SAT, DNN, CTC, LAS, CNN, RNN, LSTM, etc)?

Not because there's anything wrong with those things (except LAS, which is kind of a dumb idea) but simply because PocketSphinx does not do them, or anything like them, and there is no point in adding them to it when other systems exist. Many of them are also heavily dependent on distasteful and wasteful platforms like C++, CUDA, TensorFlow, PyTorch, and so on.

1.5 Acknowledgements

PocketSphinx was originally released by David Huggins-Daines, but is largely based on the previous Sphinx-~~II~~ II and Sphinx-III systems, developed by a large number of contributors at Carnegie Mellon University, and released as open source under a BSD-like license thanks to Kevin Lenzo. For some time, it was maintained by Nickolay Shmyrev and others at Alpha Cephei, Inc. See the `AUTHORS` file for a list of contributors.

Chapter 2

Deprecated List

Member [ps_decoder_t::ps_get_in_speech](#) ([ps_decoder_t](#) *[ps](#))

This function is retained for compatibility but should not be considered a reliable voice activity detector. It will always return 1 between calls to [ps_start_utt\(\)](#) and [ps_end_utt\(\)](#). You probably want [ps_endpoint_t](#), but for single frames of data you can also use [ps_vad_t](#).

Member [ps_decoder_t::ps_start_stream](#) ([ps_decoder_t](#) *[ps](#))

This function is retained for compatibility, but its only effect is to reset the noise removal statistics, which are otherwise retained across utterances. You do not need to call it.

Chapter 3

Directory Hierarchy

3.1 Directories

AI	??
AIAgent.h	??
HumanPlayer.cpp	??
HumanPlayer.h	??
MiniMaxAgent.cpp	??
MiniMaxAgent.h	??
NoAgent.h	??
Player.h	??
Config	??
ConfigManager.cpp	??
ConfigManager.h	??
Controller	??
TTTController.cpp	??
TTTController.h	??
examples	??
include	??
pocketsphinx	??
sphinx_config.h	??
test	??
unit	??
test_macros.h	??
config.h	??
include	??
pocketsphinx	??
sphinx_config.h	??
include	??
pocketsphinx	??
alignment.h	??
endpointer.h	??
err.h	??
export.h	??
lattice.h	??
logmath.h	??
mlr.h	??
model.h	??
prim_type.h	??

search.h	??
sphinx_config.h	??
vad.h	??
pocketsphinx.h	??
Model	??
Board.cpp	??
Board.h	??
Network	??
NetworkManager.cpp	??
NetworkManager.h	??
pocketsphinx	??
sphinx_config.h	??
pocketsphinx	??
alignment.h	??
endpointer.h	??
err.h	??
export.h	??
lattice.h	??
logmath.h	??
mlr.h	??
model.h	??
prim_type.h	??
search.h	??
sphinx_config.h	??
vad.h	??
Qt-Tic-Tac-Toe-master	??
AI	??
AIAgent.h	??
HumanPlayer.cpp	??
HumanPlayer.h	??
MiniMaxAgent.cpp	??
MiniMaxAgent.h	??
NoAgent.h	??
Player.h	??
Config	??
ConfigManager.cpp	??
ConfigManager.h	??
Controller	??
TTTController.cpp	??
TTTController.h	??
examples	??
include	??
pocketsphinx	??
sphinx_config.h	??
test	??
unit	??
test_macros.h	??
config.h	??
include	??
pocketsphinx	??
alignment.h	??
endpointer.h	??
err.h	??
export.h	??
lattice.h	??
logmath.h	??

mllr.h	??
model.h	??
prim_type.h	??
search.h	??
sphinx_config.h	??
vad.h	??
pocketsphinx.h	??
Model	??
Board.cpp	??
Board.h	??
Network	??
NetworkManager.cpp	??
NetworkManager.h	??
Speech	??
SpeechManager.cpp	??
SpeechManager.h	??
TicTacToeTest	??
main.cpp	??
tst_board.h	??
tst_minimaxagent.h	??
View	??
TicTacToeGame.cpp	??
TicTacToeGame.h	??
TitleScreen.cpp	??
TitleScreen.h	??
main.cpp	??
TTTCommonTypes.h	??
Speech	??
SpeechManager.cpp	??
SpeechManager.h	??
test	??
unit	??
test_macros.h	??
TicTacToeTest	??
main.cpp	??
tst_board.h	??
tst_minimaxagent.h	??
unit	??
test_macros.h	??
View	??
TicTacToeGame.cpp	??
TicTacToeGame.h	??
TitleScreen.cpp	??
TitleScreen.h	??

Chapter 4

Namespace Index

4.1 Namespace List

Here is a list of all namespaces with brief descriptions:

defaults		??
--------------------------	-------	----

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AIAgent Q_OBJECT	
MiniMaxAgent	??
anytype_s	??
anytype_t	??
Board	??
Cell	??
fsg_model_t	??
jsgf_rule_iter_t	??
jsgf_rule_t	??
jsgf_t	??
logadd_s	??
logadd_t	??
logmath_t	??
ngram_class_t	??
ngram_iter_t	??
ngram_model_set_iter_t	??
ngram_model_t	??
Player Q_OBJECT	
HumanPlayer	??
ps_alignment_iter_t	??
ps_alignment_t	??
ps_arg_s	??
ps_arg_t	??
ps_config_t	??
ps_decoder_t	??
ps_endpointiter_t	??
ps_latlink_iter_t	??
ps_latlink_t	??
ps_latnode_iter_t	??
ps_latnode_t	??
ps_lattice_t	??
ps_mllr_t	??
ps_nbest_t	??
ps_search_iter_t	??
ps_seg_t	??

ps_vad_t	??
QDialog	
TicTacToeGame	??
QMainWindow	
TitleScreen	??
QObject	
ConfigManager	??
NetworkManager	??
Player	??
AIAgent	??
NoAgent	??
SpeechManager	??
TTTController	??
SpeechToText	??
TTTOptions	??

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AIAgent	..	??
anytype_s	..	??
anytype_t	..	??
	Literally any type!	??
Board	..	??
	Responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board	??
Cell	..	??
	The Cell struct represents a board cell and all the necessary information needed about the cell	??
ConfigManager	..	??
	The ConfigManager class	??
fsg_model_t	..	??
	Finite-state grammar	??
HumanPlayer	..	??
jsgf_rule_iter_t	..	??
	Iterator over rules in a grammar	??
jsgf_rule_t	..	??
	Rule in a parsed JSGF grammar	??
jsgf_t	..	??
	JSGF parser	??
logadd_s	..	??
logadd_t	..	??
	Integer log math computation table	??
logmath_t	..	??
	Integer log math computation class	??
MiniMaxAgent	..	??
	Handles all the logic of picking a cell based on the MiniMax Algorithm	??
NetworkManager	..	??
ngram_class_t	..	??
	Word class in an N-Gram model	??
ngram_iter_t	..	??
	M-gram (yes, M -gram) iterator object	??
ngram_model_set_iter_t	..	??
	Iterator over language models in a set	??

ngram_model_t	
N-Gram based language model	??
NoAgent	
Empty class that does not play, it specifies the null object behaviour, so it represents the lack of an AI agent, which is useful to avoid using if conditions in code when checking on the existence of an AI agent	??
Player	
Abstract base class for different types of players in the Tic Tac Toe game, such as human players and AI players	??
ps_alignment_iter_t	
Iterator over entries in an alignment	??
ps_alignment_t	
Multi-level alignment (words, phones, states) over an utterance	??
ps_arg_s	??
ps_arg_t	
Definition of a configuration parameter	??
ps_config_t	
Configuration object	??
ps_decoder_t	
Speech recognizer object	??
ps_endpointer_t	
Simple voice activity detection based endpointing	??
ps_latlink_iter_t	
Iterator over DAG links	??
ps_latlink_t	
Link between DAG nodes	??
ps_latnode_iter_t	
Iterator over DAG nodes	??
ps_latnode_t	
Node in a word lattice	??
ps_lattice_t	
Word graph structure used in bestpath/nbest search	??
ps_mllr_t	
Linear transform object	??
ps_nbest_t	
N-best hypothesis iterator object	??
ps_search_iter_t	
Iterator over search modules	??
ps_seg_t	
Segmentation iterator object	??
ps_vad_t	
Voice activity detector	??
SpeechManager	??
SpeechToText	??
TicTacToeGame	
Responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one	??
TitleScreen	
Responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options	??
TTTTController	
Controller for a Tic Tac Toe game, it takes in game options as input, and controls the view (TicTacToeGame) and the model (Board) according to the game logic specified in this class, it is also responsible for calling the AI agent to play its turn if specified by the game options	??
TTTOptions	
The TTTOptions struct defines a tic tac toe game options	??

Chapter 7

File Index

7.1 File List

Here is a list of all files with brief descriptions:

C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/main.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TTTCommonTypes.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/AIAgent.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/HumanPlayer.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/HumanPlayer.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/MiniMaxAgent.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/MiniMaxAgent.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/NoAgent.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/Player.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config/ConfigManager.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config/ConfigManager.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Controller/TTTController.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Controller/TTTController.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/config.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/include/pocketsphinx/sphinx_config.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/test/unit/test_macros.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/alignment.h	??
Multi-level alignment structure	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/endpointer.h	??
VAD-based endpointer for PocketSphinx	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/err.h	??
Implementation of logging routines	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/export.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/lattice.h	??
Word lattices	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/logmath.h	??
Fast integer logarithmic addition operations	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/mlr.h	??
Model-space linear transforms for speaker adaptation	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/model.h	??
Public API for language models	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/prim_type.h	??
Basic type definitions used in Sphinx	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/search.h	??
Search modules	??

C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/sphinx_config.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/vad.h	
Simple voice activity detection	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model/Board.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model/Board.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network/NetworkManager.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network/NetworkManager.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech/SpeechManager.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech/SpeechManager.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToeTest/main.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToeTest/tst_board.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToeTest/tst_minimaxagent.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TicTacToeGame.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TicTacToeGame.h	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TitleScreen.cpp	??
C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TitleScreen.h	??

Chapter 8

Directory Documentation

8.1 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI Directory Reference

Files

- file [AIAgent.h](#)
- file [HumanPlayer.cpp](#)
- file [HumanPlayer.h](#)
- file [MiniMaxAgent.cpp](#)
- file [MiniMaxAgent.h](#)
- file [NoAgent.h](#)
- file [Player.h](#)

8.2 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config Directory Reference

Files

- file [ConfigManager.cpp](#)
- file [ConfigManager.h](#)

8.3 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Controller Directory Reference

Files

- file [TTTController.cpp](#)
- file [TTTController.h](#)

8.4 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples Directory Reference

Directories

- directory [include](#)
- directory [test](#)

Files

- file [config.h](#)

8.5 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/include Directory Reference

Directories

- directory [pocketsphinx](#)

8.6 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include Directory Reference

Directories

- directory [pocketsphinx](#)

Files

- file [pocketsphinx.h](#)

8.7 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model Directory Reference

Files

- file [Board.cpp](#)
- file [Board.h](#)

8.8 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network Directory Reference

Files

- file [NetworkManager.cpp](#)
- file [NetworkManager.h](#)

8.9 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/include/pocketsphinx Directory Reference

Files

- file [sphinx_config.h](#)

8.10 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx Directory Reference

Files

- file [alignment.h](#)
Multi-level alignment structure.
- file [endpointer.h](#)
VAD-based endpointer for PocketSphinx.
- file [err.h](#)
Implementation of logging routines.
- file [export.h](#)
- file [lattice.h](#)
Word lattices.
- file [logmath.h](#)
Fast integer logarithmic addition operations.
- file [mlr.h](#)
Model-space linear transforms for speaker adaptation.
- file [model.h](#)
Public API for language models.
- file [prim_type.h](#)
Basic type definitions used in Sphinx.
- file [search.h](#)
Search modules.
- file [sphinx_config.h](#)
- file [vad.h](#)
Simple voice activity detection.

8.11 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master Directory Reference

Directories

- directory [AI](#)
- directory [Config](#)
- directory [Controller](#)
- directory [examples](#)
- directory [include](#)
- directory [Model](#)
- directory [Network](#)
- directory [Speech](#)
- directory [TicTacToeTest](#)
- directory [View](#)

Files

- file [main.cpp](#)
- file [TTTCommonTypes.h](#)

8.12 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech Directory Reference

Files

- file [SpeechManager.cpp](#)
- file [SpeechManager.h](#)

8.13 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/test Directory Reference

Directories

- directory [unit](#)

8.14 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToe↵ Test Directory Reference

Files

- file [main.cpp](#)
- file [tst_board.h](#)
- file [tst_minimaxagent.h](#)

8.15 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/test/unit Directory Reference

Files

- file [test_macros.h](#)

8.16 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View Directory Reference

Files

- file [TicTacToeGame.cpp](#)
- file [TicTacToeGame.h](#)
- file [TitleScreen.cpp](#)
- file [TitleScreen.h](#)

Chapter 9

Namespace Documentation

9.1 defaults Namespace Reference

Variables

- constexpr short [INVALID_CELL](#) = -1
- constexpr size_t [DEFAULT_BOARD_SIZE](#) = 3
- constexpr unsigned short [DEFAULT_MINIMAX_DEPTH](#) = 3
- constexpr unsigned short [GUI_CELL_ROW_SPAN](#) = 1
- constexpr unsigned short [GUI_CELL_COLUMN_SPAN](#) = 1
- constexpr char [X_COLOR](#) [] = "#455A64"
- constexpr char [O_COLOR](#) [] = "#FF5722"
- constexpr char [DEFAULT_COLOR](#) [] = "#FFF"

9.1.1 Variable Documentation

9.1.1.1 DEFAULT_BOARD_SIZE

```
size_t defaults::DEFAULT_BOARD_SIZE = 3 [constexpr]
```

9.1.1.2 DEFAULT_COLOR

```
char defaults::DEFAULT_COLOR[] = "#FFF" [constexpr]
```

Referenced by [TicTacToeGame::getPlayerStyleSheet\(\)](#).

9.1.1.3 DEFAULT_MINIMAX_DEPTH

```
unsigned short defaults::DEFAULT_MINIMAX_DEPTH = 3 [constexpr]
```

9.1.1.4 GUI_CELL_COLUMN_SPAN

```
unsigned short defaults::GUI_CELL_COLUMN_SPAN = 1 [constexpr]
```

Referenced by [TicTacToeGame::buildCellButtons\(\)](#).

9.1.1.5 GUI_CELL_ROW_SPAN

```
unsigned short defaults::GUI_CELL_ROW_SPAN = 1 [constexpr]
```

Referenced by [TicTacToeGame::buildCellButtons\(\)](#).

9.1.1.6 INVALID_CELL

```
short defaults::INVALID_CELL = -1 [constexpr]
```

Referenced by [TTTController::AIAgentPlay\(\)](#), [TTTController::onSpeechRecognized\(\)](#), [TTTController::parseSpeechToCell\(\)](#), [MiniMaxAgent::play\(\)](#), and [NoAgent::play\(\)](#).

9.1.1.7 O_COLOR

```
char defaults::O_COLOR[ ] = "#FF5722" [constexpr]
```

Referenced by [TicTacToeGame::getPlayerStyleSheet\(\)](#).

9.1.1.8 X_COLOR

```
char defaults::X_COLOR[ ] = "#455A64" [constexpr]
```

Referenced by [TicTacToeGame::getPlayerStyleSheet\(\)](#).

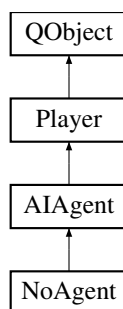
Chapter 10

Class Documentation

10.1 AIAgent Class Reference

```
#include <AIAgent.h>
```

Inheritance diagram for AIAgent:



Public Member Functions

- [AIAgent](#) ()
- virtual [~AIAgent](#) ()
- virtual int [play](#) (const [Board](#) &board)=0
Start the algorithm to choose a cell to play based on a given board.
- virtual void [reset](#) ()
reset function that will reset the state of any search algorithm.

Public Member Functions inherited from [Player](#)

- [Player](#) (QObject *parent_=nullptr, [BoardMarks](#) playerMark, const [Board](#) &board_ref)
- virtual [~Player](#) ()
- virtual void [play](#) ()=0

Additional Inherited Members

Signals inherited from [Player](#)

- void [playerFinished](#) (const int &cell)

Protected Attributes inherited from [Player](#)

- const [Board](#) * [board_](#)
- const [BoardMarks](#) [mark_](#)

10.1.1 Constructor & Destructor Documentation

10.1.1.1 [AIAgent](#)()

```
AIAgent::AIAgent () [inline]
```

10.1.1.2 [~AIAgent](#)()

```
virtual AIAgent::~~AIAgent () [inline], [virtual]
```

10.1.2 Member Function Documentation

10.1.2.1 [play](#)()

```
virtual int AIAgent::play (  
    const Board & board) [pure virtual]
```

Start the algorithm to choose a cell to play based on a given board.

Parameters

<i>board</i>	a reference to the board model.
--------------	---------------------------------

Returns

the chosen cell index (in a 1D array) that the AI played.

Implemented in [NoAgent](#).

10.1.2.2 reset()

```
virtual void AIAgent::reset () [inline], [virtual]
```

reset function that will reset the state of any search algorithm.

The documentation for this class was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[AIAgent.h](#)

10.2 anytype_s Union Reference

```
#include <prim_type.h>
```

Public Attributes

- void * [ptr](#)
- long [i](#)
- unsigned long [ui](#)
- double [fl](#)

10.2.1 Member Data Documentation

10.2.1.1 fl

```
double anytype_s::fl
```

10.2.1.2 i

```
long anytype_s::i
```

10.2.1.3 ptr

```
void* anytype_s::ptr
```

10.2.1.4 ui

```
unsigned long anytype_s::ui
```

The documentation for this union was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[prim_type.h](#)

10.3 anytype_t Union Reference

Literally any type!

```
#include <pocketsphinx/prim_type.h>
```

10.3.1 Detailed Description

Literally any type!

(correction: not literally any type, but all the ones we use for configuration)

The documentation for this union was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/prim_type.h

10.4 Board Class Reference

The [Board](#) class is responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board.

```
#include <Board.h>
```

Public Member Functions

- [Board](#) (size_t size)
Constructs a board given a board size.
- [Board](#) (Board &b)
Board copy constructor.
- [Board](#) (const Board &b)
- [Board](#) (Board &&b)
Board move constructor.
- [Board & operator=](#) (Board b)
This assignment operator implements the copy & swap idiom, so it works as a move and a copy assignment operator.
- virtual [~Board](#) ()
Board destructor.
- virtual BoardState [evaluateBoard](#) () const
evaluateBoard evaluates whether the state of the current board. the function can evaluate to X wins or O wins or a Tie or No Winner.
- virtual bool [setPlayerInput](#) (size_t row, size_t col, BoardMarks currentPlayer)
setPlayerInput sets the board with the mark given in the specified location. after boundary checking the input row and column, and if the game hasn't reached a final state.
- virtual BoardMarks [at](#) (size_t row, size_t col) const
at is a getter function that returns a cell given a row and a column.
- virtual void [reset](#) ()
reset allows resetting the whole board, marking all the cells as empty.
- virtual void [resetCell](#) (size_t row, size_t col)
resetCell allows resetting a given cell to an empty type, without the extra state checks done in the setPlayerInput function.
- size_t [size](#) () const
size is a getter functions of the board size.

Protected Attributes

- `vector< vector< BoardMarks > > board\_`
`board` is a 2D vector that represents a machine readable format to the GUI application.
- `size_t boardSize\_`
`boardSize` is the number of rows or columns in the board. a 3x3 board has a `boardSize` of 3.

10.4.1 Detailed Description

The [Board](#) class is responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board.

10.4.2 Constructor & Destructor Documentation

10.4.2.1 [Board\(\)](#) [1/4]

```
Board::Board (
    size_t size)
```

Constructs a board given a board size.

Parameters

<code>size</code>	is the number of rows or columns in the board.
-------------------	--

References [board_](#), [boardSize_](#), and [size\(\)](#).

Referenced by [Board\(\)](#), [Board\(\)](#), [Board\(\)](#), and [operator=\(\)](#).

10.4.2.2 [Board\(\)](#) [2/4]

```
Board::Board (
    Board & b)
```

[Board](#) copy constructor.

Parameters

<code>b</code>	The board to be copied.
----------------	-------------------------

References [Board\(\)](#), [board_](#), and [boardSize_](#).

10.4.2.3 [Board\(\)](#) [3/4]

```
Board::Board (
    const Board & b)
```

References [Board\(\)](#), [board_](#), and [boardSize_](#).

10.4.2.4 Board() [4/4]

```
Board::Board (  
    Board && b)
```

[Board](#) move constructor.

Parameters

<i>b</i>	The board to be moved.
----------	------------------------

References [Board\(\)](#), [board_](#), and [boardSize_](#).

10.4.2.5 ~Board()

```
Board::~~Board () [virtual]
```

[Board](#) destructor.

10.4.3 Member Function Documentation

10.4.3.1 at()

```
BoardMarks Board::at (
    size_t row,
    size_t col) const [virtual]
```

`at` is a getter function that returns a cell given a row and a column.

Parameters

<i>row</i>	of the cell target.
<i>col</i>	of the cell target.

Returns

the value mark at the cell.

Exceptions

<i>out_of_range</i>	exception when the row or column is out of bounds.
---------------------	--

References [board_](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [MiniMaxAgent::play\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), and [TEST\(\)](#).

10.4.3.2 evaluateBoard()

```
BoardState Board::evaluateBoard () const [virtual]
```

`evaluateBoard` evaluates whether the state of the current board. the function can evaluate to X wins or O wins or a Tie or No Winner.

Returns

the state of the current board.

References [board_](#), [boardSize_](#), [Empty](#), [NoWinner](#), [O](#), [OWins](#), [Tie](#), [X](#), and [XWins](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [MiniMaxAgent::play\(\)](#), [setPlayerInput\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), and [TEST\(\)](#).

10.4.3.3 operator=()

```
Board & Board::operator= (
    Board b)
```

This assignment operator implements the copy & swap idiom, so it works as a move and a copy assignment operator.

Parameters

<i>b</i>	The board to be copied or moved.
----------	----------------------------------

Returns

a reference to this object, so this operator can be chained.

References [Board\(\)](#), [board_](#), and [boardSize_](#).

10.4.3.4 reset()

```
void Board::reset () [virtual]
```

reset allows resetting the whole board, marking all the cells as empty.

References [board_](#), [boardSize_](#), and [Empty](#).

Referenced by [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), and [TEST\(\)](#).

10.4.3.5 resetCell()

```
void Board::resetCell (
    size_t row,
    size_t col) [virtual]
```

resetCell allows resetting a given cell to an empty type, without the extra state checks done in the `setPlayerInput` function.

References [board_](#), [boardSize_](#), and [Empty](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), and [MiniMaxAgent::play\(\)](#).

10.4.3.6 setPlayerInput()

```
bool Board::setPlayerInput (
    size_t row,
    size_t col,
    BoardMarks currentPlayer) [virtual]
```

setPlayerInput sets the board with the mark given in the specified location. after boundary checking the input row and column, and if the game hasn't reached a final state.

Parameters

<i>row</i>	of the cell target.
<i>col</i>	of the cell target.
<i>currentPlayer</i>	is the mark of the current player.

Returns

whether the update was successfull or not based on the boundary check and validity of input given the board state.

Exceptions

<i>out_of_range</i>	exception when the row or column is out of bounds.
---------------------	--

References [board_](#), [boardSize_](#), [Empty](#), [evaluateBoard\(\)](#), and [NoWinner](#).

Referenced by [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), [MiniMaxAgent::play\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), and [TEST\(\)](#).

10.4.3.7 size()

```
size_t Board::size () const
```

size is a getter functions of the board size.

Returns

the size of the board i.e. numebr of rows or columns.

References [boardSize_](#).

Referenced by [Board\(\)](#), [MiniMaxAgent::maxMove\(\)](#), [MiniMaxAgent::minMove\(\)](#), and [MiniMaxAgent::play\(\)](#).

10.4.4 Member Data Documentation**10.4.4.1 board_**

```
vector<vector<BoardMarks> > Board::board_ [protected]
```

board is a 2D vector that represents a machine readable format to the GUI application.

Referenced by [at\(\)](#), [Board\(\)](#), [Board\(\)](#), [Board\(\)](#), [Board\(\)](#), [evaluateBoard\(\)](#), [operator=\(\)](#), [reset\(\)](#), [resetCell\(\)](#), and [setPlayerInput\(\)](#).

10.4.4.2 boardSize_

```
size_t Board::boardSize_ [protected]
```

boardSize is the number of rows or columns in the board. a 3x3 board has a boardSize of 3.

Referenced by [Board\(\)](#), [Board\(\)](#), [Board\(\)](#), [Board\(\)](#), [evaluateBoard\(\)](#), [operator=\(\)](#), [reset\(\)](#), [resetCell\(\)](#), [setPlayerInput\(\)](#), and [size\(\)](#).

The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model/[Board.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model/[Board.cpp](#)

10.5 Cell Struct Reference

The [Cell](#) struct represents a board cell and all the necessary information needed about the cell.

```
#include <TTTCommonTypes.h>
```

Public Member Functions

- [Cell](#) (QPushButton *[cellBtn](#), int [row](#), int [col](#))

Public Attributes

- QPushButton * [cellBtn](#) = nullptr
- int [row](#) = [defaults::INVALID_CELL](#)
- int [col](#) = [defaults::INVALID_CELL](#)

10.5.1 Detailed Description

The [Cell](#) struct represents a board cell and all the necessary information needed about the cell.

10.5.2 Constructor & Destructor Documentation

10.5.2.1 Cell()

```
Cell::Cell (  
    QPushButton * cellBtn,  
    int row,  
    int col) [inline]
```

References [cellBtn](#), [col](#), and [row](#).

10.5.3 Member Data Documentation

10.5.3.1 cellBtn

```
QPushButton* Cell::cellBtn = nullptr
```

Referenced by [Cell\(\)](#), and [TicTacToeGame::updateCell\(\)](#).

10.5.3.2 col

```
int Cell::col = defaults::INVALID_CELL
```

Referenced by [TTTController::AIAgentPlay\(\)](#), [Cell\(\)](#), and [TTTController::updateGame\(\)](#).

10.5.3.3 row

```
int Cell::row = defaults::INVALID_CELL
```

Referenced by [TTTController::AIAgentPlay\(\)](#), [Cell\(\)](#), and [TTTController::updateGame\(\)](#).

The documentation for this struct was generated from the following file:

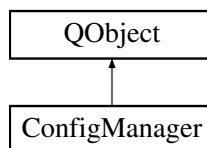
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/[TTTCommonTypes.h](#)

10.6 ConfigManager Class Reference

The [ConfigManager](#) class.

```
#include <ConfigManager.h>
```

Inheritance diagram for ConfigManager:



Public Member Functions

- void [load](#) (const QString &filePath=QString())
Carga el archivo de configuración desde la ruta indicada. Si no se especifica ruta, buscará config.ini en el directorio actual.
- QString [getValue](#) (const QString &key, const QString &defaultValue=QString()) const
Devuelve un valor del archivo INI como QString.
- int [getInt](#) (const QString &key, int defaultValue=0) const
Devuelve un valor como entero.
- bool [getBool](#) (const QString &key, bool defaultValue=false) const
Devuelve un valor como booleano.

Static Public Member Functions

- static [ConfigManager](#) & [instance](#) ()
Obtiene la instancia única del [ConfigManager](#) (Singleton).

Private Member Functions

- [ConfigManager](#) (QObject *parent=nullptr)
- [~ConfigManager](#) () override=default
- [ConfigManager](#) (const [ConfigManager](#) &)=delete
- [ConfigManager](#) & [operator=](#) (const [ConfigManager](#) &)=delete

Private Attributes

- QSettings * [settings_](#) = nullptr

10.6.1 Detailed Description

The [ConfigManager](#) class.

Gestiona la configuración de la aplicación desde un archivo config.ini. Aplica el patrón Singleton para asegurar que solo existe una instancia accesible globalmente.

10.6.2 Constructor & Destructor Documentation

10.6.2.1 [ConfigManager\(\)](#) [1/2]

```
ConfigManager::ConfigManager (  
    QObject * parent = nullptr) [explicit], [private]
```

Referenced by [ConfigManager\(\)](#), [instance\(\)](#), and [operator=\(\)](#).

10.6.2.2 [~ConfigManager\(\)](#)

```
ConfigManager::~~ConfigManager () [override], [private], [default]
```

10.6.2.3 [ConfigManager\(\)](#) [2/2]

```
ConfigManager::ConfigManager (  
    const ConfigManager & ) [private], [delete]
```

References [ConfigManager\(\)](#).

10.6.3 Member Function Documentation

10.6.3.1 getBool()

```
bool ConfigManager::getBool (
    const QString & key,
    bool defaultValue = false) const
```

Devuelve un valor como booleano.

References [settings_](#).

10.6.3.2 getInt()

```
int ConfigManager::getInt (
    const QString & key,
    int defaultValue = 0) const
```

Devuelve un valor como entero.

References [settings_](#).

10.6.3.3 getValue()

```
QString ConfigManager::getValue (
    const QString & key,
    const QString & defaultValue = QString()) const
```

Devuelve un valor del archivo INI como QString.

Parameters

<i>key</i>	Clave (por ejemplo, "network/ip")
<i>defaultValue</i>	Valor por defecto si no se encuentra la clave

Returns

Valor asociado

References [settings_](#).

10.6.3.4 instance()

```
ConfigManager & ConfigManager::instance () [static]
```

Obtiene la instancia única del [ConfigManager](#) (Singleton).

Returns

Referencia al único [ConfigManager](#).

References [ConfigManager\(\)](#), and [instance\(\)](#).

Referenced by [SpeechManager::configure\(\)](#), [instance\(\)](#), and [main\(\)](#).

10.6.3.5 load()

```
void ConfigManager::load (
    const QString & filePath = QString())
```

Carga el archivo de configuración desde la ruta indicada. Si no se especifica ruta, buscará config.ini en el directorio actual.

Parameters

<i>filePath</i>	Ruta al archivo .ini
-----------------	----------------------

References [settings_](#).

Referenced by [main\(\)](#).

10.6.3.6 operator=()

```
ConfigManager & ConfigManager::operator= (
    const ConfigManager & ) [private], [delete]
```

References [ConfigManager\(\)](#).

10.6.4 Member Data Documentation

10.6.4.1 settings_

```
QSettings* ConfigManager::settings_ = nullptr [private]
```

Referenced by [getBool\(\)](#), [getInt\(\)](#), [getValue\(\)](#), and [load\(\)](#).

The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config/[ConfigManager.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config/[ConfigManager.cpp](#)

10.7 fsg_model_t Struct Reference

Finite-state grammar.

```
#include <pocketsphinx/model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT fsg_model_t * jsgf_read_file](#) (const char *file, [logmath_t](#) *lmath, [float32](#) lw)
- [POCKETSPHINX_EXPORT fsg_model_t * jsgf_read_string](#) (const char *string, [logmath_t](#) *lmath, [float32](#) lw)
- [POCKETSPHINX_EXPORT fsg_model_t * fsg_model_retain](#) ([fsg_model_t](#) *fsg)
- [POCKETSPHINX_EXPORT int fsg_model_free](#) ([fsg_model_t](#) *fsg)
- [POCKETSPHINX_EXPORT fsg_model_t * fsg_model_readfile](#) (const char *file, [logmath_t](#) *lmath, [float32](#) lw)
- [POCKETSPHINX_EXPORT fsg_model_t * fsg_model_read](#) (FILE *fp, [logmath_t](#) *lmath, [float32](#) lw)
- [POCKETSPHINX_EXPORT int fsg_model_accept](#) ([fsg_model_t](#) *fsg, char const *words)
- [POCKETSPHINX_EXPORT void fsg_model_write](#) ([fsg_model_t](#) *fsg, FILE *fp)
- [POCKETSPHINX_EXPORT void fsg_model_writefile](#) ([fsg_model_t](#) *fsg, char const *file)
- [POCKETSPHINX_EXPORT void fsg_model_write_fsm](#) ([fsg_model_t](#) *fsg, FILE *fp)
- [POCKETSPHINX_EXPORT void fsg_model_writefile_fsm](#) ([fsg_model_t](#) *fsg, char const *file)
- [POCKETSPHINX_EXPORT void fsg_model_write_syntab](#) ([fsg_model_t](#) *fsg, FILE *file)
- [POCKETSPHINX_EXPORT void fsg_model_writefile_syntab](#) ([fsg_model_t](#) *fsg, char const *file)

10.7.1 Detailed Description

Finite-state grammar.

States are simply integers 0..n_state-1. A transition emits a word and has a given probability of being taken. There can also be null or epsilon transitions, with no associated emitted word.

10.7.2 Member Function Documentation

10.7.2.1 fsg_model_accept()

```
POCKETSPHINX_EXPORT int fsg_model_accept (
    fsg_model_t * fsg,
    char const * words)
```

Check that an FSG accepts a word sequence

Parameters

<i>words</i>	Whitespace-separated word sequence
--------------	------------------------------------

Returns

1 if accepts, 0 if not.

References [POCKETSPHINX_EXPORT](#).

10.7.2.2 fsg_model_free()

```
POCKETSPHINX_EXPORT int fsg_model_free (
    fsg_model_t * fsg)
```

Free the given word FSG.

Returns

new reference count (0 if freed completely)

References [POCKETSPHINX_EXPORT](#).

10.7.2.3 fsg_model_read()

```
POCKETSPHINX_EXPORT fsg_model_t * fsg_model_read (
    FILE * fp,
    logmath_t * lm,
    float32 lw)
```

Like [fsg_model_readfile\(\)](#), but from an already open stream.

References [POCKETSPHINX_EXPORT](#).

10.7.2.4 fsg_model_readfile()

```
POCKETSPHINX_EXPORT fsg_model_t * fsg_model_readfile (
    const char * file,
    logmath_t * lm,
    float32 lw)
```

Read a word FSG from the given file and return a pointer to the structure created. Return NULL if any error occurred.

File format:

```
Any number of comment lines; ignored
FSG_BEGIN [<fsgname>]
N <#states>
S <start-state ID>
F <final-state ID>
T <from-state> <to-state> <prob> [<word-string>]
T ...
... (any number of state transitions)
FSG_END
Any number of comment lines; ignored
```

The FSG spec begins with the line containing the keyword FSG_BEGIN. It has an optional fsg name string. If not present, the FSG has the empty string as its name.

Following the FSG_BEGIN declaration is the number of states, the start state, and the final state, each on a separate line. States are numbered in the range [0 .. <numberofstate>-1].

These are followed by all the state transitions, each on a separate line, and terminated by the FSG_END line. A state transition has the given probability of being taken, and emits the given word. The word emission is optional; if word-string omitted, it is an epsilon or null transition.

Comments can also be embedded within the FSG body proper (i.e. between FSG_BEGIN and FSG_END): any line with a # character in col 1 is treated as a comment line.

Return value: a new [fsg_model_t](#) structure if the file is successfully read, NULL otherwise.

References [POCKETSPHINX_EXPORT](#).

10.7.2.5 fsg_model_retain()

```
POCKETSPHINX_EXPORT fsg_model_t * fsg_model_retain (  
    fsg_model_t * fsg)
```

Retain ownership of an FSG.

Returns

Pointer to retained FSG.

References [POCKETSPHINX_EXPORT](#).

10.7.2.6 fsg_model_write()

```
POCKETSPHINX_EXPORT void fsg_model_write (  
    fsg_model_t * fsg,  
    FILE * fp)
```

Write FSG to a file.

References [POCKETSPHINX_EXPORT](#).

10.7.2.7 fsg_model_write_fsm()

```
POCKETSPHINX_EXPORT void fsg_model_write_fsm (  
    fsg_model_t * fsg,  
    FILE * fp)
```

Write FSG to a file in AT&T FSM format.

References [POCKETSPHINX_EXPORT](#).

10.7.2.8 fsg_model_write_syntab()

```
POCKETSPHINX_EXPORT void fsg_model_write_syntab (  
    fsg_model_t * fsg,  
    FILE * file)
```

Write FSG symbol table to a file (for AT&T FSM)

References [POCKETSPHINX_EXPORT](#).

10.7.2.9 fsg_model_writefile()

```
POCKETSPHINX_EXPORT void fsg_model_writefile (  
    fsg_model_t * fsg,  
    char const * file)
```

Write FSG to a file.

References [POCKETSPHINX_EXPORT](#).

10.7.2.10 fsg_model_writefile_fsm()

```
POCKETSPHINX_EXPORT void fsg_model_writefile_fsm (  
    fsg_model_t * fsg,  
    char const * file)
```

Write FSG to a file in AT&T FSM format.

References [POCKETSPHINX_EXPORT](#).

10.7.2.11 fsg_model_writefile_syntab()

```
POCKETSPHINX_EXPORT void fsg_model_writefile_syntab (  
    fsg_model_t * fsg,  
    char const * file)
```

Write FSG symbol table to a file (for AT&T FSM)

10.7.2.12 jsgf_read_file()

```
POCKETSPHINX_EXPORT fsg_model_t * jsgf_read_file (  
    const char * file,  
    logmath_t * lmath,  
    float32 lw)
```

Read JSGF from file and return FSG object from it.

This function looks for a first public rule in jsgf and constructs JSGF from it.

References [POCKETSPHINX_EXPORT](#).

10.7.2.13 jsgf_read_string()

```
POCKETSPHINX_EXPORT fsg_model_t * jsgf_read_string (  
    const char * string,  
    logmath_t * lmath,  
    float32 lw)
```

Read JSGF from string and return FSG object from it.

This function looks for a first public rule in jsgf and constructs JSGF from it.

References [POCKETSPHINX_EXPORT](#).

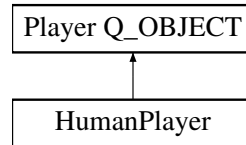
The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.8 HumanPlayer Class Reference

```
#include <HumanPlayer.h>
```

Inheritance diagram for HumanPlayer:



Signals

- void [startListening](#) ()
- void [stopListening](#) ()

Public Member Functions

- [HumanPlayer](#) (QObject &parent=nullptr, const [Board](#) &board_ref, [BoardMarks](#) playerMark=[BoardMarks::Empty](#))
virtual ~[HumanPlayer](#)()
- void [play](#) () override

Private Slots

- void [onSpeechRecognized](#) (const QString &text)

Private Member Functions

- int [parseSpeechCommand](#) (const QString &text)
- bool [validateCommand](#) (const QString &command)

Private Attributes

- [SpeechManager](#) * [stt_](#)
- QThread * [sttThread_](#)

10.8.1 Constructor & Destructor Documentation

10.8.1.1 HumanPlayer()

```
HumanPlayer::HumanPlayer (
    QObject & parent = nullptr,
    const Board & board_ref,
    BoardMarks playerMark = BoardMarks::Empty) [inline], [explicit]
```

References [Empty](#), and [HumanPlayer\(\)](#).

Referenced by [HumanPlayer\(\)](#).

10.8.2 Member Function Documentation

10.8.2.1 onSpeechRecognized

```
void HumanPlayer::onSpeechRecognized (  
    const QString & text) [private], [slot]
```

10.8.2.2 parseSpeechCommand()

```
int HumanPlayer::parseSpeechCommand (  
    const QString & text) [private]
```

10.8.2.3 play()

```
void HumanPlayer::play () [override]
```

10.8.2.4 startListening

```
void HumanPlayer::startListening () [signal]
```

10.8.2.5 stopListening

```
void HumanPlayer::stopListening () [signal]
```

10.8.2.6 validateCommand()

```
bool HumanPlayer::validateCommand (  
    const QString & command) [private]
```

10.8.3 Member Data Documentation

10.8.3.1 stt_

```
SpeechManager* HumanPlayer::stt_ [private]
```

10.8.3.2 sttThread_

```
QThread* HumanPlayer::sttThread_ [private]
```

The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[HumanPlayer.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[HumanPlayer.cpp](#)

10.9 jsgf_rule_iter_t Struct Reference

Iterator over rules in a grammar.

```
#include <model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT jsgf_rule_iter_t * jsgf_rule_iter_next \(jsgf_rule_iter_t *itor\)](#)
- [POCKETSPHINX_EXPORT jsgf_rule_t * jsgf_rule_iter_rule \(jsgf_rule_iter_t *itor\)](#)
- [POCKETSPHINX_EXPORT void jsgf_rule_iter_free \(jsgf_rule_iter_t *itor\)](#)

10.9.1 Detailed Description

Iterator over rules in a grammar.

10.9.2 Member Function Documentation

10.9.2.1 jsgf_rule_iter_free()

```
POCKETSPHINX_EXPORT void jsgf_rule_iter_free (  
    jsgf_rule_iter_t * itor)
```

Free a rule iterator (if the end hasn't been reached).

References [POCKETSPHINX_EXPORT](#).

10.9.2.2 jsgf_rule_iter_next()

```
POCKETSPHINX_EXPORT jsgf_rule_iter_t * jsgf_rule_iter_next (  
    jsgf_rule_iter_t * itor)
```

Advance an iterator to the next rule in the grammar.

References [POCKETSPHINX_EXPORT](#).

10.9.2.3 jsgf_rule_iter_rule()

```
POCKETSPHINX_EXPORT jsgf_rule_t * jsgf_rule_iter_rule (  
    jsgf_rule_iter_t * itor)
```

Get the current rule in a rule iterator.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.10 jsgf_rule_t Struct Reference

Rule in a parsed JSFG grammar.

```
#include <pocketsphinx/model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) char const * [jsgf_rule_name](#) ([jsgf_rule_t](#) *rule)
- [POCKETSPHINX_EXPORT](#) int [jsgf_rule_public](#) ([jsgf_rule_t](#) *rule)

10.10.1 Detailed Description

Rule in a parsed JSFG grammar.

10.10.2 Member Function Documentation

10.10.2.1 jsgf_rule_name()

```
POCKETSPHINX\_EXPORT char const * jsgf_rule_name (  
    jsgf\_rule\_t * rule)
```

Get the rule name from a rule.

References [POCKETSPHINX_EXPORT](#).

10.10.2.2 jsgf_rule_public()

```
POCKETSPHINX\_EXPORT int jsgf_rule_public (  
    jsgf\_rule\_t * rule)
```

Test if a rule is public or not.

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.11 jsgf_t Struct Reference

JSFG parser.

```
#include <pocketsphinx/model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT jsfg_t * jsfg_parse_file](#) (const char *filename, [jsfg_t](#) *parent)
- [POCKETSPHINX_EXPORT jsfg_t * jsfg_parse_string](#) (const char *string, [jsfg_t](#) *parent)
- [POCKETSPHINX_EXPORT](#) char const * [jsfg_grammar_name](#) ([jsfg_t](#) *jsfg)
- [POCKETSPHINX_EXPORT](#) void [jsfg_grammar_free](#) ([jsfg_t](#) *jsfg)
- [POCKETSPHINX_EXPORT jsfg_rule_t * jsfg_get_rule](#) ([jsfg_t](#) *grammar, const char *name)
- [POCKETSPHINX_EXPORT jsfg_rule_t * jsfg_get_public_rule](#) ([jsfg_t](#) *grammar)
- [POCKETSPHINX_EXPORT jsfg_rule_iter_t * jsfg_rule_iter](#) ([jsfg_t](#) *grammar)
- [POCKETSPHINX_EXPORT fsg_model_t * jsfg_build_fsg](#) ([jsfg_t](#) *grammar, [jsfg_rule_t](#) *rule, [logmath_t](#) *lmath, float32 lw)
- [POCKETSPHINX_EXPORT](#) int [jsfg_write_fsg](#) ([jsfg_t](#) *grammar, [jsfg_rule_t](#) *rule, FILE *outfh)

10.11.1 Detailed Description

JSFG parser.

10.11.2 Member Function Documentation

10.11.2.1 jsfg_build_fsg()

```
POCKETSPHINX_EXPORT fsg_model_t * jsfg_build_fsg (
    jsfg_t * grammar,
    jsfg_rule_t * rule,
    logmath_t * lmath,
    float32 lw)
```

Build a Sphinx FSG object from a JSFG rule.

References [POCKETSPHINX_EXPORT](#).

10.11.2.2 jsfg_get_public_rule()

```
POCKETSPHINX_EXPORT jsfg_rule_t * jsfg_get_public_rule (
    jsfg_t * grammar)
```

Returns the first public rule of the grammar

References [POCKETSPHINX_EXPORT](#).

10.11.2.3 jsfg_get_rule()

```
POCKETSPHINX_EXPORT jsfg_rule_t * jsfg_get_rule (
    jsfg_t * grammar,
    const char * name)
```

Get a rule by name from a grammar. Name should not contain brackets.

References [POCKETSPHINX_EXPORT](#).

10.11.2.4 jsgf_grammar_free()

```
POCKETSPHINX_EXPORT void jsgf_grammar_free (  
    jsgf_t * jsgf)
```

Free a JSGF grammar.

References [POCKETSPHINX_EXPORT](#).

10.11.2.5 jsgf_grammar_name()

```
POCKETSPHINX_EXPORT char const * jsgf_grammar_name (  
    jsgf_t * jsgf)
```

Get the grammar name from the file.

References [POCKETSPHINX_EXPORT](#).

10.11.2.6 jsgf_parse_file()

```
POCKETSPHINX_EXPORT jsgf_t * jsgf_parse_file (  
    const char * filename,  
    jsgf_t * parent)
```

Parse a JSGF grammar from a file.

Parameters

<i>filename</i>	the name of the file to parse.
<i>parent</i>	optional parent grammar for this one (NULL, usually).

Returns

new JSGF grammar object, or NULL on failure.

References [POCKETSPHINX_EXPORT](#).

10.11.2.7 jsgf_parse_string()

```
POCKETSPHINX_EXPORT jsgf_t * jsgf_parse_string (  
    const char * string,  
    jsgf_t * parent)
```

Parse a JSGF grammar from a string.

Parameters

<i>string</i>	0-terminated string with grammar.
<i>parent</i>	optional parent grammar for this one (NULL, usually).

Returns

new JSGF grammar object, or NULL on failure.

References [POCKETSPHINX_EXPORT](#).

10.11.2.8 jsgf_rule_iter()

```
POCKETSPHINX_EXPORT jsgf_rule_iter_t * jsgf_rule_iter (
    jsgf_t * grammar)
```

Get an iterator over all rules in a grammar.

References [POCKETSPHINX_EXPORT](#).

10.11.2.9 jsgf_write_fsg()

```
POCKETSPHINX_EXPORT int jsgf_write_fsg (
    jsgf_t * grammar,
    jsgf_rule_t * rule,
    FILE * outfh)
```

Convert a JSGF rule to Sphinx FSG text form.

This does a direct conversion without doing transitive closure on null transitions and so forth.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.12 logadd_s Struct Reference

```
#include <logmath.h>
```

Public Attributes

- void * [table](#)
- uint32 [table_size](#)
- uint8 [width](#)
- int8 [shift](#)

10.12.1 Member Data Documentation

10.12.1.1 shift

```
int8 logadd_s::shift
```

Right shift applied to elements in (table).

10.12.1.2 table

```
void* logadd_s::table
```

Table, in unsigned integers of (width) bytes.

10.12.1.3 table_size

```
uint32 logadd_s::table_size
```

Number of elements in (table). This is never smaller than 256 (important!)

10.12.1.4 width

```
uint8 logadd_s::width
```

Width of elements of (table).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[logmath.h](#)

10.13 logadd_t Struct Reference

Integer log math computation table.

```
#include <pocketsphinx/logmath.h>
```

10.13.1 Detailed Description

Integer log math computation table.

This is exposed here to allow log-add computations to be inlined.

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[logmath.h](#)

10.14 logmath_t Struct Reference

Integer log math computation class.

```
#include <pocketsphinx/logmath.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT logmath_t * logmath_init](#) (float64 base, int shift, int use_table)
- [POCKETSPHINX_EXPORT logmath_t * logmath_read](#) (const char *filename)
- [POCKETSPHINX_EXPORT int32 logmath_write](#) (logmath_t *lmath, const char *filename)
- [POCKETSPHINX_EXPORT int32 logmath_get_table_shape](#) (logmath_t *lmath, uint32 *out_size, uint32 *out_width, uint32 *out_shift)
- [POCKETSPHINX_EXPORT float64 logmath_get_base](#) (logmath_t *lmath)
- [POCKETSPHINX_EXPORT int logmath_get_zero](#) (logmath_t *lmath)
- [POCKETSPHINX_EXPORT int logmath_get_width](#) (logmath_t *lmath)
- [POCKETSPHINX_EXPORT int logmath_get_shift](#) (logmath_t *lmath)
- [POCKETSPHINX_EXPORT logmath_t * logmath_retain](#) (logmath_t *lmath)
- [POCKETSPHINX_EXPORT int logmath_free](#) (logmath_t *lmath)
- [POCKETSPHINX_EXPORT int logmath_add_exact](#) (logmath_t *lmath, int logb_p, int logb_q)
- [POCKETSPHINX_EXPORT int logmath_add](#) (logmath_t *lmath, int logb_p, int logb_q)
- [POCKETSPHINX_EXPORT int logmath_log](#) (logmath_t *lmath, float64 p)
- [POCKETSPHINX_EXPORT float64 logmath_exp](#) (logmath_t *lmath, int logb_p)
- [POCKETSPHINX_EXPORT int logmath_ln_to_log](#) (logmath_t *lmath, float64 log_p)
- [POCKETSPHINX_EXPORT float64 logmath_log_to_ln](#) (logmath_t *lmath, int logb_p)
- [POCKETSPHINX_EXPORT int logmath_log10_to_log](#) (logmath_t *lmath, float64 log_p)
- [POCKETSPHINX_EXPORT float logmath_log10_to_log_float](#) (logmath_t *lmath, float64 log_p)
- [POCKETSPHINX_EXPORT float64 logmath_log_to_log10](#) (logmath_t *lmath, int logb_p)
- [POCKETSPHINX_EXPORT float64 logmath_log_float_to_log10](#) (logmath_t *lmath, float log_p)

10.14.1 Detailed Description

Integer log math computation class.

10.14.2 Member Function Documentation

10.14.2.1 logmath_add()

```
POCKETSPHINX_EXPORT int logmath_add (
    logmath_t * lmath,
    int logb_p,
    int logb_q)
```

Add two values in log space (i.e. return $\log(\exp(p)+\exp(q))$)

References [POCKETSPHINX_EXPORT](#).

10.14.2.2 logmath_add_exact()

```
POCKETSPHINX_EXPORT int logmath_add_exact (  
    logmath_t * lmath,  
    int logb_p,  
    int logb_q)
```

Add two values in log space exactly and slowly (without using add table).

References [POCKETSPHINX_EXPORT](#).

10.14.2.3 logmath_exp()

```
POCKETSPHINX_EXPORT float64 logmath_exp (  
    logmath_t * lmath,  
    int logb_p)
```

Convert integer log in base B to linear floating point.

References [POCKETSPHINX_EXPORT](#).

10.14.2.4 logmath_free()

```
POCKETSPHINX_EXPORT int logmath_free (  
    logmath_t * lmath)
```

Free a log table.

Returns

new reference count (0 if freed completely)

References [POCKETSPHINX_EXPORT](#).

10.14.2.5 logmath_get_base()

```
POCKETSPHINX_EXPORT float64 logmath_get_base (  
    logmath_t * lmath)
```

Get the log base.

References [POCKETSPHINX_EXPORT](#).

10.14.2.6 logmath_get_shift()

```
POCKETSPHINX_EXPORT int logmath_get_shift (  
    logmath_t * lmath)
```

Get the shift of the values in a log table.

References [POCKETSPHINX_EXPORT](#).

10.14.2.7 logmath_get_table_shape()

```
POCKETSPHINX_EXPORT int32 logmath_get_table_shape (  
    logmath_t * lmath,  
    uint32 * out_size,  
    uint32 * out_width,  
    uint32 * out_shift)
```

Get the log table size and dimensions.

References [POCKETSPHINX_EXPORT](#).

10.14.2.8 logmath_get_width()

```
POCKETSPHINX_EXPORT int logmath_get_width (  
    logmath_t * lmath)
```

Get the width of the values in a log table.

References [POCKETSPHINX_EXPORT](#).

10.14.2.9 logmath_get_zero()

```
POCKETSPHINX_EXPORT int logmath_get_zero (  
    logmath_t * lmath)
```

Get the smallest possible value represented in this base.

References [POCKETSPHINX_EXPORT](#).

10.14.2.10 logmath_init()

```
POCKETSPHINX_EXPORT logmath_t * logmath_init (  
    float64 base,  
    int shift,  
    int use_table)
```

Initialize a log math computation table.

Parameters

<i>base</i>	The base B in which computation is to be done.
<i>shift</i>	Log values are shifted right by this many bits.
<i>use_table</i>	Whether to use an add table or not

Returns

The newly created log math table.

References [POCKETSPHINX_EXPORT](#).

10.14.2.11 logmath_ln_to_log()

```
POCKETSPHINX_EXPORT int logmath_ln_to_log (  
    logmath_t * lmath,  
    float64 log_p)
```

Convert natural log (in floating point) to integer log in base B.

References [POCKETSPHINX_EXPORT](#).

10.14.2.12 logmath_log()

```
POCKETSPHINX_EXPORT int logmath_log (  
    logmath_t * lmath,  
    float64 p)
```

Convert linear floating point number to integer log in base B.

References [POCKETSPHINX_EXPORT](#).

10.14.2.13 logmath_log10_to_log()

```
POCKETSPHINX_EXPORT int logmath_log10_to_log (  
    logmath_t * lmath,  
    float64 log_p)
```

Convert base 10 log (in floating point) to integer log in base B.

References [POCKETSPHINX_EXPORT](#).

10.14.2.14 logmath_log10_to_log_float()

```
POCKETSPHINX_EXPORT float logmath_log10_to_log_float (  
    logmath_t * lmath,  
    float64 log_p)
```

Convert base 10 log (in floating point) to float log in base B.

References [POCKETSPHINX_EXPORT](#).

10.14.2.15 logmath_log_float_to_log10()

```
POCKETSPHINX_EXPORT float64 logmath_log_float_to_log10 (  
    logmath_t * lmath,  
    float log_p)
```

Convert float log in base B to base 10 log.

10.14.2.16 logmath_log_to_ln()

```
POCKETSPHINX_EXPORT float64 logmath_log_to_ln (  
    logmath_t * lmath,  
    int logb_p)
```

Convert integer log in base B to natural log (in floating point).

References [POCKETSPHINX_EXPORT](#).

10.14.2.17 logmath_log_to_log10()

```
POCKETSPHINX_EXPORT float64 logmath_log_to_log10 (  
    logmath_t * lmath,  
    int logb_p)
```

Convert integer log in base B to base 10 log (in floating point).

References [POCKETSPHINX_EXPORT](#).

10.14.2.18 logmath_read()

```
POCKETSPHINX_EXPORT logmath_t * logmath_read (  
    const char * filename)
```

Memory-map (or read) a log table from a file.

References [POCKETSPHINX_EXPORT](#).

10.14.2.19 logmath_retain()

```
POCKETSPHINX_EXPORT logmath_t * logmath_retain (  
    logmath_t * lmath)
```

Retain ownership of a log table.

Returns

pointer to retained log table.

References [POCKETSPHINX_EXPORT](#).

10.14.2.20 logmath_write()

```
POCKETSPHINX_EXPORT int32 logmath_write (  
    logmath_t * lmath,  
    const char * filename)
```

Write a log table to a file.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

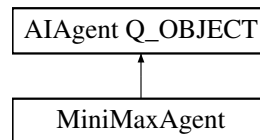
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/logmath.h

10.15 MiniMaxAgent Class Reference

The [MiniMaxAgent](#) class handles all the logic of picking a cell based on the MiniMax Algorithm.

```
#include <MiniMaxAgent.h>
```

Inheritance diagram for MiniMaxAgent:



Public Member Functions

- [MiniMaxAgent](#) (QObject *parent=nullptr, unsigned short depth, [BoardMarks](#) Almark)
MiniMaxAgent: a constructor function that creates the MiniMax Agent.
- int [play](#) () override
Start the minimax algorithm to choose a cell to play based on a given board.

Private Member Functions

- short [maxMove](#) ([Board](#) &board, unsigned short depth, short alpha, short beta) const
maxMove simulates the AI choice based on the final game states, as it wants to win it will choose the maximum score of each simulated move.
- short [minMove](#) ([Board](#) &board, unsigned short depth, short alpha, short beta) const
minMove simulates the player choice based on the final game states, as it wants to win it will choose the minimum score of each simulated move.
- short [score](#) (const [BoardState](#) state) const
score associates a score to the final states of a game.

Private Attributes

- const unsigned short [depth_](#)
depth cutoff of the minimax algorithm's search tree.
- const [BoardMarks](#) [Almark_](#)
The mark (X or O) of the AI.
- const [BoardMarks](#) [playerMark_](#)
The mark (X or O) of the human player.

Static Private Attributes

- static constexpr short [AI_WIN_SCORE](#) = 1
AI_WIN_SCORE defines the score of the final board when the AI wins.
- static constexpr short [PLAYER_WIN_SCORE](#) = -1
PLAYER_WIN_SCORE defines the score of the final board when the human player wins.
- static constexpr short [TIE_SCORE](#) = 0
TIE_SCORE defines the score of the final board when the the game is a tie.

10.15.1 Detailed Description

The [MiniMaxAgent](#) class handles all the logic of picking a cell based on the MiniMax Algorithm.

It simulates all possible plays taking turns simulating an AI turn then a [Player](#) turn until a final state is reached, then it scores the final state by giving a negative value if the [Player](#) wins and a positive value if it wins, then backtracks picking the minimum score if it is the player turn and the max score if its the AI turn.

10.15.2 Constructor & Destructor Documentation

10.15.2.1 MiniMaxAgent()

```
MiniMaxAgent::MiniMaxAgent (
    QObject * parent = nullptr,
    unsigned short depth,
    BoardMarks AImark)
```

[MiniMaxAgent](#): a constructor function that creates the MiniMax Agent.

Parameters

<i>depth</i>	cutoff of the minimax algorithm's search tree.
<i>AImark</i>	The mark (X or O) of the AI.
<i>playerMark</i>	The mark (X or O) of the human player.

References [AImark_](#), [depth_](#), and [playerMark_](#).

10.15.3 Member Function Documentation

10.15.3.1 maxMove()

```
short MiniMaxAgent::maxMove (
    Board & board,
    unsigned short depth,
    short alpha,
    short beta) const [private]
```

maxMove simulates the AI choice based on the final game states, as it wants to win it will choose the maximum score of each simulated move.

Parameters

<i>board</i>	a reference to the board model.
<i>depth</i>	The current depth of the search tree.
<i>alpha</i>	The minimum score that the maximizing player is assured of.
<i>beta</i>	The maximum score that the minimizing player is assured of.

Returns

References [AImark_](#), [Board::at\(\)](#), [Empty](#), [Board::evaluateBoard\(\)](#), [minMove\(\)](#), [NoWinner](#), [Board::resetCell\(\)](#), [score\(\)](#), [Board::setPlayerInput\(\)](#), and [Board::size\(\)](#).

Referenced by [minMove\(\)](#).

10.15.3.2 minMove()

```
short MiniMaxAgent::minMove (
    Board & board,
    unsigned short depth,
    short alpha,
    short beta) const [private]
```

minMove simulates the player choice based on the final game states, as it wants to win it will choose the minimum score of each simulated move.

Parameters

<i>board</i>	a reference to the board model.
<i>depth</i>	The current depth of the search tree.
<i>alpha</i>	The minimum score that the maximizing player is assured of.
<i>beta</i>	The maximum score that the minimizing player is assured of.

Returns

References [Board::at\(\)](#), [Empty](#), [Board::evaluateBoard\(\)](#), [maxMove\(\)](#), [NoWinner](#), [playerMark_](#), [Board::resetCell\(\)](#), [score\(\)](#), [Board::setPlayerInput\(\)](#), and [Board::size\(\)](#).

Referenced by [maxMove\(\)](#), and [play\(\)](#).

10.15.3.3 play()

```
int MiniMaxAgent::play () [override]
```

Start the minimax algorithm to choose a cell to play based on a given board.

Parameters

<i>board</i>	a reference to the board model.
--------------	---------------------------------

Returns

the chosen cell index (in a 1D array) that the AI played.

References [Almark_](#), [Board::at\(\)](#), [depth_](#), [Empty](#), [Board::evaluateBoard\(\)](#), [defaults::INVALID_CELL](#), [minMove\(\)](#), [NoWinner](#), [Board::resetCell\(\)](#), [Board::setPlayerInput\(\)](#), and [Board::size\(\)](#).

Referenced by [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), and [TEST\(\)](#).

10.15.3.4 score()

```
short MiniMaxAgent::score (
    const BoardState state) const [private]
```

score associates a score to the final states of a game.

Parameters

<code>state</code>	is an input paramater that specifies the state of the game.
--------------------	---

Returns

a score based on the state of the game.

References [AI_WIN_SCORE](#), [Almark_](#), [O](#), [OWins](#), [PLAYER_WIN_SCORE](#), [TIE_SCORE](#), [X](#), and [XWins](#).

Referenced by [maxMove\(\)](#), and [minMove\(\)](#).

10.15.4 Member Data Documentation

10.15.4.1 AI_WIN_SCORE

```
short MiniMaxAgent::AI_WIN_SCORE = 1 [static], [constexpr], [private]
```

AI_WIN_SCORE defines the score of the final board when the AI wins.

Referenced by [score\(\)](#).

10.15.4.2 Almark_

```
const BoardMarks MiniMaxAgent::Almark_ [private]
```

The mark (X or O) of the AI.

Referenced by [maxMove\(\)](#), [MiniMaxAgent\(\)](#), [play\(\)](#), and [score\(\)](#).

10.15.4.3 depth_

```
const unsigned short MiniMaxAgent::depth_ [private]
```

depth cutoff of the minimax algorithm's search tree.

Referenced by [MiniMaxAgent\(\)](#), and [play\(\)](#).

10.15.4.4 PLAYER_WIN_SCORE

```
short MiniMaxAgent::PLAYER_WIN_SCORE = -1 [static], [constexpr], [private]
```

PLAYER_WIN_SCORE defines the score of the final board when the human player wins.

Referenced by [score\(\)](#).

10.15.4.5 playerMark_

```
const BoardMarks MiniMaxAgent::playerMark_ [private]
```

The mark (X or O) of the human player.

Referenced by [MiniMaxAgent\(\)](#), and [minMove\(\)](#).

10.15.4.6 TIE_SCORE

```
short MiniMaxAgent::TIE_SCORE = 0 [static], [constexpr], [private]
```

TIE_SCORE defines the score of the final board when the the game is a tie.

Referenced by [score\(\)](#).

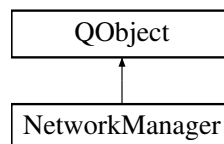
The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[MiniMaxAgent.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[MiniMaxAgent.cpp](#)

10.16 NetworkManager Class Reference

```
#include <NetworkManager.h>
```

Inheritance diagram for NetworkManager:



Signals

- void [connected](#) ()
- void [disconnected](#) ()
- void [errorOccurred](#) (const QString &error)
- void [messageSent](#) (const QString &message)

Public Member Functions

- [NetworkManager](#) (QObject *parent=nullptr)
- void [connectToServer](#) (const QString &ip, quint16 port)
- void [sendMessage](#) (const QString &message)

Private Slots

- void [onConnected](#) ()
- void [onDisconnected](#) ()
- void [onError](#) (QAbstractSocket::SocketError socketError)
- void [onSocketReadyRead](#) ()

Private Attributes

- QTcpSocket * [socket_](#)

10.16.1 Constructor & Destructor Documentation

10.16.1.1 NetworkManager()

```
NetworkManager::NetworkManager (  
    QObject * parent = nullptr) [explicit]
```

References [onConnected\(\)](#), [onDisconnected\(\)](#), [onError\(\)](#), [TTTController::onSocketReadyRead\(\)](#), and [socket_](#).

10.16.2 Member Function Documentation

10.16.2.1 connected

```
void NetworkManager::connected () [signal]
```

Referenced by [onConnected\(\)](#).

10.16.2.2 connectToServer()

```
void NetworkManager::connectToServer (  
    const QString & ip,  
    quint16 port)
```

References [socket_](#).

10.16.2.3 disconnected

```
void NetworkManager::disconnected () [signal]
```

Referenced by [onDisconnected\(\)](#).

10.16.2.4 errorOccurred

```
void NetworkManager::errorOccurred (  
    const QString & error) [signal]
```

Referenced by [onError\(\)](#), and [sendMessage\(\)](#).

10.16.2.5 messageSent

```
void NetworkManager::messageSent (  
    const QString & message) [signal]
```

Referenced by [sendMessage\(\)](#).

10.16.2.6 onConnected

```
void NetworkManager::onConnected () [private], [slot]
```

References [connected\(\)](#).

Referenced by [NetworkManager\(\)](#).

10.16.2.7 onDisconnected

```
void NetworkManager::onDisconnected () [private], [slot]
```

References [disconnected\(\)](#).

Referenced by [NetworkManager\(\)](#).

10.16.2.8 onError

```
void NetworkManager::onError (  
    QAbstractSocket::SocketError socketError) [private], [slot]
```

References [errorOccurred\(\)](#), and [socket_](#).

Referenced by [NetworkManager\(\)](#).

10.16.2.9 onSocketReadyRead

```
void NetworkManager::onSocketReadyRead () [private], [slot]
```

References [socket_](#).

10.16.2.10 sendMessage()

```
void NetworkManager::sendMessage (  
    const QString & message)
```

References [errorOccurred\(\)](#), [messageSent\(\)](#), and [socket_](#).

10.16.3 Member Data Documentation

10.16.3.1 socket_

```
QTcpSocket* NetworkManager::socket_ [private]
```

Referenced by [connectToServer\(\)](#), [NetworkManager\(\)](#), [onError\(\)](#), [onSocketReadyRead\(\)](#), and [sendMessage\(\)](#).

The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network/[NetworkManager.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network/[NetworkManager.cpp](#)

10.17 ngram_class_t Struct Reference

Word class in an N-Gram model.

```
#include <pocketsphinx/model.h>
```

10.17.1 Detailed Description

Word class in an N-Gram model.

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.18 ngram_iter_t Struct Reference

M-gram (yes, **M**-gram) iterator object.

```
#include <pocketsphinx/model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) [int32](#) const * [ngram_iter_get](#) ([ngram_iter_t](#) *itor, [int32](#) *out_score, [int32](#) *out←_bowt)
- [POCKETSPHINX_EXPORT](#) [ngram_iter_t](#) * [ngram_iter_successors](#) ([ngram_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) [ngram_iter_t](#) * [ngram_iter_next](#) ([ngram_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) void [ngram_iter_free](#) ([ngram_iter_t](#) *itor)

10.18.1 Detailed Description

M-gram (yes, **M**-gram) iterator object.

This is an iterator over the N-Gram successors of a given word or N-1-Gram, that is why it is called "M" and not "N".

10.18.2 Member Function Documentation

10.18.2.1 ngram_iter_free()

```
POCKETSPHINX_EXPORT void ngram_iter_free (  
    ngram_iter_t * itor)
```

Terminate an M-gram iterator.

References [POCKETSPHINX_EXPORT](#).

10.18.2.2 ngram_iter_get()

```
POCKETSPHINX_EXPORT int32 const * ngram_iter_get (  
    ngram_iter_t * itor,  
    int32 * out_score,  
    int32 * out_bowt)
```

Get information from the current M-gram in an iterator.

Parameters

<i>out_score</i>	Output: Score for this M-gram (including any word penalty and language weight).
<i>out_bowt</i>	Output: Backoff weight for this M-gram.

Returns

read-only array of word IDs.

References [POCKETSPHINX_EXPORT](#).

10.18.2.3 ngram_iter_next()

```
POCKETSPHINX_EXPORT ngram_iter_t * ngram_iter_next (  
    ngram_iter_t * itor)
```

Advance an M-gram iterator.

References [POCKETSPHINX_EXPORT](#).

10.18.2.4 ngram_iter_successors()

```
POCKETSPHINX_EXPORT ngram_iter_t * ngram_iter_successors (  
    ngram_iter_t * itor)
```

Iterate over all M-gram successors of an M-1-gram.

Parameters

<i>itor</i>	Iterator pointing to the M-1-gram to get successors of.
-------------	---

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.19 ngram_model_set_iter_t Struct Reference

Iterator over language models in a set.

```
#include <pocketsphinx/model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) [ngram_model_set_iter_t](#) * [ngram_model_set_iter_next](#) ([ngram_model_set_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) void [ngram_model_set_iter_free](#) ([ngram_model_set_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) [ngram_model_t](#) * [ngram_model_set_iter_model](#) ([ngram_model_set_iter_t](#) *itor, char const **lmname)

10.19.1 Detailed Description

Iterator over language models in a set.

10.19.2 Member Function Documentation

10.19.2.1 ngram_model_set_iter_free()

```
POCKETSPHINX\_EXPORT void ngram\_model\_set\_iter\_free (  
    ngram\_model\_set\_iter\_t * itor)
```

Finish iteration over a language model set.

References [POCKETSPHINX_EXPORT](#).

10.19.2.2 ngram_model_set_iter_model()

```
POCKETSPHINX\_EXPORT ngram\_model\_t * ngram\_model\_set\_iter\_model (  
    ngram\_model\_set\_iter\_t * itor,  
    char const ** lmname)
```

Get language model and associated name from an iterator.

Parameters

<i>itor</i>	the iterator
<i>lname</i>	Output: string name associated with this language model.

Returns

Language model pointed to by this iterator.

References [POCKETSPHINX_EXPORT](#).

10.19.2.3 ngram_model_set_iter_next()

```
POCKETSPHINX_EXPORT ngram_model_set_iter_t * ngram_model_set_iter_next (
    ngram_model_set_iter_t * itor)
```

Move to the next language model in a set.

Returns

iterator pointing to the next language model, or NULL if no models remain.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.20 ngram_model_t Struct Reference

N-Gram based language model.

```
#include <pocketsphinx/model.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ngram_model_t * ngram_model_read](#) ([ps_config_t](#) *config, const char *file_name, [ngram_file_type_t](#) file_type, [logmath_t](#) *lmath)
- [POCKETSPHINX_EXPORT int ngram_model_write](#) ([ngram_model_t](#) *model, const char *file_name, [ngram_file_type_t](#) format)
- [POCKETSPHINX_EXPORT ngram_file_type_t ngram_file_name_to_type](#) (const char *file_name)
- [POCKETSPHINX_EXPORT ngram_file_type_t ngram_str_to_type](#) (const char *str_name)
- [POCKETSPHINX_EXPORT char const * ngram_type_to_str](#) (int type)
- [POCKETSPHINX_EXPORT ngram_model_t * ngram_model_retain](#) ([ngram_model_t](#) *model)
- [POCKETSPHINX_EXPORT int ngram_model_free](#) ([ngram_model_t](#) *model)
- [POCKETSPHINX_EXPORT int ngram_model_casefold](#) ([ngram_model_t](#) *model, int kase)
- [POCKETSPHINX_EXPORT int ngram_model_apply_weights](#) ([ngram_model_t](#) *model, [float32](#) lw, [float32](#) wip)
- [POCKETSPHINX_EXPORT float32 ngram_model_get_weights](#) ([ngram_model_t](#) *model, [int32](#) *out_log_wip)

- `POCKETSPHINX_EXPORT int32 ngram_score (ngram_model_t *model, const char *word,...)`
- `POCKETSPHINX_EXPORT int32 ngram_tg_score (ngram_model_t *model, int32 w3, int32 w2, int32 w1, int32 *n_used)`
- `POCKETSPHINX_EXPORT int32 ngram_bg_score (ngram_model_t *model, int32 w2, int32 w1, int32 *n_used)`
- `POCKETSPHINX_EXPORT int32 ngram_ng_score (ngram_model_t *model, int32 wid, int32 *history, int32 n_hist, int32 *n_used)`
- `POCKETSPHINX_EXPORT int32 ngram_probv (ngram_model_t *model, const char *word,...)`
- `POCKETSPHINX_EXPORT int32 ngram_prob (ngram_model_t *model, const char *const *words, int32 n)`
- `POCKETSPHINX_EXPORT int32 ngram_ng_prob (ngram_model_t *model, int32 wid, int32 *history, int32 n_hist, int32 *n_used)`
- `POCKETSPHINX_EXPORT int32 ngram_score_to_prob (ngram_model_t *model, int32 score)`
- `POCKETSPHINX_EXPORT int32 ngram_wid (ngram_model_t *model, const char *word)`
- `POCKETSPHINX_EXPORT const char * ngram_word (ngram_model_t *model, int32 wid)`
- `POCKETSPHINX_EXPORT int32 ngram_unknown_wid (ngram_model_t *model)`
- `POCKETSPHINX_EXPORT int32 ngram_zero (ngram_model_t *model)`
- `POCKETSPHINX_EXPORT int32 ngram_model_get_size (ngram_model_t *model)`
- `POCKETSPHINX_EXPORT uint32 const * ngram_model_get_counts (ngram_model_t *model)`
- `POCKETSPHINX_EXPORT ngram_iter_t * ngram_model_mgrams (ngram_model_t *model, int m)`
- `POCKETSPHINX_EXPORT ngram_iter_t * ngram_iter (ngram_model_t *model, const char *word,...)`
- `POCKETSPHINX_EXPORT ngram_iter_t * ngram_ng_iter (ngram_model_t *model, int32 wid, int32 *history, int32 n_hist)`
- `POCKETSPHINX_EXPORT int32 ngram_model_add_word (ngram_model_t *model, const char *word, float32 weight)`
- `POCKETSPHINX_EXPORT int32 ngram_model_read_classdef (ngram_model_t *model, const char *file_name)`
- `POCKETSPHINX_EXPORT int32 ngram_model_add_class (ngram_model_t *model, const char *classname, float32 classweight, char **words, const float32 *weights, int32 n_words)`
- `POCKETSPHINX_EXPORT int32 ngram_model_add_class_word (ngram_model_t *model, const char *classname, const char *word, float32 weight)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_init (ps_config_t *config, ngram_model_t **models, char **names, const float32 *weights, int32 n_models)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_read (ps_config_t *config, const char *lmctfile, logmath_t *lmath)`
- `POCKETSPHINX_EXPORT int32 ngram_model_set_count (ngram_model_t *set)`
- `POCKETSPHINX_EXPORT ngram_model_set_iter_t * ngram_model_set_iter (ngram_model_t *set)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_select (ngram_model_t *set, const char *name)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_lookup (ngram_model_t *set, const char *name)`
- `POCKETSPHINX_EXPORT const char * ngram_model_set_current (ngram_model_t *set)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_interp (ngram_model_t *set, const char **names, const float32 *weights)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_add (ngram_model_t *set, ngram_model_t *model, const char *name, float32 weight, int reuse_widmap)`
- `POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_remove (ngram_model_t *set, const char *name, int reuse_widmap)`
- `POCKETSPHINX_EXPORT void ngram_model_set_map_words (ngram_model_t *set, const char **words, int32 n_words)`
- `POCKETSPHINX_EXPORT int32 ngram_model_set_current_wid (ngram_model_t *set, int32 set_wid)`
- `POCKETSPHINX_EXPORT int32 ngram_model_set_known_wid (ngram_model_t *set, int32 set_wid)`
- `POCKETSPHINX_EXPORT void ngram_model_flush (ngram_model_t *lm)`

10.20.1 Detailed Description

N-Gram based language model.

10.20.2 Member Function Documentation

10.20.2.1 ngram_bg_score()

```
POCKETSPHINX_EXPORT int32 ngram_bg_score (  
    ngram_model_t * model,  
    int32 w2,  
    int32 w1,  
    int32 * n_used)
```

Quick bigram score lookup.

References [POCKETSPHINX_EXPORT](#).

10.20.2.2 ngram_file_name_to_type()

```
POCKETSPHINX_EXPORT ngram_file_type_t ngram_file_name_to_type (  
    const char * file_name)
```

Guess the file type for an N-Gram model from the filename.

Returns

the guessed file type, or NGRAM_INVALID if none could be guessed.

References [POCKETSPHINX_EXPORT](#).

10.20.2.3 ngram_iter()

```
POCKETSPHINX_EXPORT ngram_iter_t * ngram_iter (  
    ngram_model_t * model,  
    const char * word,  
    ...)
```

Get an iterator over M-grams pointing to the specified M-gram.

References [POCKETSPHINX_EXPORT](#).

10.20.2.4 ngram_model_add_class()

```
POCKETSPHINX_EXPORT int32 ngram_model_add_class (  
    ngram_model_t * model,  
    const char * classname,  
    float32 classweight,  
    char ** words,  
    const float32 * weights,  
    int32 n_words)
```

Add a new class to a language model.

If `classname` already exists in the unigram set for `model`, then it will be converted to a class tag, and `classweight` will be ignored. Otherwise, a new unigram will be created as in [ngram_model_add_word\(\)](#).

References [POCKETSPHINX_EXPORT](#).

10.20.2.5 ngram_model_add_class_word()

```
POCKETSPHINX_EXPORT int32 ngram_model_add_class_word (  
    ngram_model_t * model,  
    const char * classname,  
    const char * word,  
    float32 weight)
```

Add a word to a class in a language model.

Parameters

<i>model</i>	The model to add a word to.
<i>classname</i>	Name of the class to add this word to.
<i>word</i>	Text of the word to add.
<i>weight</i>	Weight of this word relative to the within-class uniform distribution.

Returns

The word ID for the new word.

References [POCKETSPHINX_EXPORT](#).

10.20.2.6 ngram_model_add_word()

```
POCKETSPHINX_EXPORT int32 ngram_model_add_word (  
    ngram_model_t * model,  
    const char * word,  
    float32 weight)
```

Add a word (unigram) to the language model.

Note

The semantics of this are not particularly well-defined for model sets, and may be subject to change. Currently this will add the word to all of the submodels

Parameters

<i>model</i>	The model to add a word to.
<i>word</i>	Text of the word to add.
<i>weight</i>	Weight of this word relative to the uniform distribution.

Returns

The word ID for the new word.

References [POCKETSPHINX_EXPORT](#).

10.20.2.7 ngram_model_apply_weights()

```
POCKETSPHINX_EXPORT int ngram_model_apply_weights (  
    ngram_model_t * model,  
    float32 lw,  
    float32 wip)
```

Apply a language weight, insertion penalty, and unigram weight to a language model.

This will change the values output by [ngram_score\(\)](#) and friends. This is done for efficiency since in decoding, these are the only values we actually need. Call [ngram_prob\(\)](#) if you want the "raw" N-Gram probability estimate.

To remove all weighting, call `ngram_apply_weights(model, 1.0, 1.0)`.

References [POCKETSPHINX_EXPORT](#).

10.20.2.8 ngram_model_casefold()

```
POCKETSPHINX_EXPORT int ngram_model_casefold (  
    ngram_model_t * model,  
    int kase)
```

Case-fold word strings in an N-Gram model.

WARNING: This is not Unicode aware, so any non-ASCII characters will not be converted.

References [POCKETSPHINX_EXPORT](#).

10.20.2.9 ngram_model_flush()

```
POCKETSPHINX_EXPORT void ngram_model_flush (  
    ngram_model_t * lm)
```

Flush any cached N-Gram information

10.20.2.10 ngram_model_free()

```
POCKETSPHINX_EXPORT int ngram_model_free (  
    ngram_model_t * model)
```

Release memory associated with an N-Gram model.

Returns

new reference count (0 if freed completely)

10.20.2.11 ngram_model_get_counts()

```
POCKETSPHINX_EXPORT uint32 const * ngram_model_get_counts (  
    ngram_model_t * model)
```

Get the counts of the various N-grams in the model.

10.20.2.12 ngram_model_get_size()

```
POCKETSPHINX_EXPORT int32 ngram_model_get_size (  
    ngram_model_t * model)
```

Get the order of the N-gram model (i.e. the "N" in "N-gram")

References [POCKETSPHINX_EXPORT](#).

10.20.2.13 ngram_model_get_weights()

```
POCKETSPHINX_EXPORT float32 ngram_model_get_weights (  
    ngram_model_t * model,  
    int32 * out_log_wip)
```

Get the current weights from a language model.

Parameters

<i>model</i>	The model in question.
<i>out_log_wip</i>	Output: (optional) logarithm of word insertion penalty.

Returns

language weight.

References [POCKETSPHINX_EXPORT](#).

10.20.2.14 ngram_model_mgrams()

```
POCKETSPHINX_EXPORT ngram_iter_t * ngram_model_mgrams (  
    ngram_model_t * model,  
    int m)
```

Iterate over all M-grams.

Parameters

<i>model</i>	Language model to query.
<i>m</i>	Order of the M-Grams requested minus one (i.e. order of the history)

Returns

An iterator over the requested M, or NULL if no N-grams of order M+1 exist.

References [POCKETSPHINX_EXPORT](#).

10.20.2.15 ngram_model_read()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_read (  
    ps_config_t * config,  
    const char * file_name,  
    ngram_file_type_t file_type,  
    logmath_t * lmath)
```

Read an N-Gram model from a file on disk.

Parameters

<i>config</i>	Optional pointer to a set of command-line arguments. Recognized arguments are:
---------------	--

- -mmap (boolean) whether to use memory-mapped I/O
- -lw ([float32](#)) language weight to apply to the model
- -wip ([float32](#)) word insertion penalty to apply to the model

Parameters

<i>file_name</i>	path to the file to read.
<i>file_type</i>	type of the file, or NGRAM_AUTO to determine automatically.
<i>lmath</i>	Log-math parameters to use for probability calculations. Ownership of this object is assumed by the newly created ngram_model_t , and you should not attempt to free it manually. If you wish to reuse it elsewhere, you must retain it with <code>logmath_retain()</code> .

Returns

newly created [ngram_model_t](#).

References [POCKETSPHINX_EXPORT](#).

10.20.2.16 ngram_model_read_classdef()

```
POCKETSPHINX_EXPORT int32 ngram_model_read_classdef (
    ngram_model_t * model,
    const char * file_name)
```

Read a class definition file and add classes to a language model.

This function assumes that the class tags have already been defined as unigrams in the language model. All words in the class definition will be added to the vocabulary as special in-class words. For this reason it is necessary that they not have the same names as any words in the general unigram distribution. The convention is to suffix them with `":class_tag"`, where `class_tag` is the class tag minus the enclosing square brackets.

Returns

0 for success, <0 for error

References [POCKETSPHINX_EXPORT](#).

10.20.2.17 ngram_model_retain()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_retain (
    ngram_model_t * model)
```

Retain ownership of an N-Gram model.

Returns

Pointer to retained model.

References [POCKETSPHINX_EXPORT](#).

10.20.2.18 ngram_model_set_add()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_add (
    ngram_model_t * set,
    ngram_model_t * model,
    const char * name,
    float32 weight,
    int reuse_widmap)
```

Add a language model to a set.

Parameters

<i>set</i>	The language model set to add to.
<i>model</i>	The language model to add.
<i>name</i>	The name to associate with this model.
<i>weight</i>	Interpolation weight for this model, relative to the uniform distribution. 1.0 is a safe value.
<i>reuse_widmap</i>	Reuse the existing word-ID mapping in <i>set</i> . Any new words present in <i>model</i> will not be added to the word-ID mapping in this case.

References [POCKETSPHINX_EXPORT](#).

10.20.2.19 ngram_model_set_count()

```
POCKETSPHINX_EXPORT int32 ngram_model_set_count (
    ngram_model_t * set)
```

Returns the number of language models in a set.

10.20.2.20 ngram_model_set_current()

```
POCKETSPHINX_EXPORT const char * ngram_model_set_current (
    ngram_model_t * set)
```

Get the current language model name, if any.

References [POCKETSPHINX_EXPORT](#).

10.20.2.21 ngram_model_set_current_wid()

```
POCKETSPHINX_EXPORT int32 ngram_model_set_current_wid (
    ngram_model_t * set,
    int32 set_wid)
```

Query the word-ID mapping for the current language model.

Returns

the local word ID in the current language model, or `NGRAM_INVALID_WID` if *set_wid* is invalid or interpolation is enabled.

References [POCKETSPHINX_EXPORT](#).

10.20.2.22 ngram_model_set_init()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_init (
    ps_config_t * config,
    ngram_model_t ** models,
    char ** names,
    const float32 * weights,
    int32 n_models)
```

Create a set of language models sharing a common space of word IDs.

This function creates a meta-language model which groups together a set of language models, synchronizing word IDs between them. To use this language model, you can either select a submodel to use exclusively using [ngram_model_set_select\(\)](#), or interpolate between scores from all models. To do the latter, you can either pass a non-NULL value of the `weights` parameter, or re-activate interpolation later on by calling [ngram_model_set_interp\(\)](#).

In order to make this efficient, there are some restrictions on the models that can be grouped together. The most important (and currently the only) one is that they **must** all share the same log-math parameters.

Parameters

<i>config</i>	Any configuration parameters to be shared between models.
<i>models</i>	Array of pointers to previously created language models.
<i>names</i>	Array of strings to use as unique identifiers for LMs.
<i>weights</i>	Array of weights to use in interpolating LMs, or NULL for no interpolation.
<i>n_models</i>	Number of elements in the arrays passed to this function.

References [POCKETSPHINX_EXPORT](#).

10.20.2.23 ngram_model_set_interp()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_interp (
    ngram_model_t * set,
    const char ** names,
    const float32 * weights)
```

Set interpolation weights for a set and enables interpolation.

If `weights` is NULL, any previously initialized set of weights will be used. If no weights were specified to [ngram_model_set_init\(\)](#), then a uniform distribution will be used.

References [POCKETSPHINX_EXPORT](#).

10.20.2.24 ngram_model_set_iter()

```
POCKETSPHINX_EXPORT ngram_model_set_iter_t * ngram_model_set_iter (
    ngram_model_t * set)
```

Begin iterating over language models in a set.

Returns

iterator pointing to the first language model, or NULL if no models remain.

References [POCKETSPHINX_EXPORT](#).

10.20.2.25 ngram_model_set_known_wid()

```
POCKETSPHINX_EXPORT int32 ngram_model_set_known_wid (  
    ngram_model_t * set,  
    int32 set_wid)
```

Test whether a word ID corresponds to a known word in the current state of the language model set.

Returns

If there is a current language model, returns non-zero if `set_wid` corresponds to a known word in that language model. Otherwise, returns non-zero if `set_wid` corresponds to a known word in any language model.

References [POCKETSPHINX_EXPORT](#).

10.20.2.26 ngram_model_set_lookup()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_lookup (  
    ngram_model_t * set,  
    const char * name)
```

Look up a language model by name from a set.

Returns

language model corresponding to `name`, or NULL if no language model by that name exists.

References [POCKETSPHINX_EXPORT](#).

10.20.2.27 ngram_model_set_map_words()

```
POCKETSPHINX_EXPORT void ngram_model_set_map_words (  
    ngram_model_t * set,  
    const char ** words,  
    int32 n_words)
```

Set the word-to-ID mapping for this model set.

References [POCKETSPHINX_EXPORT](#).

10.20.2.28 ngram_model_set_read()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_read (
    ps_config_t * config,
    const char * lmctlfile,
    logmath_t * lmath)
```

Read a set of language models from a control file.

This file creates a language model set from a "control file" of the type used in Sphinx-II and Sphinx-III. File format (optional stuff is indicated by enclosing in []):

```
[{ LMClassFileName LMClassFilename ... }]
TrigramLMFileName LMName [{ LMClassName LMClassName ... }]
TrigramLMFileName LMName [{ LMClassName LMClassName ... }]
...
(There should be whitespace around the { and } delimiters.)
```

This is an extension of the older format that had only TrigramLMFileName and LMName pairs. The new format allows a set of LMClass files to be read in and referred to by the trigram LMs.

No "comments" allowed in this file.

Parameters

<i>config</i>	Configuration parameters.
<i>lmctlfile</i>	Path to the language model control file.
<i>lmath</i>	Log-math parameters to use for probability calculations. Ownership of this object is assumed by the newly created ngram_model_t , and you should not attempt to free it manually. If you wish to reuse it elsewhere, you must retain it with <code>logmath_retain()</code> .

Returns

newly created language model set.

References [POCKETSPHINX_EXPORT](#).

10.20.2.29 ngram_model_set_remove()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_remove (
    ngram_model_t * set,
    const char * name,
    int reuse_widmap)
```

Remove a language model from a set.

Parameters

<i>set</i>	The language model set to remove from.
<i>name</i>	The name associated with the model to remove.
<i>reuse_widmap</i>	Reuse the existing word-ID mapping in <i>set</i> .

References [POCKETSPHINX_EXPORT](#).

10.20.2.30 ngram_model_set_select()

```
POCKETSPHINX_EXPORT ngram_model_t * ngram_model_set_select (
    ngram_model_t * set,
    const char * name)
```

Select a single language model from a set for scoring.

Returns

the newly selected language model, or NULL if no language model by that name exists.

References [POCKETSPHINX_EXPORT](#).

10.20.2.31 ngram_model_write()

```
POCKETSPHINX_EXPORT int ngram_model_write (  
    ngram_model_t * model,  
    const char * file_name,  
    ngram_file_type_t format)
```

Write an N-Gram model to disk.

Returns

0 for success, <0 on error

References [POCKETSPHINX_EXPORT](#).

10.20.2.32 ngram_ng_iter()

```
POCKETSPHINX_EXPORT ngram_iter_t * ngram_ng_iter (  
    ngram_model_t * model,  
    int32 wid,  
    int32 * history,  
    int32 n_hist)
```

Get an iterator over M-grams pointing to the specified M-gram.

References [POCKETSPHINX_EXPORT](#).

10.20.2.33 ngram_ng_prob()

```
POCKETSPHINX_EXPORT int32 ngram_ng_prob (  
    ngram_model_t * model,  
    int32 wid,  
    int32 * history,  
    int32 n_hist,  
    int32 * n_used)
```

Quick "raw" probability lookup for a general N-Gram.

See documentation for [ngram_ng_score\(\)](#) and [ngram_apply_weights\(\)](#) for an explanation of this.

References [POCKETSPHINX_EXPORT](#).

10.20.2.34 ngram_ng_score()

```
POCKETSPHINX_EXPORT int32 ngram_ng_score (  
    ngram_model_t * model,  
    int32 wid,  
    int32 * history,  
    int32 n_hist,  
    int32 * n_used)
```

Quick general N-Gram score lookup.

References [POCKETSPHINX_EXPORT](#).

10.20.2.35 ngram_prob()

```
POCKETSPHINX_EXPORT int32 ngram_prob (  
    ngram_model_t * model,  
    const char *const * words,  
    int32 n)
```

Get the "raw" log-probability for a general N-Gram.

This returns the log-probability of an N-Gram, as defined in the language model file, before any language weighting, interpolation, or insertion penalty has been applied.

Note

When backing off to a unigram from a bigram or trigram, the unigram weight (interpolation with uniform) is not removed.

References [POCKETSPHINX_EXPORT](#).

10.20.2.36 ngram_probv()

```
POCKETSPHINX_EXPORT int32 ngram_probv (  
    ngram_model_t * model,  
    const char * word,  
    ...)
```

Get the "raw" log-probability for a general N-Gram.

This returns the log-probability of an N-Gram, as defined in the language model file, before any language weighting, interpolation, or insertion penalty has been applied.

Note

When backing off to a unigram from a bigram or trigram, the unigram weight (interpolation with uniform) is not removed.

References [POCKETSPHINX_EXPORT](#).

10.20.2.37 ngram_score()

```
POCKETSPHINX_EXPORT int32 ngram_score (  
    ngram_model_t * model,  
    const char * word,  
    ...)
```

Get the score (scaled, interpolated log-probability) for a general N-Gram.

The argument list consists of the history words (as null-terminated strings) of the N-Gram, **in reverse order**, followed by NULL. Therefore, if you wanted to get the N-Gram score for "a whole joy", you would call:

```
score = ngram_score(model, "joy", "whole", "a", NULL);
```

This is not the function to use in decoding, because it has some overhead for looking up words. Use [ngram_ng_score\(\)](#), [ngram_tg_score\(\)](#), or [ngram_bg_score\(\)](#) instead. In the future there will probably be a version that takes a general language model state object, to support suffix-array LM and things like that.

If one of the words is not in the LM's vocabulary, the result will depend on whether this is an open or closed vocabulary language model. For an open-vocabulary model, unknown words are all mapped to the unigram <UNK> which has a non-zero probability and also participates in higher-order N-Grams. Therefore, you will get a score of some sort in this case.

For a closed-vocabulary model, unknown words are impossible and thus have zero probability. Therefore, if `word` is unknown, this function will return a "zero" log-probability, i.e. a large negative number. To obtain this number for comparison, call [ngram_zero\(\)](#).

References [POCKETSPHINX_EXPORT](#).

10.20.2.38 ngram_score_to_prob()

```
POCKETSPHINX_EXPORT int32 ngram_score_to_prob (  
    ngram_model_t * model,  
    int32 score)
```

Convert score to "raw" log-probability.

Note

The unigram weight (interpolation with uniform) is not removed, since there is no way to know which order of N-Gram generated `score`.

Parameters

<i>model</i>	The N-Gram model from which score was obtained.
<i>score</i>	The N-Gram score to convert

Returns

The raw log-probability value.

References [POCKETSPHINX_EXPORT](#).

10.20.2.39 ngram_str_to_type()

```
POCKETSPHINX_EXPORT ngram_file_type_t ngram_str_to_type (  
    const char * str_name)
```

Get the N-Gram file type from a string.

Returns

file type, or NGRAM_INVALID if no such file type exists.

References [POCKETSPHINX_EXPORT](#).

10.20.2.40 ngram_tg_score()

```
POCKETSPHINX_EXPORT int32 ngram_tg_score (  
    ngram_model_t * model,  
    int32 w3,  
    int32 w2,  
    int32 w1,  
    int32 * n_used)
```

Quick trigram score lookup.

References [POCKETSPHINX_EXPORT](#).

10.20.2.41 ngram_type_to_str()

```
POCKETSPHINX_EXPORT char const * ngram_type_to_str (  
    int type)
```

Get the canonical name for an N-Gram file type.

Returns

read-only string with the name for this file type, or NULL if no such type exists.

References [POCKETSPHINX_EXPORT](#).

10.20.2.42 ngram_unknown_wid()

```
POCKETSPHINX_EXPORT int32 ngram_unknown_wid (  
    ngram_model_t * model)
```

Get the unknown word ID for a language model.

Language models can be either "open vocabulary" or "closed vocabulary". The difference is that the former assigns a fixed non-zero unigram probability to unknown words, while the latter does not allow unknown words (or, equivalently, it assigns them zero probability). If this is a closed vocabulary model, this function will return `NGRAM_INVALID_WID`.

Returns

The ID for the unknown word, or `NGRAM_INVALID_WID` if none exists.

References [POCKETSPHINX_EXPORT](#).

10.20.2.43 ngram_wid()

```
POCKETSPHINX_EXPORT int32 ngram_wid (  
    ngram_model_t * model,  
    const char * word)
```

Look up numerical word ID.

References [POCKETSPHINX_EXPORT](#).

10.20.2.44 ngram_word()

```
POCKETSPHINX_EXPORT const char * ngram_word (  
    ngram_model_t * model,  
    int32 wid)
```

Look up word string for numerical word ID.

References [POCKETSPHINX_EXPORT](#).

10.20.2.45 ngram_zero()

```
POCKETSPHINX_EXPORT int32 ngram_zero (
    ngram_model_t * model)
```

Get the "zero" log-probability value for a language model.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

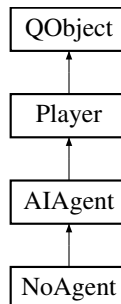
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[model.h](#)

10.21 NoAgent Class Reference

The [NoAgent](#) class is an empty class that does not play, it specifies the null object behaviour, so it represents the lack of an AI agent, which is useful to avoid using if conditions in code when checking on the existence of an AI agent.

```
#include <NoAgent.h>
```

Inheritance diagram for NoAgent:



Public Member Functions

- [NoAgent](#) ()
- [~NoAgent](#) () override
- int [play](#) (const [Board](#) &board) override
play does nothing to simulate a null agent behaviour.

Public Member Functions inherited from [AIAgent](#)

- [AIAgent](#) ()
- virtual [~AIAgent](#) ()
- virtual void [reset](#) ()
reset function that will reset the state of any search algorithm.

Public Member Functions inherited from [Player](#)

- [Player](#) (QObject *parent_=nullptr, [BoardMarks](#) playerMark, const [Board](#) &board_ref)
- virtual [~Player](#) ()
- virtual void [play](#) ()=0

Additional Inherited Members

Signals inherited from [Player](#)

- void [playerFinished](#) (const int &cell)

Protected Attributes inherited from [Player](#)

- const [Board](#) * [board_](#)
- const [BoardMarks](#) [mark_](#)

10.21.1 Detailed Description

The [NoAgent](#) class is an empty class that does not play, it specifies the null object behaviour, so it represents the lack of an AI agent, which is useful to avoid using if conditions in code when checking on the existence of an AI agent.

This class has no CPP file as it does not have any implementation details.

10.21.2 Constructor & Destructor Documentation

10.21.2.1 NoAgent()

```
NoAgent::NoAgent () [inline]
```

10.21.2.2 ~NoAgent()

```
NoAgent::~~NoAgent () [inline], [override]
```

10.21.3 Member Function Documentation

10.21.3.1 play()

```
int NoAgent::play (
    const Board & board) [inline], [override], [virtual]
```

play does nothing to simulate a null agent behaviour.

Parameters

<i>board</i>	a reference to the board model.
--------------	---------------------------------

Returns

an invalid cell location to not trigger any kind of updates.

Implements [AIAgent](#).

References [defaults::INVALID_CELL](#).

The documentation for this class was generated from the following file:

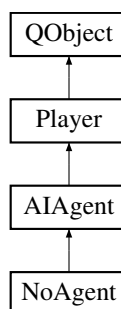
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[NoAgent.h](#)

10.22 Player Class Reference

The [Player](#) class is an abstract base class for different types of players in the Tic Tac Toe game, such as human players and AI players.

```
#include <Player.h>
```

Inheritance diagram for Player:

**Signals**

- void [playerFinished](#) (const int &cell)

Public Member Functions

- [Player](#) (QObject *parent_=nullptr, [BoardMarks](#) playerMark, const [Board](#) &borad_ref)
- virtual [~Player](#) ()
- virtual void [play](#) ()=0

Protected Attributes

- const [Board](#) * [board_](#)
- const [BoardMarks](#) [mark_](#)

10.22.1 Detailed Description

The [Player](#) class is an abstract base class for different types of players in the Tic Tac Toe game, such as human players and AI players.

10.22.2 Constructor & Destructor Documentation

10.22.2.1 Player()

```
Player::Player (
    QObject * parent_ = nullptr,
    BoardMarks playerMark,
    const Board & borad_ref) [inline]
```

References [mark_](#).

10.22.2.2 ~Player()

```
virtual Player::~~Player () [inline], [virtual]
```

10.22.3 Member Function Documentation

10.22.3.1 play()

```
virtual void Player::play () [pure virtual]
```

10.22.3.2 playerFinished

```
void Player::playerFinished (
    const int & cell) [signal]
```

10.22.4 Member Data Documentation

10.22.4.1 board_

```
const Board* Player::board_ [protected]
```

10.22.4.2 mark_

```
const BoardMarks Player::mark_ [protected]
```

Referenced by [Player\(\)](#).

The documentation for this class was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/[Player.h](#)

10.23 ps_alignment_iter_t Struct Reference

Iterator over entries in an alignment.

```
#include <pocketsphinx/alignment.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) const char * [ps_alignment_iter_name](#) ([ps_alignment_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) int [ps_alignment_iter_seg](#) ([ps_alignment_iter_t](#) *itor, int *start, int *duration)
- [POCKETSPHINX_EXPORT](#) [ps_alignment_iter_t](#) * [ps_alignment_iter_next](#) ([ps_alignment_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) [ps_alignment_iter_t](#) * [ps_alignment_iter_children](#) ([ps_alignment_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) int [ps_alignment_iter_free](#) ([ps_alignment_iter_t](#) *itor)

10.23.1 Detailed Description

Iterator over entries in an alignment.

10.23.2 Member Function Documentation

10.23.2.1 ps_alignment_iter_children()

```
POCKETSPHINX\_EXPORT ps\_alignment\_iter\_t * ps\_alignment\_iter\_children (  
    ps\_alignment\_iter\_t * itor)
```

Iterate over the children of the current alignment entry.

If there are no child nodes, NULL is returned.

References [POCKETSPHINX_EXPORT](#).

10.23.2.2 ps_alignment_iter_free()

```
POCKETSPHINX\_EXPORT int ps\_alignment\_iter\_free (  
    ps\_alignment\_iter\_t * itor)
```

Release an iterator before completing all iterations.

10.23.2.3 ps_alignment_iter_name()

```
POCKETSPHINX\_EXPORT const char * ps\_alignment\_iter\_name (  
    ps\_alignment\_iter\_t * itor)
```

Get the human-readable name of the current segment for an alignment.

Returns

Name of this segment as a string (word, phone, or state number). This pointer is owned by the iterator, do not free it yourself.

References [POCKETSPHINX_EXPORT](#).

10.23.2.4 `ps_alignment_iter_next()`

```
POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_iter_next (
    ps_alignment_iter_t * itor)
```

Move an alignment iterator forward.

If the end of the alignment is reached, this will free the iterator and return NULL.

References [POCKETSPHINX_EXPORT](#).

10.23.2.5 `ps_alignment_iter_seg()`

```
POCKETSPHINX_EXPORT int ps_alignment_iter_seg (
    ps_alignment_iter_t * itor,
    int * start,
    int * duration)
```

Get the timing and score information for the current segment of an alignment.

- start Output pointer for start frame
- duration Output pointer for duration

Returns

Acoustic score for this segment

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[alignment.h](#)

10.24 `ps_alignment_t` Struct Reference

Multi-level alignment (words, phones, states) over an utterance.

```
#include <pocketsphinx/alignment.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ps_alignment_t * ps_alignment_retain \(ps_alignment_t *al\)](#)
- [POCKETSPHINX_EXPORT int ps_alignment_free \(ps_alignment_t *al\)](#)
- [POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_words \(ps_alignment_t *al\)](#)
- [POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_phones \(ps_alignment_t *al\)](#)
- [POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_states \(ps_alignment_t *al\)](#)

10.24.1 Detailed Description

Multi-level alignment (words, phones, states) over an utterance.

10.24.2 Member Function Documentation

10.24.2.1 `ps_alignment_free()`

```
POCKETSPHINX_EXPORT int ps_alignment_free (  
    ps_alignment_t * al)
```

Release an alignment

References [POCKETSPHINX_EXPORT](#).

10.24.2.2 `ps_alignment_phones()`

```
POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_phones (  
    ps_alignment_t * al)
```

Iterate over the alignment starting at the first phone.

References [POCKETSPHINX_EXPORT](#).

10.24.2.3 `ps_alignment_retain()`

```
POCKETSPHINX_EXPORT ps_alignment_t * ps_alignment_retain (  
    ps_alignment_t * al)
```

Retain an alignment

References [POCKETSPHINX_EXPORT](#).

10.24.2.4 `ps_alignment_states()`

```
POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_states (  
    ps_alignment_t * al)
```

Iterate over the alignment starting at the first state.

References [POCKETSPHINX_EXPORT](#).

10.24.2.5 `ps_alignment_words()`

```
POCKETSPHINX_EXPORT ps_alignment_iter_t * ps_alignment_words (  
    ps_alignment_t * al)
```

Iterate over the alignment starting at the first word.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- `C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/alignment.h`

10.25 ps_arg_s Struct Reference

```
#include <pocketsphinx.h>
```

Public Attributes

- char const * [name](#)
- int [type](#)
- char const * [deflt](#)
- char const * [doc](#)

10.25.1 Member Data Documentation

10.25.1.1 deflt

```
char const* ps_arg_s::deflt
```

Default value (as a character string), or NULL if none

10.25.1.2 doc

```
char const* ps_arg_s::doc
```

Documentation/description string

10.25.1.3 name

```
char const* ps_arg_s::name
```

Name of the command line switch

10.25.1.4 type

```
int ps_arg_s::type
```

Type of the argument in question

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/[pocketsphinx.h](#)

10.26 ps_arg_t Struct Reference

Definition of a configuration parameter.

```
#include <pocketsphinx.h>
```

10.26.1 Detailed Description

Definition of a configuration parameter.

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx.h

10.27 ps_config_t Struct Reference

configuration object.

```
#include <pocketsphinx.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ps_config_t * ps_config_init](#) (const [ps_arg_t](#) *defn)
- [POCKETSPHINX_EXPORT ps_config_t * ps_config_retain](#) (ps_config_t *config)
- [POCKETSPHINX_EXPORT int ps_config_free](#) (ps_config_t *config)
- [int ps_config_validate](#) (ps_config_t *config)
- [POCKETSPHINX_EXPORT ps_config_t * ps_config_parse_json](#) (ps_config_t *config, const char *json)
- [POCKETSPHINX_EXPORT const char * ps_config_serialize_json](#) (ps_config_t *config)
- [POCKETSPHINX_EXPORT ps_type_t ps_config_typeof](#) (ps_config_t *config, char const *name)
- [POCKETSPHINX_EXPORT const anytype_t * ps_config_get](#) (ps_config_t *config, const char *name)
- [POCKETSPHINX_EXPORT const anytype_t * ps_config_set](#) (ps_config_t *config, const char *name, const anytype_t *val, ps_type_t t)
- [POCKETSPHINX_EXPORT long ps_config_int](#) (ps_config_t *config, const char *name)
- [POCKETSPHINX_EXPORT int ps_config_bool](#) (ps_config_t *config, const char *name)
- [POCKETSPHINX_EXPORT double ps_config_float](#) (ps_config_t *config, const char *name)
- [POCKETSPHINX_EXPORT const char * ps_config_str](#) (ps_config_t *config, const char *name)
- [POCKETSPHINX_EXPORT const anytype_t * ps_config_set_int](#) (ps_config_t *config, const char *name, long val)
- [POCKETSPHINX_EXPORT const anytype_t * ps_config_set_bool](#) (ps_config_t *config, const char *name, int val)
- [POCKETSPHINX_EXPORT const anytype_t * ps_config_set_float](#) (ps_config_t *config, const char *name, double val)
- [POCKETSPHINX_EXPORT const anytype_t * ps_config_set_str](#) (ps_config_t *config, const char *name, const char *val)
- [POCKETSPHINX_EXPORT int ps_config_soundfile](#) (ps_config_t *config, FILE *fh, const char *file)
- [POCKETSPHINX_EXPORT int ps_config_wavfile](#) (ps_config_t *config, FILE *infh, const char *file)
- [POCKETSPHINX_EXPORT int ps_config_nistfile](#) (ps_config_t *config, FILE *infh, const char *file)
- [POCKETSPHINX_EXPORT void ps_default_search_args](#) (ps_config_t *config)
- [POCKETSPHINX_EXPORT void ps_expand_model_config](#) (ps_config_t *config)
- [POCKETSPHINX_EXPORT ps_decoder_t * ps_init](#) (ps_config_t *config)

Related Symbols

(Note that these are not member symbols.)

- [POCKETSPHINX_EXPORT const char * ps_default_modeldir](#) (void)

10.27.1 Detailed Description

configuration object.

10.27.2 Member Function Documentation

10.27.2.1 `ps_config_bool()`

```
POCKETSPHINX_EXPORT int ps_config_bool (  
    ps_config_t * config,  
    const char * name)
```

Get a boolean-valued parameter.

If the parameter does not have an integer or boolean type, this will print an error and return 0. The return value is either 0 or 1 (if the parameter has an integer type, any non-zero value will return 1).

References [POCKETSPHINX_EXPORT](#).

10.27.2.2 `ps_config_float()`

```
POCKETSPHINX_EXPORT double ps_config_float (  
    ps_config_t * config,  
    const char * name)
```

Get a floating-point parameter.

If the parameter does not have a floating-point type, this will print an error and return 0.

References [POCKETSPHINX_EXPORT](#).

10.27.2.3 `ps_config_free()`

```
POCKETSPHINX_EXPORT int ps_config_free (  
    ps_config_t * config)
```

Release a configuration object.

10.27.2.4 `ps_config_get()`

```
POCKETSPHINX_EXPORT const anytype_t * ps_config_get (  
    ps_config_t * config,  
    const char * name)
```

Access the value of a configuration parameter.

To actually do something with the value, you will need to know its type, which can be obtained with [ps_config_typeof\(\)](#). This function is thus mainly useful for dynamic language bindings, and you should use [ps_config_int\(\)](#), [ps_config_float\(\)](#), or [ps_config_str\(\)](#) instead.

Parameters

<i>config</i>	Configuration object.
<i>name</i>	Name of the parameter to retrieve.

Returns

Pointer to the parameter's value, or NULL if the parameter does not exist. Note that a string parameter can also have NULL as a value, in which case the `ptr` field in the return value is NULL. This pointer (and any pointers inside it) is owned by the `ps_config_t`.

References [POCKETSPHINX_EXPORT](#).

10.27.2.5 `ps_config_init()`

```
POCKETSPHINX_EXPORT ps_config_t * ps_config_init (
    const ps_arg_t * defn)
```

Create a configuration with default values.

Parameters

<i>defn</i>	Array of <code>ps_arg_t</code> defining and describing parameters, terminated by an <code>ps_arg_t</code> with <code>name == NULL</code> . You should usually just pass NULL here, which will result in the standard set of parameters being used.
-------------	--

Returns

Newly created configuration or NULL on failure (should not happen, but check it anyway).

References [POCKETSPHINX_EXPORT](#).

10.27.2.6 `ps_config_int()`

```
POCKETSPHINX_EXPORT long ps_config_int (
    ps_config_t * config,
    const char * name)
```

Get an integer-valued parameter.

If the parameter does not have an integer or boolean type, this will print an error and return 0. So don't do that.

References [POCKETSPHINX_EXPORT](#).

10.27.2.7 `ps_config_nistfile()`

```
POCKETSPHINX_EXPORT int ps_config_nistfile (
    ps_config_t * config,
    FILE * infh,
    const char * file)
```

Read a NIST header and set configuration parameters.

This works like `ps_config_soundfile()` but assumes that you already know it's a NIST file.

Unlike `ps_config_soundfile()`, the file does *not* have to be seekable.

Parameters

<i>config</i>	Configuration to update from file.
<i>infh</i>	Previously opened file handle.
<i>file</i>	Name of open file handle for logging (optional, can be NULL)

References [POCKETSPHINX_EXPORT](#).

10.27.2.8 `ps_config_parse_json()`

```
POCKETSPHINX_EXPORT ps_config_t * ps_config_parse_json (
    ps_config_t * config,
    const char * json)
```

Create or update a configuration by parsing slightly extended JSON.

This function parses a JSON object in non-strict mode to produce a [ps_config_t](#). Configuration parameters are given *without* a leading dash, and do not need to be quoted, nor does the object need to be enclosed in curly braces, nor are commas necessary between key/value pairs. Basically, it's degenerate YAML. So, for example, this is accepted:

```
hmm: fr-fr
samprate: 8000
keyphrase: "hello world"
```

Of course, valid JSON is also accepted, but who wants to use that.

Well, mostly. Unicode escape sequences (e.g. `"\u0020"`) are *not* supported at the moment, so please don't use them.

- config Previously existing [ps_config_t](#) to update, or NULL to create a new one.
- json JSON serialized object as null-terminated UTF-8, containing configuration parameters.

Returns

Newly created configuration or NULL on failure (such as invalid or missing parameters).

References [POCKETSPHINX_EXPORT](#).

10.27.2.9 `ps_config_retain()`

```
POCKETSPHINX_EXPORT ps_config_t * ps_config_retain (
    ps_config_t * config)
```

Retain a pointer to a configuration object.

References [POCKETSPHINX_EXPORT](#).

10.27.2.10 ps_config_serialize_json()

```
POCKETSPHINX_EXPORT const char * ps_config_serialize_json (  
    ps_config_t * config)
```

Construct JSON from a configuration object.

Unlike [ps_config_parse_json\(\)](#), this actually produces valid JSON ;-)

- config Configuration object

Returns

Newly created null-terminated JSON string. The [ps_config_t](#) retains ownership of this pointer, which is only valid until the next call to [ps_config_serialize_json\(\)](#). You must copy it if you wish to retain it.

References [POCKETSPHINX_EXPORT](#).

10.27.2.11 ps_config_set()

```
POCKETSPHINX_EXPORT const anytype_t * ps_config_set (  
    ps_config_t * config,  
    const char * name,  
    const anytype_t * val,  
    ps_type_t t)
```

Set or unset the value of a configuration parameter.

This will coerce the value to the proper type, so you can, for example, pass it a string with ARG_STRING as the type when adding options from the command-line. Note that the return pointer will *not* be the same as the one passed in the value.

Parameters

<i>config</i>	Configuration object.
<i>name</i>	Name of the parameter to set. Must exist.
<i>val</i>	Pointer to the value (strings will be copied) inside an anytype_t union. On 64-bit little-endian platforms, you <i>can</i> cast a pointer to int, long, double, or char* here, but that doesn't necessarily mean that you <i>should</i> . As a convenience, you can pass NULL here to reset a parameter to its default value.
<i>t</i>	Type of the value in <i>val</i> , will be coerced to the type of the actual parameter if necessary.

Returns

Pointer to the parameter's value, or NULL on failure (unknown parameter, usually). This pointer (and any pointers inside it) is owned by the [ps_config_t](#).

References [POCKETSPHINX_EXPORT](#).

10.27.2.12 `ps_config_set_bool()`

```
POCKETSPHINX_EXPORT const anytype_t * ps_config_set_bool (
    ps_config_t * config,
    const char * name,
    int val)
```

Set a boolean-valued parameter.

If the parameter does not have an integer or boolean type, this will convert `val` appropriately.

References [POCKETSPHINX_EXPORT](#).

10.27.2.13 `ps_config_set_float()`

```
POCKETSPHINX_EXPORT const anytype_t * ps_config_set_float (
    ps_config_t * config,
    const char * name,
    double val)
```

Set a floating-point parameter.

If the parameter does not have a floating-point type, this will convert `val` appropriately.

References [POCKETSPHINX_EXPORT](#).

10.27.2.14 `ps_config_set_int()`

```
POCKETSPHINX_EXPORT const anytype_t * ps_config_set_int (
    ps_config_t * config,
    const char * name,
    long val)
```

Set an integer-valued parameter.

If the parameter does not have an integer or boolean type, this will convert `val` appropriately.

References [POCKETSPHINX_EXPORT](#).

10.27.2.15 `ps_config_set_str()`

```
POCKETSPHINX_EXPORT const anytype_t * ps_config_set_str (
    ps_config_t * config,
    const char * name,
    const char * val)
```

Set a string-valued parameter.

If the parameter does not have a string type, this will convert `val` appropriately. For boolean parameters, any string matching `/^[yt1] /` will be true, while any string matching `/^[nf0] /` will be false. NULL is also false.

This function is used for configuration from JSON, you may want to use it for your own configuration files too.

References [POCKETSPHINX_EXPORT](#).

10.27.2.16 ps_config_soundfile()

```
POCKETSPHINX_EXPORT int ps_config_soundfile (  
    ps_config_t * config,  
    FILE * fh,  
    const char * file)
```

Set configuration parameters (actually just sample rate) from a sound file.

If the file is unreadable, unsupported or incompatible with the existing feature extraction parameters, this will print an error message and fail (return -1).

If it is of an unknown type, it will be treated as raw data. So beware! Currently we only support WAV and NIST Sphere files. We attempt to recognize Ogg, MP3 (but not really, because it is very difficult to do reliably), and FLAC, but do not support them. For everything else, there's SoX (tm).

Currently, the file must be seekable, so you can't use this on standard input, for instance.

Parameters

<i>config</i>	Configuration to update from file.
<i>fh</i>	Previously opened file handle.
<i>file</i>	Name of open file handle for logging (optional, can be NULL)

References [POCKETSPHINX_EXPORT](#).

10.27.2.17 ps_config_str()

```
POCKETSPHINX_EXPORT const char * ps_config_str (  
    ps_config_t * config,  
    const char * name)
```

Get a string parameter.

If the parameter does not have a string type, this will print an error and return NULL. Notably, it will *NOT* format an integer or float for you, because that would involve allocating memory. So don't do that.

References [POCKETSPHINX_EXPORT](#).

10.27.2.18 ps_config_typeof()

```
POCKETSPHINX_EXPORT ps_type_t ps_config_typeof (  
    ps_config_t * config,  
    char const * name)
```

Access the type of a configuration parameter.

Parameters

<i>config</i>	Configuration object.
<i>name</i>	Name of the parameter to retrieve.

Returns

the type of the parameter (as a combination of the ARG_* bits), or 0 if no such parameter exists.

References [POCKETSPHINX_EXPORT](#).

10.27.2.19 ps_config_validate()

```
int ps_config_validate (  
    ps_config_t * config)
```

Validate configuration.

Currently this just checks that you haven't specified multiple types of grammars or language models at the same time.

Returns

0 for success, <0 for failure.

References [POCKETSPHINX_EXPORT](#).

10.27.2.20 ps_config_wavfile()

```
POCKETSPHINX_EXPORT int ps_config_wavfile (  
    ps_config_t * config,  
    FILE * infh,  
    const char * file)
```

Read a WAV header and set configuration parameters.

This works like [ps_config_soundfile\(\)](#) but assumes that you already know it's a WAV file.

Unlike [ps_config_soundfile\(\)](#), the file does *not* have to be seekable.

Parameters

<i>config</i>	Configuration to update from file.
<i>infh</i>	Previously opened file handle.
<i>file</i>	Name of open file handle for logging (optional, can be NULL)

References [POCKETSPHINX_EXPORT](#).

10.27.2.21 ps_default_search_args()

```
POCKETSPHINX_EXPORT void ps_default_search_args (  
    ps_config_t * config)
```

Sets default acoustic and language model if they are not set explicitly.

This function fills in the configuration with the default acoustic and language models and dictionary, if (and this is a badly implemented heuristic) they do not seem to be already filled in. It is preferable for you to call this *before* doing any other configuration to avoid confusion.

The default models are looked for in the directory returned by [ps_default_modeldir\(\)](#), or, if the `POCKETSPHINX_PATH` environment variable is set, this function will look there instead.

If no global model directory was defined at compilation time (this is useful for relocatable installs such as the Python module) and `POCKETSPHINX_PATH` is not set, this will simply do nothing.

References [POCKETSPHINX_EXPORT](#).

10.27.2.22 ps_expand_model_config()

```
POCKETSPHINX_EXPORT void ps_expand_model_config (  
    ps_config_t * config)
```

Sets default file paths and parameters based on configuration.

References [POCKETSPHINX_EXPORT](#).

10.27.2.23 ps_init()

```
POCKETSPHINX_EXPORT ps_decoder_t * ps_init (  
    ps_config_t * config)
```

Initialize the decoder from a configuration object.

Note

The decoder retains ownership of the pointer `config`, so if you are not going to use it elsewhere, you can free it.

Parameters

<i>config</i>	a configuration object. If NULL, the decoder will be allocated but not initialized. You can proceed to initialize it with <code>ps_reinit()</code> .
---------------	--

References [POCKETSPHINX_EXPORT](#).

10.27.3 Friends And Related Symbol Documentation

10.27.3.1 ps_default_modeldir()

```
POCKETSPHINX_EXPORT const char * ps_default_modeldir (
    void ) [related]
```

Gets the system default model directory, if any exists.

Returns

system model directory defined at compile time, or NULL if not defined (usually in a relocatable installation such as a Python module).

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/[pocketsphinx.h](#)

10.28 ps_decoder_t Struct Reference

Speech recognizer object.

```
#include <pocketsphinx.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) int [ps_reinit](#) ([ps_decoder_t](#) *ps, [ps_config_t](#) *config)
- [POCKETSPHINX_EXPORT](#) int [ps_reinit_feat](#) ([ps_decoder_t](#) *ps, [ps_config_t](#) *config)
- [POCKETSPHINX_EXPORT](#) const char * [ps_get_cmn](#) ([ps_decoder_t](#) *ps, int update)
- [POCKETSPHINX_EXPORT](#) int [ps_set_cmn](#) ([ps_decoder_t](#) *ps, const char *cmn)
- [POCKETSPHINX_EXPORT](#) [ps_decoder_t](#) * [ps_retain](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) int [ps_free](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) [ps_config_t](#) * [ps_get_config](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) [logmath_t](#) * [ps_get_logmath](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) [ps_mllr_t](#) * [ps_update_mllr](#) ([ps_decoder_t](#) *ps, [ps_mllr_t](#) *mllr)
- [POCKETSPHINX_EXPORT](#) int [ps_load_dict](#) ([ps_decoder_t](#) *ps, char const *dictfile, char const *dictfile, char const *format)
- [POCKETSPHINX_EXPORT](#) int [ps_save_dict](#) ([ps_decoder_t](#) *ps, char const *dictfile, char const *format)
- [POCKETSPHINX_EXPORT](#) int [ps_add_word](#) ([ps_decoder_t](#) *ps, char const *word, char const *phones, int update)
- [POCKETSPHINX_EXPORT](#) char * [ps_lookup_word](#) ([ps_decoder_t](#) *ps, const char *word)
- [POCKETSPHINX_EXPORT](#) long [ps_decode_raw](#) ([ps_decoder_t](#) *ps, FILE *rawfh, long maxsamps)
- [POCKETSPHINX_EXPORT](#) int [ps_decode_senscr](#) ([ps_decoder_t](#) *ps, FILE *senfh)
- [POCKETSPHINX_EXPORT](#) int [ps_start_stream](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) int [ps_get_in_speech](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) int [ps_start_utt](#) ([ps_decoder_t](#) *ps)
- [POCKETSPHINX_EXPORT](#) int [ps_process_raw](#) ([ps_decoder_t](#) *ps, [int16](#) const *data, [size_t](#) n_samples, int no_search, int full_utt)

- `POCKETSPHINX_EXPORT` int `ps_process_cep` (`ps_decoder_t` *ps, float32 **data, int n_frames, int no_← search, int full_utt)
- `POCKETSPHINX_EXPORT` int `ps_get_n_frames` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` int `ps_end_utt` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` char const * `ps_get_hyp` (`ps_decoder_t` *ps, int32 *out_best_score)
- `POCKETSPHINX_EXPORT` int32 `ps_get_prob` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` `ps_lattice_t` * `ps_get_lattice` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` `ps_seg_t` * `ps_seg_iter` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` `ps_nbest_t` * `ps_nbest` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` void `ps_get_utt_time` (`ps_decoder_t` *ps, double *out_nspeech, double *out_← ncpu, double *out_nwall)
- `POCKETSPHINX_EXPORT` void `ps_get_all_time` (`ps_decoder_t` *ps, double *out_nspeech, double *out_← ncpu, double *out_nwall)
- `POCKETSPHINX_EXPORT` int `ps_activate_search` (`ps_decoder_t` *ps, const char *name)
- `POCKETSPHINX_EXPORT` const char * `ps_current_search` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` int `ps_remove_search` (`ps_decoder_t` *ps, const char *name)
- `POCKETSPHINX_EXPORT` `ps_search_iter_t` * `ps_search_iter` (`ps_decoder_t` *ps)
- `POCKETSPHINX_EXPORT` `ngram_model_t` * `ps_get_lm` (`ps_decoder_t` *ps, const char *name)
- `POCKETSPHINX_EXPORT` int `ps_add_lm` (`ps_decoder_t` *ps, const char *name, `ngram_model_t` *lm)
- `POCKETSPHINX_EXPORT` int `ps_add_lm_file` (`ps_decoder_t` *ps, const char *name, const char *path)
- `POCKETSPHINX_EXPORT` `fsg_model_t` * `ps_get_fsg` (`ps_decoder_t` *ps, const char *name)
- `POCKETSPHINX_EXPORT` int `ps_add_fsg` (`ps_decoder_t` *ps, const char *name, `fsg_model_t` *fsg)
- `POCKETSPHINX_EXPORT` int `ps_add_jsgf_file` (`ps_decoder_t` *ps, const char *name, const char *path)
- `POCKETSPHINX_EXPORT` int `ps_add_jsgf_string` (`ps_decoder_t` *ps, const char *name, const char *jsgf_← string)
- `POCKETSPHINX_EXPORT` const char * `ps_get_kws` (`ps_decoder_t` *ps, const char *name)
- `POCKETSPHINX_EXPORT` int `ps_add_kws` (`ps_decoder_t` *ps, const char *name, const char *keyfile)
- `POCKETSPHINX_EXPORT` int `ps_add_keyphrase` (`ps_decoder_t` *ps, const char *name, const char *keyphrase)
- `POCKETSPHINX_EXPORT` int `ps_add_allphone` (`ps_decoder_t` *ps, const char *name, `ngram_model_t` *lm)
- `POCKETSPHINX_EXPORT` int `ps_add_allphone_file` (`ps_decoder_t` *ps, const char *name, const char *path)
- `POCKETSPHINX_EXPORT` int `ps_set_align_text` (`ps_decoder_t` *ps, const char *words)
- `POCKETSPHINX_EXPORT` int `ps_set_alignment` (`ps_decoder_t` *ps, `ps_alignment_t` *al)
- `POCKETSPHINX_EXPORT` `ps_alignment_t` * `ps_get_alignment` (`ps_decoder_t` *ps)

10.28.1 Detailed Description

Speech recognizer object.

10.28.2 Member Function Documentation

10.28.2.1 ps_activate_search()

```
POCKETSPHINX_EXPORT int ps_activate_search (
    ps_decoder_t * ps,
    const char * name)
```

Activates search with the provided name.

Parameters

<i>name</i>	Name of search module to activate. This must have been previously added by either ps_add_fsg() , ps_add_lm() , or ps_add_kws() . If NULL, it will re-activate the default search, which is useful when running second-pass alignment, for instance.
-------------	---

Returns

0 on success, -1 on failure

References [POCKETSPHINX_EXPORT](#).

10.28.2.2 ps_add_allphone()

```
POCKETSPHINX_EXPORT int ps_add_allphone (  
    ps_decoder_t * ps,  
    const char * name,  
    ngram_model_t * lm)
```

Adds new search based on phone N-gram language model.

Associates N-gram search with the provided name. The search can be activated using [ps_activate_search\(\)](#).

See also

[ps_activate_search](#).

References [POCKETSPHINX_EXPORT](#).

10.28.2.3 ps_add_allphone_file()

```
POCKETSPHINX_EXPORT int ps_add_allphone_file (  
    ps_decoder_t * ps,  
    const char * name,  
    const char * path)
```

Adds new search based on phone N-gram language model.

Convenient method to load N-gram model and create a search.

See also

[ps_add_allphone](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.4 `ps_add_fsg()`

```
POCKETSPHINX_EXPORT int ps_add_fsg (  
    ps_decoder_t * ps,  
    const char * name,  
    fsg_model_t * fsg)
```

Adds new search based on finite state grammar.

Associates FSG search with the provided name. The search can be activated using [ps_activate_search\(\)](#).

See also

[ps_activate_search](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.5 `ps_add_jsgf_file()`

```
POCKETSPHINX_EXPORT int ps_add_jsgf_file (  
    ps_decoder_t * ps,  
    const char * name,  
    const char * path)
```

Adds new search using JSJGF model.

Convenient method to load JSJGF model and create a search.

See also

[ps_add_fsg](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.6 `ps_add_jsgf_string()`

```
POCKETSPHINX_EXPORT int ps_add_jsgf_string (  
    ps_decoder_t * ps,  
    const char * name,  
    const char * jsjgf_string)
```

Adds new search using JSJGF model.

Convenience method to parse JSJGF model from string and create a search.

See also

[ps_add_fsg](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.7 `ps_add_keyphrase()`

```
POCKETSPHINX_EXPORT int ps_add_keyphrase (  
    ps_decoder_t * ps,  
    const char * name,  
    const char * keyphrase)
```

Adds new keyphrase to spot

Associates KWS search with the provided name. The search can be activated using [ps_activate_search\(\)](#).

See also

[ps_activate_search](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.8 `ps_add_kws()`

```
POCKETSPHINX_EXPORT int ps_add_kws (  
    ps_decoder_t * ps,  
    const char * name,  
    const char * keyfile)
```

Adds keyphrases from a file to spotting

Associates KWS search with the provided name. The search can be activated using [ps_activate_search\(\)](#).

See also

[ps_activate_search](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.9 `ps_add_lm()`

```
POCKETSPHINX_EXPORT int ps_add_lm (  
    ps_decoder_t * ps,  
    const char * name,  
    ngram_model_t * lm)
```

Adds new search based on N-gram language model.

Associates N-gram search with the provided name. The search can be activated using [ps_activate_search\(\)](#).

See also

[ps_activate_search](#).

References [POCKETSPHINX_EXPORT](#).

10.28.2.10 `ps_add_lm_file()`

```
POCKETSPHINX_EXPORT int ps_add_lm_file (
    ps_decoder_t * ps,
    const char * name,
    const char * path)
```

Adds new search based on N-gram language model.

Convenient method to load N-gram model and create a search.

See also

[`ps\_add\_lm`](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.11 `ps_add_word()`

```
POCKETSPHINX_EXPORT int ps_add_word (
    ps_decoder_t * ps,
    char const * word,
    char const * phones,
    int update)
```

Add a word to the pronunciation dictionary.

This function adds a word to the pronunciation dictionary and the current language model (but, obviously, not to the current FSG if FSG mode is enabled). If the word is already present in one or the other, it does whatever is necessary to ensure that the word can be recognized.

Parameters

<i>word</i>	Word string to add.
<i>phones</i>	Whitespace-separated list of phoneme strings describing pronunciation of <i>word</i> .
<i>update</i>	If TRUE, update the search module (whichever one is currently active) to recognize the newly added word. If adding multiple words, it is more efficient to pass FALSE here in all but the last word.

Returns

The internal ID (≥ 0) of the newly added word, or <0 on failure.

References [POCKETSPHINX_EXPORT](#).

10.28.2.12 `ps_current_search()`

```
POCKETSPHINX_EXPORT const char * ps_current_search (
    ps_decoder_t * ps)
```

Returns name of current search in decoder

See also

[`ps\_activate\_search`](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.13 `ps_decode_raw()`

```
POCKETSPHINX_EXPORT long ps_decode_raw (  
    ps_decoder_t * ps,  
    FILE * rawfh,  
    long maxsamps)
```

Decode a raw audio stream.

No headers are recognized in this files. The configuration parameters `-samprate` and `-input_endian` are used to determine the sampling rate and endianness of the stream, respectively. Audio is always assumed to be 16-bit signed PCM.

Parameters

<i>ps</i>	Decoder.
<i>rawfh</i>	Previously opened file stream.
<i>maxsamps</i>	Maximum number of samples to read from rawfh, or -1 to read until end-of-file.

Returns

Number of samples of audio.

References [POCKETSPHINX_EXPORT](#).

10.28.2.14 `ps_decode_senscr()`

```
POCKETSPHINX_EXPORT int ps_decode_senscr (  
    ps_decoder_t * ps,  
    FILE * senfh)
```

Decode a senone score dump file.

Parameters

<i>ps</i>	Decoder
<i>senfh</i>	Previously opened file handle positioned at start of file.

Returns

Number of frames read.

References [POCKETSPHINX_EXPORT](#).

10.28.2.15 `ps_end_utt()`

```
POCKETSPHINX_EXPORT int ps_end_utt (  
    ps_decoder_t * ps)
```

End utterance processing.

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

0 for success, <0 on error

References [POCKETSPHINX_EXPORT](#).

10.28.2.16 ps_free()

```
POCKETSPHINX_EXPORT int ps_free (
    ps_decoder_t * ps)
```

Finalize the decoder.

This releases all resources associated with the decoder.

Parameters

<i>ps</i>	Decoder to be freed.
-----------	----------------------

Returns

New reference count (0 if freed).

References [POCKETSPHINX_EXPORT](#).

10.28.2.17 ps_get_alignment()

```
POCKETSPHINX_EXPORT ps_alignment_t * ps_get_alignment (
    ps_decoder_t * ps)
```

Get the alignment associated with the current search module.

As noted above, if decoding has not been run, this will contain invalid durations, but that may still be useful if you just want to know the state sequence.

Returns

Current alignment, or NULL if none. This pointer is owned by the decoder, so you must call `ps_alignment_↵retain()` on it if you wish to keep it outside the lifetime of the decoder.

10.28.2.18 ps_get_all_time()

```
POCKETSPHINX_EXPORT void ps_get_all_time (
    ps_decoder_t * ps,
    double * out_nspeech,
    double * out_ncpu,
    double * out_nwall)
```

Get overall performance information.

Parameters

<i>ps</i>	Decoder.
<i>out_nspeech</i>	Output: Number of seconds of speech.
<i>out_ncpu</i>	Output: Number of seconds of CPU time used.
<i>out_nwall</i>	Output: Number of seconds of wall time used.

10.28.2.19 `ps_get_cmn()`

```
POCKETSPHINX_EXPORT const char * ps_get_cmn (
    ps_decoder_t * ps,
    int update)
```

Get the current cepstral mean as a string.

This is the string representation of the current cepstral mean, which represents the acoustic channel conditions in live recognition. This can be used to initialize the decoder with the `cmninit` option, e.g.:

```
config = ps_config_parse_json(NULL, "cmninit: 42,-1,0");
```

Parameters

<i>ps</i>	Decoder
<i>update</i>	Update the cepstral mean using data processed so far.

Returns

String representation of cepstral mean, as `ps_config_get_int(config, "ceplen")` comma-separated numbers. This pointer is owned by the decoder and only valid until the next call to `ps_get_cmn()`, `ps_set_cmn()` or `ps_end_utt()`.

References [POCKETSPHINX_EXPORT](#).

10.28.2.20 `ps_get_config()`

```
POCKETSPHINX_EXPORT ps_config_t * ps_get_config (
    ps_decoder_t * ps)
```

Get the configuration object for this decoder.

Returns

The configuration object for this decoder. The decoder owns this pointer, so you should not attempt to free it manually. Use `ps_config_retain()` if you wish to reuse it elsewhere.

References [POCKETSPHINX_EXPORT](#).

10.28.2.21 ps_get_fsg()

```
POCKETSPHINX_EXPORT fsg_model_t * ps_get_fsg (
    ps_decoder_t * ps,
    const char * name)
```

Get the finite-state grammar set object associated with a search.

- name Name of FSG search, or NULL for current search.

Returns

The current FSG set object for this decoder, or NULL if `name` does not correspond to an FSG search.

References [POCKETSPHINX_EXPORT](#).

10.28.2.22 ps_get_hyp()

```
POCKETSPHINX_EXPORT char const * ps_get_hyp (
    ps_decoder_t * ps,
    int32 * out_best_score)
```

Get hypothesis string and path score.

Parameters

<i>ps</i>	Decoder.
<i>out_best_score</i>	Output: path score corresponding to returned string.

Returns

String containing best hypothesis at this point in decoding. NULL if no hypothesis is available. This string is owned by the decoder and only valid for the current hypothesis, so you should copy it if you need to hold onto it.

References [POCKETSPHINX_EXPORT](#).

10.28.2.23 ps_get_in_speech()

```
POCKETSPHINX_EXPORT int ps_get_in_speech (
    ps_decoder_t * ps)
```

Check in-speech status of decoder.

Deprecated This function is retained for compatibility but should not be considered a reliable voice activity detector. It will always return 1 between calls to [ps_start_utt\(\)](#) and [ps_end_utt\(\)](#). You probably want [ps_endpoint_t](#), but for single frames of data you can also use [ps_vad_t](#).

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

1 if last buffer contained speech, 0 - otherwise

References [POCKETSPHINX_EXPORT](#).

10.28.2.24 `ps_get_kws()`

```
POCKETSPHINX_EXPORT const char * ps_get_kws (  
    ps_decoder_t * ps,  
    const char * name)
```

Get the keyphrase associated with a KWS search

- name Name of KWS search, or NULL for current search.

Returns

The current keyphrase to spot, or NULL if `name` does not correspond to a KWS search

References [POCKETSPHINX_EXPORT](#).

10.28.2.25 `ps_get_lattice()`

```
POCKETSPHINX_EXPORT ps_lattice_t * ps_get_lattice (  
    ps_decoder_t * ps)
```

Get word lattice.

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

Word lattice object containing all hypotheses so far. NULL if no hypotheses are available. This pointer is owned by the decoder and you should not attempt to free it manually. It is only valid until the next utterance, unless you use `ps_lattice_retain()` to retain it.

References [POCKETSPHINX_EXPORT](#).

10.28.2.26 ps_get_lm()

```
POCKETSPHINX_EXPORT ngram_model_t * ps_get_lm (  
    ps_decoder_t * ps,  
    const char * name)
```

Get the language model or lmset object associated with a search.

- name Name of language model search, or NULL for current search.

Returns

The language model (possibly set of language models) object for this decoder. The decoder retains ownership of this pointer, so you should not attempt to free it manually. Use `ngram_model_retain()` if you wish to reuse it elsewhere.

References [POCKETSPHINX_EXPORT](#).

10.28.2.27 ps_get_logmath()

```
POCKETSPHINX_EXPORT logmath_t * ps_get_logmath (  
    ps_decoder_t * ps)
```

Get the log-math computation object for this decoder.

Returns

The log-math object for this decoder. The decoder owns this pointer, so you should not attempt to free it manually. Use `logmath_retain()` if you wish to reuse it elsewhere.

References [POCKETSPHINX_EXPORT](#).

10.28.2.28 ps_get_n_frames()

```
POCKETSPHINX_EXPORT int ps_get_n_frames (  
    ps_decoder_t * ps)
```

Get the number of frames of data searched.

Note that there is a delay between this and the number of frames of audio which have been input to the system. This is due to the fact that acoustic features are computed using a sliding window of audio, and dynamic features are computed over a sliding window of acoustic features.

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

Number of frames of speech data which have been recognized so far.

References [POCKETSPHINX_EXPORT](#).

10.28.2.29 ps_get_prob()

```
POCKETSPHINX_EXPORT int32 ps_get_prob (
    ps_decoder_t * ps)
```

Get posterior probability.

Note

Unless the `-bestpath` option is enabled, this function will always return zero (corresponding to a posterior probability of 1.0). Even if `-bestpath` is enabled, it will also return zero when called on a partial result. Ongoing research into effective confidence annotation for partial hypotheses may result in these restrictions being lifted in future versions.

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

Posterior probability of the best hypothesis.

References [POCKETSPHINX_EXPORT](#).

10.28.2.30 ps_get_utt_time()

```
POCKETSPHINX_EXPORT void ps_get_utt_time (
    ps_decoder_t * ps,
    double * out_nspeech,
    double * out_ncpu,
    double * out_nwall)
```

Get performance information for the current utterance.

Parameters

<i>ps</i>	Decoder.
<i>out_nspeech</i>	Output: Number of seconds of speech.
<i>out_ncpu</i>	Output: Number of seconds of CPU time used.
<i>out_nwall</i>	Output: Number of seconds of wall time used.

References [POCKETSPHINX_EXPORT](#).

10.28.2.31 ps_load_dict()

```
POCKETSPHINX_EXPORT int ps_load_dict (
    ps_decoder_t * ps,
    char const * dictfile,
    char const * fdictfile,
    char const * format)
```

Reload the pronunciation dictionary from a file.

This function replaces the current pronunciation dictionary with the one stored in `dictfile`. This also causes the active search module(s) to be reinitialized, in the same manner as calling [ps_add_word\(\)](#) with `update=TRUE`.

Parameters

<i>dictfile</i>	Path to dictionary file to load.
<i>fdictfile</i>	Path to filler dictionary to load, or NULL to keep the existing filler dictionary.
<i>format</i>	Format of the dictionary file, or NULL to determine automatically (currently unused, should be NULL)

References [POCKETSPHINX_EXPORT](#).

10.28.2.32 ps_lookup_word()

```
POCKETSPHINX_EXPORT char * ps_lookup_word (  
    ps_decoder_t * ps,  
    const char * word)
```

Look up a word in the dictionary and return phone transcription for it.

Parameters

<i>ps</i>	Pocketsphinx decoder
<i>word</i>	Word to look for

Returns

Whitespace-separated phone string describing the pronunciation of the `word` or NULL if word is not present in the dictionary. The string is allocated and must be freed by the user.

References [POCKETSPHINX_EXPORT](#).

10.28.2.33 ps_nbest()

```
POCKETSPHINX_EXPORT ps_nbest_t * ps_nbest (  
    ps_decoder_t * ps)
```

Get an iterator over the best hypotheses. The function may also return a NULL which means that there is no hypothesis available for this utterance.

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

Iterator over N-best hypotheses or NULL if no hypothesis is available

References [POCKETSPHINX_EXPORT](#).

10.28.2.34 ps_process_cep()

```
POCKETSPHINX_EXPORT int ps_process_cep (  
    ps_decoder_t * ps,  
    float32 ** data,  
    int n_frames,  
    int no_search,  
    int full_utt)
```

Decode acoustic feature data.

Parameters

<i>ps</i>	Decoder.
<i>data</i>	Acoustic feature data, a 2-dimensional array of 32-bit floating-point values. Note that this is not a standard 2-dimensional C array but rather an array of pointers to floats, each of which is one vector (or frame) of <code>ps_config_get_int("ceplen")</code> values.
<i>n_frames</i>	Number of vectors in <i>data</i> .
<i>no_search</i>	If non-zero, perform feature extraction but don't do any recognition yet. This may be necessary if your processor has trouble doing recognition in real-time.
<i>full_utt</i>	If non-zero, this block of data is a full utterance worth of data. This may allow the recognizer to produce more accurate results.

Returns

Number of frames of data searched, or <0 for error.

References [POCKETSPHINX_EXPORT](#).

10.28.2.35 ps_process_raw()

```
POCKETSPHINX_EXPORT int ps_process_raw (  
    ps_decoder_t * ps,  
    int16 const * data,  
    size_t n_samples,  
    int no_search,  
    int full_utt)
```

Decode raw audio data.

Parameters

<i>ps</i>	Decoder.
<i>data</i>	Audio data, as 16-bit linear PCM.
<i>n_samples</i>	Number of samples (not bytes) in <i>data</i> .
<i>no_search</i>	If non-zero, perform feature extraction but don't do any recognition yet. This may be necessary if your processor has trouble doing recognition in real-time.
<i>full_utt</i>	If non-zero, this block of data is a full utterance worth of data. This may allow the recognizer to produce more accurate results.

Returns

Number of frames of data searched, or <0 for error.

References [POCKETSPHINX_EXPORT](#).

10.28.2.36 `ps_reinit()`

```
POCKETSPHINX_EXPORT int ps_reinit (  
    ps_decoder_t * ps,  
    ps_config_t * config)
```

Reinitialize the decoder with updated configuration.

This function allows you to switch the acoustic model, dictionary, or other configuration without creating an entirely new decoding object.

Note

Since the acoustic model will be reloaded, changes made to feature extraction parameters may be overridden if a `feat.params` file is present.

Any searches created with `ps_set_search()` or words added to the dictionary with `ps_add_word()` will also be lost. To avoid this you can use `ps_reinit_feat()`.

The decoder retains ownership of the pointer `config`, so you should free it when no longer used.

Parameters

<i>ps</i>	Decoder.
<i>config</i>	An optional new configuration to use. If this is NULL, the previous configuration will be reloaded, with any changes applied.

Returns

0 for success, <0 for failure.

References [POCKETSPHINX_EXPORT](#).

10.28.2.37 `ps_reinit_feat()`

```
POCKETSPHINX_EXPORT int ps_reinit_feat (  
    ps_decoder_t * ps,  
    ps_config_t * config)
```

Reinitialize only the feature computation with updated configuration.

This function allows you to switch the feature computation parameters without otherwise affecting the decoder configuration. For example, if you change the sample rate or the frame rate, and do not want to reconfigure the rest of the decoder.

Note that if you have set a custom cepstral mean with `ps_set_cmn()`, it will be overridden.

Note

The decoder retains ownership of the pointer `config`, so you should free it when no longer used.

Parameters

<i>ps</i>	Decoder.
<i>config</i>	An optional new configuration to use. If this is NULL, the previous configuration will be reloaded, with any changes to feature computation applied.

Returns

0 for success, <0 for failure (usually an invalid parameter)

References [POCKETSPHINX_EXPORT](#).

10.28.2.38 `ps_remove_search()`

```
POCKETSPHINX_EXPORT int ps_remove_search (
    ps_decoder_t * ps,
    const char * name)
```

Removes a search module and releases its resources.

Removes a search module previously added with using [ps_add_fsg\(\)](#), [ps_add_lm\(\)](#), [ps_add_kws\(\)](#), etc.

See also

[ps_add_fsg](#)
[ps_add_lm](#)
[ps_add_kws](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.39 `ps_retain()`

```
POCKETSPHINX_EXPORT ps_decoder_t * ps_retain (
    ps_decoder_t * ps)
```

Retain a pointer to the decoder.

This increments the reference count on the decoder, allowing it to be shared between multiple parent objects. In general you will not need to use this function, ever. It is mainly here for the convenience of scripting language bindings.

Returns

pointer to retained decoder.

References [POCKETSPHINX_EXPORT](#).

10.28.2.40 `ps_save_dict()`

```
POCKETSPHINX_EXPORT int ps_save_dict (
    ps_decoder_t * ps,
    char const * dictfile,
    char const * format)
```

Dump the current pronunciation dictionary to a file.

This function dumps the current pronunciation dictionary to a text file.

Parameters

<i>dictfile</i>	Path to file where dictionary will be written.
<i>format</i>	Format of the dictionary file, or NULL for the default (text) format (currently unused, should be NULL)

References [POCKETSPHINX_EXPORT](#).

10.28.2.41 `ps_search_iter()`

```
POCKETSPHINX_EXPORT ps_search_iter_t * ps_search_iter (
    ps_decoder_t * ps)
```

Returns iterator over current searches

See also

[ps_activate_search](#)

References [POCKETSPHINX_EXPORT](#).

10.28.2.42 `ps_seg_iter()`

```
POCKETSPHINX_EXPORT ps_seg_t * ps_seg_iter (
    ps_decoder_t * ps)
```

Get an iterator over the word segmentation for the best hypothesis.

Parameters

<i>ps</i>	Decoder.
-----------	----------

Returns

Iterator over the best hypothesis at this point in decoding. NULL if no hypothesis is available.

References [POCKETSPHINX_EXPORT](#).

10.28.2.43 `ps_set_align_text()`

```
POCKETSPHINX_EXPORT int ps_set_align_text (
    ps_decoder_t * ps,
    const char * words)
```

Set up decoder to force-align a word sequence.

Unlike the `ps_add_*` functions, this activates the search module immediately, since force-alignment is nearly always a single shot. Currently "under the hood" this is an FSG search but you shouldn't depend on that. The search module activated is *not* the default search, so you can return to that one by calling `ps_activate_search` with [NULL](#).

Decoding proceeds as normal, though only this word sequence will be recognized, with silences and alternate pronunciations inserted. Word alignments are available with [ps_seg_iter\(\)](#). To obtain phoneme or state segmentations, you must subsequently call [ps_set_alignment\(\)](#) and re-run decoding. It's tough son, but it's life.

Parameters

<i>ps</i>	Decoder
<i>words</i>	String containing whitespace-separated words for alignment. These words are assumed to exist in the current dictionary.

References [POCKETSPHINX_EXPORT](#).

10.28.2.44 ps_set_alignment()

```
POCKETSPHINX_EXPORT int ps_set_alignment (
    ps_decoder_t * ps,
    ps_alignment_t * al)
```

Set up decoder to run phone and state-level alignment.

Unlike the `ps_add_*` functions, this activates the search module immediately, since force-alignment is nearly always a single shot.

To align, run or re-run decoding as usual, then call [ps_get_alignment\(\)](#) to get the resulting alignment. Note that if you call this function *before* rerunning decoding, you can obtain the phone and state sequence, but the durations will be invalid (phones and states will inherit the parent word's duration).

Parameters

<i>ps</i>	Decoder object.
<i>al</i>	Usually NULL, which means to construct an alignment from the current search hypothesis (this does not work with allphone or keyword spotting). You can also pass a ps_alignment_t here if you have one. The search will retain but not copy it, so after running decoding it will be updated with new durations. You can set starts and durations for words or phones (not states) to constrain the alignment.

Returns

0 for success, -1 for error (if there is no search hypothesis, or it cannot be aligned due to missing word IDs)

References [POCKETSPHINX_EXPORT](#).

10.28.2.45 ps_set_cmn()

```
POCKETSPHINX_EXPORT int ps_set_cmn (
    ps_decoder_t * ps,
    const char * cmn)
```

Set the current cepstral mean from a string.

This does the same thing as setting `cmninit` with `ps_config_set_string()` and running `ps_reinit_feat()` but is more efficient, and can also be done in the middle of an utterance if you like.

Parameters

<i>ps</i>	Decoder
<i>cmn</i>	String representation of cepstral mean, as up to <code>ps_config_get_int(config, "ceplen")</code> -separated numbers (any missing values will be zero-filled).

Returns

0 for success or -1 for invalid input.

References [POCKETSPHINX_EXPORT](#).

10.28.2.46 ps_start_stream()

```
POCKETSPHINX_EXPORT int ps_start_stream (
    ps_decoder_t * ps)
```

Start processing of the stream of speech.

Deprecated This function is retained for compatibility, but its only effect is to reset the noise removal statistics, which are otherwise retained across utterances. You do not need to call it.

Returns

0 for success, <0 on error.

References [POCKETSPHINX_EXPORT](#).

10.28.2.47 ps_start_utt()

```
POCKETSPHINX_EXPORT int ps_start_utt (
    ps_decoder_t * ps)
```

Start utterance processing.

This function should be called before any utterance data is passed to the decoder. It marks the start of a new utterance and reinitializes internal data structures.

Parameters

<i>ps</i>	Decoder to be started.
-----------	------------------------

Returns

0 for success, <0 on error.

References [POCKETSPHINX_EXPORT](#).

10.28.2.48 ps_update_mllr()

```
POCKETSPHINX_EXPORT ps_mllr_t * ps_update_mllr (
    ps_decoder_t * ps,
    ps_mllr_t * mllr)
```

Adapt current acoustic model using a linear transform.

Parameters

<i>ml/r</i>	The new transform to use, or NULL to update the existing transform. The decoder retains ownership of this pointer, so you may free it if you no longer need it.
-------------	---

Returns

The updated transform object for this decoder, or NULL on failure.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/[pocketsphinx.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[search.h](#)

10.29 ps_endpoint_t Struct Reference

Simple voice activity detection based endpointing.

```
#include <pocketsphinx/endpointer.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) [ps_endpoint_t](#) * [ps_endpoint_init](#) (double window, double ratio, [ps_vad_mode_t](#) mode, int sample_rate, double frame_length)
- [POCKETSPHINX_EXPORT](#) [ps_endpoint_t](#) * [ps_endpoint_retain](#) ([ps_endpoint_t](#) *ep)
- [POCKETSPHINX_EXPORT](#) int [ps_endpoint_free](#) ([ps_endpoint_t](#) *ep)
- [POCKETSPHINX_EXPORT](#) [ps_vad_t](#) * [ps_endpoint_vad](#) ([ps_endpoint_t](#) *ep)
- [POCKETSPHINX_EXPORT](#) const int16 * [ps_endpoint_process](#) ([ps_endpoint_t](#) *ep, const int16 *frame)
- [POCKETSPHINX_EXPORT](#) const int16 * [ps_endpoint_end_stream](#) ([ps_endpoint_t](#) *ep, const int16 *frame, size_t nsamp, size_t *out_nsamp)
- [POCKETSPHINX_EXPORT](#) int [ps_endpoint_in_speech](#) ([ps_endpoint_t](#) *ep)
- [POCKETSPHINX_EXPORT](#) double [ps_endpoint_speech_start](#) ([ps_endpoint_t](#) *ep)
- [POCKETSPHINX_EXPORT](#) double [ps_endpoint_speech_end](#) ([ps_endpoint_t](#) *ep)
- [POCKETSPHINX_EXPORT](#) int [ps_endpoint_set_timestamp_func](#) ([ps_endpoint_t](#) *ep, [ps_endpoint_timestamp_cb_t](#) cb, void *user_data)
- [POCKETSPHINX_EXPORT](#) double [ps_endpoint_timestamp](#) ([ps_endpoint_t](#) *ep)

10.29.1 Detailed Description

Simple voice activity detection based endpointing.

10.29.2 Member Function Documentation

10.29.2.1 ps_endpoint_end_stream()

```
POCKETSPHINX\_EXPORT const int16 * ps\_endpoint\_end\_stream (
    ps\_endpoint\_t * ep,
    const int16 * frame,
    size_t nsamp,
    size_t * out_nsamp)
```

Process remaining samples at end of stream.

Note that the endpointer is *not* thread-safe. You must call all endpointer functions from the same thread.

Parameters

<i>ep</i>	Endpointer.
<i>frame</i>	Frame of data, must contain ps_endpoint_frame_size() samples or less.
<i>nsamp</i>	Number of samples in frame.
<i>out_nsamp</i>	Output, number of samples available.

Returns

Pointer to available samples, or NULL if none available.

References [POCKETSPHINX_EXPORT](#).

10.29.2.2 ps_endpoint_free()

```
POCKETSPHINX_EXPORT int ps_endpoint_free (
    ps_endpoint_t * ep)
```

Release a pointer to endpointer.

Parameters

<i>ep</i>	Endpointer
-----------	------------

Returns

New reference count (0 if freed).

References [POCKETSPHINX_EXPORT](#).

10.29.2.3 ps_endpoint_in_speech()

```
POCKETSPHINX_EXPORT int ps_endpoint_in_speech (
    ps_endpoint_t * ep)
```

Get the current state (speech/not-speech) of the endpointer.

This function can be used to detect speech/non-speech transitions. If it returns 0, and a subsequent call to [ps_endpoint_process\(\)](#) returns non-NULL, this indicates a transition to speech. Conversely, if [ps_endpoint_process\(\)](#) returns non-NULL and a subsequent call to this function returns 0, this indicates a transition to non-speech.

Parameters

<i>ep</i>	Endpointer.
-----------	-------------

Returns

non-zero if in a speech segment after processing the last frame of data.

References [POCKETSPHINX_EXPORT](#).

10.29.2.4 ps_endpoint_init()

```
POCKETSPHINX_EXPORT ps_endpoint_t * ps_endpoint_init (
    double window,
    double ratio,
    ps_vad_mode_t mode,
    int sample_rate,
    double frame_length)
```

Initialize endpointing.

Parameters

<i>window</i>	Seconds of audio to use in speech start/end decision, or 0 to use the default (PS_ENDPOINTER_DEFAULT_WINDOW).
<i>ratio</i>	Ratio of frames needed to trigger start/end decision, or 0 for the default (PS_ENDPOINTER_DEFAULT_RATIO).
<i>mode</i>	"Aggressiveness" of voice activity detection. Stricter values (see ps_vad_mode_t) are less likely to misclassify non-speech as speech.
<i>sample_rate</i>	Sampling rate of input, or 0 for default (which can be obtained with ps_vad_sample_rate()). Only 8000, 16000, 32000, 48000 are directly supported, others will use the closest supported rate (within reason). Note that this means that the actual frame length may not be exactly the one requested, so you must always use the one returned by ps_endpoint_frame_size() (in samples) or ps_endpoint_frame_length() (in seconds).
<i>frame_length</i>	Requested frame length in seconds, or 0.0 for the default. Only 0.01, 0.02, 0.03 currently supported. Actual frame length may be different, you must always use ps_endpoint_frame_length() to obtain it.

Returns

Endpointer object or NULL on failure (invalid parameter for instance).

References [POCKETSPHINX_EXPORT](#).

10.29.2.5 ps_endpoint_process()

```
POCKETSPHINX_EXPORT const int16 * ps_endpoint_process (
    ps_endpoint_t * ep,
    const int16 * frame)
```

Process a frame of audio, returning a frame if in a speech region.

Note that the endpointer is *not* thread-safe. You must call all endpointer functions from the same thread.

Parameters

<i>ep</i>	Endpointer.
<i>frame</i>	Frame of data, must contain ps_endpoint_frame_size() samples.

Returns

NULL if no speech available, or pointer to a frame of [ps_endpoint_frame_size\(\)](#) samples (no more and no less).

References [POCKETSPHINX_EXPORT](#).

10.29.2.6 ps_endpoint_retain()

```
POCKETSPHINX_EXPORT ps_endpoint_t * ps_endpoint_retain (  
    ps_endpoint_t * ep)
```

Retain a pointer to endpointer

Parameters

<i>ep</i>	Endpointer.
-----------	-------------

Returns

Endpointer with incremented reference count.

References [POCKETSPHINX_EXPORT](#).

10.29.2.7 ps_endpoint_set_timestamp_func()

```
POCKETSPHINX_EXPORT int ps_endpoint_set_timestamp_func (  
    ps_endpoint_t * ep,  
    ps_endpoint_timestamp_cb_t cb,  
    void * user_data)
```

Set a callback function to provide external timestamps.

By default, the endpointer uses audio sample counts to calculate timestamps. This can cause drift between audio time and system time when the audio clock is not synchronized with the system clock.

This function allows you to provide an external timestamp source (e.g., system time, monotonic clock) to avoid this drift.

Parameters

<i>ep</i>	Endpointer.
<i>cb</i>	Callback function that returns current time in seconds, or NULL to revert to audio-based timestamps.
<i>user_data</i>	User data to pass to the callback function.

Returns

0 for success, -1 for error.

References [POCKETSPHINX_EXPORT](#).

10.29.2.8 ps_endpoint_speech_end()

```
POCKETSPHINX_EXPORT double ps_endpoint_speech_end (  
    ps_endpoint_t * ep)
```

Get the end time of the last speech segment

References [POCKETSPHINX_EXPORT](#).

10.29.2.9 ps_endpointter_speech_start()

```
POCKETSPHINX_EXPORT double ps_endpointter_speech_start (
    ps_endpointter_t * ep)
```

Get the start time of the last speech segment.

References [POCKETSPHINX_EXPORT](#).

10.29.2.10 ps_endpointter_timestamp()

```
POCKETSPHINX_EXPORT double ps_endpointter_timestamp (
    ps_endpointter_t * ep)
```

Get the current timestamp from the endpointter.

This returns the current timestamp used by the endpointter, which may be based on audio samples or an external timestamp source if one has been set with [ps_endpointter_set_timestamp_func\(\)](#).

Parameters

<i>ep</i>	Endpointter.
-----------	--------------

Returns

Current timestamp in seconds.

10.29.2.11 ps_endpointter_vad()

```
POCKETSPHINX_EXPORT ps_vad_t * ps_endpointter_vad (
    ps_endpointter_t * ep)
```

Get the voice activity detector used by the endpointter.

Returns

VAD object. The endpointter retains ownership of this object, so you must use [ps_vad_retain\(\)](#) if you wish to use it outside of the lifetime of the endpointter.

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[endpointter.h](#)

10.30 ps_latlink_iter_t Struct Reference

Iterator over DAG links.

```
#include <pocketsphinx/lattice.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) [ps_latlink_iter_t](#) * [ps_latlink_iter_next](#) ([ps_latlink_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) void [ps_latlink_iter_free](#) ([ps_latlink_iter_t](#) *itor)
- [POCKETSPHINX_EXPORT](#) [ps_latlink_t](#) * [ps_latlink_iter_link](#) ([ps_latlink_iter_t](#) *itor)

10.30.1 Detailed Description

Iterator over DAG links.

10.30.2 Member Function Documentation

10.30.2.1 [ps_latlink_iter_free\(\)](#)

```
POCKETSPHINX\_EXPORT void ps\_latlink\_iter\_free (
    ps\_latlink\_iter\_t * itor)
```

Stop iterating over links.

Parameters

<i>itor</i>	Link iterator.
-------------	----------------

References [POCKETSPHINX_EXPORT](#).

10.30.2.2 [ps_latlink_iter_link\(\)](#)

```
POCKETSPHINX\_EXPORT ps\_latlink\_t * ps\_latlink\_iter\_link (
    ps\_latlink\_iter\_t * itor)
```

Get link from iterator.

References [POCKETSPHINX_EXPORT](#).

10.30.2.3 [ps_latlink_iter_next\(\)](#)

```
POCKETSPHINX\_EXPORT ps\_latlink\_iter\_t * ps\_latlink\_iter\_next (
    ps\_latlink\_iter\_t * itor)
```

Get next link from a lattice link iterator.

Parameters

<i>itor</i>	Iterator.
-------------	-----------

Returns

Updated iterator, or NULL if finished.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[lattice.h](#)

10.31 ps_latlink_t Struct Reference

Link between DAG nodes.

```
#include <pocketsphinx/lattice.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) int [ps_latlink_times](#) ([ps_latlink_t](#) *link, int16 *out_sf)
- [POCKETSPHINX_EXPORT](#) [ps_latnode_t](#) * [ps_latlink_nodes](#) ([ps_latlink_t](#) *link, [ps_latnode_t](#) **out_src)
- [POCKETSPHINX_EXPORT](#) char const * [ps_latlink_word](#) ([ps_lattice_t](#) *dag, [ps_latlink_t](#) *link)
- [POCKETSPHINX_EXPORT](#) char const * [ps_latlink_baseword](#) ([ps_lattice_t](#) *dag, [ps_latlink_t](#) *link)
- [POCKETSPHINX_EXPORT](#) [ps_latlink_t](#) * [ps_latlink_pred](#) ([ps_latlink_t](#) *link)
- [POCKETSPHINX_EXPORT](#) int32 [ps_latlink_prob](#) ([ps_lattice_t](#) *dag, [ps_latlink_t](#) *link, int32 *out_ascr)

10.31.1 Detailed Description

Link between DAG nodes.

A link corresponds to a single hypothesized instance of a word with a given start and end point.

10.31.2 Member Function Documentation

10.31.2.1 ps_latlink_baseword()

```
POCKETSPHINX\_EXPORT char const * ps\_latlink\_baseword (  
    ps\_lattice\_t * dag,  
    ps\_latlink\_t * link)
```

Get base word string from a lattice link.

Parameters

<i>dag</i>	Lattice to which node belongs.
<i>link</i>	Link inquired about

Returns

Base word string for this link

References [POCKETSPHINX_EXPORT](#).

10.31.2.2 ps_latlink_nodes()

```
POCKETSPHINX\_EXPORT ps\_latnode\_t * ps\_latlink\_nodes (  
    ps\_latlink\_t * link,  
    ps\_latnode\_t ** out_src)
```

Get destination and source nodes from a lattice link

Parameters

<i>link</i>	Link inquired about
<i>out_src</i>	Output: (optional) source node.

Returns

destination node

References [POCKETSPHINX_EXPORT](#).

10.31.2.3 ps_latlink_pred()

```
POCKETSPHINX_EXPORT ps_latlink_t * ps_latlink_pred (
    ps_latlink_t * link)
```

Get predecessor link in best path.

Parameters

<i>link</i>	Link inquired about
-------------	---------------------

Returns

Best previous link from bestpath search, if any. Otherwise NULL

References [POCKETSPHINX_EXPORT](#).

10.31.2.4 ps_latlink_prob()

```
POCKETSPHINX_EXPORT int32 ps_latlink_prob (
    ps_lattice_t * dag,
    ps_latlink_t * link,
    int32 * out_ascr)
```

Get acoustic score and posterior probability from a lattice link.

Parameters

<i>dag</i>	Lattice to which node belongs.
<i>link</i>	Link inquired about
<i>out_ascr</i>	Output: (optional) acoustic score.

Returns

Posterior probability for this link. Log is expressed in the log-base used in the decoder. To convert to linear floating-point, use `logmath_exp(ps_lattice_get_logmath(), pprob)`.

References [POCKETSPHINX_EXPORT](#).

10.31.2.5 ps_latlink_times()

```
POCKETSPHINX_EXPORT int ps_latlink_times (
    ps_latlink_t * link,
    int16 * out_sf)
```

Get start and end times from a lattice link.

Note

these are **inclusive** - i.e. the last frame of this word is ef, not ef-1.

Parameters

<i>link</i>	Link inquired about.
<i>out_sf</i>	Output: (optional) start frame of this link.

Returns

End frame of this link.

References [POCKETSPHINX_EXPORT](#).

10.31.2.6 ps_latlink_word()

```
POCKETSPHINX_EXPORT char const * ps_latlink_word (
    ps_lattice_t * dag,
    ps_latlink_t * link)
```

Get word string from a lattice link.

Parameters

<i>dag</i>	Lattice to which node belongs.
<i>link</i>	Link inquired about

Returns

Word string for this link (possibly a pronunciation variant).

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[lattice.h](#)

10.32 ps_latnode_iter_t Struct Reference

Iterator over DAG nodes.

```
#include <pocketsphinx/lattice.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ps_latnode_iter_t * ps_latnode_iter_next \(ps_latnode_iter_t *itor\)](#)
- [POCKETSPHINX_EXPORT void ps_latnode_iter_free \(ps_latnode_iter_t *itor\)](#)
- [POCKETSPHINX_EXPORT ps_latnode_t * ps_latnode_iter_node \(ps_latnode_iter_t *itor\)](#)
- [POCKETSPHINX_EXPORT int ps_latnode_times \(ps_latnode_t *node, int16 *out_fef, int16 *out_lef\)](#)

10.32.1 Detailed Description

Iterator over DAG nodes.

10.32.2 Member Function Documentation

10.32.2.1 ps_latnode_iter_free()

```
POCKETSPHINX_EXPORT void ps_latnode_iter_free (
    ps_latnode_iter_t * itor)
```

Stop iterating over nodes.

Parameters

<i>itor</i>	Node iterator.
-------------	----------------

References [POCKETSPHINX_EXPORT](#).

10.32.2.2 ps_latnode_iter_next()

```
POCKETSPHINX_EXPORT ps_latnode_iter_t * ps_latnode_iter_next (
    ps_latnode_iter_t * itor)
```

Move to next node in iteration.

Parameters

<i>itor</i>	Node iterator.
-------------	----------------

Returns

Updated node iterator, or NULL if finished

References [POCKETSPHINX_EXPORT](#).

10.32.2.3 ps_latnode_iter_node()

```
POCKETSPHINX_EXPORT ps_latnode_t * ps_latnode_iter_node (
    ps_latnode_iter_t * itor)
```

Get node from iterator.

References [POCKETSPHINX_EXPORT](#).

10.32.2.4 ps_latnode_times()

```
POCKETSPHINX_EXPORT int ps_latnode_times (
    ps_latnode_t * node,
    int16 * out_fef,
    int16 * out_lef)
```

Get start and end time range for a node.

Parameters

<i>node</i>	Node inquired about.
<i>out_fef</i>	Output: End frame of first exit from this node.
<i>out_lef</i>	Output: End frame of last exit from this node.

Returns

Start frame for all edges exiting this node.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/lattice.h

10.33 ps_latnode_t Struct Reference

Node in a word lattice.

```
#include <pocketsphinx/lattice.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) char const * [ps_latnode_word](#) ([ps_lattice_t](#) *dag, [ps_latnode_t](#) *node)
- [POCKETSPHINX_EXPORT](#) char const * [ps_latnode_baseword](#) ([ps_lattice_t](#) *dag, [ps_latnode_t](#) *node)
- [POCKETSPHINX_EXPORT](#) [ps_latlink_iter_t](#) * [ps_latnode_exits](#) ([ps_latnode_t](#) *node)
- [POCKETSPHINX_EXPORT](#) [ps_latlink_iter_t](#) * [ps_latnode_entries](#) ([ps_latnode_t](#) *node)
- [POCKETSPHINX_EXPORT](#) int32 [ps_latnode_prob](#) ([ps_lattice_t](#) *dag, [ps_latnode_t](#) *node, [ps_latlink_t](#) **out_link)

10.33.1 Detailed Description

Node in a word lattice.

A node corresponds to a number of hypothesized instances of a word which all share the same starting point.

10.33.2 Member Function Documentation

10.33.2.1 `ps_latnode_baseword()`

```
POCKETSPHINX_EXPORT char const * ps_latnode_baseword (  
    ps_lattice_t * dag,  
    ps_latnode_t * node)
```

Get base word string for this node.

Parameters

<i>dag</i>	Lattice to which node belongs.
<i>node</i>	Node inquired about.

Returns

Base word string for this node.

References [POCKETSPHINX_EXPORT](#).

10.33.2.2 `ps_latnode_entries()`

```
POCKETSPHINX_EXPORT ps_latlink_iter_t * ps_latnode_entries (  
    ps_latnode_t * node)
```

Iterate over entries to this node.

Parameters

<i>node</i>	Node inquired about.
-------------	----------------------

Returns

Iterator over entry links to this node.

References [POCKETSPHINX_EXPORT](#).

10.33.2.3 `ps_latnode_exits()`

```
POCKETSPHINX_EXPORT ps_latlink_iter_t * ps_latnode_exits (  
    ps_latnode_t * node)
```

Iterate over exits from this node.

Parameters

<i>node</i>	Node inquired about.
-------------	----------------------

Returns

Iterator over exit links from this node.

References [POCKETSPHINX_EXPORT](#).

10.33.2.4 ps_latnode_prob()

```
POCKETSPHINX_EXPORT int32 ps_latnode_prob (
    ps_lattice_t * dag,
    ps_latnode_t * node,
    ps_latlink_t ** out_link)
```

Get best posterior probability and associated acoustic score from a lattice node.

Parameters

<i>dag</i>	Lattice to which node belongs.
<i>node</i>	Node inquired about.
<i>out_link</i>	Output: exit link with highest posterior probability

Returns

Posterior probability of the best link exiting this node. Log is expressed in the log-base used in the decoder. To convert to linear floating-point, use `logmath_exp(ps_lattice_get_logmath(), pprob)`.

References [POCKETSPHINX_EXPORT](#).

10.33.2.5 ps_latnode_word()

```
POCKETSPHINX_EXPORT char const * ps_latnode_word (
    ps_lattice_t * dag,
    ps_latnode_t * node)
```

Get word string for this node.

Parameters

<i>dag</i>	Lattice to which node belongs.
<i>node</i>	Node inquired about.

Returns

Word string for this node (possibly a pronunciation variant).

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[lattice.h](#)

10.34 ps_lattice_t Struct Reference

Word graph structure used in bestpath/nbest search.

```
#include <pocketsphinx/lattice.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ps_lattice_t * ps_lattice_read](#) (struct ps_decoder_s *ps, char const *file)
- [POCKETSPHINX_EXPORT ps_lattice_t * ps_lattice_retain](#) (ps_lattice_t *dag)
- [POCKETSPHINX_EXPORT int ps_lattice_free](#) (ps_lattice_t *dag)
- [POCKETSPHINX_EXPORT int ps_lattice_write](#) (ps_lattice_t *dag, char const *filename)
- [POCKETSPHINX_EXPORT int ps_lattice_write_htk](#) (ps_lattice_t *dag, char const *filename)
- [POCKETSPHINX_EXPORT logmath_t * ps_lattice_get_logmath](#) (ps_lattice_t *dag)
- [POCKETSPHINX_EXPORT ps_latnode_iter_t * ps_latnode_iter](#) (ps_lattice_t *dag)
- [POCKETSPHINX_EXPORT void ps_lattice_link](#) (ps_lattice_t *dag, ps_latnode_t *from, ps_latnode_t *to, int32 score, int32 ef)
- [POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_traverse_edges](#) (ps_lattice_t *dag, ps_latnode_t *start, ps_latnode_t *end)
- [POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_traverse_next](#) (ps_lattice_t *dag, ps_latnode_t *end)
- [POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_reverse_edges](#) (ps_lattice_t *dag, ps_latnode_t *start, ps_latnode_t *end)
- [POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_reverse_next](#) (ps_lattice_t *dag, ps_latnode_t *start)
- [POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_bestpath](#) (ps_lattice_t *dag, ngram_model_t *lmset, float32 lwf, float32 ascale)
- [POCKETSPHINX_EXPORT int32 ps_lattice_posterior](#) (ps_lattice_t *dag, ngram_model_t *lmset, float32 ascale)
- [POCKETSPHINX_EXPORT int32 ps_lattice_posterior_prune](#) (ps_lattice_t *dag, int32 beam)
- [POCKETSPHINX_EXPORT int ps_lattice_n_frames](#) (ps_lattice_t *dag)

10.34.1 Detailed Description

Word graph structure used in bestpath/nbest search.

10.34.2 Member Function Documentation

10.34.2.1 ps_latnode_iter()

```
POCKETSPHINX_EXPORT ps_latnode_iter_t * ps_latnode_iter (
    ps_lattice_t * dag)
```

Start iterating over nodes in the lattice.

Note

No particular order of traversal is guaranteed, and you should not depend on this.

Parameters

<i>dag</i>	Lattice to iterate over.
------------	--------------------------

Returns

Iterator over lattice nodes.

References [POCKETSPHINX_EXPORT](#).

10.34.2.2 `ps_lattice_bestpath()`

```
POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_bestpath (  
    ps_lattice_t * dag,  
    ngram_model_t * lmset,  
    float32 lwf,  
    float32 ascale)
```

Do N-Gram based best-path search on a word graph.

This function calculates both the best path as well as the forward probability used in confidence estimation.

Returns

Final link in best path, NULL on error.

References [POCKETSPHINX_EXPORT](#).

10.34.2.3 `ps_lattice_free()`

```
POCKETSPHINX_EXPORT int ps_lattice_free (  
    ps_lattice_t * dag)
```

Free a lattice.

Returns

new reference count (0 if dag was freed)

References [POCKETSPHINX_EXPORT](#).

10.34.2.4 `ps_lattice_get_logmath()`

```
POCKETSPHINX_EXPORT logmath_t * ps_lattice_get_logmath (  
    ps_lattice_t * dag)
```

Get the log-math computation object for this lattice

Returns

The log-math object for this lattice. The lattice retains ownership of this pointer, so you should not attempt to free it manually. Use `logmath_retain()` if you wish to reuse it elsewhere.

References [POCKETSPHINX_EXPORT](#).

10.34.2.5 `ps_lattice_link()`

```
POCKETSPHINX_EXPORT void ps_lattice_link (
    ps_lattice_t * dag,
    ps_latnode_t * from,
    ps_latnode_t * to,
    int32 score,
    int32 ef)
```

Create a directed link between "from" and "to" nodes, but if a link already exists, choose one with the best link_scr.

References [POCKETSPHINX_EXPORT](#).

10.34.2.6 `ps_lattice_n_frames()`

```
POCKETSPHINX_EXPORT int ps_lattice_n_frames (
    ps_lattice_t * dag)
```

Get the number of frames in the lattice.

Parameters

<i>dag</i>	The lattice in question.
------------	--------------------------

Returns

Number of frames in this lattice.

10.34.2.7 `ps_lattice_posterior()`

```
POCKETSPHINX_EXPORT int32 ps_lattice_posterior (
    ps_lattice_t * dag,
    ngram_model_t * lmset,
    float32 ascale)
```

Calculate link posterior probabilities on a word graph.

This function assumes that bestpath search has already been done.

Returns

Posterior probability of the utterance as a whole.

References [POCKETSPHINX_EXPORT](#).

10.34.2.8 `ps_lattice_posterior_prune()`

```
POCKETSPHINX_EXPORT int32 ps_lattice_posterior_prune (
    ps_lattice_t * dag,
    int32 beam)
```

Prune all links (and associated nodes) below a certain posterior probability.

This function assumes that [ps_lattice_posterior\(\)](#) has already been called.

Parameters

<i>beam</i>	Minimum posterior probability for links. This is expressed in the log-base used in the decoder. To convert from linear floating-point, use <code>logmath_log(ps_lattice_get_logmath(), prob)</code> .
-------------	---

Returns

number of arcs removed.

References [POCKETSPHINX_EXPORT](#).

10.34.2.9 `ps_lattice_read()`

```
POCKETSPHINX_EXPORT ps_lattice_t * ps_lattice_read (
    struct ps_decoder_s * ps,
    char const * file)
```

Read a lattice from a file on disk.

Parameters

<i>ps</i>	Decoder to use for processing this lattice, or NULL.
<i>file</i>	Path to lattice file.

Returns

Newly created lattice, or NULL for failure.

References [POCKETSPHINX_EXPORT](#).

10.34.2.10 `ps_lattice_retain()`

```
POCKETSPHINX_EXPORT ps_lattice_t * ps_lattice_retain (
    ps_lattice_t * dag)
```

Retain a lattice.

This function retains ownership of a lattice for the caller, preventing it from being freed automatically. You must call [ps_lattice_free\(\)](#) to free it after having called this function.

Returns

pointer to the retained lattice.

References [POCKETSPHINX_EXPORT](#).

10.34.2.11 `ps_lattice_reverse_edges()`

```
POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_reverse_edges (
    ps_lattice_t * dag,
    ps_latnode_t * start,
    ps_latnode_t * end)
```

Start a reverse traversal of edges in a word graph.

Note

See [ps_lattice_traverse_edges\(\)](#) for why this API is the way it is.

Parameters

<i>dag</i>	Lattice to be traversed.
<i>start</i>	Start node (goal) of traversal.
<i>end</i>	End node (source) of traversal.

Returns

First link in traversal.

References [POCKETSPHINX_EXPORT](#).

10.34.2.12 ps_lattice_reverse_next()

```
POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_reverse_next (
    ps_lattice_t * dag,
    ps_latnode_t * start)
```

Get the next link in reverse traversal.

Parameters

<i>dag</i>	Lattice to be traversed.
<i>start</i>	Start node (goal) of traversal.

Returns

Next link in traversal.

References [POCKETSPHINX_EXPORT](#).

10.34.2.13 ps_lattice_traverse_edges()

```
POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_traverse_edges (
    ps_lattice_t * dag,
    ps_latnode_t * start,
    ps_latnode_t * end)
```

Start a forward traversal of edges in a word graph.

Note

A keen eye will notice an inconsistency in this API versus other types of iterators in PocketSphinx. The reason for this is that the traversal algorithm is much more efficient when it is able to modify the lattice structure. Therefore, to avoid giving the impression that multiple traversals are possible at once, no separate iterator structure is provided.

Parameters

<i>dag</i>	Lattice to be traversed.
<i>start</i>	Start node (source) of traversal.
<i>end</i>	End node (goal) of traversal.

Returns

First link in traversal.

References [POCKETSPHINX_EXPORT](#).

10.34.2.14 ps_lattice_traverse_next()

```
POCKETSPHINX_EXPORT ps_latlink_t * ps_lattice_traverse_next (  
    ps_lattice_t * dag,  
    ps_latnode_t * end)
```

Get the next link in forward traversal.

Parameters

<i>dag</i>	Lattice to be traversed.
<i>end</i>	End node (goal) of traversal.

Returns

Next link in traversal.

References [POCKETSPHINX_EXPORT](#).

10.34.2.15 ps_lattice_write()

```
POCKETSPHINX_EXPORT int ps_lattice_write (  
    ps_lattice_t * dag,  
    char const * filename)
```

Write a lattice to disk.

Returns

0 for success, <0 on failure.

References [POCKETSPHINX_EXPORT](#).

10.34.2.16 ps_lattice_write_htk()

```
POCKETSPHINX_EXPORT int ps_lattice_write_htk (  
    ps_lattice_t * dag,  
    char const * filename)
```

Write a lattice to disk in HTK format

Returns

0 for success, <0 on failure.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[lattice.h](#)

10.35 ps_mllr_t Struct Reference

Linear transform object.

```
#include <pocketsphinx/mllr.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ps_mllr_t * ps_mllr_read](#) (char const *file)
- [POCKETSPHINX_EXPORT ps_mllr_t * ps_mllr_retain](#) ([ps_mllr_t](#) *mllr)
- [POCKETSPHINX_EXPORT int ps_mllr_free](#) ([ps_mllr_t](#) *mllr)

10.35.1 Detailed Description

Linear transform object.

10.35.2 Member Function Documentation

10.35.2.1 ps_mllr_free()

```
POCKETSPHINX_EXPORT int ps_mllr_free (  
    ps_mllr_t * mllr)
```

Release a pointer to a linear transform.

10.35.2.2 ps_mllr_read()

```
POCKETSPHINX_EXPORT ps_mllr_t * ps_mllr_read (
    char const * file)
```

Read a speaker-adaptive linear transform from a file.

References [POCKETSPHINX_EXPORT](#).

10.35.2.3 ps_mllr_retain()

```
POCKETSPHINX_EXPORT ps_mllr_t * ps_mllr_retain (
    ps_mllr_t * mllr)
```

Retain a pointer to a linear transform.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[mllr.h](#)

10.36 ps_nbest_t Struct Reference

N-best hypothesis iterator object.

```
#include <pocketsphinx.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT ps_nbest_t * ps_nbest_next](#) ([ps_nbest_t](#) *nbest)
- [POCKETSPHINX_EXPORT char const * ps_nbest_hyp](#) ([ps_nbest_t](#) *nbest, [int32](#) *out_score)
- [POCKETSPHINX_EXPORT ps_seg_t * ps_nbest_seg](#) ([ps_nbest_t](#) *nbest)
- [POCKETSPHINX_EXPORT void ps_nbest_free](#) ([ps_nbest_t](#) *nbest)

10.36.1 Detailed Description

N-best hypothesis iterator object.

10.36.2 Member Function Documentation

10.36.2.1 ps_nbest_free()

```
POCKETSPHINX_EXPORT void ps_nbest_free (
    ps_nbest_t * nbest)
```

Finish N-best search early, releasing resources.

Parameters

<i>nbest</i>	N-best iterator.
--------------	------------------

References [POCKETSPHINX_EXPORT](#).

10.36.2.2 `ps_nbest_hyp()`

```
POCKETSPHINX_EXPORT char const * ps_nbest_hyp (  
    ps_nbest_t * nbest,  
    int32 * out_score)
```

Get the hypothesis string from an N-best list iterator.

Parameters

<i>nbest</i>	N-best iterator.
<i>out_score</i>	Output: Path score for this hypothesis.

Returns

String containing next best hypothesis. Note that this pointer is only valid for the current iteration.

References [POCKETSPHINX_EXPORT](#).

10.36.2.3 `ps_nbest_next()`

```
POCKETSPHINX_EXPORT ps_nbest_t * ps_nbest_next (  
    ps_nbest_t * nbest)
```

Move an N-best list iterator forward.

Parameters

<i>nbest</i>	N-best iterator.
--------------	------------------

Returns

Updated N-best iterator, or NULL if no more hypotheses are available (iterator is freed in this case).

References [POCKETSPHINX_EXPORT](#).

10.36.2.4 `ps_nbest_seg()`

```
POCKETSPHINX_EXPORT ps_seg_t * ps_nbest_seg (  
    ps_nbest_t * nbest)
```

Get the word segmentation from an N-best list iterator.

Parameters

--

<i>nbest</i>	N-best iterator.
--------------	------------------

Returns

Iterator over the next best hypothesis.

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- `C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx.h`

10.37 `ps_search_iter_t` Struct Reference

Iterator over search modules.

```
#include <pocketsphinx/search.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) `ps_search_iter_t * ps_search_iter_next (ps_search_iter_t *itor)`
- [POCKETSPHINX_EXPORT](#) `const char * ps_search_iter_val (ps_search_iter_t *itor)`
- [POCKETSPHINX_EXPORT](#) `void ps_search_iter_free (ps_search_iter_t *itor)`

10.37.1 Detailed Description

Iterator over search modules.

10.37.2 Member Function Documentation

10.37.2.1 `ps_search_iter_free()`

```
POCKETSPHINX\_EXPORT void ps_search_iter_free (  
    ps\_search\_iter\_t * itor)
```

Delete an unfinished search iterator

See also

`ps_activate_search`

References [POCKETSPHINX_EXPORT](#), and [ps_search_iter_val\(\)](#).

10.37.2.2 `ps_search_iter_next()`

```
POCKETSPHINX_EXPORT ps_search_iter_t * ps_search_iter_next (  
    ps_search_iter_t * itor)
```

Updates search iterator to point to the next position.

This function automatically frees the iterator object upon reaching the final entry.

See also

`ps_activate_search`

References [POCKETSPHINX_EXPORT](#).

10.37.2.3 `ps_search_iter_val()`

```
POCKETSPHINX_EXPORT const char * ps_search_iter_val (  
    ps_search_iter_t * itor)
```

Retrieves the name of the search the iterator points to.

See also

`ps_activate_search`

Updates search iterator to point to the next position.

This function automatically frees the iterator object upon reaching the final entry.

See also

`ps_activate_search`

References [POCKETSPHINX_EXPORT](#).

Referenced by [ps_search_iter_free\(\)](#).

The documentation for this struct was generated from the following file:

- `C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/search.h`

10.38 `ps_seg_t` Struct Reference

Segmentation iterator object.

```
#include <pocketsphinx.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) [ps_seg_t](#) * [ps_seg_next](#) ([ps_seg_t](#) *seg)
- [POCKETSPHINX_EXPORT](#) char const * [ps_seg_word](#) ([ps_seg_t](#) *seg)
- [POCKETSPHINX_EXPORT](#) void [ps_seg_frames](#) ([ps_seg_t](#) *seg, int *out_sf, int *out_ef)
- [POCKETSPHINX_EXPORT](#) int32 [ps_seg_prob](#) ([ps_seg_t](#) *seg, int32 *out_ascr, int32 *out_lscr, int32 *out_lback)
- [POCKETSPHINX_EXPORT](#) void [ps_seg_free](#) ([ps_seg_t](#) *seg)

10.38.1 Detailed Description

Segmentation iterator object.

10.38.2 Member Function Documentation**10.38.2.1 ps_seg_frames()**

```
POCKETSPHINX\_EXPORT void ps\_seg\_frames (
    ps\_seg\_t * seg,
    int * out_sf,
    int * out_ef)
```

Get inclusive start and end frames from a segmentation iterator.

Note

These frame numbers are inclusive, i.e. the end frame refers to the last frame in which the given word or other segment was active. Therefore, the actual duration is *out_ef - *out_sf + 1.

Parameters

<i>seg</i>	Segment iterator.
<i>out_sf</i>	Output: First frame index in segment.
<i>out_ef</i>	Output: Last frame index in segment.

References [POCKETSPHINX_EXPORT](#).

10.38.2.2 ps_seg_free()

```
POCKETSPHINX\_EXPORT void ps\_seg\_free (
    ps\_seg\_t * seg)
```

Finish iterating over a word segmentation early, freeing resources.

References [POCKETSPHINX_EXPORT](#).

10.38.2.3 ps_seg_next()

```
POCKETSPHINX\_EXPORT ps\_seg\_t * ps\_seg\_next (
    ps\_seg\_t * seg)
```

Get the next segment in a word segmentation.

Parameters

<i>seg</i>	Segment iterator.
------------	-------------------

Returns

Updated iterator with the next segment. NULL at end of utterance (the iterator will be freed in this case).

References [POCKETSPHINX_EXPORT](#).

10.38.2.4 ps_seg_prob()

```
POCKETSPHINX_EXPORT int32 ps_seg_prob (
    ps_seg_t * seg,
    int32 * out_ascr,
    int32 * out_lscr,
    int32 * out_lback)
```

Get language, acoustic, and posterior probabilities from a segmentation iterator.

Note

Unless the -bestpath option is enabled, this function will always return zero (corresponding to a posterior probability of 1.0). Even if -bestpath is enabled, it will also return zero when called on a partial result. Ongoing research into effective confidence annotation for partial hypotheses may result in these restrictions being lifted in future versions.

Parameters

<i>out_ascr</i>	Output: acoustic model score for this segment.
<i>out_lscr</i>	Output: language model score for this segment.
<i>out_lback</i>	Output: language model backoff mode for this segment (i.e. the number of words used in calculating lscr). This field is, of course, only meaningful for N-Gram models.

Returns

Log posterior probability of current segment. Log is expressed in the log-base used in the decoder. To convert to linear floating-point, use `logmath_exp(ps_get_logmath(), pprob)`.

References [POCKETSPHINX_EXPORT](#).

10.38.2.5 ps_seg_word()

```
POCKETSPHINX_EXPORT char const * ps_seg_word (
    ps_seg_t * seg)
```

Get word string from a segmentation iterator.

Parameters

<i>seg</i>	Segment iterator.
------------	-------------------

Returns

Read-only string giving string name of this segment. This is only valid until the next call to [ps_seg_next\(\)](#).

References [POCKETSPHINX_EXPORT](#).

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/[pocketsphinx.h](#)

10.39 ps_vad_t Struct Reference

Voice activity detector.

```
#include <pocketsphinx/vad.h>
```

Public Member Functions

- [POCKETSPHINX_EXPORT](#) [ps_vad_t](#) * [ps_vad_init](#) ([ps_vad_mode_t](#) mode, int sample_rate, double frame↵_length)
- [POCKETSPHINX_EXPORT](#) [ps_vad_t](#) * [ps_vad_retain](#) ([ps_vad_t](#) *vad)
- [POCKETSPHINX_EXPORT](#) int [ps_vad_free](#) ([ps_vad_t](#) *vad)
- [POCKETSPHINX_EXPORT](#) int [ps_vad_set_input_params](#) ([ps_vad_t](#) *vad, int sample_rate, double frame↵_length)
- [POCKETSPHINX_EXPORT](#) int [ps_vad_sample_rate](#) ([ps_vad_t](#) *vad)
- [POCKETSPHINX_EXPORT](#) size_t [ps_vad_frame_size](#) ([ps_vad_t](#) *vad)
- [POCKETSPHINX_EXPORT](#) [ps_vad_class_t](#) [ps_vad_classify](#) ([ps_vad_t](#) *vad, const [int16](#) *frame)

10.39.1 Detailed Description

Voice activity detector.

10.39.2 Member Function Documentation

10.39.2.1 ps_vad_classify()

```
POCKETSPHINX\_EXPORT ps\_vad\_class\_t ps\_vad\_classify (
    ps\_vad\_t * vad,
    const int16 * frame)
```

Classify a frame as speech or not speech.

Parameters

<i>vad</i>	Voice activity detector.
<i>frame</i>	Frame of input, must contain the number of samples returned by ps_vad_frame_size() .

Returns

PS_VAD_SPEECH, PS_VAD_NOT_SPEECH, or PS_VAD_ERROR (see [ps_vad_class_t](#)).

10.39.2.2 ps_vad_frame_size()

```
POCKETSPHINX_EXPORT size_t ps_vad_frame_size (
    ps_vad_t * vad)
```

Get the number of samples expected by voice activity detection.

You **must** always ensure that the buffers passed to [ps_vad_classify\(\)](#) contain this number of samples (zero-pad them if necessary).

Parameters

<i>vad</i>	Voice activity detector.
------------	--------------------------

Returns

Size, in samples, of the frames passed to [ps_vad_classify\(\)](#).

10.39.2.3 ps_vad_free()

```
POCKETSPHINX_EXPORT int ps_vad_free (
    ps_vad_t * vad)
```

Release a pointer to voice activity detector.

Parameters

<i>vad</i>	Voice activity detector.
------------	--------------------------

Returns

New reference count (0 if freed).

References [POCKETSPHINX_EXPORT](#).

10.39.2.4 ps_vad_init()

```
POCKETSPHINX_EXPORT ps_vad_t * ps_vad_init (
    ps_vad_mode_t mode,
    int sample_rate,
    double frame_length)
```

Initialize voice activity detection.

Parameters

<i>mode</i>	"Aggressiveness" of voice activity detection. Stricter values (see ps_vad_mode_t) are less likely to misclassify non-speech as speech.
<i>sample_rate</i>	Sampling rate of input, or 0 for default (which can be obtained with ps_vad_sample_rate()). Only 8000, 16000, 32000, 48000 are directly supported. See ps_vad_set_input_params() for more information.
<i>frame_length</i>	Frame length in seconds, or 0.0 for the default. Only 0.01, 0.02, 0.03 currently supported. Actual value may differ, you must use ps_vad_frame_length() to obtain it.

Returns

VAD object or NULL on failure (invalid parameter for instance).

References [POCKETSPHINX_EXPORT](#).

10.39.2.5 ps_vad_retain()

```
POCKETSPHINX_EXPORT ps_vad_t * ps_vad_retain (
    ps_vad_t * vad)
```

Retain a pointer to voice activity detector.

Parameters

<i>vad</i>	Voice activity detector.
------------	--------------------------

Returns

Voice activity detector with incremented reference count.

References [POCKETSPHINX_EXPORT](#).

10.39.2.6 ps_vad_sample_rate()

```
POCKETSPHINX_EXPORT int ps_vad_sample_rate (
    ps_vad_t * vad)
```

Get the sampling rate expected by voice activity detection.

Parameters

<i>vad</i>	Voice activity detector.
------------	--------------------------

Returns

Expected sampling rate.

References [POCKETSPHINX_EXPORT](#).

10.39.2.7 ps_vad_set_input_params()

```
POCKETSPHINX_EXPORT int ps_vad_set_input_params (
    ps_vad_t * vad,
    int sample_rate,
    double frame_length)
```

Set the input parameters for voice activity detection.

Parameters

<i>sample_rate</i>	Sampling rate of input, or 0 for default (which can be obtained with ps_vad_sample_rate()). Only 8000, 16000, 32000, 48000 are directly supported, others will use the closest supported rate (within reason). Note that this means that the actual frame length may not be exactly the one requested, so you must always use the one returned by ps_vad_frame_size() (in samples) or ps_vad_frame_length() (in seconds).
<i>frame_length</i>	Requested frame length in seconds, or 0.0 for the default. Only 0.01, 0.02, 0.03 currently supported. Actual frame length may be different, you must always use ps_vad_frame_length() to obtain it.

Returns

0 for success or -1 on error.

References [POCKETSPHINX_EXPORT](#).

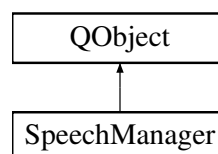
The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/[vad.h](#)

10.40 SpeechManager Class Reference

```
#include <SpeechManager.h>
```

Inheritance diagram for SpeechManager:



Signals

- void [speechRecognized](#) (const QString &text)
- void [speechReadyToProcess](#) (const char *hyp)

Public Member Functions

- [SpeechManager](#) (QObject *parent=nullptr)
- [~SpeechManager](#) ()
- void [configure](#) ()
- void [init](#) ()
- void [startListening](#) ()

Private Slots

- void [processRecognitionResult](#) (const char *hyp)

Private Member Functions

- bool [hypIsValid](#) ()
- void [freeHeap](#) ()

Private Attributes

- PaStream * [stream_](#)
- PaError [err_](#)
- [ps_decoder_t](#) * [decoder_](#)
- [ps_config_t](#) * [config_](#)
- [ps_endpointiter_t](#) * [ep_](#)
- short * [frame_](#)
- size_t [frame_size_](#)
- const char * [hyp_](#)

10.40.1 Constructor & Destructor Documentation**10.40.1.1 SpeechManager()**

```
SpeechManager::SpeechManager (
    QObject * parent = nullptr) [explicit]
```

DEV-SPEECH-0003 @title [SpeechManager](#) bloquea el hilo principal Solved @severity High ratty [SpeechManager](#) debe moverse a QThread TASK-THREAD-0002

References [config_](#), [decoder_](#), [ep_](#), [frame_](#), [frame_size_](#), [processRecognitionResult\(\)](#), [speechReadyToProcess\(\)](#), [speechRecognized\(\)](#), and [stream_](#).

10.40.1.2 ~SpeechManager()

```
SpeechManager::~SpeechManager ()
```

References [freeHeap\(\)](#).

10.40.2 Member Function Documentation

10.40.2.1 `configure()`

```
void SpeechManager::configure ()
```

References [config_](#), and [ConfigManager::instance\(\)](#).

10.40.2.2 `freeHeap()`

```
void SpeechManager::freeHeap () [private]
```

References [decoder_](#), and [ep_](#).

Referenced by [~SpeechManager\(\)](#).

10.40.2.3 `hypIsValid()`

```
bool SpeechManager::hypIsValid () [private]
```

References [hyp_](#).

Referenced by [startListening\(\)](#).

10.40.2.4 `init()`

```
void SpeechManager::init ()
```

References [E_FATAL](#), [E_FATAL_SYSTEM](#), [NULL](#), [ps_endpointter_frame_size](#), and [PS_VAD_LOOSE](#).

10.40.2.5 `processRecognitionResult`

```
void SpeechManager::processRecognitionResult (  
    const char * hyp) [private], [slot]
```

Referenced by [SpeechManager\(\)](#).

10.40.2.6 `speechReadyToProcess`

```
void SpeechManager::speechReadyToProcess (  
    const char * hyp) [signal]
```

Referenced by [SpeechManager\(\)](#).

10.40.2.7 speechRecognized

```
void SpeechManager::speechRecognized (
    const QString & text) [signal]
```

Referenced by [SpeechManager\(\)](#).

10.40.2.8 startListening()

```
void SpeechManager::startListening ()
```

References [E_ERROR](#), [E_FATAL](#), [FALSE](#), [hyplsValid\(\)](#), and [NULL](#).

10.40.3 Member Data Documentation

10.40.3.1 config_

```
ps_config_t* SpeechManager::config_ [private]
```

Referenced by [configure\(\)](#), and [SpeechManager\(\)](#).

10.40.3.2 decoder_

```
ps_decoder_t* SpeechManager::decoder_ [private]
```

Referenced by [freeHeap\(\)](#), and [SpeechManager\(\)](#).

10.40.3.3 ep_

```
ps_endpointinter_t* SpeechManager::ep_ [private]
```

Referenced by [freeHeap\(\)](#), and [SpeechManager\(\)](#).

10.40.3.4 err_

```
PaError SpeechManager::err_ [private]
```

10.40.3.5 frame_

```
short* SpeechManager::frame_ [private]
```

Referenced by [SpeechManager\(\)](#).

10.40.3.6 frame_size_

```
size_t SpeechManager::frame_size_ [private]
```

Referenced by [SpeechManager\(\)](#).

10.40.3.7 hyp_

```
const char* SpeechManager::hyp_ [private]
```

Referenced by [hyplsValid\(\)](#).

10.40.3.8 stream_

```
PaStream* SpeechManager::stream_ [private]
```

Referenced by [SpeechManager\(\)](#).

The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech/[SpeechManager.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech/[SpeechManager.cpp](#)

10.41 SpeechToText Struct Reference

```
#include <TTTCommonTypes.h>
```

Public Member Functions

- void [configuration](#) ()
- void [init](#) ()
- bool [hyplsValid](#) ()
- void [listen](#) ()
- void [free](#) ()

Static Public Member Functions

- static void [catch_sig](#) (int signum)

Public Attributes

- PaStream * [stream](#)
- PaError [err](#)
- [ps_decoder_t](#) * [decoder](#)
- [ps_config_t](#) * [config](#)
- [ps_endpointiter_t](#) * [ep](#)
- short * [frame](#)
- size_t [frame_size](#)
- const char * [hyp](#)

10.41.1 Member Function Documentation

10.41.1.1 `catch_sig()`

```
void SpeechToText::catch_sig (  
    int signum)    [inline], [static]
```

References [global_done](#).

Referenced by [init\(\)](#).

10.41.1.2 `configuration()`

```
void SpeechToText::configuration ()    [inline]
```

References [config](#), and [NULL](#).

Referenced by [TTTController::HumanPlay\(\)](#).

10.41.1.3 `free()`

```
void SpeechToText::free ()    [inline]
```

References [config](#), [decoder](#), and [ep](#).

Referenced by [TTTController::HumanPlay\(\)](#).

10.41.1.4 `hypIsValid()`

```
bool SpeechToText::hypIsValid ()    [inline]
```

References [hyp](#), and [NULL](#).

Referenced by [listen\(\)](#).

10.41.1.5 `init()`

```
void SpeechToText::init ()    [inline]
```

References [catch_sig\(\)](#), [config](#), [decoder](#), [E_FATAL](#), [E_FATAL_SYSTEM](#), [ep](#), [err](#), [frame](#), [frame_size](#), [NULL](#), [ps_endpointer_frame_size](#), [PS_VAD_LOOSE](#), and [stream](#).

Referenced by [TTTController::HumanPlay\(\)](#).

10.41.1.6 `listen()`

```
void SpeechToText::listen ()    [inline]
```

References [decoder](#), [E_ERROR](#), [E_FATAL](#), [ep](#), [err](#), [FALSE](#), [frame](#), [frame_size](#), [hyp](#), [hypIsValid\(\)](#), [NULL](#), and [stream](#).

Referenced by [TTTController::HumanPlay\(\)](#).

10.41.2 Member Data Documentation

10.41.2.1 config

`ps_config_t*` `SpeechToText::config`

Referenced by [configuration\(\)](#), [free\(\)](#), and [init\(\)](#).

10.41.2.2 decoder

`ps_decoder_t*` `SpeechToText::decoder`

Referenced by [free\(\)](#), [init\(\)](#), and [listen\(\)](#).

10.41.2.3 ep

`ps_endpointiter_t*` `SpeechToText::ep`

Referenced by [free\(\)](#), [init\(\)](#), and [listen\(\)](#).

10.41.2.4 err

`PaError` `SpeechToText::err`

Referenced by [init\(\)](#), and [listen\(\)](#).

10.41.2.5 frame

`short*` `SpeechToText::frame`

Referenced by [init\(\)](#), and [listen\(\)](#).

10.41.2.6 frame_size

`size_t` `SpeechToText::frame_size`

Referenced by [init\(\)](#), and [listen\(\)](#).

10.41.2.7 hyp

`const char*` `SpeechToText::hyp`

Referenced by [TTTController::HumanPlay\(\)](#), [hypIsValid\(\)](#), and [listen\(\)](#).

10.41.2.8 stream

```
PaStream* SpeechToText::stream
```

Referenced by [init\(\)](#), and [listen\(\)](#).

The documentation for this struct was generated from the following file:

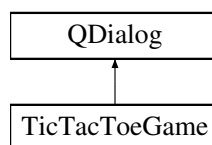
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/[TTTCommonTypes.h](#)

10.42 TicTacToeGame Class Reference

The [TicTacToeGame](#) class is responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one.

```
#include <TicTacToeGame.h>
```

Inheritance diagram for TicTacToeGame:



Signals

- void [newGame](#) ()
newGame is emitted when the New Game button is clicked and some reset logic must be executed.

Public Member Functions

- [TicTacToeGame](#) (QWidget *parent=nullptr)
TicTacToeGame constructs a view of a game.
- void [updateCell](#) (Cell &cell, BoardMarks currentPlayer)
updateCell sets the Text of the cell to the current player's mark.
- void [declareGameState](#) (BoardState boardState)
declareGameState outputs the board's state to a QMessageBox.
- vector< Cell > [buildCellButtons](#) (size_t boardSize)
buildCellButtons builds the GUI cells as push buttons.
- void [reset](#) (vector< Cell > &cells)
reset clears out the text of all push button cells.

Private Member Functions

- void [setConnections](#) ()
setConnections handles setting the connection to emit if cells have clicked, and the navigational back button.
- QString [getPlayerText](#) ([BoardMarks](#) currentPlayer)
getPlayerText converts a board mark to a QString.
- QString [getPlayerStyleSheet](#) ([BoardMarks](#) currentPlayer)
getPlayerStyleSheet gets a stylesheet associated with each player.
- QString [getBoardFinalStateText](#) ([BoardState](#) boardState)
getBoardFinalStateText converts a state to a declaration text, that will be used when declaring the final state of a game.

Private Attributes

- std::unique_ptr< Ui::TicTacToeGame > [ui](#)
ui is a reference to the ui object of the tic tac toe game.

10.42.1 Detailed Description

The [TicTacToeGame](#) class is responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one.

10.42.2 Constructor & Destructor Documentation

10.42.2.1 TicTacToeGame()

```
TicTacToeGame::TicTacToeGame (
    QWidget * parent = nullptr) [explicit]
```

[TicTacToeGame](#) constructs a view of a game.

Parameters

<i>parent</i>	is a QWidget type parent that owns this instance of the controller.
---------------	---

References [setConnections\(\)](#), [TicTacToeGame\(\)](#), and [ui](#).

Referenced by [TicTacToeGame\(\)](#).

10.42.3 Member Function Documentation

10.42.3.1 buildCellButtons()

```
vector< Cell > TicTacToeGame::buildCellButtons (
    size_t boardSize)
```

buildCellButtons builds the GUI cells as push buttons.

Parameters

<i>boardSize</i>	number of rows or columns of a board.
------------------	---------------------------------------

Returns

a collection of cells containing push button references and their corresponding row and column.

References [defaults::GUI_CELL_COLUMN_SPAN](#), [defaults::GUI_CELL_ROW_SPAN](#), and [ui](#).

10.42.3.2 declareGameState()

```
void TicTacToeGame::declareGameState (  
    BoardState boardState)
```

declareGameState outputs the board's state to a QMessageBox.

Parameters

<i>boardState</i>	the current board state.
-------------------	--------------------------

References [getBoardFinalStateText\(\)](#).

10.42.3.3 getBoardFinalStateText()

```
QString TicTacToeGame::getBoardFinalStateText (  
    BoardState boardState) [private]
```

getBoardFinalStateText converts a state to a declaration text, that will be used when declaring the final state of a game.

Parameters

<i>boardState</i>	the current board state.
-------------------	--------------------------

Returns

QString declaration text, that will be used when declaring the final state of a game.

References [OWins](#), [Tie](#), and [XWins](#).

Referenced by [declareGameState\(\)](#).

10.42.3.4 getPlayerStyleSheet()

```
QString TicTacToeGame::getPlayerStyleSheet (  
    BoardMarks currentPlayer) [private]
```

getPlayerStyleSheet gets a stylesheet associated with each player.

Parameters

<i>currentPlayer</i>	the current player's mark (X or O).
----------------------	-------------------------------------

Returns

a QString stylesheet of the player given in the input.

References [defaults::DEFAULT_COLOR](#), [O](#), [defaults::O_COLOR](#), [X](#), and [defaults::X_COLOR](#).

Referenced by [updateCell\(\)](#).

10.42.3.5 getPlayerText()

```
QString TicTacToeGame::getPlayerText (
    BoardMarks currentPlayer) [private]
```

getPlayerText converts a board mark to a QString.

Parameters

<i>currentPlayer</i>	the current player's mark (X or O).
----------------------	-------------------------------------

Returns

a QString of the player's mark.

References [O](#), and [X](#).

Referenced by [updateCell\(\)](#).

10.42.3.6 newGame

```
void TicTacToeGame::newGame () [signal]
```

newGame is emitted when the New Game button is clicked and some reset logic must be executed.

Referenced by [TTTController::setConnections\(\)](#).

10.42.3.7 reset()

```
void TicTacToeGame::reset (
    vector< Cell > & cells)
```

reset clears out the text of all push button cells.

Parameters

<i>cells</i>	a collection of cells containing push button references and their corresponding row and column.
--------------	---

10.42.3.8 setConnections()

```
void TicTacToeGame::setConnections () [private]
```

setConnections handles setting the connection to emit if cells have clicked, and the navigational back button.

References [ui](#).

Referenced by [TicTacToeGame\(\)](#).

10.42.3.9 updateCell()

```
void TicTacToeGame::updateCell (
    Cell & cell,
    BoardMarks currentPlayer)
```

updateCell sets the Text of the cell to the current player's mark.

Parameters

<i>cell</i>	a refernece to the clicked cell.
<i>currentPlayer</i>	the current player's mark (X or O).

References [Cell::cellBtn](#), [getPlayerStyleSheet\(\)](#), and [getPlayerText\(\)](#).

10.42.4 Member Data Documentation

10.42.4.1 ui

```
std::unique_ptr<Ui::TicTacToeGame> TicTacToeGame::ui [private]
```

ui is a reference to the ui object of the tic tac toe game.

Referenced by [buildCellButtons\(\)](#), [setConnections\(\)](#), and [TicTacToeGame\(\)](#).

The documentation for this class was generated from the following files:

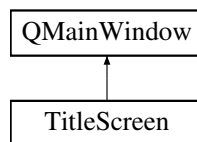
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/[TicTacToeGame.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/[TicTacToeGame.cpp](#)

10.43 TitleScreen Class Reference

The [TitleScreen](#) class is responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options.

```
#include <TitleScreen.h>
```

Inheritance diagram for TitleScreen:



Public Slots

- void [updateBoardSize](#) (int size)
updateBoardSize updates the board size in the current options.
- void [updateSinglePlayer](#) (bool checked)
updateSinglePlayer updates the AI existence flag to true as there is one player.
- void [updateTwoPlayers](#) (bool checked)
updateTwoPlayers updates the AI existence flag to false as there are two player.
- void [updateAIstartsGame](#) (bool checked)
updateAIstartsGame sets the AI starting the first play or not.
- void [updateMiniMaxDepth](#) (int depth)
updateMiniMaxDepth sets the value of the depth cutoff for the minimax algorithm.
- void [startGame](#) ()
startGame hides the title screen and creates a Tic tac toe controller to start a game.

Signals

- void [exited](#) ()
exited is emitted to manually handle the close functionality.

Public Member Functions

- [TitleScreen](#) (QWidget *parent=nullptr)
[TitleScreen](#) constructs the view of the title screen.

Private Member Functions

- void [setConnections](#) ()
setConnections sets connections to gather game options from the GUI elements.
- void [closeEvent](#) (QCloseEvent *event) override
closeEvent manually handles the closing event defined in the base class QWidget.

Private Attributes

- `std::unique_ptr< Ui::TitleScreen > ui`
ui is a reference to the ui object of the tic tac toe game.
- `TTTOptions options_`
options are the values and flags that control a tic tac toe game.

10.43.1 Detailed Description

The [TitleScreen](#) class is responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options.

10.43.2 Constructor & Destructor Documentation

10.43.2.1 TitleScreen()

```
TitleScreen::TitleScreen (
    QWidget * parent = nullptr) [explicit]
```

[TitleScreen](#) constructs the view of the title screen.

Parameters

<i>parent</i>	is a QWidget type parent that owns this instance of the controller.
---------------	---

References [setConnections\(\)](#), [TitleScreen\(\)](#), and [ui](#).

Referenced by [TitleScreen\(\)](#).

10.43.3 Member Function Documentation

10.43.3.1 closeEvent()

```
void TitleScreen::closeEvent (
    QCloseEvent * event) [override], [private]
```

`closeEvent` manually handles the closing event defined in the base class `QWidget`.

Parameters

<i>event</i>	the closing event.
--------------	--------------------

References [exited\(\)](#).

10.43.3.2 exited

```
void TitleScreen::exited () [signal]
```

exited is emitted to manually handle the close functionality.

Referenced by [closeEvent\(\)](#), and [main\(\)](#).

10.43.3.3 setConnections()

```
void TitleScreen::setConnections () [private]
```

setConnections sets connections to gather game options from the GUI elements.

References [startGame\(\)](#), [ui](#), [updateAStartsGame\(\)](#), [updateBoardSize\(\)](#), [updateMiniMaxDepth\(\)](#), [updateSinglePlayer\(\)](#), and [updateTwoPlayers\(\)](#).

Referenced by [TitleScreen\(\)](#).

10.43.3.4 startGame

```
void TitleScreen::startGame () [slot]
```

startGame hides the title screen and creates a Tic tac toe controller to start a game.

References [options_](#).

Referenced by [setConnections\(\)](#).

10.43.3.5 updateAStartsGame

```
void TitleScreen::updateAStartsGame (
    bool checked) [slot]
```

updateAStartsGame sets the AI starting the first play or not.

Parameters

<i>checked</i>	wether the checkbox is checked or not.
----------------	--

References [options_](#).

Referenced by [setConnections\(\)](#).

10.43.3.6 updateBoardSize

```
void TitleScreen::updateBoardSize (
    int size) [slot]
```

updateBoardSize updates the board size in the current options.

Parameters

<i>size</i>	the number of rows or columns in the board.
-------------	---

References [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

10.43.3.7 updateMinMaxDepth

```
void TitleScreen::updateMinMaxDepth (  
    int depth) [slot]
```

updateMinMaxDepth sets the value of the depth cutoff for the minimax algorithm.

Parameters

<i>depth</i>	the depth cutoff for the minimax algorithm.
--------------	---

References [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

10.43.3.8 updateSinglePlayer

```
void TitleScreen::updateSinglePlayer (  
    bool checked) [slot]
```

updateSinglePlayer updates the AI existence flag to true as there is one player.

Parameters

<i>checked</i>	wether the radio button is checked or not.
----------------	--

References [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

10.43.3.9 updateTwoPlayers

```
void TitleScreen::updateTwoPlayers (  
    bool checked) [slot]
```

updateTwoPlayers updates the AI existence flag to false as there are two player.

Parameters

<i>checked</i>	wether the radio button is checked or not.
----------------	--

References [options_](#), and [ui](#).

Referenced by [setConnections\(\)](#).

10.43.4 Member Data Documentation

10.43.4.1 options_

```
TTOptions TitleScreen::options_ [private]
```

options are the values and flags that control a tic tac toe game.

Referenced by [startGame\(\)](#), [updateAlstartsGame\(\)](#), [updateBoardSize\(\)](#), [updateMiniMaxDepth\(\)](#), [updateSinglePlayer\(\)](#), and [updateTwoPlayers\(\)](#).

10.43.4.2 ui

```
std::unique_ptr<Ui::TitleScreen> TitleScreen::ui [private]
```

ui is a reference to the ui object of the tic tac toe game.

Referenced by [setConnections\(\)](#), [TitleScreen\(\)](#), [updateBoardSize\(\)](#), [updateMiniMaxDepth\(\)](#), [updateSinglePlayer\(\)](#), and [updateTwoPlayers\(\)](#).

The documentation for this class was generated from the following files:

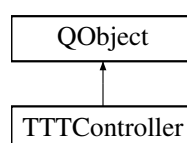
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/[TitleScreen.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/[TitleScreen.cpp](#)

10.44 TTTController Class Reference

The [TTTController](#) class is the controller for a Tic Tac Toe game, it takes in game options as input, and controls the view ([TicTacToeGame](#)) and the model ([Board](#)) according to the game logic specified in this class, it is also responsible for calling the AI agent to play its turn if specified by the game options.

```
#include <TTTController.h>
```

Inheritance diagram for TTTController:



Public Slots

- virtual void [updateGame](#) ([Cell](#) &cell)
updateGame sets the cell clicked in the model (board).

Signals

- void [AIFinished](#) ()
turnFinished is emitted when the human player is finished with the cell input, and the internal game state is supposed to be updated.
- void [humanFinished](#) ()
- void [speechRecognized](#) (QString)

Public Member Functions

- [TTTController](#) (const [TTTOptions](#) &options, QObject *parent=nullptr)
TTTController constructs a game controller.
- virtual void [startGame](#) ()
startGame executes this QDialog to show the GUI and start the gameplay.
- int [parseSpeechToCell](#) (const QString &text)
- QWidget * [getView](#) ()

Protected Member Functions

- virtual void [updateGameState](#) ([Cell](#) &cell)
updateGameState updates the view and the model and switches the current player.
- virtual void [AIAgentPlay](#) ()
AIAgentPlay calls the AI agent to play given the current board inputs.
- virtual void [HumanPlay](#) ()
HumanPlay calls the Speech Recognition Engine for decoding input into [Cell](#).
- virtual void [switchPlayer](#) ()
switchPlayer switches the current player mark (X to O and O to X).

Protected Attributes

- [TicTacToeGame](#) [view_](#)
view is the view class of a tic tac toe game.
- [Board](#) [board_](#)
board is the model that represents the underlying layer of a game.
- vector< [Cell](#) > [cells_](#)
cells are the collection of cell buttons built by the view class.
- const [TTTOptions](#) & [options_](#)
options is the collection of values and flags that controls the game.
- [BoardMarks](#) [currentPlayer_](#)
currentPlayer is the player that will play the current turn.
- unique_ptr< [AIAgent](#) > [agent_](#)
agent is the AI agent chosen to play against the human player.

Private Slots

- void [onSpeechRecognized](#) (const QString &text)
- void [onSocketReadyRead](#) ()

Private Member Functions

- void [setConnections](#) ()
setConnections will set a connection to all GUI elements to slots defined to handle these events.
- void [reset](#) ()
resets all internal components, the view, the model and the AI agent. then if AI is supposed to start a game it will play first.
- void [setupNetwork](#) ()
- void [sendMove](#) (int row, int col, char player)

Private Attributes

- QTcpSocket * [socket_](#)

10.44.1 Detailed Description

The [TTTController](#) class is the controller for a Tic Tac Toe game, it takes in game options as input, and controls the view ([TicTacToeGame](#)) and the model ([Board](#)) according to the game logic specified in this class, it is also responsible for calling the AI agent to play its turn if specified by the game options.

10.44.2 Constructor & Destructor Documentation

10.44.2.1 TTTController()

```
TTTController::TTTController (
    const TTTOptions & options,
    QObject * parent = nullptr) [explicit]
```

[TTTController](#) constructs a game controller.

Parameters

<i>options</i>	is the collection of values and flags that controls the game.
<i>parent</i>	is a QObject type parent that owns this instance of the controller.

References [agent_](#), [board_](#), [TTTOptions::boardSize](#), [cells_](#), [currentPlayer_](#), [O](#), [options_](#), [reset\(\)](#), [setConnections\(\)](#), [view_](#), and [X](#).

10.44.3 Member Function Documentation

10.44.3.1 AIAgentPlay()

```
void TTTController::AIAgentPlay () [protected], [virtual]
```

AIAgentPlay calls the AI agent to play given the current board inputs.

References [agent_](#), [board_](#), [cells_](#), [Cell::col](#), [currentPlayer_](#), [defaults::INVALID_CELL](#), [Cell::row](#), and [updateGame\(\)](#).

Referenced by [reset\(\)](#), and [setConnections\(\)](#).

10.44.3.2 AIFinished

```
void TTTController::AIFinished () [signal]
```

turnFinished is emitted when the human player is finished with the cell input, and the internal game state is supposed to be updated.

Referenced by [onSocketReadyRead\(\)](#), and [setConnections\(\)](#).

10.44.3.3 getView()

```
QWidget * TTTController::getView () [inline]
```

References [view_](#).

10.44.3.4 humanFinished

```
void TTTController::humanFinished () [signal]
```

Referenced by [onSocketReadyRead\(\)](#), and [setConnections\(\)](#).

10.44.3.5 HumanPlay()

```
void TTTController::HumanPlay () [protected], [virtual]
```

HumanPlay calls the Speech Recognition Engine for decoding input into [Cell](#).

References [SpeechToText::configuration\(\)](#), [SpeechToText::free\(\)](#), [SpeechToText::hyp](#), [SpeechToText::init\(\)](#), [SpeechToText::listen\(\)](#), and [speechRecognized\(\)](#).

Referenced by [onSpeechRecognized\(\)](#), [reset\(\)](#), and [setConnections\(\)](#).

10.44.3.6 onSocketReadyRead

```
void TTTController::onSocketReadyRead () [private], [slot]
```

References [AIFinished\(\)](#), [currentPlayer_](#), [humanFinished\(\)](#), [O](#), [options_](#), [socket_](#), [switchPlayer\(\)](#), and [X](#).

Referenced by [NetworkManager::NetworkManager\(\)](#), and [setupNetwork\(\)](#).

10.44.3.7 onSpeechRecognized

```
void TTTController::onSpeechRecognized (  
    const QString & text) [private], [slot]
```

References [cells_](#), [HumanPlay\(\)](#), [defaults::INVALID_CELL](#), [parseSpeechToCell\(\)](#), and [updateGame\(\)](#).

Referenced by [setConnections\(\)](#).

10.44.3.8 parseSpeechToCell()

```
int TTTController::parseSpeechToCell (  
    const QString & text) [inline]
```

References [defaults::INVALID_CELL](#).

Referenced by [onSpeechRecognized\(\)](#).

10.44.3.9 reset()

```
void TTTController::reset () [private]
```

resets all internal components, the view, the model and the AI agent. then if AI is supposed to start a game it will play first.

References [agent_](#), [AIAgentPlay\(\)](#), [board_](#), [cells_](#), [currentPlayer_](#), [HumanPlay\(\)](#), [options_](#), [view_](#), and [X](#).

Referenced by [setConnections\(\)](#), and [TTTController\(\)](#).

10.44.3.10 sendMove()

```
void TTTController::sendMove (  
    int row,  
    int col,  
    char player) [private]
```

References [socket_](#).

Referenced by [updateGame\(\)](#).

10.44.3.11 setConnections()

```
void TTTController::setConnections () [private]
```

setConnections will set a connection to all GUI elements to slots defined to handle these events.

References [AIAgentPlay\(\)](#), [AIFinished\(\)](#), [humanFinished\(\)](#), [HumanPlay\(\)](#), [TicTacToeGame::newGame\(\)](#), [onSpeechRecognized\(\)](#), [reset\(\)](#), [setupNetwork\(\)](#), [speechRecognized\(\)](#), and [view_](#).

Referenced by [TTTController\(\)](#).

10.44.3.12 setupNetwork()

```
void TTTController::setupNetwork () [private]
```

References [onSocketReadyRead\(\)](#), and [socket_](#).

Referenced by [setConnections\(\)](#).

10.44.3.13 speechRecognized

```
void TTTController::speechRecognized (
    QString ) [signal]
```

Referenced by [HumanPlay\(\)](#), and [setConnections\(\)](#).

10.44.3.14 startGame()

```
void TTTController::startGame () [virtual]
```

startGame executes this QDialog to show the GUI and start the gameplay.

References [view_](#).

10.44.3.15 switchPlayer()

```
void TTTController::switchPlayer () [protected], [virtual]
```

switchPlayer switches the current player mark (X to O and O to X).

References [currentPlayer_](#), [O](#), and [X](#).

Referenced by [onSocketReadyRead\(\)](#).

10.44.3.16 updateGame

```
void TTTController::updateGame (
    Cell & cell) [virtual], [slot]
```

updateGame sets the cell clicked in the model (board).

Parameters

<i>cell</i>	the clicked cell that the player chooses.
-------------	---

References [board_](#), [Cell::col](#), [currentPlayer_](#), [Cell::row](#), [sendMove\(\)](#), [updateGameState\(\)](#), and [X](#).

Referenced by [AIAgentPlay\(\)](#), and [onSpeechRecognized\(\)](#).

10.44.3.17 updateGameState()

```
void TTTController::updateGameState (
    Cell & cell) [protected], [virtual]
```

updateGameState updates the view and the model and switches the current player.

Parameters

<i>cell</i>	the clicked cell that the player chooses.
-------------	---

References [board_](#), [currentPlayer_](#), [NoWinner](#), [view_](#), and [X](#).

Referenced by [updateGame\(\)](#).

10.44.4 Member Data Documentation

10.44.4.1 agent_

```
unique_ptr<AIAgent> TTTController::agent_ [protected]
```

agent is the AI agent chosen to play against the human player.

Referenced by [AIAgentPlay\(\)](#), [reset\(\)](#), and [TTTController\(\)](#).

10.44.4.2 board_

```
Board TTTController::board_ [protected]
```

board is the model that represents the underlying layer of a game.

Referenced by [AIAgentPlay\(\)](#), [reset\(\)](#), [TTTController\(\)](#), [updateGame\(\)](#), and [updateGameState\(\)](#).

10.44.4.3 cells_

```
vector<Cell> TTTController::cells_ [protected]
```

cells are the collection of cell buttons built by the view class.

Referenced by [AIAgentPlay\(\)](#), [onSpeechRecognized\(\)](#), [reset\(\)](#), and [TTTController\(\)](#).

10.44.4.4 currentPlayer_

`BoardMarks` `TTTController::currentPlayer_` [protected]

`currentPlayer` is the player that will play the current turn.

Referenced by [AIAgentPlay\(\)](#), [onSocketReadyRead\(\)](#), [reset\(\)](#), [switchPlayer\(\)](#), [TTTController\(\)](#), [updateGame\(\)](#), and [updateGameState\(\)](#).

10.44.4.5 options_

`const TTTOptions&` `TTTController::options_` [protected]

`options` is the collection of values and flags that controls the game.

Referenced by [onSocketReadyRead\(\)](#), [reset\(\)](#), and [TTTController\(\)](#).

10.44.4.6 socket_

`QTcpSocket*` `TTTController::socket_` [private]

Referenced by [onSocketReadyRead\(\)](#), [sendMove\(\)](#), and [setupNetwork\(\)](#).

10.44.4.7 view_

`TicTacToeGame` `TTTController::view_` [protected]

`view` is the view class of a tic tac toe game.

Referenced by [getView\(\)](#), [reset\(\)](#), [setConnections\(\)](#), [startGame\(\)](#), [TTTController\(\)](#), and [updateGameState\(\)](#).

The documentation for this class was generated from the following files:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Controller/[TTTController.h](#)
- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Controller/[TTTController.cpp](#)

10.45 TTTOptions Struct Reference

The [TTTOptions](#) struct defines a tic tac toe game options.

```
#include <TTTCommonTypes.h>
```

Public Attributes

- `size_t` `boardSize` = [defaults::DEFAULT_BOARD_SIZE](#)
- `unsigned short` `miniMaxDepth` = [defaults::DEFAULT_MINIMAX_DEPTH](#)
- `bool` `Alloponent` = `true`
- `bool` `Alstarts` = `false`

10.45.1 Detailed Description

The [TTTOptions](#) struct defines a tic tac toe game options.

10.45.2 Member Data Documentation

10.45.2.1 Alopponent

```
bool TTTOptions::Alopponent = true
```

10.45.2.2 Astarts

```
bool TTTOptions::Astarts = false
```

10.45.2.3 boardSize

```
size_t TTTOptions::boardSize = defaults::DEFAULT_BOARD_SIZE
```

Referenced by [TTTController::TTTController\(\)](#).

10.45.2.4 miniMaxDepth

```
unsigned short TTTOptions::miniMaxDepth = defaults::DEFAULT_MINIMAX_DEPTH
```

The documentation for this struct was generated from the following file:

- C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/[TTTCommonTypes.h](#)

Chapter 11

File Documentation

11.1 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/AIAgent.h File Reference

```
#include "Player.h"
```

Classes

- class [AIAgent](#)

11.2 AIAgent.h

[Go to the documentation of this file.](#)

```
00001 #ifndef AIAgent_H
00002 #define AIAgent_H
00003 #include "Player.h"
00008
00009 class Board;
00010 class AIAgent : public Player
00011 {
00012 public:
00013     AIAgent() {}
00014     virtual ~AIAgent() {}
00020     virtual int play(const Board &board) = 0;
00024     virtual void reset() {}
00025 };
00026
00027 #endif // AIAgent_H
```

11.3 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/HumanPlayer.cpp ↩

File Reference

```
#include "HumanPlayer.h"
```

11.4 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/HumanPlayer.h File Reference ↩

```
#include "Player.h"
#include "SpeechManager.h"
#include <QThread>
```

Classes

- class [HumanPlayer](#)

11.5 HumanPlayer.h

[Go to the documentation of this file.](#)

```
00001 #ifndef HUMANPLAYER_H
00002 #define HUMANPLAYER_H
00003 #include "Player.h"
00004 #include "SpeechManager.h"
00005 #include <QThread>
00006
00007 class HumanPlayer final : public Player
00008     Q_OBJECT
00009 {
00010 public:
00011     explicit HumanPlayer(QObject& parent = nullptr, const Board& board_ref, BoardMarks playerMark =
        BoardMarks::Empty)
00012     virtual ~HumanPlayer() {}
00013     void play() override;
00014 signals:
00015     void startListening();
00016     void stopListening();
00017 private slots:
00018     void onSpeechRecognized(const QString &text);
00019
00020 private:
00021     int parseSpeechCommand(const QString &text);
00022     bool validateCommand(const QString &command);
00023     SpeechManager* stt_;
00024     QThread* sttThread_;
00025 };
00026
00027 #endif // HUMANPLAYER_H
```

11.6 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/MiniMaxAgent.cpp File Reference ↩

```
#include "MiniMaxAgent.h"
#include "Board.h"
#include <QPair>
#include <limits.h>
```

11.7 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/MiniMaxAgent.h File Reference ↩

```
#include "TTTCommonTypes.h"
#include "AIAgent.h"
```

Classes

- class [MiniMaxAgent](#)

The *MiniMaxAgent* class handles all the logic of picking a cell based on the MiniMax Algorithm.

11.8 MiniMaxAgent.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MINIMAXAGENT_H
00002 #define MINIMAXAGENT_H
00003 #include "TTTCommonTypes.h"
00004 #include "AIAgent.h"
00005
00018
00019 class MiniMaxAgent final : public AIAgent
00020 Q_OBJECT
00021 {
00022 public:
00029     MiniMaxAgent(QObject* parent = nullptr, unsigned short depth, BoardMarks AImark);
00035     int play() override;
00036
00037 private:
00041     const unsigned short depth_;
00045     const BoardMarks AImark_;
00049     const BoardMarks playerMark_;
00050
00054     static constexpr short AI_WIN_SCORE = 1;
00058     static constexpr short PLAYER_WIN_SCORE = -1;
00062     static constexpr short TIE_SCORE = 0;
00063
00073     short maxMove(Board &board, unsigned short depth, short alpha, short beta) const;
00083     short minMove(Board &board, unsigned short depth, short alpha, short beta) const;
00089     short score(const BoardState state) const;
00090 };
00091
00092 #endif // MINIMAXAGENT_H
```

11.9 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/NoAgent.h File Reference

```
#include "AIAgent.h"
#include "TTTCommonTypes.h"
#include <qglobal.h>
```

Classes

- class [NoAgent](#)

The *NoAgent* class is an empty class that does not play, it specifies the null object behaviour, so it represents the lack of an AI agent, which is useful to avoid using if conditions in code when checking on the existence of an AI agent.

11.10 NoAgent.h

[Go to the documentation of this file.](#)

```

00001 #ifndef NOAGENT_H
00002 #define NOAGENT_H
00003 #include "AIAgent.h"
00004 #include "TTTCommonTypes.h"
00005 #include <qglobal.h>
00006
00017
00018 class NoAgent final : public AIAgent
00019 {
00020 public:
00021     NoAgent() {}
00022     ~NoAgent() override {}
00028     int play(const Board& board) override {
00029         Q_UNUSED(board);
00030         return defaults::INVALID_CELL;
00031     }
00032 };
00033 #endif // NOAGENT_H

```

11.11 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/AI/Player.h

File Reference

```

#include <QObject>
#include "Board.h"
#include "TTTCommonTypes.h"

```

Classes

- class [Player](#)

The [Player](#) class is an abstract base class for different types of players in the Tic Tac Toe game, such as human players and AI players.

11.12 Player.h

[Go to the documentation of this file.](#)

```

00001 #ifndef PLAYER_H
00002 #define PLAYER_H
00003 #include <QObject>
00004 #include "Board.h"
00005 #include "TTTCommonTypes.h"
00010 class Player : public QObject
00011 {
00012     public:
00013     Player(QObject *parent_ = nullptr, BoardMarks playerMark, const Board& borad_ref) :
00014         parent(parent_),
00015         mark_(playerMark) {}
00016     virtual ~Player() {}
00017     virtual void play() = 0;
00018     signals:
00019     void playerFinished(const int& cell);
00020
00021     protected:
00022
00023     const Board* board_;
00024     const BoardMarks mark_;
00025
00026 };
00027
00028 #endif // PLAYER_H

```

11.13 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config/ConfigManager.cpp File Reference

```
#include "ConfigManager.h"
```

11.14 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Config/ConfigManager.h File Reference

```
#include <QObject>
#include <QSettings>
#include <QDir>
#include <QDebug>
```

Classes

- class [ConfigManager](#)
The *ConfigManager* class.

11.15 ConfigManager.h

[Go to the documentation of this file.](#)

```
00001 #ifndef CONFIGMANAGER_H
00002 #define CONFIGMANAGER_H
00003
00004 #pragma once
00005 #include <QObject>
00006 #include <QSettings>
00007 #include <QDir>
00008 #include <QDebug>
00009
00017 class ConfigManager final : public QObject
00018 {
00019     Q_OBJECT
00020
00021 public:
00027     static ConfigManager& instance();
00028
00035     void load(const QString& filePath = QString());
00036
00044     QString getValue(const QString& key, const QString& defaultValue = QString()) const;
00045
00049     int getInt(const QString& key, int defaultValue = 0) const;
00050
00054     bool getBool(const QString& key, bool defaultValue = false) const;
00055
00056 private:
00057     explicit ConfigManager(QObject *parent = nullptr);
00058     ~ConfigManager() override = default;
00059
00060     // Bloquear copia y asignación
00061     ConfigManager(const ConfigManager&) = delete;
00062     ConfigManager& operator=(const ConfigManager&) = delete;
00063
00064     QSettings* settings_ = nullptr;
00065 };
00066
00067 #endif // CONFIGMANAGER_H
```

11.16 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/↵ Controller/TTTController.cpp File Reference

```
#include "TTTController.h"
#include "MiniMaxAgent.h"
#include "NoAgent.h"
#include "TicTacToeGame.h"
#include <QPushButton>
#include <QtConcurrent/QtConcurrentRun>
#include <QFuture>
```

11.17 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/↵ Controller/TTTController.h File Reference

```
#include "AIAgent.h"
#include "Board.h"
#include "TTTCommonTypes.h"
#include "TicTacToeGame.h"
#include "NetworkManager.h"
#include <QTcpSocket>
#include <QObject>
#include <memory>
#include <vector>
#include <portaudio.h>
#include <pocketsphinx.h>
#include <signal.h>
```

Classes

- class [TTTController](#)

The [TTTController](#) class is the controller for a Tic Tac Toe game, it takes in game options as input, and controls the view ([TicTacToeGame](#)) and the model ([Board](#)) according to the game logic specified in this class, it is also responsible for calling the AI agent to play its turn if specified by the game options.

11.18 TTTController.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TTTCONTROLLER_H
00002 #define TTTCONTROLLER_H
00003 #include "AIAgent.h"
00004 #include "Board.h"
00005 #include "TTTCommonTypes.h"
00006 #include "TicTacToeGame.h"
00007 #include "NetworkManager.h"
00008 #include <QTcpSocket>
00009 #include <QObject>
00010 #include <memory>
00011 #include <vector>
00012 #include <portaudio.h>
00013 #include <pocketsphinx.h>
00014 #include <signal.h>
00015
00016 using std::unique_ptr;
```



```

00017 using std::vector;
00018
00026
00027 class TTController : public QObject
00028 {
00029     Q_OBJECT
00030 protected:
00034     TicTacToeGame view_;
00038     Board board_;
00042     vector<Cell> cells_;
00046     const TTOptions& options_;
00050     BoardMarks currentPlayer_;
00054     unique_ptr<AIAgent> agent_;
00055
00056 private: // Methods
00061     void setConnections();
00066     void reset();
00067
00068     QTcpSocket *socket_;
00069     void setupNetwork();
00070     void sendMove(int row, int col, char player);
00071
00072 protected: // Methods
00077     virtual void updateGameState(Cell &cell);
00081     virtual void AIAgentPlay();
00085     virtual void HumanPlay();
00089     virtual void switchPlayer();
00090
00091 public:
00097     explicit TTController(const TTOptions &options, QObject *parent = nullptr);
00101     virtual void startGame();
00102     int parseSpeechToCell(const QString &text)
00103     {
00104         QStringList keywords = text.split(' ', Qt::SkipEmptyParts);
00105         if (keywords.size() < 2) return defaults::INVALID_CELL;
00106
00107         QString k1 = keywords[0];
00108         QString k2 = keywords[1];
00109
00110         if (k1 == "arriba") {
00111             if (k2 == "izquierda") return 0;
00112             if (k2 == "arriba") return 1;
00113             if (k2 == "derecha") return 2;
00114         } else if (k1 == "centro") {
00115             if (k2 == "izquierda") return 3;
00116             if (k2 == "centro") return 4;
00117             if (k2 == "derecha") return 5;
00118             if (k2 == "arriba") return 1;
00119             if (k2 == "abajo") return 7;
00120         } else if (k1 == "abajo") {
00121             if (k2 == "izquierda") return 6;
00122             if (k2 == "centro") return 7;
00123             if (k2 == "derecha") return 8;
00124         }
00125         return defaults::INVALID_CELL;
00126     }
00127     QWidget* getView() { return &view_; }
00128 signals:
00133     void AIFinished();
00134
00135     void humanFinished();
00136     void speechRecognized(QString);
00137
00138
00139 public slots:
00144     virtual void updateGame(Cell &cell);
00145 private slots:
00146     void onSpeechRecognized(const QString &text);
00147     void onSocketReadyRead();
00148 };
00149
00150 #endif // TTCONTROLLER_H

```

11.19 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/config.h File Reference

Macros

- #define `DEFAULT_RADIX` 12

- `#define HAVE_LONG_LONG`
- `#define HAVE_POPEN`
- `#define HAVE_SNPRINTF`
- `#define HAVE_SYS_STAT_H`
- `#define HAVE_SYS_TYPES_H`
- `#define HAVE_UNISTD_H`
- `#define HAVE_STDINT_H`
- `#define SIZEOF_LONG 8`
- `#define SIZEOF_LONG_LONG 8`
- `#define PACKAGE_BUGREPORT "dhdaines@gmail.com"`
- `#define PACKAGE_NAME "PocketSphinx"`
- `#define PACKAGE_STRING "PocketSphinx 5.0.4"`
- `#define PACKAGE_TARNAME "pocketsphinx-5.0.4"`
- `#define PACKAGE_URL "https://github.com/cmusphinx/pocketsphinx"`
- `#define PACKAGE_VERSION "5.0.4"`

11.19.1 Macro Definition Documentation

11.19.1.1 DEFAULT_RADIX

```
#define DEFAULT_RADIX 12
```

11.19.1.2 HAVE_LONG_LONG

```
#define HAVE_LONG_LONG
```

11.19.1.3 HAVE_POPEN

```
#define HAVE_POPEN
```

11.19.1.4 HAVE_SNPRINTF

```
#define HAVE_SNPRINTF
```

11.19.1.5 HAVE_STDINT_H

```
#define HAVE_STDINT_H
```

11.19.1.6 HAVE_SYS_STAT_H

```
#define HAVE_SYS_STAT_H
```

11.19.1.7 HAVE_SYS_TYPES_H

```
#define HAVE_SYS_TYPES_H
```

11.19.1.8 HAVE_UNISTD_H

```
#define HAVE_UNISTD_H
```

11.19.1.9 PACKAGE_BUGREPORT

```
#define PACKAGE_BUGREPORT "dhdaines@gmail.com"
```

11.19.1.10 PACKAGE_NAME

```
#define PACKAGE_NAME "PocketSphinx"
```

11.19.1.11 PACKAGE_STRING

```
#define PACKAGE_STRING "PocketSphinx 5.0.4"
```

11.19.1.12 PACKAGE_TARNAME

```
#define PACKAGE_TARNAME "pocketsphinx-5.0.4"
```

11.19.1.13 PACKAGE_URL

```
#define PACKAGE_URL "https://github.com/cmusphinx/pocketsphinx"
```

11.19.1.14 PACKAGE_VERSION

```
#define PACKAGE_VERSION "5.0.4"
```

11.19.1.15 SIZEOF_LONG

```
#define SIZEOF_LONG 8
```

11.19.1.16 SIZEOF_LONG_LONG

```
#define SIZEOF_LONG_LONG 8
```

11.20 config.h

[Go to the documentation of this file.](#)

```

00001 /* include/config.h.in.  Generated from configure.ac by autoheader.  */
00002
00003 /* Default radix point for fixed-point */
00004 #define DEFAULT_RADIX 12
00005
00006 /* Define to use fixed-point computation */
00007 /* #undef FIXED_POINT */
00008
00009 /* Define to use thread-local storage for random number generator */
00010 /* #undef PS_USE_THREAD_LOCAL_RNG */
00011
00012 /* Define if the system has the type `long long'. */
00013 #define HAVE_LONG_LONG
00014
00015 /* Define if you have the `popen' function. */
00016 #define HAVE_POPEN
00017
00018 /* Define if you have the `snprintf' function. */
00019 #define HAVE_SNPRINTF
00020
00021 /* Define if you have the <sys/stat.h> header file. */
00022 #define HAVE_SYS_STAT_H
00023
00024 /* Define if you have the <sys/types.h> header file. */
00025 #define HAVE_SYS_TYPES_H
00026
00027 /* Define if you have the <unistd.h> header file. */
00028 #define HAVE_UNISTD_H
00029
00030 /* Define if you have the <inttypes.h> header file. */
00031 /* #undef HAVE_INTTYPES_H */
00032
00033 /* Define if you have the <stdint.h> header file. */
00034 #define HAVE_STDINT_H
00035
00036 /* The size of `long', as computed by sizeof. */
00037 #define SIZEOF_LONG 8
00038
00039 /* The size of `long long', as computed by sizeof. */
00040 #define SIZEOF_LONG_LONG 8
00041
00042 /* Define WORDS_BIGENDIAN if your processor stores words with the most
00043    significant byte first (like Motorola and SPARC, unlike Intel). */
00044 #if defined __BIG_ENDIAN__
00045 #   define WORDS_BIGENDIAN
00046 #else
00047 /* #undef WORDS_BIGENDIAN */
00048 #endif
00049
00050 /* Define to the address where bug reports for this package should be sent. */
00051 #define PACKAGE_BUGREPORT "dhdaines@gmail.com"
00052
00053 /* Define to the full name of this package. */
00054 #define PACKAGE_NAME "PocketSphinx"
00055
00056 /* Define to the full name and version of this package. */
00057 #define PACKAGE_STRING "PocketSphinx 5.0.4"
00058
00059 /* Define to the one symbol short name of this package. */
00060 #define PACKAGE_TARNAME "pocketsphinx-5.0.4"
00061
00062 /* Define to the home page for this package. */
00063 #define PACKAGE_URL "https://github.com/cmusphinx/pocketsphinx"
00064
00065 /* Define to the version of this package. */
00066 #define PACKAGE_VERSION "5.0.4"
00067

```

11.21 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/include/pocketsphinx/sphinx_config.h File Reference

Macros

- #define [DEFAULT_RADIX](#) 12

- `#define SIZEOF_LONG 8`
- `#define HAVE_LONG_LONG`
- `#define SIZEOF_LONG_LONG 8`

11.21.1 Macro Definition Documentation

11.21.1.1 DEFAULT_RADIX

```
#define DEFAULT_RADIX 12
```

11.21.1.2 HAVE_LONG_LONG

```
#define HAVE_LONG_LONG
```

11.21.1.3 SIZEOF_LONG

```
#define SIZEOF_LONG 8
```

11.21.1.4 SIZEOF_LONG_LONG

```
#define SIZEOF_LONG_LONG 8
```

11.22 sphinx_config.h

[Go to the documentation of this file.](#)

```
00001 /* sphinx_config.h: Externally visible configuration parameters */
00002
00003 /* Define to use fixed-point computation */
00004 /* #undef FIXED_POINT */
00005
00006 /* Default radix point for fixed-point */
00007 #define DEFAULT_RADIX 12
00008
00009 /* Define if you have the <stdint.h> header file. */
00010 #ifndef HAVE_STDINT_H
00011 #define HAVE_STDINT_H
00012 #endif
00013
00014 /* The size of `long', as computed by sizeof. */
00015 #define SIZEOF_LONG 8
00016
00017 /* Define if the system has the type `long long'. */
00018 #ifndef HAVE_LONG_LONG /* sometimes it is already defined */
00019 #define HAVE_LONG_LONG
00020 #endif
00021
00022 /* The size of `long long', as computed by sizeof. */
00023 #define SIZEOF_LONG_LONG 8
00024
00025 /* Enable debugging output */
00026 /* #undef SPHINX_DEBUG */
```

11.23 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/sphinx_config.h File Reference

Macros

- #define [DEFAULT_RADIX](#) 12
- #define [SIZEOF_LONG](#) 8
- #define [HAVE_LONG_LONG](#)
- #define [SIZEOF_LONG_LONG](#) 8

11.23.1 Macro Definition Documentation

11.23.1.1 DEFAULT_RADIX

```
#define DEFAULT_RADIX 12
```

11.23.1.2 HAVE_LONG_LONG

```
#define HAVE_LONG_LONG
```

11.23.1.3 SIZEOF_LONG

```
#define SIZEOF_LONG 8
```

11.23.1.4 SIZEOF_LONG_LONG

```
#define SIZEOF_LONG_LONG 8
```

11.24 sphinx_config.h

[Go to the documentation of this file.](#)

```
00001 /* sphinx_config.h: Externally visible configuration parameters */
00002
00003 /* Define to use fixed-point computation */
00004 /* #undef FIXED_POINT */
00005
00006 /* Default radix point for fixed-point */
00007 #define DEFAULT_RADIX 12
00008
00009 /* Define if you have the <stdint.h> header file. */
00010 #ifndef HAVE_STDINT_H
00011 #define HAVE_STDINT_H
00012 #endif
00013
00014 /* The size of `long', as computed by sizeof. */
00015 #define SIZEOF_LONG 8
00016
00017 /* Define if the system has the type `long long'. */
00018 #ifndef HAVE_LONG_LONG /* sometimes it is already defined */
00019 #define HAVE_LONG_LONG
00020 #endif
00021
00022 /* The size of `long long', as computed by sizeof. */
00023 #define SIZEOF_LONG_LONG 8
00024
00025 /* Enable debugging output */
00026 /* #undef SPHINX_DEBUG */
```

11.25 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/examples/test/unit/test_macros.h File Reference

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <math.h>
#include <pocketsphinx/logmath.h>
```

Macros

- `#define EPSILON 0.01`
- `#define TEST_ASSERT(x)`
- `#define TEST_EQUAL(a, b)`
- `#define TEST_EQUAL_FLOAT(a, b)`
- `#define TEST_EQUAL_MFCC(a, b)`
- `#define TEST_EQUAL_STRING(a, b)`
- `#define LOG_EPSILON 200`
- `#define TEST_EQUAL_LOG(a, b)`
- `#define MODELDIR "/home/ratty/Downloads/pocketsphinx-main/model"`
- `#define DATADIR "/home/ratty/Downloads/pocketsphinx-main/test/data"`
- `#define TESTDATADIR "/home/ratty/Downloads/pocketsphinx-main/test/regression"`

11.25.1 Macro Definition Documentation

11.25.1.1 DATADIR

```
#define DATADIR "/home/ratty/Downloads/pocketsphinx-main/test/data"
```

11.25.1.2 EPSILON

```
#define EPSILON 0.01
```

11.25.1.3 LOG_EPSILON

```
#define LOG_EPSILON 200
```

11.25.1.4 MODELDIR

```
#define MODELDIR "/home/ratty/Downloads/pocketsphinx-main/model"
```

11.25.1.5 TEST_ASSERT

```
#define TEST_ASSERT(  
    x)
```

Value:

```
if (!x) { fprintf(stderr, "FAIL: %s\n", #x); exit(1); }
```

11.25.1.6 TEST_EQUAL

```
#define TEST_EQUAL(  
    a,  
    b)
```

Value:

```
TEST_ASSERT((a) == (b))
```

11.25.1.7 TEST_EQUAL_FLOAT

```
#define TEST_EQUAL_FLOAT(  
    a,  
    b)
```

Value:

```
TEST_ASSERT(fabs((a) - (b)) < EPSILON)
```

11.25.1.8 TEST_EQUAL_LOG

```
#define TEST_EQUAL_LOG(  
    a,  
    b)
```

Value:

```
TEST_ASSERT(abs((a) - (b)) < LOG_EPSILON)
```

11.25.1.9 TEST_EQUAL_MFCC

```
#define TEST_EQUAL_MFCC(  
    a,  
    b)
```

Value:

```
TEST_ASSERT(fabs((a) - (b)) < EPSILON)
```

11.25.1.10 TEST_EQUAL_STRING

```
#define TEST_EQUAL_STRING(  
    a,  
    b)
```

Value:

```
TEST_ASSERT(0 == strcmp((a), (b)))
```


11.25.1.11 TESTDATADIR

```
#define TESTDATADIR "/home/ratty/Downloads/pocketsphinx-main/test/regression"
```

11.26 test_macros.h

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <string.h>
00003 #include <stdlib.h>
00004 #include <math.h>
00005
00006 #include <pocketsphinx/logmath.h>
00007
00008 #define EPSILON 0.01
00009 #define TEST_ASSERT(x) if (!(x)) { fprintf(stderr, "FAIL: %s\n", #x); exit(1); }
00010 #define TEST_EQUAL(a,b) TEST_ASSERT((a) == (b))
00011 #define TEST_EQUAL_FLOAT(a,b) TEST_ASSERT(fabs((a) - (b)) < EPSILON)
00012 #ifndef FIXED_POINT
00013 #define TEST_EQUAL_MFCC(a,b) TEST_EQUAL(a,b)
00014 #else
00015 #define TEST_EQUAL_MFCC(a,b) TEST_ASSERT(fabs((a) - (b)) < EPSILON)
00016 #endif
00017 #define TEST_EQUAL_STRING(a,b) TEST_ASSERT(0 == strcmp((a), (b)))
00018 #define LOG_EPSILON 200
00019 #define TEST_EQUAL_LOG(a,b) TEST_ASSERT(abs((a) - (b)) < LOG_EPSILON)
00020 #define MODELDIR "/home/ratty/Downloads/pocketsphinx-main/model"
00021 #define DATADIR "/home/ratty/Downloads/pocketsphinx-main/test/data"
00022 #define TESTDATADIR "/home/ratty/Downloads/pocketsphinx-main/test/regression"
```

11.27 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx.h File Reference

```
#include <stdio.h>
#include <pocketsphinx/sphinx_config.h>
#include <pocketsphinx/prim_type.h>
#include <pocketsphinx/logmath.h>
#include <pocketsphinx/err.h>
#include <pocketsphinx/vad.h>
#include <pocketsphinx/endpointer.h>
#include <pocketsphinx/model.h>
#include <pocketsphinx/search.h>
#include <pocketsphinx/export.h>
#include <pocketsphinx/lattice.h>
#include <pocketsphinx/alignment.h>
#include <pocketsphinx/mlr.h>
```

Classes

- struct [ps_arg_s](#)

Typedefs

- typedef enum [ps_type_e](#) [ps_type_t](#)
Types of configuration parameters.
- typedef struct [ps_arg_s](#) [ps_arg_t](#)
- typedef struct cmd_ln_s [ps_config_t](#)
- typedef struct ps_decoder_s [ps_decoder_t](#)
- typedef struct ps_astar_s [ps_nbest_t](#)
- typedef struct ps_seg_s [ps_seg_t](#)

Enumerations

- enum [ps_type_e](#) {
[ARG_REQUIRED](#) = (1<<0) , [ARG_INTEGER](#) = (1<<1) , [ARG_FLOATING](#) = (1<<2) , [ARG_STRING](#) = (1<<3) ,
[ARG_BOOLEAN](#) = (1<<4) , [REQARG_INTEGER](#) = (ARG_INTEGER | ARG_REQUIRED) , [REQARG_FLOATING](#) = (ARG_FLOATING | ARG_REQUIRED) , [REQARG_STRING](#) = (ARG_STRING | ARG_REQUIRED) ,
[REQARG_BOOLEAN](#) = (ARG_BOOLEAN | ARG_REQUIRED) }

Types of configuration parameters.

Functions

- [POCKETSPHINX_EXPORT ps_arg_t](#) const * [ps_args](#) (void)

11.27.1 Detailed Description

Main header file for the PocketSphinx decoder.

This is the only header file you should need to include in order to write code using PocketSphinx. The documentation for its various functions and structures is actually located on the pages for those structures, and because Doxygen does not seem smart enough to put links in the "Typedefs" list above, here they are for your convenience:

- [ps_config_t](#)
- [ps_arg_t](#)
- [ps_decoder_t](#)
- [ps_nbest_t](#)
- [ps_seg_t](#)

There are also a few other structures you should be aware of, which can be useful in writing speech applications:

- [ps_endpoint_t](#)
- [ps_vad_t](#)
- [jsgf_t](#)
- [ngram_model_t](#)

Finally, to learn about switching language models and grammars, see [<pocketsphinx/search.h>](#)

11.27.2 Typedef Documentation

11.27.2.1 [ps_arg_t](#)

```
typedef struct ps\_arg\_s ps\_arg\_t
```

11.27.2.2 ps_config_t

```
typedef struct cmd_ln_s ps_config_t
```

11.27.2.3 ps_decoder_t

```
typedef struct ps_decoder_s ps_decoder_t
```

11.27.2.4 ps_nbest_t

```
typedef struct ps_astar_s ps_nbest_t
```

11.27.2.5 ps_seg_t

```
typedef struct ps_seg_s ps_seg_t
```

11.27.2.6 ps_type_t

```
typedef enum ps_type_e ps_type_t
```

Types of configuration parameters.

11.27.3 Enumeration Type Documentation

11.27.3.1 ps_type_e

```
enum ps_type_e
```

Types of configuration parameters.

Enumerator

ARG_REQUIRED	
ARG_INTEGER	
ARG_FLOATING	
ARG_STRING	
ARG_BOOLEAN	
REQARG_INTEGER	
REQARG_FLOATING	
REQARG_STRING	
REQARG_BOOLEAN	

11.27.4 Function Documentation

11.27.4.1 `ps_args()`

```
POCKETSPHINX_EXPORT ps_arg_t const * ps_args (
    void )
```

Returns the argument definitions used in `ps_config_init()`.

This is here to avoid exporting global data, which is problematic on Win32 and Symbian (and possibly other platforms).

[ps_config_t](#)

References [POCKETSPHINX_EXPORT](#).

11.28 `pocketsphinx.h`

[Go to the documentation of this file.](#)

```
00001 /* -*- c-basic-offset:4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 1999-2008 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  *
00019  * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS" AND
00020  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00021  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00022  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00023  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00024  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00025  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00026  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00027  * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00028  * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00029  * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00030  *
00031  * =====
00032  */
00033
00061
00062 #ifndef __POCKETSPHINX_H__
00063 #define __POCKETSPHINX_H__
00064
00065 /* System headers */
00066 #include <stdio.h>
00067
00068 /* PocketSphinx utility headers */
00069 #include <pocketsphinx/sphinx_config.h>
00070 #include <pocketsphinx/prim_type.h>
00071 #include <pocketsphinx/logmath.h>
00072 #include <pocketsphinx/err.h>
00073
00074 /* PocketSphinx API headers */
00075 #include <pocketsphinx/vad.h>
00076 #include <pocketsphinx/endpointer.h>
00077 #include <pocketsphinx/model.h>
00078 #include <pocketsphinx/search.h>
00079 #include <pocketsphinx/export.h>
00080 #include <pocketsphinx/lattice.h>
```

```

00081 #include <pocketsphinx/alignment.h>
00082 #include <pocketsphinx/mlr.h>
00083
00084 /* Namum manglium ii domum */
00085 #ifdef __cplusplus
00086 extern "C" {
00087 #endif
00088 #if 0
00089 }
00090 #endif
00091
00092 /* Transparent structures */
00093 typedef enum ps_type_e {
00094     ARG_REQUIRED = (1<<0), /*« Bit indicating required argument. */
00095     ARG_INTEGER = (1<<1), /*« Integer up to 64 bits. */
00096     ARG_FLOATING = (1<<2), /*« Double-precision floating point. */
00097     ARG_STRING = (1<<3), /*« String. */
00098     ARG_BOOLEAN = (1<<4), /*« Boolean (true/false). */
00099     REQARG_INTEGER = (ARG_INTEGER | ARG_REQUIRED),
00100     REQARG_FLOATING = (ARG_FLOATING | ARG_REQUIRED),
00101     REQARG_STRING = (ARG_STRING | ARG_REQUIRED),
00102     REQARG_BOOLEAN = (ARG_BOOLEAN | ARG_REQUIRED)
00103 } ps_type_t;
00104
00105 typedef struct ps_arg_s {
00106     char const *name;
00107     int type;
00108     char const *deflt;
00109     char const *doc;
00110 } ps_arg_t;
00111
00112 /* Opaque structures */
00113
00114 typedef struct cmd_ln_s ps_config_t;
00115
00116 typedef struct ps_decoder_s ps_decoder_t;
00117
00118 typedef struct ps_astar_s ps_nbest_t;
00119
00120 typedef struct ps_seg_s ps_seg_t;
00121
00122 POCKETSPHINX_EXPORT
00123 ps_config_t *ps_config_init(const ps_arg_t *defn);
00124
00125 POCKETSPHINX_EXPORT
00126 ps_config_t *ps_config_retain(ps_config_t *config);
00127
00128 POCKETSPHINX_EXPORT
00129 int ps_config_free(ps_config_t *config);
00130
00131 int ps_config_validate(ps_config_t *config);
00132
00133 POCKETSPHINX_EXPORT
00134 ps_config_t *ps_config_parse_json(ps_config_t *config, const char *json);
00135
00136 POCKETSPHINX_EXPORT
00137 const char *ps_config_serialize_json(ps_config_t *config);
00138
00139 POCKETSPHINX_EXPORT
00140 ps_type_t ps_config_typeof(ps_config_t *config, char const *name);
00141
00142 POCKETSPHINX_EXPORT
00143 const anytype_t *ps_config_get(ps_config_t *config, const char *name);
00144
00145 POCKETSPHINX_EXPORT
00146 const anytype_t *ps_config_set(ps_config_t *config, const char *name,
00147                                const anytype_t *val, ps_type_t t);
00148
00149 POCKETSPHINX_EXPORT
00150 long ps_config_int(ps_config_t *config, const char *name);
00151
00152 POCKETSPHINX_EXPORT
00153 int ps_config_bool(ps_config_t *config, const char *name);
00154
00155 POCKETSPHINX_EXPORT
00156 double ps_config_float(ps_config_t *config, const char *name);
00157
00158 POCKETSPHINX_EXPORT
00159 const char *ps_config_str(ps_config_t *config, const char *name);
00160
00161 POCKETSPHINX_EXPORT
00162 const anytype_t *ps_config_set_int(ps_config_t *config, const char *name, long val);
00163
00164 POCKETSPHINX_EXPORT
00165 const anytype_t *ps_config_set_bool(ps_config_t *config, const char *name, int val);
00166
00167 POCKETSPHINX_EXPORT

```

```
00370 const anytype_t *ps_config_set_float(ps_config_t *config, const char *name, double val);
00371
00385 POCKETSPHINX_EXPORT
00386 const anytype_t *ps_config_set_str(ps_config_t *config, const char *name, const char *val);
00387
00410 POCKETSPHINX_EXPORT
00411 int ps_config_soundfile(ps_config_t *config, FILE *fh, const char *file);
00412
00426 POCKETSPHINX_EXPORT
00427 int ps_config_wavfile(ps_config_t *config, FILE *infh, const char *file);
00428
00442 POCKETSPHINX_EXPORT
00443 int ps_config_nistfile(ps_config_t *config, FILE *infh, const char *file);
00444
00464 POCKETSPHINX_EXPORT
00465 void ps_default_search_args(ps_config_t *config);
00466
00472 POCKETSPHINX_EXPORT
00473 void ps_expand_model_config(ps_config_t *config);
00474
00483 POCKETSPHINX_EXPORT
00484 const char *ps_default_modeldir(void);
00485
00497 POCKETSPHINX_EXPORT
00498 ps_decoder_t *ps_init(ps_config_t *config);
00499
00523 POCKETSPHINX_EXPORT
00524 int ps_reinit(ps_decoder_t *ps, ps_config_t *config);
00525
00547 POCKETSPHINX_EXPORT
00548 int ps_reinit_feat(ps_decoder_t *ps, ps_config_t *config);
00549
00569 POCKETSPHINX_EXPORT
00570 const char *ps_get_cmfn(ps_decoder_t *ps, int update);
00571
00587 POCKETSPHINX_EXPORT
00588 int ps_set_cmfn(ps_decoder_t *ps, const char *cmfn);
00589
00598 POCKETSPHINX_EXPORT
00599 ps_arg_t const *ps_args(void);
00600
00612 POCKETSPHINX_EXPORT
00613 ps_decoder_t *ps_retain(ps_decoder_t *ps);
00614
00624 POCKETSPHINX_EXPORT
00625 int ps_free(ps_decoder_t *ps);
00626
00636 POCKETSPHINX_EXPORT
00637 ps_config_t *ps_get_config(ps_decoder_t *ps);
00638
00648 POCKETSPHINX_EXPORT
00649 logmath_t *ps_get_logmath(ps_decoder_t *ps);
00650
00662 POCKETSPHINX_EXPORT
00663 ps_mllr_t *ps_update_mllr(ps_decoder_t *ps, ps_mllr_t *mllr);
00664
00680 POCKETSPHINX_EXPORT
00681 int ps_load_dict(ps_decoder_t *ps, char const *dictfile,
00682                char const *fdictfile, char const *format);
00683
00694 POCKETSPHINX_EXPORT
00695 int ps_save_dict(ps_decoder_t *ps, char const *dictfile, char const *format);
00696
00717 POCKETSPHINX_EXPORT
00718 int ps_add_word(ps_decoder_t *ps,
00719                char const *word,
00720                char const *phones,
00721                int update);
00722
00735 POCKETSPHINX_EXPORT
00736 char *ps_lookup_word(ps_decoder_t *ps,
00737                     const char *word);
00738
00754 POCKETSPHINX_EXPORT
00755 long ps_decode_raw(ps_decoder_t *ps, FILE *rawfh,
00756                  long maxsamps);
00757
00766 POCKETSPHINX_EXPORT
00767 int ps_decode_senscr(ps_decoder_t *ps, FILE *senfh);
00768
00779 POCKETSPHINX_EXPORT
00780 int ps_start_stream(ps_decoder_t *ps);
00781
00795 POCKETSPHINX_EXPORT
00796 int ps_get_in_speech(ps_decoder_t *ps);
00797
00809 POCKETSPHINX_EXPORT
```

```

00810 int ps_start_utt(ps_decoder_t *ps);
00811
00828 POCKETSPHINX_EXPORT
00829 int ps_process_raw(ps_decoder_t *ps,
00830                   int16 const *data,
00831                   size_t n_samples,
00832                   int no_search,
00833                   int full_utt);
00834
00855 POCKETSPHINX_EXPORT
00856 int ps_process_cep(ps_decoder_t *ps,
00857                   float32 **data,
00858                   int n_frames,
00859                   int no_search,
00860                   int full_utt);
00861
00876 POCKETSPHINX_EXPORT
00877 int ps_get_n_frames(ps_decoder_t *ps);
00878
00886 POCKETSPHINX_EXPORT
00887 int ps_end_utt(ps_decoder_t *ps);
00888
00900 POCKETSPHINX_EXPORT
00901 char const *ps_get_hyp(ps_decoder_t *ps, int32 *out_best_score);
00902
00917 POCKETSPHINX_EXPORT
00918 int32 ps_get_prob(ps_decoder_t *ps);
00919
00931 POCKETSPHINX_EXPORT
00932 ps_lattice_t *ps_get_lattice(ps_decoder_t *ps);
00933
00942 POCKETSPHINX_EXPORT
00943 ps_seg_t *ps_seg_iter(ps_decoder_t *ps);
00944
00953 POCKETSPHINX_EXPORT
00954 ps_seg_t *ps_seg_next(ps_seg_t *seg);
00955
00964 POCKETSPHINX_EXPORT
00965 char const *ps_seg_word(ps_seg_t *seg);
00966
00979 POCKETSPHINX_EXPORT
00980 void ps_seg_frames(ps_seg_t *seg, int *out_sf, int *out_ef);
00981
01005 POCKETSPHINX_EXPORT
01006 int32 ps_seg_prob(ps_seg_t *seg, int32 *out_ascr, int32 *out_lscr, int32 *out_lback);
01007
01012 POCKETSPHINX_EXPORT
01013 void ps_seg_free(ps_seg_t *seg);
01014
01024 POCKETSPHINX_EXPORT
01025 ps_nbest_t *ps_nbest(ps_decoder_t *ps);
01026
01035 POCKETSPHINX_EXPORT
01036 ps_nbest_t *ps_nbest_next(ps_nbest_t *nbest);
01037
01047 POCKETSPHINX_EXPORT
01048 char const *ps_nbest_hyp(ps_nbest_t *nbest, int32 *out_score);
01049
01057 POCKETSPHINX_EXPORT
01058 ps_seg_t *ps_nbest_seg(ps_nbest_t *nbest);
01059
01066 POCKETSPHINX_EXPORT
01067 void ps_nbest_free(ps_nbest_t *nbest);
01068
01078 POCKETSPHINX_EXPORT
01079 void ps_get_utt_time(ps_decoder_t *ps, double *out_nspeech,
01080                    double *out_ncpu, double *out_nwall);
01081
01091 POCKETSPHINX_EXPORT
01092 void ps_get_all_time(ps_decoder_t *ps, double *out_nspeech,
01093                    double *out_ncpu, double *out_nwall);
01094
01320
01321 #ifdef __cplusplus
01322 } /* extern "C" */
01323 #endif
01324
01325 #endif /* __POCKETSPHINX_H__ */

```

11.29 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/alignment.h File Reference

Multi-level alignment structure.

```
#include <pocketsphinx/prim_type.h>
```

Macros

- `#define PS_ALIGNMENT_NONE -1`

Typedefs

- `typedef struct ps_alignment_s ps\_alignment\_t`
- `typedef struct ps_alignment_iter_s ps\_alignment\_iter\_t`

11.29.1 Detailed Description

Multi-level alignment structure.

Because doxygen is Bad Software, the actual documentation can only exist in [ps_alignment_t](#). Sorry about that.

11.29.2 Macro Definition Documentation

11.29.2.1 PS_ALIGNMENT_NONE

```
#define PS_ALIGNMENT_NONE -1
```

Value indicating no parent or child for an entry. [ps_alignment_t](#)

11.29.3 Typedef Documentation

11.29.3.1 ps_alignment_iter_t

```
typedef struct ps_alignment_iter_s ps_alignment_iter_t
```

11.29.3.2 ps_alignment_t

```
typedef struct ps_alignment_s ps_alignment_t
```


11.30 alignment.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset: 4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2010 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  * This work was supported in part by funding from the Defense Advanced
00019  * Research Projects Agency and the National Science Foundation of the
00020  * United States of America, and the CMU Sphinx Speech Consortium.
00021  *
00022  * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS'' AND
00023  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00024  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00025  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00026  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00027  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00028  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00029  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00030  * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00031  * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00032  * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00033  *
00034  * =====
00035  */
00036 */
00037
00045
00046 #ifndef __PS_ALIGNMENT_H__
00047 #define __PS_ALIGNMENT_H__
00048
00049 #include <pocketsphinx/prim_type.h>
00050
00051 #ifdef __cplusplus
00052 extern "C" {
00053 #endif
00054 #if 0
00055 }
00056 #endif
00057
00062 #define PS_ALIGNMENT_NONE -1
00063
00068 typedef struct ps_alignment_s ps_alignment_t;
00069
00074 typedef struct ps_alignment_iter_s ps_alignment_iter_t;
00075
00080 POCKETSPHINX_EXPORT
00081 ps_alignment_t *ps_alignment_retain(ps_alignment_t *al);
00082
00087 POCKETSPHINX_EXPORT
00088 int ps_alignment_free(ps_alignment_t *al);
00089
00094 POCKETSPHINX_EXPORT
00095 ps_alignment_iter_t *ps_alignment_words(ps_alignment_t *al);
00096
00101 POCKETSPHINX_EXPORT
00102 ps_alignment_iter_t *ps_alignment_phones(ps_alignment_t *al);
00103
00108 POCKETSPHINX_EXPORT
00109 ps_alignment_iter_t *ps_alignment_states(ps_alignment_t *al);
00110
00119 POCKETSPHINX_EXPORT
00120 const char *ps_alignment_iter_name(ps_alignment_iter_t *itor);
00121
00130 POCKETSPHINX_EXPORT
00131 int ps_alignment_iter_seg(ps_alignment_iter_t *itor, int *start, int *duration);
00132
00141 POCKETSPHINX_EXPORT
00142 ps_alignment_iter_t *ps_alignment_iter_next(ps_alignment_iter_t *itor);
00143
00151 POCKETSPHINX_EXPORT
00152 ps_alignment_iter_t *ps_alignment_iter_children(ps_alignment_iter_t *itor);

```

```

00153
00159 POCKETSPHINX_EXPORT
00160 int ps_alignment_iter_free(ps_alignment_iter_t *itor);
00161
00162 #ifdef __cplusplus
00163 } /* extern "C" */
00164 #endif
00165
00166 #endif /* __PS_ALIGNMENT_H__ */

```

11.31 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/endpointer.h File Reference

VAD-based endpointer for PocketSphinx.

```

#include <pocketsphinx/prim_type.h>
#include <pocketsphinx/export.h>
#include <pocketsphinx/vad.h>

```

Macros

- #define [PS_ENDPOINTER_DEFAULT_WINDOW](#) 0.3
- #define [PS_ENDPOINTER_DEFAULT_RATIO](#) 0.9
- #define [ps_endpoint_frame_size](#)(ep)
- #define [ps_endpoint_frame_length](#)(ep)
- #define [ps_endpoint_sample_rate](#)(ep)

Typedefs

- typedef struct ps_endpointer_s [ps_endpointer_t](#)
- typedef double(* [ps_endpointer_timestamp_cb_t](#)) (void *user_data)

11.31.1 Detailed Description

VAD-based endpointer for PocketSphinx.

Because doxygen is Bad Software, the actual documentation can only exist in [ps_endpointer_t](#). Sorry about that.

11.31.2 Macro Definition Documentation

11.31.2.1 PS_ENDPOINTER_DEFAULT_RATIO

```
#define PS_ENDPOINTER_DEFAULT_RATIO 0.9
```

Default ratio of frames in window to trigger start/end decision.

11.31.2.2 PS_ENDPOINTER_DEFAULT_WINDOW

```
#define PS_ENDPOINTER_DEFAULT_WINDOW 0.3
```

Default window in seconds of audio to use for speech start/end decision.

11.31.2.3 ps_endpointter_frame_length

```
#define ps_endpointter_frame_length(  
    ep)
```

Value:

```
ps_vad_frame_length(ps_endpointter_vad(ep))
```

Get the frame length (in seconds) consumed by the endpointter.

11.31.2.4 ps_endpointter_frame_size

```
#define ps_endpointter_frame_size(  
    ep)
```

Value:

```
ps_vad_frame_size(ps_endpointter_vad(ep))
```

Get the frame size (in samples) consumed by the endpointter.

Multiply this by 2 to get the size of the frame buffer required.

Referenced by [SpeechManager::init\(\)](#), and [SpeechToText::init\(\)](#).

11.31.2.5 ps_endpointter_sample_rate

```
#define ps_endpointter_sample_rate(  
    ep)
```

Value:

```
ps_vad_sample_rate(ps_endpointter_vad(ep))
```

Get the sample rate required by the endpointter.

11.31.3 Typedef Documentation

11.31.3.1 ps_endpointter_t

```
typedef struct ps_endpointter_s ps_endpointter_t
```

11.31.3.2 ps_endpointter_timestamp_cb_t

```
typedef double(* ps_endpointter_timestamp_cb_t) (void *user_data)
```

Callback function type for external timestamp sources.

This callback allows applications to provide their own timestamp source (e.g., system time, monotonic clock) to avoid timestamp drift between audio clock and system clock.

Parameters

<i>user_data</i>	User-provided data passed to <code>ps_endpoint_set_timestamp_func()</code>
------------------	--

Returns

Current timestamp in seconds

11.32 endpointer.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset:4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2022 David Huggins-Daines. All rights reserved.
00004  *
00005  * Redistribution and use in source and binary forms, with or without
00006  * modification, are permitted provided that the following conditions
00007  * are met:
00008  *
00009  * 1. Redistributions of source code must retain the above copyright
00010  * notice, this list of conditions and the following disclaimer.
00011  *
00012  * 2. Redistributions in binary form must reproduce the above copyright
00013  * notice, this list of conditions and the following disclaimer in
00014  * the documentation and/or other materials provided with the
00015  * distribution.
00016  *
00017  * THIS SOFTWARE IS PROVIDED ``AS IS'' AND ANY EXPRESSED OR IMPLIED
00018  * WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
00019  * OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
00020  * DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT,
00021  * INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
00022  * (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
00023  * SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
00024  * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
00025  * STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
00026  * ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
00027  * OF THE POSSIBILITY OF SUCH DAMAGE.
00028  * =====
00029  */
00030
00031 #ifndef __PS_ENDPOINTER_H__
00032 #define __PS_ENDPOINTER_H__
00033
00034 #include <pocketsphinx/prim_type.h>
00035 #include <pocketsphinx/export.h>
00036
00037 #ifdef __cplusplus
00038 extern "C" {
00039 #endif
00040 #if 0
00041 }
00042 #endif
00043
00044 #include <pocketsphinx/prim_type.h>
00045 #include <pocketsphinx/export.h>
00046 #include <pocketsphinx/vad.h>
00047
00048 typedef struct ps_endpointer_s ps_endpointer_t;
00049
00050 typedef double (*ps_endpointer_timestamp_cb_t)(void *user_data);
00051
00052 #define PS_ENDPOINTER_DEFAULT_WINDOW 0.3
00053 #define PS_ENDPOINTER_DEFAULT_RATIO 0.9
00054
00055 POCKETSPHINX_EXPORT
00056 ps_endpointer_t *ps_endpointer_init(double window,
00057                                     double ratio,
00058                                     ps_vad_mode_t mode,
00059                                     int sample_rate, double frame_length);
00060
00061 POCKETSPHINX_EXPORT
00062 ps_endpointer_t *ps_endpointer_retain(ps_endpointer_t *ep);
00063
00064 POCKETSPHINX_EXPORT
00065 int ps_endpointer_free(ps_endpointer_t *ep);
00066

```

```

00145 POCKETSPHINX_EXPORT
00146 ps_vad_t *ps_endpoint_vad(ps_endpoint_t *ep);
00147
00153 #define ps_endpoint_frame_size(ep) ps_vad_frame_size(ps_endpoint_vad(ep))
00154
00158 #define ps_endpoint_frame_length(ep) ps_vad_frame_length(ps_endpoint_vad(ep))
00159
00163 #define ps_endpoint_sample_rate(ep) ps_vad_sample_rate(ps_endpoint_vad(ep))
00164
00178 POCKETSPHINX_EXPORT
00179 const int16_t *ps_endpoint_process(ps_endpoint_t *ep,
00180                                   const int16_t *frame);
00181
00196 POCKETSPHINX_EXPORT
00197 const int16_t *ps_endpoint_end_stream(ps_endpoint_t *ep,
00198                                       const int16_t *frame,
00199                                       size_t nsamp,
00200                                       size_t *out_nsamp);
00201
00217 POCKETSPHINX_EXPORT
00218 int ps_endpoint_in_speech(ps_endpoint_t *ep);
00219
00224 POCKETSPHINX_EXPORT
00225 double ps_endpoint_speech_start(ps_endpoint_t *ep);
00226
00231 POCKETSPHINX_EXPORT
00232 double ps_endpoint_speech_end(ps_endpoint_t *ep);
00233
00251 POCKETSPHINX_EXPORT
00252 int ps_endpoint_set_timestamp_func(ps_endpoint_t *ep,
00253                                   ps_endpoint_timestamp_cb_t cb,
00254                                   void *user_data);
00255
00267 POCKETSPHINX_EXPORT
00268 double ps_endpoint_timestamp(ps_endpoint_t *ep);
00269
00270 #ifdef __cplusplus
00271 }
00272 #endif
00273
00274 #endif /* __PS_ENDPOINTER_H__ */

```

11.33 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/err.h File Reference

Implementation of logging routines.

```

#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <pocketsphinx/export.h>

```

Macros

- #define [E_SYSCALL](#)(stmt, ...)
- #define [FILELINE](#) __FILE__ , __LINE__
- #define [E_FATAL](#)(...)
- #define [E_FATAL_SYSTEM](#)(...)
- #define [E_ERROR_SYSTEM](#)(...)
- #define [E_ERROR](#)(...)
- #define [E_WARN](#)(...)
- #define [E_INFO](#)(...)
- #define [E_INFOCONT](#)(...)
- #define [E_INFO_NOFN](#)(...)
- #define [E_DEBUG](#)(...)

Typedefs

- typedef enum [err_e](#) [err_lvl_t](#)
- typedef void(* [err_cb_f](#)) (void *user_data, [err_lvl_t](#), const char *,...)

Enumerations

- enum [err_e](#) {
[ERR_DEBUG](#) , [ERR_INFO](#) , [ERR_WARN](#) , [ERR_ERROR](#) ,
[ERR_FATAL](#) , [ERR_MAX](#) }

Functions

- [POCKETSPHINX_EXPORT](#) void [err_msg](#) ([err_lvl_t](#) lvl, const char *path, long ln, const char *fmt,...)
- [POCKETSPHINX_EXPORT](#) void [err_msg_system](#) ([err_lvl_t](#) lvl, const char *path, long ln, const char *fmt,...)
- [POCKETSPHINX_EXPORT](#) void [err_logfp_cb](#) (void *user_data, [err_lvl_t](#) level, const char *fmt,...)
- [POCKETSPHINX_EXPORT](#) int [err_set_loglevel](#) ([err_lvl_t](#) lvl)
- [POCKETSPHINX_EXPORT](#) const char * [err_set_loglevel_str](#) (const char *lvl)
- [POCKETSPHINX_EXPORT](#) void [err_set_callback](#) ([err_cb_f](#) callback, void *user_data)
- [POCKETSPHINX_EXPORT](#) void [err_set_logfp](#) (FILE *stream)
- [POCKETSPHINX_EXPORT](#) FILE * [err_get_logfp](#) (void)
- [POCKETSPHINX_EXPORT](#) int [err_set_logfile](#) (const char *path)

11.33.1 Detailed Description

Implementation of logging routines.

Logging, warning, debug and error message output functionality is provided in this file. Sphinxbase defines several level of logging messages - INFO, WARNING, ERROR, FATAL. By default output goes to standard error output.

Logging is implemented through macros. They take same arguments as printf: format string and values. By default source file name and source line are prepended to the message. Log output could be redirected to any file using [err_set_logfp\(\)](#) and [err_set_logfile\(\)](#) functions. To disable logging in your application, call [err_set_logfp\(NULL\)](#).

It's possible to log multiline info messages, to do that you need to start message with [E_INFO](#) and output other lines with [E_INFOCONT](#).

11.33.2 Macro Definition Documentation

11.33.2.1 E_DEBUG

```
#define E_DEBUG(  
    ...)
```

Debug is disabled by default

11.33.2.2 E_ERROR

```
#define E_ERROR(
    ...)
```

Value:

```
err_msg(ERR_ERROR, FILELINE, __VA_ARGS__)
```

Print error message to error log

Referenced by [SpeechToText::listen\(\)](#), and [SpeechManager::startListening\(\)](#).

11.33.2.3 E_ERROR_SYSTEM

```
#define E_ERROR_SYSTEM(
    ...)
```

Value:

```
err_msg_system(ERR_ERROR, FILELINE, __VA_ARGS__)
```

Print error text; Call perror("");

11.33.2.4 E_FATAL

```
#define E_FATAL(
    ...)
```

Value:

```
do {
    err_msg(ERR_FATAL, FILELINE, __VA_ARGS__); \
    exit(EXIT_FAILURE); \
} while (0)
```

Exit with non-zero status after error message

Referenced by [SpeechManager::init\(\)](#), [SpeechToText::init\(\)](#), [SpeechToText::listen\(\)](#), and [SpeechManager::startListening\(\)](#).

11.33.2.5 E_FATAL_SYSTEM

```
#define E_FATAL_SYSTEM(
    ...)
```

Value:

```
do {
    err_msg_system(ERR_FATAL, FILELINE, __VA_ARGS__); \
    exit(EXIT_FAILURE); \
} while (0)
```

Print error text; Call perror(""); exit(errno);

Referenced by [SpeechManager::init\(\)](#), and [SpeechToText::init\(\)](#).

11.33.2.6 E_INFO

```
#define E_INFO(  
    ...)
```

Value:

```
err_msg(ERR_INFO, FILELINE, __VA_ARGS__)
```

Print logging information to standard error stream

11.33.2.7 E_INFO_NOFN

```
#define E_INFO_NOFN(  
    ...)
```

Value:

```
err_msg(ERR_INFO, NULL, 0, __VA_ARGS__)
```

Print logging information without filename.

11.33.2.8 E_INFOCONT

```
#define E_INFOCONT(  
    ...)
```

Value:

```
err_msg(ERR_INFO, NULL, 0, __VA_ARGS__)
```

Continue printing the information to standard error stream

11.33.2.9 E_SYSCALL

```
#define E_SYSCALL(  
    stmt,  
    ...)
```

Value:

```
if (stmt) E_FATAL_SYSTEM(__VA_ARGS__);
```

11.33.2.10 E_WARN

```
#define E_WARN(  
    ...)
```

Value:

```
err_msg(ERR_WARN, FILELINE, __VA_ARGS__)
```

Print warning message to error log

11.33.2.11 FILELINE

```
#define FILELINE __FILE__ , __LINE__
```

11.33.3 Typedef Documentation

11.33.3.1 err_cb_f

```
typedef void(* err_cb_f) (void *user_data, err_lvl_t, const char *,...)
```

11.33.3.2 err_lvl_t

```
typedef enum err_e err_lvl_t
```

11.33.4 Enumeration Type Documentation

11.33.4.1 err_e

```
enum err_e
```

Enumerator

ERR_DEBUG	
ERR_INFO	
ERR_WARN	
ERR_ERROR	
ERR_FATAL	
ERR_MAX	

11.33.5 Function Documentation

11.33.5.1 err_get_logfp()

```
POCKETSPHINX_EXPORT FILE * err_get_logfp (  
    void )
```

Get the current logging filehandle.

Returns

Current logging filehandle, NULL if logging is disabled. Initially it returns stderr

References [POCKETSPHINX_EXPORT](#).

11.33.5.2 `err_logfp_cb()`

```
POCKETSPHINX_EXPORT void err_logfp_cb (
    void * user_data,
    err_lvl_t level,
    const char * fmt,
    ...)
```

11.33.5.3 `err_msg()`

```
POCKETSPHINX_EXPORT void err_msg (
    err_lvl_t lvl,
    const char * path,
    long ln,
    const char * fmt,
    ...)
```

References [POCKETSPHINX_EXPORT](#).

11.33.5.4 `err_msg_system()`

```
POCKETSPHINX_EXPORT void err_msg_system (
    err_lvl_t lvl,
    const char * path,
    long ln,
    const char * fmt,
    ...)
```

References [POCKETSPHINX_EXPORT](#).

11.33.5.5 `err_set_callback()`

```
POCKETSPHINX_EXPORT void err_set_callback (
    err_cb_f callback,
    void * user_data)
```

Sets function to output error messages. Use it to redirect the logging to your application. By default the handler which dumps messages to stderr is set.

Parameters

<i>callback</i>	callback to pass messages too
<i>user_data</i>	data to pass to callback

References [POCKETSPHINX_EXPORT](#).

11.33.5.6 `err_set_logfile()`

```
POCKETSPHINX_EXPORT int err_set_logfile (
    const char * path)
```

Append all log messages to a given file.

Previous logging filehandle is closed (unless it was stdout or stderr).

Parameters

<i>path</i>	File path to send log messages to
-------------	-----------------------------------

Returns

0 for success, <0 for failure (e.g. if file does not exist)

11.33.5.7 err_set_logfp()

```
POCKETSPHINX_EXPORT void err_set_logfp (
    FILE * stream)
```

Direct all logging to a given filehandle if default logfp callback is set.

Parameters

<i>stream</i>	Filehandle to send log messages to, or NULL to disable logging.
---------------	---

References [POCKETSPHINX_EXPORT](#).

11.33.5.8 err_set_loglevel()

```
POCKETSPHINX_EXPORT int err_set_loglevel (
    err_lvl_t lvl)
```

Set minimum logging level.

Parameters

<i>lvl</i>	Level below which messages will not be logged (note ERR_DEBUG messages are not logged unless compiled in debugging mode)
------------	--

Returns

previous log level.

References [POCKETSPHINX_EXPORT](#).

11.33.5.9 err_set_loglevel_str()

```
POCKETSPHINX_EXPORT const char * err_set_loglevel_str (
    const char * lvl)
```

Set minimum logging level from a string

Parameters

/v/	Level below which messages will not be logged (note ERR_DEBUG messages are not logged unless compiled in debugging mode). A string corresponding to the names in enum err_e , but without the leading "ERR_" prefix.
-----	--

Returns

previous log level string, or NULL for invalid argument.

References [POCKETSPHINX_EXPORT](#).

11.34 err.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset: 4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 1999-2004 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  * This work was supported in part by funding from the Defense Advanced
00019  * Research Projects Agency and the National Science Foundation of the
00020  * United States of America, and the CMU Sphinx Speech Consortium.
00021  *
00022  * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS" AND
00023  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00024  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00025  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00026  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00027  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00028  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00029  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00030  * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00031  * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00032  * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00033  *
00034  * =====
00035  */
00036 */
00037
00038 #ifndef _LIBUTIL_ERR_H_
00039 #define _LIBUTIL_ERR_H_
00040
00041 #include <stdarg.h>
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <errno.h>
00045
00046 #include <pocketsphinx/export.h>
00047
00048
00049 #ifdef __cplusplus
00050 extern "C" {
00051 #endif
00052 #if 0
00053 /* Fool Emacs. */
00054 }
00055 #endif
00056
00057 #define E_SYSCALL(stmt, ...) if (stmt) E_FATAL_SYSTEM(__VA_ARGS__);
00058
00059 #define FILELINE __FILE__ , __LINE__
00060

```

```

00080 #define E_FATAL(...) \
00081     do { \
00082         err_msg(ERR_FATAL, FILELINE, __VA_ARGS__); \
00083         exit(EXIT_FAILURE); \
00084     } while (0)
00085
00089 #define E_FATAL_SYSTEM(...) \
00090     do { \
00091         err_msg_system(ERR_FATAL, FILELINE, __VA_ARGS__); \
00092         exit(EXIT_FAILURE); \
00093     } while (0)
00094
00098 #define E_ERROR_SYSTEM(...) err_msg_system(ERR_ERROR, FILELINE, __VA_ARGS__)
00099
00103 #define E_ERROR(...) err_msg(ERR_ERROR, FILELINE, __VA_ARGS__)
00104
00108 #define E_WARN(...) err_msg(ERR_WARN, FILELINE, __VA_ARGS__)
00109
00113 #define E_INFO(...) err_msg(ERR_INFO, FILELINE, __VA_ARGS__)
00114
00118 #define E_INFOCONT(...) err_msg(ERR_INFO, NULL, 0, __VA_ARGS__)
00119
00123 #define E_INFO_NOFN(...) err_msg(ERR_INFO, NULL, 0, __VA_ARGS__)
00124
00128 #ifdef SPHINX_DEBUG
00129 #define E_DEBUG(...) err_msg(ERR_DEBUG, NULL, 0, __VA_ARGS__)
00130 #else
00131 #define E_DEBUG(...)
00132 #endif
00133
00134 typedef enum err_e {
00135     ERR_DEBUG,
00136     ERR_INFO,
00137     ERR_WARN,
00138     ERR_ERROR,
00139     ERR_FATAL,
00140     ERR_MAX
00141 } err_lvl_t;
00142
00143 POCKETSPHINX_EXPORT
00144 void err_msg(err_lvl_t lvl, const char *path, long ln, const char *fmt, ...);
00145
00146 POCKETSPHINX_EXPORT
00147 void err_msg_system(err_lvl_t lvl, const char *path, long ln, const char *fmt, ...);
00148
00149 POCKETSPHINX_EXPORT
00150 void err_logfp_cb(void * user_data, err_lvl_t level, const char *fmt, ...);
00151
00152 typedef void (*err_cb_f)(void* user_data, err_lvl_t, const char *, ...);
00153
00162 POCKETSPHINX_EXPORT
00163 int err_set_loglevel(err_lvl_t lvl);
00164
00174 POCKETSPHINX_EXPORT
00175 const char *err_set_loglevel_str(const char *lvl);
00176
00185 POCKETSPHINX_EXPORT
00186 void err_set_callback(err_cb_f callback, void *user_data);
00187
00193 POCKETSPHINX_EXPORT
00194 void err_set_logfp(FILE *stream);
00195
00202 POCKETSPHINX_EXPORT
00203 FILE *err_get_logfp(void);
00204
00213 POCKETSPHINX_EXPORT
00214 int err_set_logfile(const char *path);
00215
00216 #ifdef __cplusplus
00217 }
00218 #endif
00219
00220 #endif /* !_ERR_H */

```

11.35 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/export.h File Reference

Macros

- #define [POCKETSPHINX_EXPORT](#)

11.35.1 Macro Definition Documentation

11.35.1.1 POCKETSPHINX_EXPORT

```
#define POCKETSPHINX_EXPORT
```

Referenced by [err_get_logfp\(\)](#), [err_msg\(\)](#), [err_msg_system\(\)](#), [err_set_callback\(\)](#), [err_set_logfp\(\)](#), [err_set_loglevel\(\)](#), [err_set_loglevel_str\(\)](#), [fsg_model_t::fsg_model_accept\(\)](#), [fsg_model_t::fsg_model_free\(\)](#), [fsg_model_t::fsg_model_read\(\)](#), [fsg_model_t::fsg_model_readfile\(\)](#), [fsg_model_t::fsg_model_retain\(\)](#), [fsg_model_t::fsg_model_write\(\)](#), [fsg_model_t::fsg_model_writefile_fsm\(\)](#), [fsg_model_t::fsg_model_write_syntab\(\)](#), [fsg_model_t::fsg_model_writefile\(\)](#), [fsg_model_t::fsg_model_writefile_fsm\(\)](#), [jsgf_t::jsgf_build_fsg\(\)](#), [jsgf_t::jsgf_get_public_rule\(\)](#), [jsgf_t::jsgf_get_rule\(\)](#), [jsgf_t::jsgf_grammar_free\(\)](#), [jsgf_t::jsgf_grammar_name\(\)](#), [jsgf_t::jsgf_parse_file\(\)](#), [jsgf_t::jsgf_parse_string\(\)](#), [fsg_model_t::jsgf_read_file\(\)](#), [fsg_model_t::jsgf_read_string\(\)](#), [jsgf_t::jsgf_rule_iter\(\)](#), [jsgf_rule_iter_t::jsgf_rule_iter_free\(\)](#), [jsgf_rule_iter_t::jsgf_rule_iter_next\(\)](#), [jsgf_rule_iter_t::jsgf_rule_iter_rule\(\)](#), [jsgf_rule_t::jsgf_rule_name\(\)](#), [jsgf_t::jsgf_write_fsg\(\)](#), [logmath_t::logmath_add\(\)](#), [logmath_t::logmath_add_exact\(\)](#), [logmath_t::logmath_exp\(\)](#), [logmath_t::logmath_free\(\)](#), [logmath_t::logmath_get_base\(\)](#), [logmath_t::logmath_get_shift\(\)](#), [logmath_t::logmath_get_table_shape\(\)](#), [logmath_t::logmath_get_width\(\)](#), [logmath_t::logmath_get_zero\(\)](#), [logmath_t::logmath_init\(\)](#), [logmath_t::logmath_ln_to_log\(\)](#), [logmath_t::logmath_log\(\)](#), [logmath_t::logmath_log10_to_log\(\)](#), [logmath_t::logmath_log10_to_log_float\(\)](#), [logmath_t::logmath_log_to_ln\(\)](#), [logmath_t::logmath_log_to_log10\(\)](#), [logmath_t::logmath_read\(\)](#), [logmath_t::logmath_retain\(\)](#), [logmath_t::logmath_write\(\)](#), [ngram_model_t::ngram_bg_score\(\)](#), [ngram_model_t::ngram_file_name_to_type\(\)](#), [ngram_model_t::ngram_iter\(\)](#), [ngram_iter_t::ngram_iter_free\(\)](#), [ngram_iter_t::ngram_iter_get\(\)](#), [ngram_iter_t::ngram_iter_next\(\)](#), [ngram_iter_t::ngram_iter_successors\(\)](#), [ngram_model_t::ngram_model_add_class\(\)](#), [ngram_model_t::ngram_model_add_class_word\(\)](#), [ngram_model_t::ngram_model_add_word\(\)](#), [ngram_model_t::ngram_model_apply_weights\(\)](#), [ngram_model_t::ngram_model_casefold\(\)](#), [ngram_model_t::ngram_model_get_size\(\)](#), [ngram_model_t::ngram_model_get_weights\(\)](#), [ngram_model_t::ngram_model_mgrams\(\)](#), [ngram_model_t::ngram_model_read\(\)](#), [ngram_model_t::ngram_model_read_classdef\(\)](#), [ngram_model_t::ngram_model_retain\(\)](#), [ngram_model_t::ngram_model_set_add\(\)](#), [ngram_model_t::ngram_model_set_current\(\)](#), [ngram_model_t::ngram_model_set_current_iter\(\)](#), [ngram_model_t::ngram_model_set_init\(\)](#), [ngram_model_t::ngram_model_set_interp\(\)](#), [ngram_model_t::ngram_model_set_iter\(\)](#), [ngram_model_set_iter_t::ngram_model_set_iter_free\(\)](#), [ngram_model_set_iter_t::ngram_model_set_iter_model\(\)](#), [ngram_model_set_iter_t::ngram_model_set_iter_next\(\)](#), [ngram_model_t::ngram_model_set_known_wid\(\)](#), [ngram_model_t::ngram_model_set_map_words\(\)](#), [ngram_model_t::ngram_model_set_read\(\)](#), [ngram_model_t::ngram_model_set_reorder\(\)](#), [ngram_model_t::ngram_model_set_select\(\)](#), [ngram_model_t::ngram_model_write\(\)](#), [ngram_model_t::ngram_ng_iter\(\)](#), [ngram_model_t::ngram_ng_prob\(\)](#), [ngram_model_t::ngram_ng_score\(\)](#), [ngram_model_t::ngram_prob\(\)](#), [ngram_model_t::ngram_prob_iter\(\)](#), [ngram_model_t::ngram_score\(\)](#), [ngram_model_t::ngram_score_to_prob\(\)](#), [ngram_model_t::ngram_str_to_type\(\)](#), [ngram_model_t::ngram_tg_score\(\)](#), [ngram_model_t::ngram_type_to_str\(\)](#), [ngram_model_t::ngram_unknown_wid\(\)](#), [ngram_model_t::ngram_wid\(\)](#), [ngram_model_t::ngram_word\(\)](#), [ngram_model_t::ngram_zero\(\)](#), [ps_decoder_t::ps_activate_search\(\)](#), [ps_decoder_t::ps_add_allphone\(\)](#), [ps_decoder_t::ps_add_allphone_file\(\)](#), [ps_decoder_t::ps_add_fsg\(\)](#), [ps_decoder_t::ps_add_jsgf_fsg\(\)](#), [ps_decoder_t::ps_add_jsgf_string\(\)](#), [ps_decoder_t::ps_add_keyphrase\(\)](#), [ps_decoder_t::ps_add_kws\(\)](#), [ps_decoder_t::ps_add_lm\(\)](#), [ps_decoder_t::ps_add_lm_file\(\)](#), [ps_decoder_t::ps_add_word\(\)](#), [ps_alignment_t::ps_alignment_free\(\)](#), [ps_alignment_iter_t::ps_alignment_iter_name\(\)](#), [ps_alignment_iter_t::ps_alignment_iter_next\(\)](#), [ps_alignment_iter_t::ps_alignment_iter_rule\(\)](#), [ps_alignment_t::ps_alignment_phones\(\)](#), [ps_alignment_t::ps_alignment_retain\(\)](#), [ps_alignment_t::ps_alignment_states\(\)](#), [ps_alignment_t::ps_alignment_words\(\)](#), [ps_args\(\)](#), [ps_config_t::ps_config_bool\(\)](#), [ps_config_t::ps_config_float\(\)](#), [ps_config_t::ps_config_get\(\)](#), [ps_config_t::ps_config_init\(\)](#), [ps_config_t::ps_config_int\(\)](#), [ps_config_t::ps_config_nistfile\(\)](#), [ps_config_t::ps_config_parse_json\(\)](#), [ps_config_t::ps_config_retain\(\)](#), [ps_config_t::ps_config_serialize_json\(\)](#), [ps_config_t::ps_config_set\(\)](#), [ps_config_t::ps_config_set_bool\(\)](#), [ps_config_t::ps_config_set_float\(\)](#), [ps_config_t::ps_config_set_int\(\)](#), [ps_config_t::ps_config_set_str\(\)](#), [ps_config_t::ps_config_soundfile\(\)](#), [ps_config_t::ps_config_str\(\)](#), [ps_config_t::ps_config_typeof\(\)](#), [ps_config_t::ps_config_validate\(\)](#), [ps_config_t::ps_config_wavfile\(\)](#), [ps_decoder_t::ps_current_search\(\)](#), [ps_decoder_t::ps_decode_raw\(\)](#), [ps_decoder_t::ps_decode_senscr\(\)](#), [ps_config_t::ps_default_model_dir\(\)](#), [ps_config_t::ps_default_search_args\(\)](#), [ps_decoder_t::ps_end_utt\(\)](#), [ps_endpointiter_t::ps_endpointiter_end_stream\(\)](#), [ps_endpointiter_t::ps_endpointiter_free\(\)](#), [ps_endpointiter_t::ps_endpointiter_in_speech\(\)](#), [ps_endpointiter_t::ps_endpointiter_init\(\)](#), [ps_endpointiter_t::ps_endpointiter_process\(\)](#), [ps_endpointiter_t::ps_endpointiter_retain\(\)](#), [ps_endpointiter_t::ps_endpointiter_set_timestamp_func\(\)](#), [ps_endpointiter_t::ps_endpointiter_set_timestamp\(\)](#), [ps_endpointiter_t::ps_endpointiter_speech_start\(\)](#), [ps_config_t::ps_expand_model_config\(\)](#), [ps_decoder_t::ps_free\(\)](#), [ps_decoder_t::ps_get_cmfn\(\)](#), [ps_decoder_t::ps_get_config\(\)](#), [ps_decoder_t::ps_get_fsg\(\)](#), [ps_decoder_t::ps_get_hyp\(\)](#), [ps_decoder_t::ps_get_in_speech\(\)](#), [ps_decoder_t::ps_get_kws\(\)](#), [ps_decoder_t::ps_get_lattice\(\)](#), [ps_decoder_t::ps_get_lm\(\)](#), [ps_decoder_t::ps_get_logmath\(\)](#), [ps_decoder_t::ps_get_n_frames\(\)](#), [ps_decoder_t::ps_get_prob\(\)](#), [ps_decoder_t::ps_get_utt_time\(\)](#), [ps_config_t::ps_init\(\)](#), [ps_latlink_t::ps_latlink_baseword\(\)](#), [ps_latlink_iter_t::ps_latlink_iter_free\(\)](#), [ps_latlink_iter_t::ps_latlink_iter_link\(\)](#), [ps_latlink_iter_t::ps_latlink_iter_next\(\)](#), [ps_latlink_t::ps_latlink_nodes\(\)](#), [ps_latlink_t::ps_latlink_pred\(\)](#), [ps_latlink_t::ps_latlink_prob\(\)](#), [ps_latlink_t::ps_latlink_times\(\)](#), [ps_latlink_t::ps_latlink_word\(\)](#), [ps_latnode_t::ps_latnode_baseword\(\)](#), [ps_latnode_t::ps_latnode_entry\(\)](#), [ps_latnode_t::ps_latnode_exits\(\)](#), [ps_lattice_t::ps_latnode_iter\(\)](#), [ps_latnode_iter_t::ps_latnode_iter_free\(\)](#),

```

ps_latnode_iter_t::ps_latnode_iter_next(), ps_latnode_iter_t::ps_latnode_iter_node(), ps_latnode_t::ps_latnode_prob(),
ps_latnode_iter_t::ps_latnode_times(), ps_latnode_t::ps_latnode_word(), ps_lattice_t::ps_lattice_bestpath(),
ps_lattice_t::ps_lattice_free(), ps_lattice_t::ps_lattice_get_logmath(), ps_lattice_t::ps_lattice_link(), ps_lattice_t::ps_lattice_posterior(),
ps_lattice_t::ps_lattice_posterior_prune(), ps_lattice_t::ps_lattice_read(), ps_lattice_t::ps_lattice_retain(), ps_lattice_t::ps_lattice_reve,
ps_lattice_t::ps_lattice_reverse_next(), ps_lattice_t::ps_lattice_traverse_edges(), ps_lattice_t::ps_lattice_traverse_next(),
ps_lattice_t::ps_lattice_write(), ps_lattice_t::ps_lattice_write_htk(), ps_decoder_t::ps_load_dict(), ps_decoder_t::ps_lookup_word(),
ps_mllr_t::ps_mllr_read(), ps_mllr_t::ps_mllr_retain(), ps_decoder_t::ps_nbest(), ps_nbest_t::ps_nbest_free(),
ps_nbest_t::ps_nbest_hyp(), ps_nbest_t::ps_nbest_next(), ps_nbest_t::ps_nbest_seg(), ps_decoder_t::ps_process_cep(),
ps_decoder_t::ps_process_raw(), ps_decoder_t::ps_reinit(), ps_decoder_t::ps_reinit_feat(), ps_decoder_t::ps_remove_search(),
ps_decoder_t::ps_retain(), ps_decoder_t::ps_save_dict(), ps_decoder_t::ps_search_iter(), ps_search_iter_t::ps_search_iter_free(),
ps_search_iter_t::ps_search_iter_next(), ps_search_iter_t::ps_search_iter_val(), ps_seg_t::ps_seg_frames(),
ps_seg_t::ps_seg_free(), ps_decoder_t::ps_seg_iter(), ps_seg_t::ps_seg_next(), ps_seg_t::ps_seg_prob(),
ps_seg_t::ps_seg_word(), ps_decoder_t::ps_set_align_text(), ps_decoder_t::ps_set_alignment(), ps_decoder_t::ps_set_cmnn(),
ps_decoder_t::ps_start_stream(), ps_decoder_t::ps_start_utt(), ps_decoder_t::ps_update_mllr(), ps_vad_t::ps_vad_free(),
ps_vad_t::ps_vad_init(), ps_vad_t::ps_vad_retain(), ps_vad_t::ps_vad_sample_rate(), and ps_vad_t::ps_vad_set_input_params().

```

11.36 export.h

[Go to the documentation of this file.](#)

```

00001 #ifndef __POCKETSPHINX_EXPORT_H__
00002 #define __POCKETSPHINX_EXPORT_H__
00003
00004 /* Win32 DLL gunk */
00005 #if defined(_WIN32) && defined(SPHINX_DLL)
00006 #if defined(POCKETSPHINX_EXPORTS) /* DLL itself */
00007 #define POCKETSPHINX_EXPORT __declspec(dllexport)
00008 #else
00009 #define POCKETSPHINX_EXPORT __declspec(dllimport)
00010 #endif
00011 #else /* No DLL things*/
00012 #define POCKETSPHINX_EXPORT
00013 #endif
00014
00015 #endif /* __POCKETSPHINX_EXPORT_H__ */

```

11.37 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/lattice.h File Reference

Word lattices.

```

#include <pocketsphinx/prim_type.h>
#include <pocketsphinx/model.h>
#include <pocketsphinx/export.h>

```

Typedefs

- typedef struct ps_lattice_s [ps_lattice_t](#)
- typedef struct ps_latnode_s [ps_latnode_t](#)
- typedef struct ps_latnode_s [ps_latnode_iter_t](#)
- typedef struct ps_latlink_s [ps_latlink_t](#)
- typedef struct latlink_list_s [ps_latlink_iter_t](#)

11.37.1 Detailed Description

Word lattices.

Because doxygen is Bad Software, the actual documentation can only exist in [ps_lattice_t](#). Sorry about that.

11.37.2 Typedef Documentation

11.37.2.1 ps_latlink_iter_t

```
typedef struct latlink_list_s ps_latlink_iter_t
```

11.37.2.2 ps_latlink_t

```
typedef struct ps_latlink_s ps_latlink_t
```

11.37.2.3 ps_latnode_iter_t

```
typedef struct ps_latnode_s ps_latnode_iter_t
```

11.37.2.4 ps_latnode_t

```
typedef struct ps_latnode_s ps_latnode_t
```

11.37.2.5 ps_lattice_t

```
typedef struct ps_lattice_s ps_lattice_t
```

11.38 lattice.h

[Go to the documentation of this file.](#)

```
00001 /* -*- c-basic-offset: 4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2008 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  * This work was supported in part by funding from the Defense Advanced
00019  * Research Projects Agency and the National Science Foundation of the
00020  * United States of America, and the CMU Sphinx Speech Consortium.
00021  *
00022  * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS'' AND
```



```

00023  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00024  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00025  * PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00026  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00027  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00028  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00029  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00030  * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00031  * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00032  * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00033  *
00034  * =====
00035  *
00036  */
00037
00045
00046 #ifndef __PS_LATTICE_H__
00047 #define __PS_LATTICE_H__
00048
00049 #include <pocketsphinx/prim_type.h>
00050 #include <pocketsphinx/model.h>
00051 #include <pocketsphinx/export.h>
00052
00053 #ifdef __cplusplus
00054 extern "C" {
00055 #endif
00056 #if 0
00057 }
00058 #endif
00059
00064 typedef struct ps_lattice_s ps_lattice_t;
00065
00073 typedef struct ps_latnode_s ps_latnode_t;
00074
00079 typedef struct ps_latnode_s ps_latnode_iter_t; /* pay no attention to the man behind the curtain */
00080
00088 typedef struct ps_latlink_s ps_latlink_t;
00089
00094 typedef struct latlink_list_s ps_latlink_iter_t;
00095
00096 /* Forward declaration needed to avoid circular includes */
00097 struct ps_decoder_s;
00098
00107 POCKETSPHINX_EXPORT
00108 ps_lattice_t *ps_lattice_read(struct ps_decoder_s *ps,
00109                             char const *file);
00110
00121 POCKETSPHINX_EXPORT
00122 ps_lattice_t *ps_lattice_retain(ps_lattice_t *dag);
00123
00130 POCKETSPHINX_EXPORT
00131 int ps_lattice_free(ps_lattice_t *dag);
00132
00139 POCKETSPHINX_EXPORT
00140 int ps_lattice_write(ps_lattice_t *dag, char const *filename);
00141
00148 POCKETSPHINX_EXPORT
00149 int ps_lattice_write_htk(ps_lattice_t *dag, char const *filename);
00150
00160 POCKETSPHINX_EXPORT
00161 logmath_t *ps_lattice_get_logmath(ps_lattice_t *dag);
00162
00163
00174 POCKETSPHINX_EXPORT
00175 ps_latnode_iter_t *ps_latnode_iter(ps_lattice_t *dag);
00176
00183 POCKETSPHINX_EXPORT
00184 ps_latnode_iter_t *ps_latnode_iter_next(ps_latnode_iter_t *itor);
00185
00191 POCKETSPHINX_EXPORT
00192 void ps_latnode_iter_free(ps_latnode_iter_t *itor);
00193
00198 POCKETSPHINX_EXPORT
00199 ps_latnode_t *ps_latnode_iter_node(ps_latnode_iter_t *itor);
00200
00210 POCKETSPHINX_EXPORT
00211 int ps_latnode_times(ps_latnode_t *node, int16 *out_fef, int16 *out_lef);
00212
00221 POCKETSPHINX_EXPORT
00222 char const *ps_latnode_word(ps_lattice_t *dag, ps_latnode_t *node);
00223
00232 POCKETSPHINX_EXPORT
00233 char const *ps_latnode_baseword(ps_lattice_t *dag, ps_latnode_t *node);
00234
00242 POCKETSPHINX_EXPORT
00243 ps_latlink_iter_t *ps_latnode_exits(ps_latnode_t *node);
00244

```

```

00252 POCKETSPHINX_EXPORT
00253 ps_latlink_iter_t *ps_latnode_entries(ps_latnode_t *node);
00254
00267 POCKETSPHINX_EXPORT
00268 int32 ps_latnode_prob(ps_lattice_t *dag, ps_latnode_t *node,
00269                      ps_latlink_t **out_link);
00270
00278 POCKETSPHINX_EXPORT
00279 ps_latlink_iter_t *ps_latlink_iter_next(ps_latlink_iter_t *itor);
00280
00286 POCKETSPHINX_EXPORT
00287 void ps_latlink_iter_free(ps_latlink_iter_t *itor);
00288
00293 POCKETSPHINX_EXPORT
00294 ps_latlink_t *ps_latlink_iter_link(ps_latlink_iter_t *itor);
00295
00307 POCKETSPHINX_EXPORT
00308 int ps_latlink_times(ps_latlink_t *link, int16 *out_sf);
00309
00318 POCKETSPHINX_EXPORT
00319 ps_latnode_t *ps_latlink_nodes(ps_latlink_t *link, ps_latnode_t **out_src);
00320
00329 POCKETSPHINX_EXPORT
00330 char const *ps_latlink_word(ps_lattice_t *dag, ps_latlink_t *link);
00331
00340 POCKETSPHINX_EXPORT
00341 char const *ps_latlink_baseword(ps_lattice_t *dag, ps_latlink_t *link);
00342
00350 POCKETSPHINX_EXPORT
00351 ps_latlink_t *ps_latlink_pred(ps_latlink_t *link);
00352
00364 POCKETSPHINX_EXPORT
00365 int32 ps_latlink_prob(ps_lattice_t *dag, ps_latlink_t *link, int32 *out_ascr);
00366
00373 POCKETSPHINX_EXPORT
00374 void ps_lattice_link(ps_lattice_t *dag, ps_latnode_t *from, ps_latnode_t *to,
00375                     int32 score, int32 ef);
00376
00393 POCKETSPHINX_EXPORT
00394 ps_latlink_t *ps_lattice_traverse_edges(ps_lattice_t *dag, ps_latnode_t *start, ps_latnode_t *end);
00395
00404 POCKETSPHINX_EXPORT
00405 ps_latlink_t *ps_lattice_traverse_next(ps_lattice_t *dag, ps_latnode_t *end);
00406
00418 POCKETSPHINX_EXPORT
00419 ps_latlink_t *ps_lattice_reverse_edges(ps_lattice_t *dag, ps_latnode_t *start, ps_latnode_t *end);
00420
00429 POCKETSPHINX_EXPORT
00430 ps_latlink_t *ps_lattice_reverse_next(ps_lattice_t *dag, ps_latnode_t *start);
00431
00441 POCKETSPHINX_EXPORT
00442 ps_latlink_t *ps_lattice_bestpath(ps_lattice_t *dag, ngram_model_t *lmset,
00443                                  float32 lwf, float32 ascale);
00444
00453 POCKETSPHINX_EXPORT
00454 int32 ps_lattice_posterior(ps_lattice_t *dag, ngram_model_t *lmset,
00455                           float32 ascale);
00456
00469 POCKETSPHINX_EXPORT
00470 int32 ps_lattice_posterior_prune(ps_lattice_t *dag, int32 beam);
00471
00472 #ifdef NOT_IMPLEMENTED_YET
00479 POCKETSPHINX_EXPORT
00480 int32 ps_lattice_ngram_expand(ps_lattice_t *dag, ngram_model_t *lm);
00481 #endif
00482
00490 POCKETSPHINX_EXPORT
00491 int ps_lattice_n_frames(ps_lattice_t *dag);
00492
00493 #ifdef __cplusplus
00494 }
00495 #endif
00496
00497 #endif /* __PS_LATTICE_H__ */

```

11.39 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/logmath.h File Reference

Fast integer logarithmic addition operations.

```
#include <pocketsphinx/export.h>
#include <pocketsphinx/prim_type.h>
```

Classes

- struct [logadd_s](#)

Macros

- #define [LOGMATH_TABLE](#)(lm)

Typedefs

- typedef struct [logadd_s](#) [logadd_t](#)
- typedef struct [logmath_s](#) [logmath_t](#)

11.39.1 Detailed Description

Fast integer logarithmic addition operations.

Because doxygen is Bad Software, the actual documentation can only exist in [logmath_t](#). Sorry about that.

In evaluating HMM models, probability values are often kept in log domain, to avoid overflow. To enable these logprob values to be held in [int32](#) variables without significant loss of precision, a logbase of (1+epsilon) (where epsilon < 0.01 or so) is used. This module maintains this logbase (B).

However, maintaining probabilities in log domain creates a problem when adding two probability values. This problem can be solved by table lookup. Note that:

- $b^z = b^x + b^y$
- $b^z = b^x(1 + b^{y-x}) = b^y(1 + e^{x-y})$
- $z = x + \log_b(1 + b^{y-x}) = y + \log_b(1 + b^{x-y})$

So:

- when $y > x$, $z = y + \logadd_table[-(x - y)]$
- when $x > y$, $z = x + \logadd_table[-(y - x)]$
- where $\logadd_table[n] = \log_b(1 + b^{-n})$

The first entry in *logadd_table* is simply $\log_b(2.0)$, for the case where $y = x$ and thus $z = \log_b(2x) = \log_b(2) + x$. The last entry is zero, where $\log_b(x + y) = x = y$ due to loss of precision.

Since this table can be quite large particularly for small logbases, an option is provided to compress it by dropping the least significant bits of the table.

11.39.2 Macro Definition Documentation

11.39.2.1 LOGMATH_TABLE

```
#define LOGMATH_TABLE(  
    lm)
```

Value:

```
((logadd_t *)lm)
```

Obtain the log-add table from a `logmath_t` *

11.39.3 Typedef Documentation

11.39.3.1 logadd_t

```
typedef struct logadd_s logadd_t
```

11.39.3.2 logmath_t

```
typedef struct logmath_s logmath_t
```

11.40 logmath.h

[Go to the documentation of this file.](#)

```
00001 /* -*- c-basic-offset: 4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2007 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  * This work was supported in part by funding from the Defense Advanced
00019  * Research Projects Agency and the National Science Foundation of the
00020  * United States of America, and the CMU Sphinx Speech Consortium.
00021  *
00022  * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS" AND
00023  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00024  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00025  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00026  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00027  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00028  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00029  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00030  * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00031  * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00032  * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00033  *
00034  * =====
00035  */
00036 */
00074
00075 #ifndef __LOGMATH_H__
```

```

00076 #define __LOGMATH_H__
00077
00078 #include <pocketsphinx/export.h>
00079 #include <pocketsphinx/prim_type.h>
00080
00081 #ifdef __cplusplus
00082 extern "C" {
00083 #endif
00084 #if 0
00085 /* Fool Emacs. */
00086 }
00087 #endif
00088
00095 typedef struct logadd_s logadd_t;
00096 struct logadd_s {
00097     void *table;
00100     uint32 table_size;
00102     uint8 width;
00104     int8 shift;
00105 };
00106
00111 typedef struct logmath_s logmath_t;
00112
00116 #define LOGMATH_TABLE(lm) ((logadd_t *)lm)
00117
00126 POCKETSPHINX_EXPORT
00127 logmath_t *logmath_init(float64 base, int shift, int use_table);
00128
00133 POCKETSPHINX_EXPORT
00134 logmath_t *logmath_read(const char *filename);
00135
00140 POCKETSPHINX_EXPORT
00141 int32 logmath_write(logmath_t *lmath, const char *filename);
00142
00147 POCKETSPHINX_EXPORT
00148 int32 logmath_get_table_shape(logmath_t *lmath, uint32 *out_size,
00149                               uint32 *out_width, uint32 *out_shift);
00150
00155 POCKETSPHINX_EXPORT
00156 float64 logmath_get_base(logmath_t *lmath);
00157
00162 POCKETSPHINX_EXPORT
00163 int logmath_get_zero(logmath_t *lmath);
00164
00169 POCKETSPHINX_EXPORT
00170 int logmath_get_width(logmath_t *lmath);
00171
00176 POCKETSPHINX_EXPORT
00177 int logmath_get_shift(logmath_t *lmath);
00178
00185 POCKETSPHINX_EXPORT
00186 logmath_t *logmath_retain(logmath_t *lmath);
00187
00194 POCKETSPHINX_EXPORT
00195 int logmath_free(logmath_t *lmath);
00196
00201 POCKETSPHINX_EXPORT
00202 int logmath_add_exact(logmath_t *lmath, int logb_p, int logb_q);
00203
00208 POCKETSPHINX_EXPORT
00209 int logmath_add(logmath_t *lmath, int logb_p, int logb_q);
00210
00215 POCKETSPHINX_EXPORT
00216 int logmath_log(logmath_t *lmath, float64 p);
00217
00222 POCKETSPHINX_EXPORT
00223 float64 logmath_exp(logmath_t *lmath, int logb_p);
00224
00229 POCKETSPHINX_EXPORT
00230 int logmath_ln_to_log(logmath_t *lmath, float64 log_p);
00231
00236 POCKETSPHINX_EXPORT
00237 float64 logmath_log_to_ln(logmath_t *lmath, int logb_p);
00238
00243 POCKETSPHINX_EXPORT
00244 int logmath_log10_to_log(logmath_t *lmath, float64 log_p);
00245
00250 POCKETSPHINX_EXPORT
00251 float logmath_log10_to_log_float(logmath_t *lmath, float64 log_p);
00252
00257 POCKETSPHINX_EXPORT
00258 float64 logmath_log_to_log10(logmath_t *lmath, int logb_p);
00259
00264 POCKETSPHINX_EXPORT
00265 float64 logmath_log_float_to_log10(logmath_t *lmath, float log_p);
00266
00267 #ifdef __cplusplus

```

```

00268 }
00269 #endif
00270
00271
00272 #endif /* __LOGMATH_H__ */

```

11.41 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/mlr.h File Reference

Model-space linear transforms for speaker adaptation.

```

#include <pocketsphinx/prim_type.h>
#include <pocketsphinx/export.h>

```

Typedefs

- typedef struct ps_mllr_s [ps_mllr_t](#)

11.41.1 Detailed Description

Model-space linear transforms for speaker adaptation.

Because doxygen is Bad Software, the actual documentation can only exist in [ps_mllr_t](#). Sorry about that.

11.41.2 Typedef Documentation

11.41.2.1 ps_mllr_t

```
typedef struct ps_mllr_s ps_mllr_t
```

11.42 mllr.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset: 4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2008 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  * This work was supported in part by funding from the Defense Advanced
00019  * Research Projects Agency and the National Science Foundation of the
00020  * United States of America, and the CMU Sphinx Speech Consortium.
00021  *

```

```

00022 * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS" AND
00023 * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00024 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00025 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00026 * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00027 * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00028 * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00029 * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00030 * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00031 * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00032 * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00033 *
00034 * =====
00035 *
00036 */
00037
00045
00046 #ifndef __PS_MLLR_H__
00047 #define __PS_MLLR_H__
00048
00049 #include <pocketsphinx/prim_type.h>
00050 #include <pocketsphinx/export.h>
00051
00052 #ifdef __cplusplus
00053 extern "C" {
00054 #endif
00055 #if 0
00056 }
00057 #endif
00058
00063 typedef struct ps_mllr_s ps_mllr_t;
00064
00069 POCKETSPHINX_EXPORT
00070 ps_mllr_t *ps_mllr_read(char const *file);
00071
00076 POCKETSPHINX_EXPORT
00077 ps_mllr_t *ps_mllr_retain(ps_mllr_t *mllr);
00078
00083 POCKETSPHINX_EXPORT
00084 int ps_mllr_free(ps_mllr_t *mllr);
00085
00086 #ifdef __cplusplus
00087 }
00088 #endif
00089
00090 #endif /* __PS_MLLR_H__ */

```

11.43 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/model.h File Reference

Public API for language models.

```

#include <stdio.h>
#include <pocketsphinx/prim_type.h>
#include <pocketsphinx/logmath.h>
#include <pocketsphinx/export.h>

```

Macros

- #define [NGRAM_INVALID_WID](#) -1

Typedefs

- typedef struct jsgf_s [jsgf_t](#)
- typedef struct jsgf_rule_s [jsgf_rule_t](#)
- typedef struct fsg_model_s [fsg_model_t](#)
- typedef struct ngram_model_s [ngram_model_t](#)

- typedef struct hash_iter_s jsgf_rule_iter_t
- typedef struct ngram_class_s ngram_class_t
- typedef enum ngram_file_type_e ngram_file_type_t
- typedef enum ngram_case_e ngram_case_t
- typedef struct ngram_iter_s ngram_iter_t
- typedef struct ngram_model_set_iter_s ngram_model_set_iter_t

Enumerations

- enum ngram_file_type_e { NGRAM_INVALID = -1 , NGRAM_AUTO , NGRAM_ARPA , NGRAM_BIN }
File types for N-Gram files.
- enum ngram_case_e { NGRAM_UPPER , NGRAM_LOWER }
Constants for case folding.

11.43.1 Detailed Description

Public API for language models.

Because doxygen is Bad Software, the actual documentation can only exist in [jsgf_t](#), [fsg_model_t](#), and [ngram_model_t](#). Sorry about that.

11.43.2 Macro Definition Documentation

11.43.2.1 NGRAM_INVALID_WID

```
#define NGRAM_INVALID_WID -1
```

Impossible word ID

11.43.3 Typedef Documentation

11.43.3.1 fsg_model_t

```
typedef struct fsg_model_s fsg_model_t
```

11.43.3.2 jsgf_rule_iter_t

```
typedef struct hash_iter_s jsgf_rule_iter_t
```

11.43.3.3 jsgf_rule_t

```
typedef struct jsgf_rule_s jsgf_rule_t
```


11.43.3.4 jsgf_t

```
typedef struct jsgf_s jsgf_t
```

11.43.3.5 ngram_case_t

```
typedef enum ngram_case_e ngram_case_t
```

11.43.3.6 ngram_class_t

```
typedef struct ngram_class_s ngram_class_t
```

11.43.3.7 ngram_file_type_t

```
typedef enum ngram_file_type_e ngram_file_type_t
```

11.43.3.8 ngram_iter_t

```
typedef struct ngram_iter_s ngram_iter_t
```

11.43.3.9 ngram_model_set_iter_t

```
typedef struct ngram_model_set_iter_s ngram_model_set_iter_t
```

11.43.3.10 ngram_model_t

```
typedef struct ngram_model_s ngram_model_t
```

11.43.4 Enumeration Type Documentation

11.43.4.1 ngram_case_e

```
enum ngram_case_e
```

Constants for case folding.

[pocketsphinx/model.h](#)

Enumerator

NGRAM_UPPER	Upper case
-------------	------------

NGRAM_LOWER	Lower case
-------------	------------

11.43.4.2 ngram_file_type_e

enum [ngram_file_type_e](#)

File types for N-Gram files.

[pocketsphinx/model.h](#)

Enumerator

NGRAM_INVALID	Not a valid file type.
NGRAM_AUTO	Determine file type automatically.
NGRAM_ARPA	ARPABO text format (the standard).
NGRAM_BIN	Sphinx .DMP format.

11.44 model.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset:4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 1999-2022 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  * notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  * notice, this list of conditions and the following disclaimer in
00015  * the documentation and/or other materials provided with the
00016  * distribution.
00017  *
00018  * THIS SOFTWARE IS PROVIDED ``AS IS'' AND ANY EXPRESSED OR IMPLIED
00019  * WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
00020  * OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
00021  * DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT,
00022  * INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
00023  * (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
00024  * SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
00025  * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
00026  * STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
00027  * ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
00028  * OF THE POSSIBILITY OF SUCH DAMAGE.
00029  * =====
00030  */
00031
00032 #ifndef __PS_MODEL_H__
00033 #define __PS_MODEL_H__
00034
00035 #include <stdio.h>
00036
00037 #include <pocketsphinx/prim_type.h>
00038 #include <pocketsphinx/logmath.h>
00039 #include <pocketsphinx/export.h>
00040
00041 #ifdef __cplusplus
00042 extern "C" {
00043 #endif
00044 #if 0
00045 }
00046 #endif

```

```

00055
00056 /* Forward declaration to avoid include loops */
00057 typedef struct cmd_ln_s ps_config_t;
00058
00063 typedef struct jsgf_s jsgf_t;
00064
00069 typedef struct jsgf_rule_s jsgf_rule_t;
00070
00080 typedef struct fsg_model_s fsg_model_t;
00081
00086 typedef struct ngram_model_s ngram_model_t;
00087
00096 POCKETSPHINX_EXPORT
00097 jsgf_t *jsgf_parse_file(const char *filename, jsgf_t *parent);
00098
00107 POCKETSPHINX_EXPORT
00108 jsgf_t *jsgf_parse_string(const char *string, jsgf_t *parent);
00109
00114 POCKETSPHINX_EXPORT
00115 char const *jsgf_grammar_name(jsgf_t *jsgf);
00116
00121 POCKETSPHINX_EXPORT
00122 void jsgf_grammar_free(jsgf_t *jsgf);
00123
00128 POCKETSPHINX_EXPORT
00129 jsgf_rule_t *jsgf_get_rule(jsgf_t *grammar, const char *name);
00130
00135 POCKETSPHINX_EXPORT
00136 jsgf_rule_t *jsgf_get_public_rule(jsgf_t *grammar);
00137
00142 POCKETSPHINX_EXPORT
00143 char const *jsgf_rule_name(jsgf_rule_t *rule);
00144
00149 POCKETSPHINX_EXPORT
00150 int jsgf_rule_public(jsgf_rule_t *rule);
00151
00156 typedef struct hash_iter_s jsgf_rule_iter_t;
00157
00162 POCKETSPHINX_EXPORT
00163 jsgf_rule_iter_t *jsgf_rule_iter(jsgf_t *grammar);
00164
00169 POCKETSPHINX_EXPORT
00170 jsgf_rule_iter_t *jsgf_rule_iter_next(jsgf_rule_iter_t *itor);
00171
00176 POCKETSPHINX_EXPORT
00177 jsgf_rule_t *jsgf_rule_iter_rule(jsgf_rule_iter_t *itor);
00178
00183 POCKETSPHINX_EXPORT
00184 void jsgf_rule_iter_free(jsgf_rule_iter_t *itor);
00185
00190 POCKETSPHINX_EXPORT
00191 fsg_model_t *jsgf_build_fsg(jsgf_t *grammar, jsgf_rule_t *rule,
00192                             logmath_t *lmath, float32 lw);
00193
00200 POCKETSPHINX_EXPORT
00201 fsg_model_t *jsgf_read_file(const char *file, logmath_t *lmath, float32 lw);
00202
00209 POCKETSPHINX_EXPORT
00210 fsg_model_t *jsgf_read_string(const char *string, logmath_t *lmath, float32 lw);
00211
00219 POCKETSPHINX_EXPORT
00220 int jsgf_write_fsg(jsgf_t *grammar, jsgf_rule_t *rule, FILE *outfh);
00221
00228 POCKETSPHINX_EXPORT
00229 fsg_model_t *fsg_model_retain(fsg_model_t *fsg);
00230
00237 POCKETSPHINX_EXPORT
00238 int fsg_model_free(fsg_model_t *fsg);
00239
00280 POCKETSPHINX_EXPORT
00281 fsg_model_t *fsg_model_readfile(const char *file, logmath_t *lmath, float32 lw);
00282
00287 POCKETSPHINX_EXPORT
00288 fsg_model_t *fsg_model_read(FILE *fp, logmath_t *lmath, float32 lw);
00289
00297 POCKETSPHINX_EXPORT
00298 int fsg_model_accept(fsg_model_t *fsg, char const *words);
00299
00304 POCKETSPHINX_EXPORT
00305 void fsg_model_write(fsg_model_t *fsg, FILE *fp);
00306
00311 POCKETSPHINX_EXPORT
00312 void fsg_model_writefile(fsg_model_t *fsg, char const *file);
00313
00318 POCKETSPHINX_EXPORT
00319 void fsg_model_write_fsm(fsg_model_t *fsg, FILE *fp);
00320

```

```
00325 POCKETSPHINX_EXPORT
00326 void fsg_model_writefile_fsm(fsg_model_t *fsg, char const *file);
00327
00332 POCKETSPHINX_EXPORT
00333 void fsg_model_write_syntab(fsg_model_t *fsg, FILE *file);
00334
00339 POCKETSPHINX_EXPORT
00340 void fsg_model_writefile_syntab(fsg_model_t *fsg, char const *file);
00341
00346 typedef struct ngram_class_s ngram_class_t;
00347
00352 typedef enum ngram_file_type_e {
00353     NGRAM_INVALID = -1,
00354     NGRAM_AUTO,
00355     NGRAM_ARPA,
00356     NGRAM_BIN
00357 } ngram_file_type_t;
00358
00359 #define NGRAM_INVALID_WID -1
00360
00381 POCKETSPHINX_EXPORT
00382 ngram_model_t *ngram_model_read(ps_config_t *config,
00383     const char *file_name,
00384     ngram_file_type_t file_type,
00385     logmath_t *lmath);
00386
00393 POCKETSPHINX_EXPORT
00394 int ngram_model_write(ngram_model_t *model, const char *file_name,
00395     ngram_file_type_t format);
00396
00403 POCKETSPHINX_EXPORT
00404 ngram_file_type_t ngram_file_name_to_type(const char *file_name);
00405
00412 POCKETSPHINX_EXPORT
00413 ngram_file_type_t ngram_str_to_type(const char *str_name);
00414
00422 POCKETSPHINX_EXPORT
00423 char const *ngram_type_to_str(int type);
00424
00431 POCKETSPHINX_EXPORT
00432 ngram_model_t *ngram_model_retain(ngram_model_t *model);
00433
00440 POCKETSPHINX_EXPORT
00441 int ngram_model_free(ngram_model_t *model);
00442
00447 typedef enum ngram_case_e {
00448     NGRAM_UPPER,
00449     NGRAM_LOWER
00450 } ngram_case_t;
00451
00460 POCKETSPHINX_EXPORT
00461 int ngram_model_casefold(ngram_model_t *model, int kase);
00462
00476 POCKETSPHINX_EXPORT
00477 int ngram_model_apply_weights(ngram_model_t *model,
00478     float32 lw, float32 wip);
00479
00488 POCKETSPHINX_EXPORT
00489 float32 ngram_model_get_weights(ngram_model_t *model, int32 *out_log_wip);
00490
00525 POCKETSPHINX_EXPORT
00526 int32 ngram_score(ngram_model_t *model, const char *word, ...);
00527
00533 POCKETSPHINX_EXPORT
00534 int32 ngram_tg_score(ngram_model_t *model,
00535     int32 w3, int32 w2, int32 w1,
00536     int32 *n_used);
00537
00543 POCKETSPHINX_EXPORT
00544 int32 ngram_bg_score(ngram_model_t *model,
00545     int32 w2, int32 w1,
00546     int32 *n_used);
00547
00553 POCKETSPHINX_EXPORT
00554 int32 ngram_ng_score(ngram_model_t *model, int32 wid, int32 *history,
00555     int32 n_hist, int32 *n_used);
00556
00568 POCKETSPHINX_EXPORT
00569 int32 ngram_probv(ngram_model_t *model, const char *word, ...);
00570
00582 POCKETSPHINX_EXPORT
00583 int32 ngram_prob(ngram_model_t *model, const char* const *words, int32 n);
00584
00593 POCKETSPHINX_EXPORT
00594 int32 ngram_ng_prob(ngram_model_t *model, int32 wid, int32 *history,
00595     int32 n_hist, int32 *n_used);
00596
```

```
00609 POCKETSPHINX_EXPORT
00610 int32 ngram_score_to_prob(ngram_model_t *model, int32 score);
00611
00617 POCKETSPHINX_EXPORT
00618 int32 ngram_wid(ngram_model_t *model, const char *word);
00619
00625 POCKETSPHINX_EXPORT
00626 const char *ngram_word(ngram_model_t *model, int32 wid);
00627
00642 POCKETSPHINX_EXPORT
00643 int32 ngram_unknown_wid(ngram_model_t *model);
00644
00650 POCKETSPHINX_EXPORT
00651 int32 ngram_zero(ngram_model_t *model);
00652
00658 POCKETSPHINX_EXPORT
00659 int32 ngram_model_get_size(ngram_model_t *model);
00660
00666 POCKETSPHINX_EXPORT
00667 uint32 const *ngram_model_get_counts(ngram_model_t *model);
00668
00676 typedef struct ngram_iter_s ngram_iter_t;
00677
00687 POCKETSPHINX_EXPORT
00688 ngram_iter_t *ngram_model_mgrams(ngram_model_t *model, int m);
00689
00695 POCKETSPHINX_EXPORT
00696 ngram_iter_t *ngram_iter(ngram_model_t *model, const char *word, ...);
00697
00703 POCKETSPHINX_EXPORT
00704 ngram_iter_t *ngram_ng_iter(ngram_model_t *model, int32 wid, int32 *history, int32 n_hist);
00705
00715 POCKETSPHINX_EXPORT
00716 int32 const *ngram_iter_get(ngram_iter_t *itor,
00717                             int32 *out_score,
00718                             int32 *out_bowt);
00719
00726 POCKETSPHINX_EXPORT
00727 ngram_iter_t *ngram_iter_successors(ngram_iter_t *itor);
00728
00733 POCKETSPHINX_EXPORT
00734 ngram_iter_t *ngram_iter_next(ngram_iter_t *itor);
00735
00740 POCKETSPHINX_EXPORT
00741 void ngram_iter_free(ngram_iter_t *itor);
00742
00756 POCKETSPHINX_EXPORT
00757 int32 ngram_model_add_word(ngram_model_t *model,
00758                           const char *word, float32 weight);
00759
00774 POCKETSPHINX_EXPORT
00775 int32 ngram_model_read_classdef(ngram_model_t *model,
00776                                const char *file_name);
00777
00787 POCKETSPHINX_EXPORT
00788 int32 ngram_model_add_class(ngram_model_t *model,
00789                            const char *classname,
00790                            float32 classweight,
00791                            char **words,
00792                            const float32 *weights,
00793                            int32 n_words);
00794
00805 POCKETSPHINX_EXPORT
00806 int32 ngram_model_add_class_word(ngram_model_t *model,
00807                                 const char *classname,
00808                                 const char *word,
00809                                 float32 weight);
00810
00836 POCKETSPHINX_EXPORT
00837 ngram_model_t *ngram_model_set_init(ps_config_t *config,
00838                                     ngram_model_t **models,
00839                                     char **names,
00840                                     const float32 *weights,
00841                                     int32 n_models);
00842
00874 POCKETSPHINX_EXPORT
00875 ngram_model_t *ngram_model_set_read(ps_config_t *config,
00876                                     const char *lmctlfile,
00877                                     logmath_t *lmath);
00878
00883 POCKETSPHINX_EXPORT
00884 int32 ngram_model_set_count(ngram_model_t *set);
00885
00890 typedef struct ngram_model_set_iter_s ngram_model_set_iter_t;
00891
00898 POCKETSPHINX_EXPORT
00899 ngram_model_set_iter_t *ngram_model_set_iter(ngram_model_t *set);
```

```

00900
00907 POCKETSPHINX_EXPORT
00908 ngram_model_set_iter_t *ngram_model_set_iter_next(ngram_model_set_iter_t *itor);
00909
00914 POCKETSPHINX_EXPORT
00915 void ngram_model_set_iter_free(ngram_model_set_iter_t *itor);
00916
00925 POCKETSPHINX_EXPORT
00926 ngram_model_t *ngram_model_set_iter_model(ngram_model_set_iter_t *itor,
00927                                           char const **lmname);
00928
00936 POCKETSPHINX_EXPORT
00937 ngram_model_t *ngram_model_set_select(ngram_model_t *set,
00938                                       const char *name);
00939
00947 POCKETSPHINX_EXPORT
00948 ngram_model_t *ngram_model_set_lookup(ngram_model_t *set,
00949                                       const char *name);
00950
00955 POCKETSPHINX_EXPORT
00956 const char *ngram_model_set_current(ngram_model_t *set);
00957
00967 POCKETSPHINX_EXPORT
00968 ngram_model_t *ngram_model_set_interp(ngram_model_t *set,
00969                                       const char **names,
00970                                       const float32 *weights);
00971
00985 POCKETSPHINX_EXPORT
00986 ngram_model_t *ngram_model_set_add(ngram_model_t *set,
00987                                    ngram_model_t *model,
00988                                    const char *name,
00989                                    float32 weight,
00990                                    int reuse_widmap);
00991
01001 POCKETSPHINX_EXPORT
01002 ngram_model_t *ngram_model_set_remove(ngram_model_t *set,
01003                                       const char *name,
01004                                       int reuse_widmap);
01005
01010 POCKETSPHINX_EXPORT
01011 void ngram_model_set_map_words(ngram_model_t *set,
01012                               const char **words,
01013                               int32 n_words);
01014
01023 POCKETSPHINX_EXPORT
01024 int32 ngram_model_set_current_wid(ngram_model_t *set,
01025                                   int32 set_wid);
01026
01037 POCKETSPHINX_EXPORT
01038 int32 ngram_model_set_known_wid(ngram_model_t *set, int32 set_wid);
01039
01044 POCKETSPHINX_EXPORT
01045 void ngram_model_flush(ngram_model_t *lm);
01046
01047 #ifdef __cplusplus
01048 }
01049 #endif
01050
01051 #endif /* __PS_MODEL_H__ */

```

11.45 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/prim_type.h File Reference

Basic type definitions used in Sphinx.

```
#include <pocketsphinx/sphinx_config.h>
```

Classes

- union [anytype_s](#)

Macros

- `#define __BIGSTACKVARIABLE__`
- `#define TRUE 1`
- `#define FALSE 0`
- `#define NULL (void *)0`
- `#define MAX_INT32 ((int32) 0x7fffffff)`
- `#define MAX_INT16 ((int16) 0x00007fff)`
- `#define MAX_INT8 ((int8) 0x000000ff)`
- `#define MAX_NEG_INT32 ((int32) 0x80000000)`
- `#define MAX_NEG_INT16 ((int16) 0xffff8000)`
- `#define MAX_NEG_INT8 ((int8) 0xffff80)`
- `#define MAX_UINT32 ((uint32) 0xffffffff)`
- `#define MAX_UINT16 ((uint16) 0x0000ffff)`
- `#define MAX_UINT8 ((uint8) 0x000000ff)`
- `#define MAX_POS_FLOAT32 3.4e+38f`
- `#define MIN_POS_FLOAT32 1.2e-38f /* But not 0 */`
- `#define MAX_POS_FLOAT64 1.8e+307`
- `#define MIN_POS_FLOAT64 2.2e-308`
- `#define MAX_IEEE_NORM_POS_FLOAT32 3.4e+38f`
- `#define MIN_IEEE_NORM_POS_FLOAT32 1.2e-38f`
- `#define MIN_IEEE_NORM_NEG_FLOAT32 -3.4e+38f`
- `#define MAX_IEEE_NORM_POS_FLOAT64 1.8e+307`
- `#define MIN_IEEE_NORM_POS_FLOAT64 2.2e-308`
- `#define MIN_IEEE_NORM_NEG_FLOAT64 -1.8e+307`
- `#define MIN_NEG_FLOAT32 ((float32) (-MIN_POS_FLOAT32))`
- `#define MIN_NEG_FLOAT64 ((float64) (-MIN_POS_FLOAT64))`

Typedefs

- `typedef union anytype_s anytype_t`
- `typedef int int32`
- `typedef short int16`
- `typedef signed char int8`
- `typedef unsigned int uint32`
- `typedef unsigned short uint16`
- `typedef unsigned char uint8`
- `typedef long long int64`
- `typedef unsigned long long uint64`
- `typedef float float32`
- `typedef double float64`

11.45.1 Detailed Description

Basic type definitions used in Sphinx.

11.45.2 Macro Definition Documentation

11.45.2.1 __BIGSTACKVARIABLE__

```
#define __BIGSTACKVARIABLE__
```

11.45.2.2 FALSE

```
#define FALSE 0
```

Referenced by [SpeechToText::listen\(\)](#), and [SpeechManager::startListening\(\)](#).

11.45.2.3 MAX_IEEE_NORM_POS_FLOAT32

```
#define MAX_IEEE_NORM_POS_FLOAT32 3.4e+38f
```

11.45.2.4 MAX_IEEE_NORM_POS_FLOAT64

```
#define MAX_IEEE_NORM_POS_FLOAT64 1.8e+307
```

11.45.2.5 MAX_INT16

```
#define MAX_INT16 ((int16) 0x00007fff)
```

11.45.2.6 MAX_INT32

```
#define MAX_INT32 ((int32) 0x7fffffff)
```

11.45.2.7 MAX_INT8

```
#define MAX_INT8 ((int8) 0x0000007f)
```

11.45.2.8 MAX_NEG_INT16

```
#define MAX_NEG_INT16 ((int16) 0xffff8000)
```

11.45.2.9 MAX_NEG_INT32

```
#define MAX_NEG_INT32 ((int32) 0x80000000)
```

11.45.2.10 MAX_NEG_INT8

```
#define MAX_NEG_INT8 ((int8) 0xfffffff8)
```

11.45.2.11 MAX_POS_FLOAT32

```
#define MAX_POS_FLOAT32 3.4e+38f
```


11.45.2.12 MAX_POS_FLOAT64

```
#define MAX_POS_FLOAT64 1.8e+307
```

11.45.2.13 MAX_UINT16

```
#define MAX_UINT16 ((uint16) 0x0000ffff)
```

11.45.2.14 MAX_UINT32

```
#define MAX_UINT32 ((uint32) 0xffffffff)
```

11.45.2.15 MAX_UINT8

```
#define MAX_UINT8 ((uint8) 0x000000ff)
```

11.45.2.16 MIN_IEEE_NORM_NEG_FLOAT32

```
#define MIN_IEEE_NORM_NEG_FLOAT32 -3.4e+38f
```

11.45.2.17 MIN_IEEE_NORM_NEG_FLOAT64

```
#define MIN_IEEE_NORM_NEG_FLOAT64 -1.8e+307
```

11.45.2.18 MIN_IEEE_NORM_POS_FLOAT32

```
#define MIN_IEEE_NORM_POS_FLOAT32 1.2e-38f
```

11.45.2.19 MIN_IEEE_NORM_POS_FLOAT64

```
#define MIN_IEEE_NORM_POS_FLOAT64 2.2e-308
```

11.45.2.20 MIN_NEG_FLOAT32

```
#define MIN_NEG_FLOAT32 ((float32) (-MIN_POS_FLOAT32))
```

11.45.2.21 MIN_NEG_FLOAT64

```
#define MIN_NEG_FLOAT64 ((float64) (-MIN_POS_FLOAT64))
```

11.45.2.22 MIN_POS_FLOAT32

```
#define MIN_POS_FLOAT32 1.2e-38f /* But not 0 */
```

11.45.2.23 MIN_POS_FLOAT64

```
#define MIN_POS_FLOAT64 2.2e-308
```

11.45.2.24 NULL

```
#define NULL (void *)0
```

Referenced by [SpeechToText::configuration\(\)](#), [SpeechToText::hyplsValid\(\)](#), [SpeechManager::init\(\)](#), [SpeechToText::init\(\)](#), [SpeechToText::listen\(\)](#), and [SpeechManager::startListening\(\)](#).

11.45.2.25 TRUE

```
#define TRUE 1
```

11.45.3 Typedef Documentation

11.45.3.1 anytype_t

```
typedef union anytype\_s anytype_t
```

11.45.3.2 float32

```
typedef float float32
```

11.45.3.3 float64

```
typedef double float64
```

11.45.3.4 int16

```
typedef short int16
```

11.45.3.5 int32

```
typedef int int32
```

11.45.3.6 int64

```
typedef long long int64
```

11.45.3.7 int8

```
typedef signed char int8
```

11.45.3.8 uint16

```
typedef unsigned short uint16
```

11.45.3.9 uint32

```
typedef unsigned int uint32
```

11.45.3.10 uint64

```
typedef unsigned long long uint64
```

11.45.3.11 uint8

```
typedef unsigned char uint8
```

11.46 prim_type.h

[Go to the documentation of this file.](#)

```
00001 /* -*- c-basic-offset: 4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 1999-2004 Carnegie Mellon University. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  * notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  * notice, this list of conditions and the following disclaimer in
00015  * the documentation and/or other materials provided with the
00016  * distribution.
00017  *
00018  * This work was supported in part by funding from the Defense Advanced
00019  * Research Projects Agency and the National Science Foundation of the
00020  * United States of America, and the CMU Sphinx Speech Consortium.
00021  *
00022  * THIS SOFTWARE IS PROVIDED BY CARNEGIE MELLON UNIVERSITY ``AS IS" AND
00023  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00024  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00025  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00026  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00027  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00028  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00029  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
```

```

00030 * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00031 * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00032 * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00033 *
00034 * =====
00035 *
00036 */
00037 /*
00038 * prim_type.h -- Primitive types; more machine-independent.
00039 *
00040 * *****
00041 * CMU ARPA Speech Project
00042 *
00043 * Copyright (c) 1999 Carnegie Mellon University.
00044 * ALL RIGHTS RESERVED.
00045 * *****
00046 *
00047 * HISTORY
00048 * $Log: prim_type.h,v $
00049 * Revision 1.12 2005/10/05 00:31:14 dhdfu
00050 * Make int8 be explicitly signed (signedness of 'char' is
00051 * architecture-dependent). Then make a bunch of things use uint8 where
00052 * signedness is unimportant, because on the architecture where 'char' is
00053 * unsigned, it is that way for a reason (signed chars are slower).
00054 *
00055 * Revision 1.11 2005/06/22 03:10:23 arthchan2003
00056 * Added keyword.
00057 *
00058 * Revision 1.3 2005/03/30 01:22:48 archan
00059 * Fixed mistakes in last updates. Add
00060 *
00061 *
00062 * 12-Mar-1999 M K Ravishankar (rkm@cs.cmu.edu) at Carnegie Mellon
00063 * Added arraysize_t, point_t, fpoint_t.
00064 *
00065 * 01-Feb-1999 M K Ravishankar (rkm@cs.cmu.edu) at Carnegie Mellon
00066 * Added anytype_t.
00067 *
00068 * 08-31-95 M K Ravishankar (rkm@cs.cmu.edu) at Carnegie Mellon
00069 * Created.
00070 */
00071
00072
00073 #ifndef _LIBUTIL_PRIM_TYPE_H_
00074 #define _LIBUTIL_PRIM_TYPE_H_
00075
00076
00080
00081 #ifdef __cplusplus
00082 extern "C" {
00083 #endif
00084 #if 0
00085 } /* Fool Emacs into not indenting things. */
00086 #endif
00087
00088 #include <pocketsphinx/sphinx_config.h>
00089
00090 /* Define some things for VisualDSP++ */
00091 #if defined(__ADSPBLACKFIN__) && !defined(__GNUC__)
00092 # ifnndef HAVE_LONG_LONG
00093 # define HAVE_LONG_LONG
00094 # endif
00095 # ifnndef ssize_t
00096 typedef signed int ssize_t;
00097 # endif
00098 # define SIZEOF_LONG_LONG 8
00099 # define __BIGSTACKVARIABLE__ static
00100 #else /* Not VisualDSP++ */
00101 # define __BIGSTACKVARIABLE__
00102 #endif
00103
00110 typedef union anytype_s {
00111     void *ptr;
00112     long i;
00113     unsigned long ui;
00114     double fl;
00115 } anytype_t;
00116
00117 /* Use C99 types if available */
00118 #if defined(HAVE_STDINT_H) || (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L)
00119 #include <stdint.h>
00120 typedef int32_t int32;
00121 typedef int16_t int16;
00122 typedef int8_t int8;
00123 typedef uint32_t uint32;
00124 typedef uint16_t uint16;
00125 typedef uint8_t uint8;
00126 typedef int64_t int64;

```

```

00127 typedef uint64_t      uint64;
00128 /* Take a wild guess otherwise */
00129 #else
00130 typedef int            int32;
00131 typedef short          int16;
00132 typedef signed char    int8;
00133 typedef unsigned int    uint32;
00134 typedef unsigned short  uint16;
00135 typedef unsigned char   uint8;
00136 # if defined(_MSC_VER)
00137 typedef __int64         int64;
00138 typedef unsigned __int64 uint64;
00139 # else
00140 typedef long long       int64;
00141 typedef unsigned long long uint64;
00142 # endif
00143 #endif /* not C99 or POSIX */
00144
00145 /* We should maybe stop using these as there isn't any good way to
00146    know their exact size, but it's 99% certain they are 32 and 64
00147    bits. */
00148 typedef float          float32;
00149 typedef double          float64;
00150
00151 #ifndef TRUE
00152 #define TRUE 1
00153 #endif
00154 #ifndef FALSE
00155 #define FALSE 0
00156 #endif
00157
00158 #ifndef NULL
00159 #define NULL (void *)0
00160 #endif
00161
00162 /* These really ought to come from <limits.h>, but not everybody has that. */
00163 /* Useful constants */
00164 #define MAX_INT32      ((int32) 0x7fffffff)
00165 #define MAX_INT16      ((int16) 0x00007fff)
00166 #define MAX_INT8       ((int8)  0x0000007f)
00167
00168 #define MAX_NEG_INT32   ((int32) 0x80000000)
00169 #define MAX_NEG_INT16   ((int16) 0xffff8000)
00170 #define MAX_NEG_INT8    ((int8)  0xfffffff8)
00171
00172 #define MAX_UINT32      ((uint32) 0xffffffff)
00173 #define MAX_UINT16      ((uint16) 0x0000ffff)
00174 #define MAX_UINT8       ((uint8)  0x000000ff)
00175
00176 /* The following are approximate; IEEE floating point standards might quibble! */
00177 #define MAX_POS_FLOAT32  3.4e+38f
00178 #define MIN_POS_FLOAT32  1.2e-38f /* But not 0 */
00179 #define MAX_POS_FLOAT64  1.8e+307
00180 #define MIN_POS_FLOAT64  2.2e-308
00181
00182 #define MAX_IEEE_NORM_POS_FLOAT32  3.4e+38f
00183 #define MIN_IEEE_NORM_POS_FLOAT32  1.2e-38f
00184 #define MIN_IEEE_NORM_NEG_FLOAT32 -3.4e+38f
00185 #define MAX_IEEE_NORM_POS_FLOAT64  1.8e+307
00186 #define MIN_IEEE_NORM_POS_FLOAT64  2.2e-308
00187 #define MIN_IEEE_NORM_NEG_FLOAT64 -1.8e+307
00188
00189 /* Will the following really work?? */
00190 #define MIN_NEG_FLOAT32  ((float32) (-MIN_POS_FLOAT32))
00191 #define MIN_NEG_FLOAT64  ((float64) (-MIN_POS_FLOAT64))
00192
00193
00194 #ifdef __cplusplus
00195 }
00196 #endif
00197
00198 #endif

```

11.47 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/search.h File Reference

Search modules.

```

#include <pocketsphinx/model.h>
#include <pocketsphinx/alignment.h>

```

Typedefs

- typedef struct ps_search_iter_s [ps_search_iter_t](#)

11.47.1 Detailed Description

Search modules.

User can configure several "search" objects with different grammars and language models and switch them in runtime to provide interactive experience for the user.

There are different possible search modes:

- keyphrase - efficiently looks for keyphrase and ignores other speech. allows to configure detection threshold.
- grammar - recognizes speech according to JSGF grammar. Unlike keyphrase grammar search doesn't ignore words which are not in grammar but tries to recognize them.
- ngram/lm - recognizes natural speech with a language model.
- allphone - recognizes phonemes with a phonetic language model.
- align - creates time alignments for a fixed word sequence.

Each search module has a name and can be referenced by name. These names are application-specific. The function `ps_activate_search()` activates a search module previously added by a name. Only one search module can be activated at time.

To add the search module one needs to point to the grammar/language model describing the search. The location of the grammar is specific to the application.

The exact design of a searches depends on your application. For example, you might want to listen for activation keyphrase first and once keyphrase is recognized switch to ngram search to recognize actual command. Once you have recognized the command, you can switch to grammar search to recognize the confirmation and then switch back to keyphrase listening mode to wait for another command.

If only a simple recognition is required it is sufficient to add a single search or just configure the required mode with configuration options.

Because Doxygen is Bad Software, the actual function definitions can only exist in [ps_decoder_t](#). Sorry about that.

11.47.2 Typedef Documentation

11.47.2.1 `ps_search_iter_t`

```
typedef struct ps_search_iter_s ps_search_iter_t
```

11.48 search.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset:4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2014 Alpha Cephei Inc.. All rights
00004  * reserved.
00005  *
00006  * Redistribution and use in source and binary forms, with or without
00007  * modification, are permitted provided that the following conditions
00008  * are met:
00009  *
00010  * 1. Redistributions of source code must retain the above copyright
00011  *    notice, this list of conditions and the following disclaimer.
00012  *
00013  * 2. Redistributions in binary form must reproduce the above copyright
00014  *    notice, this list of conditions and the following disclaimer in
00015  *    the documentation and/or other materials provided with the
00016  *    distribution.
00017  *
00018  *
00019  * THIS SOFTWARE IS PROVIDED BY ALPHA CEPHEI INC. ``AS IS" AND
00020  * ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
00021  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
00022  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL CARNEGIE MELLON UNIVERSITY
00023  * NOR ITS EMPLOYEES BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
00024  * SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
00025  * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
00026  * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
00027  * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00028  * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
00029  * OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00030  *
00031  * =====
00032  *
00033  */
00034
00035 #ifndef __PS_SEARCH_H__
00036 #define __PS_SEARCH_H__
00037
00038 #include <pocketsphinx/model.h>
00039 #include <pocketsphinx/alignment.h>
00040
00041 #ifdef __cplusplus
00042 extern "C" {
00043 #endif
00044 #if 0
00045 }
00046 #endif
00047
00048 typedef struct ps_search_iter_s ps_search_iter_t;
00049
00050 /* Forward-declare this because header files are an atrocity. */
00051 typedef struct ps_decoder_s ps_decoder_t;
00052
00053 POCKETSPHINX_EXPORT
00054 int ps_activate_search(ps_decoder_t *ps, const char *name);
00055
00056 POCKETSPHINX_EXPORT
00057 const char* ps_current_search(ps_decoder_t *ps);
00058
00059 POCKETSPHINX_EXPORT
00060 int ps_remove_search(ps_decoder_t *ps, const char *name);
00061
00062 POCKETSPHINX_EXPORT
00063 ps_search_iter_t *ps_search_iter(ps_decoder_t *ps);
00064
00065 POCKETSPHINX_EXPORT
00066 ps_search_iter_t *ps_search_iter_next(ps_search_iter_t *itor);
00067
00068 POCKETSPHINX_EXPORT
00069 const char* ps_search_iter_val(ps_search_iter_t *itor);
00070
00071 POCKETSPHINX_EXPORT
00072 void ps_search_iter_free(ps_search_iter_t *itor);
00073
00074 POCKETSPHINX_EXPORT
00075 const char* ps_search_iter_val(ps_search_iter_t *itor);
00076
00077 POCKETSPHINX_EXPORT
00078 ngram_model_t *ps_get_lm(ps_decoder_t *ps, const char *name);
00079
00080 POCKETSPHINX_EXPORT
00081 int ps_add_lm(ps_decoder_t *ps, const char *name, ngram_model_t *lm);
00082

```

```

00217 POCKETSPHINX_EXPORT
00218 int ps_add_lm_file(ps_decoder_t *ps, const char *name, const char *path);
00219
00228 POCKETSPHINX_EXPORT
00229 fsg_model_t *ps_get_fsg(ps_decoder_t *ps, const char *name);
00230
00240 POCKETSPHINX_EXPORT
00241 int ps_add_fsg(ps_decoder_t *ps, const char *name, fsg_model_t *fsg);
00242
00251 POCKETSPHINX_EXPORT
00252 int ps_add_jsgf_file(ps_decoder_t *ps, const char *name, const char *path);
00253
00262 POCKETSPHINX_EXPORT
00263 int ps_add_jsgf_string(ps_decoder_t *ps, const char *name, const char *jsgf_string);
00264
00273 POCKETSPHINX_EXPORT
00274 const char* ps_get_kws(ps_decoder_t *ps, const char *name);
00275
00285 POCKETSPHINX_EXPORT
00286 int ps_add_kws(ps_decoder_t *ps, const char *name, const char *keyfile);
00287
00297 POCKETSPHINX_EXPORT
00298 int ps_add_keyphrase(ps_decoder_t *ps, const char *name, const char *keyphrase);
00299
00309 POCKETSPHINX_EXPORT
00310 int ps_add_allphone(ps_decoder_t *ps, const char *name, ngram_model_t *lm);
00311
00320 POCKETSPHINX_EXPORT
00321 int ps_add_allphone_file(ps_decoder_t *ps, const char *name, const char *path);
00322
00345 POCKETSPHINX_EXPORT
00346 int ps_set_align_text(ps_decoder_t *ps, const char *words);
00347
00374 POCKETSPHINX_EXPORT
00375 int ps_set_alignment(ps_decoder_t *ps, ps_alignment_t *al);
00376
00390 POCKETSPHINX_EXPORT
00391 ps_alignment_t *ps_get_alignment(ps_decoder_t *ps);
00392
00393 #ifdef __cplusplus
00394 }
00395 #endif
00396
00397 #endif /* __PS_SEARCH_H__ */

```

11.49 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/include/pocketsphinx/vad.h File Reference

Simple voice activity detection.

```

#include <pocketsphinx/prim_type.h>
#include <pocketsphinx/export.h>

```

Macros

- #define [PS_VAD_DEFAULT_SAMPLE_RATE](#) 16000
- #define [PS_VAD_DEFAULT_FRAME_LENGTH](#) 0.03
- #define [ps_vad_frame_length](#)(vad)

Typedefs

- typedef struct ps_vad_s [ps_vad_t](#)
- typedef enum [ps_vad_mode_e](#) [ps_vad_mode_t](#)
- typedef enum [ps_vad_class_e](#) [ps_vad_class_t](#)

Enumerations

- enum [ps_vad_mode_e](#) { [PS_VAD_LOOSE](#) = 0 , [PS_VAD_MEDIUM_LOOSE](#) = 1 , [PS_VAD_MEDIUM_STRICT](#) = 2 , [PS_VAD_STRICT](#) = 3 }
Voice activity detection "aggressiveness" levels.
- enum [ps_vad_class_e](#) { [PS_VAD_ERROR](#) = -1 , [PS_VAD_NOT_SPEECH](#) = 0 , [PS_VAD_SPEECH](#) = 1 }
Classification of input frames returned by [ps_vad_classify\(\)](#).

11.49.1 Detailed Description

Simple voice activity detection.

Because doxygen is Bad Software, the actual documentation can only exist in [ps_vad_t](#). Sorry about that.

11.49.2 Macro Definition Documentation

11.49.2.1 PS_VAD_DEFAULT_FRAME_LENGTH

```
#define PS_VAD_DEFAULT_FRAME_LENGTH 0.03
```

Default frame length for voice activity detector

11.49.2.2 PS_VAD_DEFAULT_SAMPLE_RATE

```
#define PS_VAD_DEFAULT_SAMPLE_RATE 16000
```

Default sampling rate for voice activity detector

11.49.2.3 ps_vad_frame_length

```
#define ps_vad_frame_length(  
    vad)
```

Value:

```
((double)ps_vad_frame_size(vad) / ps_vad_sample_rate(vad))
```

Get the *actual* length of a frame in seconds.

This may differ from the value requested in [ps_vad_set_input_params\(\)](#).

11.49.3 Typedef Documentation

11.49.3.1 ps_vad_class_t

```
typedef enum ps\_vad\_class\_e ps\_vad\_class\_t
```

11.49.3.2 ps_vad_mode_t

```
typedef enum ps_vad_mode_e ps_vad_mode_t
```

11.49.3.3 ps_vad_t

```
typedef struct ps_vad_s ps_vad_t
```

11.49.4 Enumeration Type Documentation

11.49.4.1 ps_vad_class_e

```
enum ps_vad_class_e
```

Classification of input frames returned by `ps_vad_classify()`.

[pocketsphinx/vad.h](http://pocketsphinx.org/vad.h)

Enumerator

PS_VAD_ERROR	
PS_VAD_NOT_SPEECH	
PS_VAD_SPEECH	

11.49.4.2 ps_vad_mode_e

```
enum ps_vad_mode_e
```

Voice activity detection "aggressiveness" levels.

[pocketsphinx/vad.h](http://pocketsphinx.org/vad.h)

Enumerator

PS_VAD_LOOSE	
PS_VAD_MEDIUM_LOOSE	
PS_VAD_MEDIUM_STRICT	
PS_VAD_STRICT	

11.50 vad.h

[Go to the documentation of this file.](#)

```

00001 /* -*- c-basic-offset:4; indent-tabs-mode: nil -*- */
00002 /* =====
00003  * Copyright (c) 2022 David Huggins-Daines. All rights reserved.
00004  *
00005  * Redistribution and use in source and binary forms, with or without
00006  * modification, are permitted provided that the following conditions
00007  * are met:
00008  *
00009  * 1. Redistributions of source code must retain the above copyright
00010  * notice, this list of conditions and the following disclaimer.
00011  *
00012  * 2. Redistributions in binary form must reproduce the above copyright
00013  * notice, this list of conditions and the following disclaimer in
00014  * the documentation and/or other materials provided with the
00015  * distribution.
00016  *
00017  * THIS SOFTWARE IS PROVIDED ``AS IS'' AND ANY EXPRESSED OR IMPLIED
00018  * WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
00019  * OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
00020  * DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT,
00021  * INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
00022  * (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
00023  * SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
00024  * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
00025  * STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
00026  * ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
00027  * OF THE POSSIBILITY OF SUCH DAMAGE.
00028  * =====
00029  */
00037
00038 #ifndef __PS_VAD_H__
00039 #define __PS_VAD_H__
00040
00041
00042 #include <pocketsphinx/prim_type.h>
00043 #include <pocketsphinx/export.h>
00044
00045 #ifdef __cplusplus
00046 extern "C" {
00047 #endif
00048 #if 0
00049 }
00050 #endif
00051
00056 typedef struct ps_vad_s ps_vad_t;
00057
00062 typedef enum ps_vad_mode_e {
00063     PS_VAD_LOOSE = 0,
00064     PS_VAD_MEDIUM_LOOSE = 1,
00065     PS_VAD_MEDIUM_STRICT = 2,
00066     PS_VAD_STRICT = 3
00067 } ps_vad_mode_t;
00068
00073 typedef enum ps_vad_class_e {
00074     PS_VAD_ERROR = -1,
00075     PS_VAD_NOT_SPEECH = 0,
00076     PS_VAD_SPEECH = 1
00077 } ps_vad_class_t;
00078
00082 #define PS_VAD_DEFAULT_SAMPLE_RATE 16000
00086 #define PS_VAD_DEFAULT_FRAME_LENGTH 0.03
00087
00105 POCKETSPHINX_EXPORT
00106 ps_vad_t *ps_vad_init(ps_vad_mode_t mode, int sample_rate, double frame_length);
00107
00115 POCKETSPHINX_EXPORT
00116 ps_vad_t *ps_vad_retain(ps_vad_t *vad);
00117
00125 POCKETSPHINX_EXPORT
00126 int ps_vad_free(ps_vad_t *vad);
00127
00146 POCKETSPHINX_EXPORT
00147 int ps_vad_set_input_params(ps_vad_t *vad, int sample_rate, double frame_length);
00148
00156 POCKETSPHINX_EXPORT
00157 int ps_vad_sample_rate(ps_vad_t *vad);
00158
00170 POCKETSPHINX_EXPORT
00171 size_t ps_vad_frame_size(ps_vad_t *vad);
00172
00178 #define ps_vad_frame_length(vad) ((double)ps_vad_frame_size(vad) / ps_vad_sample_rate(vad))
00179

```

```

00190 POCKETSPHINX_EXPORT
00191 ps_vad_class_t ps_vad_classify(ps_vad_t *vad, const int16 *frame);
00192
00193 #ifdef __cplusplus
00194 }
00195 #endif
00196
00197 #endif /* __PS_VAD_H__ */

```

11.51 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/main.cpp File Reference

```

#include "TicTacToeGame.h"
#include "TitleScreen.h"
#include "ConfigManager.h"
#include <QApplication>

```

Functions

- int [main](#) (int argc, char *argv[])

11.51.1 Function Documentation

11.51.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

References [TitleScreen::exited\(\)](#), [ConfigManager::instance\(\)](#), and [ConfigManager::load\(\)](#).

11.52 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToe↵ Test/main.cpp File Reference

```

#include "tst_board.h"
#include "tst_minimaxagent.h"
#include <gtest/gtest.h>

```

Functions

- int [main](#) (int argc, char *argv[])

11.52.1 Function Documentation

11.52.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

11.53 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model/Board.cpp File Reference

```
#include "Board.h"
#include <QString>
#include <stdexcept>
```

11.54 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Model/Board.h File Reference

```
#include "TTTCommonTypes.h"
#include <QtGlobal>
#include <vector>
```

Classes

- class [Board](#)

The [Board](#) class is responsible for representing the underlying model of the game, it is a 2D matrix representation of the GUI that is machine readable, and can be used when running any sort of algorithms on the board.

11.55 Board.h

[Go to the documentation of this file.](#)

```
00001 #ifndef BOARD_H
00002 #define BOARD_H
00003 #include "TTTCommonTypes.h"
00004 #include <QtGlobal>
00005 #include <vector>
00006
00007 using std::vector;
00008
00009
00010
00011
00012
00013
00014
00015
00016 class Board
00017 {
00018
00019 protected: // Data
00020     vector<vector<BoardMarks>> board_;
00021     size_t boardSize_;
00022
00023
00024
00025
00026
00027
00028
00029
00030
00031 public:
00032     Board(size_t size);
00033     Board(Board& b);
00034
00035
00036
00037
00038
00039
00040
00041
00042
```

```

00043     Board(const Board& b);
00048     Board(Board&& b);
00055     Board& operator=(Board b);
00059     virtual ~Board();
00065     virtual BoardState evaluateBoard() const;
00076     virtual bool setPlayerInput(size_t row, size_t col, BoardMarks currentPlayer);
00084     virtual BoardMarks at(size_t row, size_t col) const;
00088     virtual void reset();
00093     virtual void resetCell(size_t row, size_t col);
00098     size_t size() const;
00099 #ifdef QT_DEBUG
00100     virtual void printBoard () const;
00101 #endif
00102 };
00103
00104 #endif // BOARD_H

```

11.56 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network/↵ NetworkManager.cpp File Reference

```

#include "NetworkManager.h"
#include <QDebug>

```

11.57 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Network/↵ NetworkManager.h File Reference

```

#include <QObject>
#include <QTcpSocket>
#include <QTimer>

```

Classes

- class [NetworkManager](#)

11.58 NetworkManager.h

[Go to the documentation of this file.](#)

```

00001 #ifndef NETWORKMANAGER_H
00002 #define NETWORKMANAGER_H
00003 #pragma once
00004 #include <QObject>
00005 #include <QTcpSocket>
00006 #include <QTimer>
00007
00008 class NetworkManager : public QObject
00009 {
00010     Q_OBJECT
00011
00012 public:
00013     explicit NetworkManager(QObject *parent = nullptr);
00014
00015     void connectToServer(const QString &ip, quint16 port);
00016     void sendMessage(const QString &message);
00017
00018 signals:
00019     void connected();
00020     void disconnected();
00021     void errorOccurred(const QString &error);

```

```
00022     void messageSent(const QString &message);
00023
00024 private slots:
00025     void onConnected();
00026     void onDisconnected();
00027     void onError(QAbstractSocket::SocketError socketError);
00028     void onSocketReadyRead();
00029
00030 private:
00031     QTcpSocket *socket_;
00032 };
00033
00034 #endif // NETWORKMANAGER_H
```

11.59 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech/SpeechManager.cpp File Reference ↩

```
#include "SpeechManager.h"
```

11.60 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/Speech/SpeechManager.h File Reference ↩

```
#include <QObject>
#include <QString>
#include <QDebug>
#include <QtConcurrent/QtConcurrentRun>
#include <QFuture>
#include <cstdint>
#include <portaudio.h>
#include <pocketsphinx.h>
#include <signal.h>
#include "ConfigManager.h"
```

Classes

- class [SpeechManager](#)

11.61 SpeechManager.h

[Go to the documentation of this file.](#)

```
00001 #ifndef SPEECHMANAGER_H
00002 #define SPEECHMANAGER_H
00003 #include <QObject>
00004 #include <QString>
00005 #include <QDebug>
00006 #include <QtConcurrent/QtConcurrentRun>
00007 #include <QFuture>
00008 #include <cstdint>
00009 #include <portaudio.h>
00010 #include <pocketsphinx.h>
00011 #include <signal.h>
00012 #include "ConfigManager.h"
00013
00014
00015 class SpeechManager : public QObject
```

```

00016 {
00017     Q_OBJECT
00018 public:
00019     explicit SpeechManager(QObject *parent = nullptr);
00020     ~SpeechManager();
00021     void configure();
00022     void init();
00023     void startListening();
00024 signals:
00025     void speechRecognized(const QString &text);
00026     void speechReadyToProcess(const char *hyp);
00027
00028 private slots:
00029     void processRecognitionResult(const char *hyp);
00030 private:
00031     PaStream *stream_;
00032     PaError err_;
00033     ps_decoder_t *decoder_;
00034     ps_config_t *config_;
00035     ps_endpoint_t *ep_;
00036     short *frame_;
00037     size_t frame_size_;
00038     const char *hyp_;
00039     bool hypIsValid();
00040     void freeHeap();
00041 };
00042 #endif // SPEECHMANAGER_H

```

11.62 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToe↵ Test/tst_board.h File Reference

```

#include <gtest/gtest.h>
#include <gmock/gmock-matchers.h>
#include "Board.h"

```

Functions

- [TEST](#) (BoardTest, resetBoard)
- [TEST](#) (BoardTest, evaluateBoardXWins3x3)
- [TEST](#) (BoardTest, evaluateBoardOWins3x3)
- [TEST](#) (BoardTest, evaluateBoardTie3x3)
- [TEST](#) (BoardTest, evaluateBoardNoWinner3x3)
- [TEST](#) (BoardTest, setPlayerInputValid)
- [TEST](#) (BoardTest, setPlayerInputOutOfBounds)
- [TEST](#) (BoardTest, setPlayerInputOverwrite)
- [TEST](#) (BoardTest, setPlayerInputAfterGameOver)
- [TEST](#) (BoardTest, boardAccessValid)
- [TEST](#) (BoardTest, boardAccessInvalid)

11.62.1 Function Documentation

11.62.1.1 [TEST\(\)](#) [1/11]

```

TEST (
    BoardTest ,
    boardAccessInvalid )

```

References [Board::at\(\)](#).

11.62.1.2 TEST() [2/11]

```
TEST (
    BoardTest ,
    boardAccessValid )
```

References [Board::at\(\)](#), [Empty](#), [O](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.3 TEST() [3/11]

```
TEST (
    BoardTest ,
    evaluateBoardNoWinner3x3 )
```

References [Empty](#), [Board::evaluateBoard\(\)](#), [NoWinner](#), [O](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.4 TEST() [4/11]

```
TEST (
    BoardTest ,
    evaluateBoardOWins3x3 )
```

References [Empty](#), [Board::evaluateBoard\(\)](#), [O](#), [OWins](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.5 TEST() [5/11]

```
TEST (
    BoardTest ,
    evaluateBoardTie3x3 )
```

References [Board::evaluateBoard\(\)](#), [O](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), [Tie](#), and [X](#).

11.62.1.6 TEST() [6/11]

```
TEST (
    BoardTest ,
    evaluateBoardXWins3x3 )
```

References [Empty](#), [Board::evaluateBoard\(\)](#), [O](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), [X](#), and [XWins](#).

11.62.1.7 TEST() [7/11]

```
TEST (
    BoardTest ,
    resetBoard )
```

References [Board::at\(\)](#), [Empty](#), [O](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.8 TEST() [8/11]

```
TEST (
    BoardTest ,
    setPlayerInputAfterGameOver )
```

References [Empty](#), [O](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.9 TEST() [9/11]

```
TEST (
    BoardTest ,
    setPlayerInputOutOfBounds )
```

References [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.10 TEST() [10/11]

```
TEST (
    BoardTest ,
    setPlayerInputOverwrite )
```

References [O](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.62.1.11 TEST() [11/11]

```
TEST (
    BoardTest ,
    setPlayerInputValid )
```

References [Board::setPlayerInput\(\)](#), and [X](#).

11.63 tst_board.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include <gtest/gtest.h>
00004 #include <gmock/gmock-matchers.h>
00005 #include "Board.h"
00006
00007 using namespace testing;
00008
00009 TEST(BoardTest, resetBoard)
00010 {
00011     Board board(3);
00012
00013     board.setPlayerInput(0, 0, BoardMarks::X);
00014     board.setPlayerInput(0, 1, BoardMarks::X);
00015     board.setPlayerInput(0, 2, BoardMarks::X);
00016
00017     board.setPlayerInput(1, 0, BoardMarks::O);
00018     board.setPlayerInput(1, 1, BoardMarks::O);
00019     board.setPlayerInput(1, 2, BoardMarks::X);
00020
00021     board.setPlayerInput(2, 0, BoardMarks::Empty);
00022     board.setPlayerInput(2, 1, BoardMarks::O);
```

```

00023     board.setPlayerInput(2, 2, BoardMarks::Empty);
00024
00025     board.reset();
00026
00027     EXPECT_EQ(board.at(0, 0), BoardMarks::Empty);
00028     EXPECT_EQ(board.at(0, 1), BoardMarks::Empty);
00029     EXPECT_EQ(board.at(0, 2), BoardMarks::Empty);
00030
00031     EXPECT_EQ(board.at(1, 0), BoardMarks::Empty);
00032     EXPECT_EQ(board.at(1, 1), BoardMarks::Empty);
00033     EXPECT_EQ(board.at(1, 2), BoardMarks::Empty);
00034
00035     EXPECT_EQ(board.at(2, 0), BoardMarks::Empty);
00036     EXPECT_EQ(board.at(2, 1), BoardMarks::Empty);
00037     EXPECT_EQ(board.at(2, 2), BoardMarks::Empty);
00038 }
00039
00040 TEST(BoardTest, evaluateBoardXWins3x3)
00041 {
00042     Board board(3);
00043
00044     board.setPlayerInput(0, 0, BoardMarks::X);
00045     board.setPlayerInput(0, 1, BoardMarks::X);
00046     board.setPlayerInput(0, 2, BoardMarks::X);
00047
00048     board.setPlayerInput(1, 0, BoardMarks::O);
00049     board.setPlayerInput(1, 1, BoardMarks::O);
00050     board.setPlayerInput(1, 2, BoardMarks::X);
00051
00052     board.setPlayerInput(2, 0, BoardMarks::Empty);
00053     board.setPlayerInput(2, 1, BoardMarks::O);
00054     board.setPlayerInput(2, 2, BoardMarks::Empty);
00055
00056     /* X X X
00057      * O O X
00058      * - O - */
00059     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00060
00061     board.reset();
00062
00063     board.setPlayerInput(0, 0, BoardMarks::O);
00064     board.setPlayerInput(0, 1, BoardMarks::O);
00065     board.setPlayerInput(0, 2, BoardMarks::X);
00066
00067     board.setPlayerInput(1, 0, BoardMarks::X);
00068     board.setPlayerInput(1, 1, BoardMarks::X);
00069     board.setPlayerInput(1, 2, BoardMarks::X);
00070
00071     board.setPlayerInput(2, 0, BoardMarks::Empty);
00072     board.setPlayerInput(2, 1, BoardMarks::O);
00073     board.setPlayerInput(2, 2, BoardMarks::Empty);
00074
00075     /* O O X
00076      * X X X
00077      * - O - */
00078     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00079
00080     board.reset();
00081
00082     board.setPlayerInput(0, 0, BoardMarks::O);
00083     board.setPlayerInput(0, 1, BoardMarks::O);
00084     board.setPlayerInput(0, 2, BoardMarks::X);
00085
00086     board.setPlayerInput(1, 0, BoardMarks::Empty);
00087     board.setPlayerInput(1, 1, BoardMarks::O);
00088     board.setPlayerInput(1, 2, BoardMarks::Empty);
00089
00090     board.setPlayerInput(2, 0, BoardMarks::X);
00091     board.setPlayerInput(2, 1, BoardMarks::X);
00092     board.setPlayerInput(2, 2, BoardMarks::X);
00093
00094     /* O O X
00095      * - O -
00096      * X X X */
00097     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00098
00099     board.reset();
00100
00101     board.setPlayerInput(0, 0, BoardMarks::X);
00102     board.setPlayerInput(0, 1, BoardMarks::O);
00103     board.setPlayerInput(0, 2, BoardMarks::O);
00104
00105     board.setPlayerInput(1, 0, BoardMarks::X);
00106     board.setPlayerInput(1, 1, BoardMarks::O);
00107     board.setPlayerInput(1, 2, BoardMarks::X);
00108
00109     board.setPlayerInput(2, 0, BoardMarks::X);

```

```

00110     board.setPlayerInput(2, 1, BoardMarks::X);
00111     board.setPlayerInput(2, 2, BoardMarks::O);
00112
00113     /* X O O
00114      * X O X
00115      * X X O */
00116     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00117
00118     board.reset();
00119
00120     board.setPlayerInput(0, 0, BoardMarks::O);
00121     board.setPlayerInput(0, 1, BoardMarks::X);
00122     board.setPlayerInput(0, 2, BoardMarks::O);
00123
00124     board.setPlayerInput(1, 0, BoardMarks::O);
00125     board.setPlayerInput(1, 1, BoardMarks::X);
00126     board.setPlayerInput(1, 2, BoardMarks::X);
00127
00128     board.setPlayerInput(2, 0, BoardMarks::X);
00129     board.setPlayerInput(2, 1, BoardMarks::X);
00130     board.setPlayerInput(2, 2, BoardMarks::O);
00131
00132     /* O X O
00133      * O X X
00134      * X X O */
00135     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00136
00137     board.reset();
00138
00139     board.setPlayerInput(0, 0, BoardMarks::O);
00140     board.setPlayerInput(0, 1, BoardMarks::O);
00141     board.setPlayerInput(0, 2, BoardMarks::X);
00142
00143     board.setPlayerInput(1, 0, BoardMarks::O);
00144     board.setPlayerInput(1, 1, BoardMarks::X);
00145     board.setPlayerInput(1, 2, BoardMarks::X);
00146
00147     board.setPlayerInput(2, 0, BoardMarks::X);
00148     board.setPlayerInput(2, 1, BoardMarks::O);
00149     board.setPlayerInput(2, 2, BoardMarks::X);
00150
00151     /* O O X
00152      * O X X
00153      * X O X */
00154     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00155
00156     board.reset();
00157
00158     board.setPlayerInput(0, 0, BoardMarks::X);
00159     board.setPlayerInput(0, 1, BoardMarks::O);
00160     board.setPlayerInput(0, 2, BoardMarks::O);
00161
00162     board.setPlayerInput(1, 0, BoardMarks::O);
00163     board.setPlayerInput(1, 1, BoardMarks::X);
00164     board.setPlayerInput(1, 2, BoardMarks::X);
00165
00166     board.setPlayerInput(2, 0, BoardMarks::X);
00167     board.setPlayerInput(2, 1, BoardMarks::O);
00168     board.setPlayerInput(2, 2, BoardMarks::X);
00169
00170     /* X O O
00171      * O X X
00172      * X O X */
00173     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00174
00175     board.setPlayerInput(0, 0, BoardMarks::O);
00176     board.setPlayerInput(0, 1, BoardMarks::O);
00177     board.setPlayerInput(0, 2, BoardMarks::X);
00178
00179     board.setPlayerInput(1, 0, BoardMarks::O);
00180     board.setPlayerInput(1, 1, BoardMarks::X);
00181     board.setPlayerInput(1, 2, BoardMarks::X);
00182
00183     board.setPlayerInput(2, 0, BoardMarks::X);
00184     board.setPlayerInput(2, 1, BoardMarks::O);
00185     board.setPlayerInput(2, 2, BoardMarks::X);
00186
00187     /* O O X
00188      * O X X
00189      * X O X */
00190     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00191 }
00192
00193 TEST(BoardTest, evaluateBoardOWins3x3)
00194 {
00195     Board board(3);
00196

```

```

00197     board.setPlayerInput(0, 0, BoardMarks::O);
00198     board.setPlayerInput(0, 1, BoardMarks::O);
00199     board.setPlayerInput(0, 2, BoardMarks::O);
00200
00201     board.setPlayerInput(1, 0, BoardMarks::X);
00202     board.setPlayerInput(1, 1, BoardMarks::X);
00203     board.setPlayerInput(1, 2, BoardMarks::O);
00204
00205     board.setPlayerInput(2, 0, BoardMarks::Empty);
00206     board.setPlayerInput(2, 1, BoardMarks::X);
00207     board.setPlayerInput(2, 2, BoardMarks::Empty);
00208
00209     /* O O O
00210      * X X O
00211      * - X - */
00212     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00213
00214     board.reset();
00215
00216     board.setPlayerInput(0, 0, BoardMarks::X);
00217     board.setPlayerInput(0, 1, BoardMarks::X);
00218     board.setPlayerInput(0, 2, BoardMarks::O);
00219
00220     board.setPlayerInput(1, 0, BoardMarks::O);
00221     board.setPlayerInput(1, 1, BoardMarks::O);
00222     board.setPlayerInput(1, 2, BoardMarks::O);
00223
00224     board.setPlayerInput(2, 0, BoardMarks::Empty);
00225     board.setPlayerInput(2, 1, BoardMarks::X);
00226     board.setPlayerInput(2, 2, BoardMarks::Empty);
00227
00228     /* X X O
00229      * O O O
00230      * - X - */
00231     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00232
00233     board.reset();
00234
00235     board.setPlayerInput(0, 0, BoardMarks::X);
00236     board.setPlayerInput(0, 1, BoardMarks::X);
00237     board.setPlayerInput(0, 2, BoardMarks::O);
00238
00239     board.setPlayerInput(1, 0, BoardMarks::Empty);
00240     board.setPlayerInput(1, 1, BoardMarks::X);
00241     board.setPlayerInput(1, 2, BoardMarks::Empty);
00242
00243     board.setPlayerInput(2, 0, BoardMarks::O);
00244     board.setPlayerInput(2, 1, BoardMarks::O);
00245     board.setPlayerInput(2, 2, BoardMarks::O);
00246
00247     /* X X O
00248      * - X -
00249      * O O O */
00250     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00251
00252     board.reset();
00253
00254     board.setPlayerInput(0, 0, BoardMarks::O);
00255     board.setPlayerInput(0, 1, BoardMarks::X);
00256     board.setPlayerInput(0, 2, BoardMarks::X);
00257
00258     board.setPlayerInput(1, 0, BoardMarks::O);
00259     board.setPlayerInput(1, 1, BoardMarks::X);
00260     board.setPlayerInput(1, 2, BoardMarks::O);
00261
00262     board.setPlayerInput(2, 0, BoardMarks::O);
00263     board.setPlayerInput(2, 1, BoardMarks::O);
00264     board.setPlayerInput(2, 2, BoardMarks::X);
00265
00266     /* O X X
00267      * O X O
00268      * O O X */
00269     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00270
00271     board.reset();
00272
00273     board.setPlayerInput(0, 0, BoardMarks::X);
00274     board.setPlayerInput(0, 1, BoardMarks::O);
00275     board.setPlayerInput(0, 2, BoardMarks::X);
00276
00277     board.setPlayerInput(1, 0, BoardMarks::X);
00278     board.setPlayerInput(1, 1, BoardMarks::O);
00279     board.setPlayerInput(1, 2, BoardMarks::O);
00280
00281     board.setPlayerInput(2, 0, BoardMarks::O);
00282     board.setPlayerInput(2, 1, BoardMarks::O);
00283     board.setPlayerInput(2, 2, BoardMarks::X);

```

```

00284
00285     /* X O X
00286      * X O O
00287      * O O X */
00288     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00289
00290     board.reset();
00291
00292     board.setPlayerInput(0, 0, BoardMarks::X);
00293     board.setPlayerInput(0, 1, BoardMarks::X);
00294     board.setPlayerInput(0, 2, BoardMarks::O);
00295
00296     board.setPlayerInput(1, 0, BoardMarks::X);
00297     board.setPlayerInput(1, 1, BoardMarks::O);
00298     board.setPlayerInput(1, 2, BoardMarks::O);
00299
00300     board.setPlayerInput(2, 0, BoardMarks::O);
00301     board.setPlayerInput(2, 1, BoardMarks::X);
00302     board.setPlayerInput(2, 2, BoardMarks::O);
00303
00304     /* X X O
00305      * X O O
00306      * O X O */
00307     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00308
00309     board.reset();
00310
00311     board.setPlayerInput(0, 0, BoardMarks::O);
00312     board.setPlayerInput(0, 1, BoardMarks::X);
00313     board.setPlayerInput(0, 2, BoardMarks::X);
00314
00315     board.setPlayerInput(1, 0, BoardMarks::X);
00316     board.setPlayerInput(1, 1, BoardMarks::O);
00317     board.setPlayerInput(1, 2, BoardMarks::O);
00318
00319     board.setPlayerInput(2, 0, BoardMarks::O);
00320     board.setPlayerInput(2, 1, BoardMarks::X);
00321     board.setPlayerInput(2, 2, BoardMarks::O);
00322
00323     /* O X X
00324      * X O O
00325      * O X O */
00326     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00327
00328     board.setPlayerInput(0, 0, BoardMarks::X);
00329     board.setPlayerInput(0, 1, BoardMarks::X);
00330     board.setPlayerInput(0, 2, BoardMarks::O);
00331
00332     board.setPlayerInput(1, 0, BoardMarks::X);
00333     board.setPlayerInput(1, 1, BoardMarks::O);
00334     board.setPlayerInput(1, 2, BoardMarks::O);
00335
00336     board.setPlayerInput(2, 0, BoardMarks::O);
00337     board.setPlayerInput(2, 1, BoardMarks::X);
00338     board.setPlayerInput(2, 2, BoardMarks::O);
00339
00340     /* X X O
00341      * X O O
00342      * O X O */
00343     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00344 }
00345
00346 TEST(BoardTest, evaluateBoardTie3x3){
00347     Board board(3);
00348
00349     board.setPlayerInput(0, 0, BoardMarks::X);
00350     board.setPlayerInput(0, 1, BoardMarks::O);
00351     board.setPlayerInput(0, 2, BoardMarks::X);
00352
00353     board.setPlayerInput(1, 0, BoardMarks::O);
00354     board.setPlayerInput(1, 1, BoardMarks::X);
00355     board.setPlayerInput(1, 2, BoardMarks::X);
00356
00357     board.setPlayerInput(2, 0, BoardMarks::O);
00358     board.setPlayerInput(2, 1, BoardMarks::X);
00359     board.setPlayerInput(2, 2, BoardMarks::O);
00360
00361     /* X O X
00362      * O X X
00363      * O X O */
00364     EXPECT_EQ(BoardState::Tie, board.evaluateBoard());
00365
00366     board.reset();
00367
00368     board.setPlayerInput(0, 0, BoardMarks::O);
00369     board.setPlayerInput(0, 1, BoardMarks::X);
00370     board.setPlayerInput(0, 2, BoardMarks::O);

```

```

00371
00372     board.setPlayerInput(1, 0, BoardMarks::X);
00373     board.setPlayerInput(1, 1, BoardMarks::O);
00374     board.setPlayerInput(1, 2, BoardMarks::X);
00375
00376     board.setPlayerInput(2, 0, BoardMarks::X);
00377     board.setPlayerInput(2, 1, BoardMarks::O);
00378     board.setPlayerInput(2, 2, BoardMarks::X);
00379
00380     /* O X O
00381      * X O X
00382      * X O X */
00383     EXPECT_EQ(BoardState::Tie, board.evaluateBoard());
00384
00385     board.reset();
00386
00387     board.setPlayerInput(0, 0, BoardMarks::X);
00388     board.setPlayerInput(0, 1, BoardMarks::X);
00389     board.setPlayerInput(0, 2, BoardMarks::O);
00390
00391     board.setPlayerInput(1, 0, BoardMarks::O);
00392     board.setPlayerInput(1, 1, BoardMarks::X);
00393     board.setPlayerInput(1, 2, BoardMarks::X);
00394
00395     board.setPlayerInput(2, 0, BoardMarks::X);
00396     board.setPlayerInput(2, 1, BoardMarks::O);
00397     board.setPlayerInput(2, 2, BoardMarks::O);
00398
00399     /* X X O
00400      * O X X
00401      * X O O */
00402     EXPECT_EQ(BoardState::Tie, board.evaluateBoard());
00403 }
00404
00405 TEST(BoardTest, evaluateBoardNoWinner3x3) {
00406     Board board(3);
00407
00408     board.setPlayerInput(0, 0, BoardMarks::X);
00409     board.setPlayerInput(0, 1, BoardMarks::O);
00410     board.setPlayerInput(0, 2, BoardMarks::X);
00411
00412     board.setPlayerInput(1, 0, BoardMarks::O);
00413     board.setPlayerInput(1, 1, BoardMarks::X);
00414     board.setPlayerInput(1, 2, BoardMarks::X);
00415
00416     board.setPlayerInput(2, 0, BoardMarks::Empty);
00417     board.setPlayerInput(2, 1, BoardMarks::X);
00418     board.setPlayerInput(2, 2, BoardMarks::O);
00419
00420     /* X O X
00421      * O X X
00422      * - X O */
00423     EXPECT_EQ(BoardState::NoWinner, board.evaluateBoard());
00424
00425     board.reset();
00426
00427     board.setPlayerInput(0, 0, BoardMarks::Empty);
00428     board.setPlayerInput(0, 1, BoardMarks::Empty);
00429     board.setPlayerInput(0, 2, BoardMarks::Empty);
00430
00431     board.setPlayerInput(1, 0, BoardMarks::Empty);
00432     board.setPlayerInput(1, 1, BoardMarks::Empty);
00433     board.setPlayerInput(1, 2, BoardMarks::Empty);
00434
00435     board.setPlayerInput(2, 0, BoardMarks::Empty);
00436     board.setPlayerInput(2, 1, BoardMarks::Empty);
00437     board.setPlayerInput(2, 2, BoardMarks::Empty);
00438
00439     /* - - -
00440      * - - -
00441      * - - - */
00442     EXPECT_EQ(BoardState::NoWinner, board.evaluateBoard());
00443
00444     board.reset();
00445
00446     board.setPlayerInput(0, 0, BoardMarks::Empty);
00447     board.setPlayerInput(0, 1, BoardMarks::X);
00448     board.setPlayerInput(0, 2, BoardMarks::Empty);
00449
00450     board.setPlayerInput(1, 0, BoardMarks::Empty);
00451     board.setPlayerInput(1, 1, BoardMarks::X);
00452     board.setPlayerInput(1, 2, BoardMarks::O);
00453
00454     board.setPlayerInput(2, 0, BoardMarks::X);
00455     board.setPlayerInput(2, 1, BoardMarks::Empty);
00456     board.setPlayerInput(2, 2, BoardMarks::O);
00457

```

```

00458     /* - X -
00459     * - X O
00460     * X - O */
00461     EXPECT_EQ(BoardState::NoWinner, board.evaluateBoard());
00462 }
00463
00464 TEST(BoardTest, setPlayerInputValid){
00465     Board board(4);
00466     EXPECT_EQ(true, board.setPlayerInput(0, 0, BoardMarks::X));
00467     EXPECT_EQ(true, board.setPlayerInput(3, 3, BoardMarks::X));
00468     EXPECT_EQ(true, board.setPlayerInput(2, 1, BoardMarks::X));
00469     EXPECT_EQ(true, board.setPlayerInput(1, 0, BoardMarks::X));
00470 }
00471
00472 TEST(BoardTest, setPlayerInputOutOfBounds){
00473     Board board(4);
00474     EXPECT_ANY_THROW(board.setPlayerInput(0, 4, BoardMarks::X));
00475     EXPECT_ANY_THROW(board.setPlayerInput(-1, 2, BoardMarks::X));
00476     EXPECT_ANY_THROW(board.setPlayerInput(5, 2, BoardMarks::X));
00477     EXPECT_ANY_THROW(board.setPlayerInput(-1, -1, BoardMarks::X));
00478 }
00479
00480 TEST(BoardTest, setPlayerInputOverwrite){
00481     Board board(4);
00482     EXPECT_EQ(true, board.setPlayerInput(0, 0, BoardMarks::X));
00483     EXPECT_EQ(false, board.setPlayerInput(0, 0, BoardMarks::X));
00484     EXPECT_EQ(true, board.setPlayerInput(3, 3, BoardMarks::O));
00485     EXPECT_EQ(false, board.setPlayerInput(3, 3, BoardMarks::X));
00486 }
00487
00488 TEST(BoardTest, setPlayerInputAfterGameOver){
00489     Board board(3);
00490
00491     // O Wins
00492     board.setPlayerInput(0, 0, BoardMarks::X);
00493     board.setPlayerInput(0, 1, BoardMarks::X);
00494     board.setPlayerInput(0, 2, BoardMarks::O);
00495
00496     board.setPlayerInput(1, 0, BoardMarks::O);
00497     board.setPlayerInput(1, 1, BoardMarks::O);
00498     board.setPlayerInput(1, 2, BoardMarks::O);
00499
00500     board.setPlayerInput(2, 0, BoardMarks::Empty);
00501     board.setPlayerInput(2, 1, BoardMarks::X);
00502     board.setPlayerInput(2, 2, BoardMarks::Empty);
00503
00504     /* X X O
00505     * O O O
00506     * - X - */
00507     EXPECT_EQ(false, board.setPlayerInput(0, 0, BoardMarks::O));
00508     EXPECT_EQ(false, board.setPlayerInput(3, 3, BoardMarks::X));
00509     EXPECT_EQ(false, board.setPlayerInput(2, 1, BoardMarks::O));
00510     EXPECT_EQ(false, board.setPlayerInput(1, 0, BoardMarks::X));
00511
00512     board.reset();
00513
00514     // X wins
00515     board.setPlayerInput(0, 0, BoardMarks::X);
00516     board.setPlayerInput(0, 1, BoardMarks::O);
00517     board.setPlayerInput(0, 2, BoardMarks::O);
00518
00519     board.setPlayerInput(1, 0, BoardMarks::X);
00520     board.setPlayerInput(1, 1, BoardMarks::O);
00521     board.setPlayerInput(1, 2, BoardMarks::X);
00522
00523     board.setPlayerInput(2, 0, BoardMarks::X);
00524     board.setPlayerInput(2, 1, BoardMarks::X);
00525     board.setPlayerInput(2, 2, BoardMarks::O);
00526
00527     /* X O O
00528     * X O X
00529     * X X O */
00530     EXPECT_EQ(false, board.setPlayerInput(0, 0, BoardMarks::O));
00531     EXPECT_EQ(false, board.setPlayerInput(3, 3, BoardMarks::X));
00532     EXPECT_EQ(false, board.setPlayerInput(2, 1, BoardMarks::O));
00533     EXPECT_EQ(false, board.setPlayerInput(1, 0, BoardMarks::X));
00534
00535     board.reset();
00536
00537     // Tie
00538     board.setPlayerInput(0, 0, BoardMarks::O);
00539     board.setPlayerInput(0, 1, BoardMarks::X);
00540     board.setPlayerInput(0, 2, BoardMarks::O);
00541
00542     board.setPlayerInput(1, 0, BoardMarks::X);
00543     board.setPlayerInput(1, 1, BoardMarks::O);
00544     board.setPlayerInput(1, 2, BoardMarks::X);

```



```

00545
00546     board.setPlayerInput(2, 0, BoardMarks::X);
00547     board.setPlayerInput(2, 1, BoardMarks::O);
00548     board.setPlayerInput(2, 2, BoardMarks::X);
00549
00550     /* O X O
00551        * X O X
00552        * X O X */
00553     EXPECT_EQ(false, board.setPlayerInput(0, 0, BoardMarks::O));
00554     EXPECT_EQ(false, board.setPlayerInput(3, 3, BoardMarks::O));
00555     EXPECT_EQ(false, board.setPlayerInput(2, 1, BoardMarks::X));
00556     EXPECT_EQ(false, board.setPlayerInput(1, 0, BoardMarks::X));
00557 }
00558
00559 TEST(BoardTest, boardAccessValid){
00560     Board board(3);
00561
00562     board.setPlayerInput(0, 0, BoardMarks::O);
00563     board.setPlayerInput(0, 1, BoardMarks::O);
00564     board.setPlayerInput(0, 2, BoardMarks::Empty);
00565
00566     board.setPlayerInput(1, 0, BoardMarks::X);
00567     board.setPlayerInput(1, 1, BoardMarks::X);
00568     board.setPlayerInput(1, 2, BoardMarks::O);
00569
00570     board.setPlayerInput(2, 0, BoardMarks::Empty);
00571     board.setPlayerInput(2, 1, BoardMarks::X);
00572     board.setPlayerInput(2, 2, BoardMarks::Empty);
00573
00574     EXPECT_EQ(BoardMarks::X, board.at(1, 1));
00575     EXPECT_EQ(BoardMarks::O, board.at(1, 2));
00576     EXPECT_EQ(BoardMarks::Empty, board.at(2, 2));
00577 }
00578
00579 TEST(BoardTest, boardAccessInvalid){
00580     Board board(3);
00581
00582     EXPECT_ANY_THROW(board.at(1, 4));
00583     EXPECT_ANY_THROW(board.at(-1, 2));
00584     EXPECT_ANY_THROW(board.at(-10, -10));
00585     EXPECT_ANY_THROW(board.at(5, 9));
00586 }

```

11.64 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TicTacToeTest/tst_minimaxagent.h File Reference

```

#include <gtest/gtest.h>
#include <gmock/gmock-matchers.h>
#include "Board.h"
#include "MiniMaxAgent.h"

```

Functions

- [TEST](#) (MiniMaxAgentTest, PlayerStartAIWin)
- [TEST](#) (MiniMaxAgentTest, AIStartAIWin)
- [TEST](#) (MiniMaxAgentTest, PlayerStartAIBlock)
- [TEST](#) (MiniMaxAgentTest, AIStartAIBlock)

11.64.1 Function Documentation

11.64.1.1 TEST() [1/4]

```

TEST (
    MiniMaxAgentTest ,
    AIStartAIBlock )

```

References [Board::at\(\)](#), [Empty](#), [O](#), [MiniMaxAgent::play\(\)](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.64.1.2 TEST() [2/4]

```
TEST (
    MiniMaxAgentTest ,
    AIStartAIWin )
```

References [Empty](#), [Board::evaluateBoard\(\)](#), [O](#), [MiniMaxAgent::play\(\)](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), [X](#), and [XWins](#).

11.64.1.3 TEST() [3/4]

```
TEST (
    MiniMaxAgentTest ,
    PlayerStartAIBlock )
```

References [Board::at\(\)](#), [Empty](#), [O](#), [MiniMaxAgent::play\(\)](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.64.1.4 TEST() [4/4]

```
TEST (
    MiniMaxAgentTest ,
    PlayerStartAIWin )
```

References [Empty](#), [Board::evaluateBoard\(\)](#), [O](#), [OWins](#), [MiniMaxAgent::play\(\)](#), [Board::reset\(\)](#), [Board::setPlayerInput\(\)](#), and [X](#).

11.65 tst_minimaxagent.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TST_MINIMAXAGENT_H
00002 #define TST_MINIMAXAGENT_H
00003
00004 #endif // TST_MINIMAXAGENT_H
00005 #include <gtest/gtest.h>
00006 #include <gmock/gmock-matchers.h>
00007 #include "Board.h"
00008 #include "MiniMaxAgent.h"
00009
00010 using namespace testing;
00011
00012 TEST(MiniMaxAgentTest, PlayerStartAIWin)
00013 {
00014     Board board(3);
00015     MiniMaxAgent ai(9, BoardMarks::O, BoardMarks::X);
00016
00017     board.setPlayerInput(0, 0, BoardMarks::X);
00018     board.setPlayerInput(0, 1, BoardMarks::X);
00019     board.setPlayerInput(0, 2, BoardMarks::O);
00020
00021     board.setPlayerInput(1, 0, BoardMarks::Empty);
00022     board.setPlayerInput(1, 1, BoardMarks::X);
00023     board.setPlayerInput(1, 2, BoardMarks::Empty);
00024
00025     board.setPlayerInput(2, 0, BoardMarks::O);
00026     board.setPlayerInput(2, 1, BoardMarks::O);
00027     board.setPlayerInput(2, 2, BoardMarks::Empty);
00028
00029     /* X X O
00030      * - X -
00031      * O O - */
00032     ai.play(board);
00033     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00034 }
```

```

00035     board.reset();
00036
00037     board.setPlayerInput(0, 0, BoardMarks::X);
00038     board.setPlayerInput(0, 1, BoardMarks::X);
00039     board.setPlayerInput(0, 2, BoardMarks::O);
00040
00041     board.setPlayerInput(1, 0, BoardMarks::Empty);
00042     board.setPlayerInput(1, 1, BoardMarks::O);
00043     board.setPlayerInput(1, 2, BoardMarks::X);
00044
00045     board.setPlayerInput(2, 0, BoardMarks::Empty);
00046     board.setPlayerInput(2, 1, BoardMarks::X);
00047     board.setPlayerInput(2, 2, BoardMarks::Empty);
00048
00049     /* X X O
00050      * - O X
00051      * - X O */
00052     ai.play(board);
00053     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00054
00055
00056     board.reset();
00057
00058     board.setPlayerInput(0, 0, BoardMarks::X);
00059     board.setPlayerInput(0, 1, BoardMarks::X);
00060     board.setPlayerInput(0, 2, BoardMarks::O);
00061
00062     board.setPlayerInput(1, 0, BoardMarks::Empty);
00063     board.setPlayerInput(1, 1, BoardMarks::O);
00064     board.setPlayerInput(1, 2, BoardMarks::Empty);
00065
00066     board.setPlayerInput(2, 0, BoardMarks::X);
00067     board.setPlayerInput(2, 1, BoardMarks::X);
00068     board.setPlayerInput(2, 2, BoardMarks::O);
00069
00070     /* X X O
00071      * - O -
00072      * X X O */
00073     ai.play(board);
00074     EXPECT_EQ(BoardState::OWins, board.evaluateBoard());
00075 }
00076
00077 TEST(MiniMaxAgentTest, AIStartAIWin)
00078 {
00079     Board board(3);
00080     MiniMaxAgent ai(9, BoardMarks::X, BoardMarks::O);
00081
00082     board.setPlayerInput(0, 0, BoardMarks::X);
00083     board.setPlayerInput(0, 1, BoardMarks::X);
00084     board.setPlayerInput(0, 2, BoardMarks::O);
00085
00086     board.setPlayerInput(1, 0, BoardMarks::Empty);
00087     board.setPlayerInput(1, 1, BoardMarks::X);
00088     board.setPlayerInput(1, 2, BoardMarks::Empty);
00089
00090     board.setPlayerInput(2, 0, BoardMarks::O);
00091     board.setPlayerInput(2, 1, BoardMarks::O);
00092     board.setPlayerInput(2, 2, BoardMarks::Empty);
00093
00094     /* X X O
00095      * - X -
00096      * O O - */
00097     ai.play(board);
00098     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00099
00100     board.reset();
00101
00102     board.setPlayerInput(0, 0, BoardMarks::X);
00103     board.setPlayerInput(0, 1, BoardMarks::X);
00104     board.setPlayerInput(0, 2, BoardMarks::O);
00105
00106     board.setPlayerInput(1, 0, BoardMarks::Empty);
00107     board.setPlayerInput(1, 1, BoardMarks::O);
00108     board.setPlayerInput(1, 2, BoardMarks::Empty);
00109
00110     board.setPlayerInput(2, 0, BoardMarks::X);
00111     board.setPlayerInput(2, 1, BoardMarks::X);
00112     board.setPlayerInput(2, 2, BoardMarks::Empty);
00113
00114     /* X X O
00115      * - O -
00116      * - X O */
00117     ai.play(board);
00118     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00119
00120
00121     board.reset();

```

```

00122
00123     board.setPlayerInput(0, 0, BoardMarks::X);
00124     board.setPlayerInput(0, 1, BoardMarks::X);
00125     board.setPlayerInput(0, 2, BoardMarks::Empty);
00126
00127     board.setPlayerInput(1, 0, BoardMarks::Empty);
00128     board.setPlayerInput(1, 1, BoardMarks::O);
00129     board.setPlayerInput(1, 2, BoardMarks::O);
00130
00131     board.setPlayerInput(2, 0, BoardMarks::X);
00132     board.setPlayerInput(2, 1, BoardMarks::X);
00133     board.setPlayerInput(2, 2, BoardMarks::O);
00134
00135     /* X X O
00136        * - O O
00137        * X X O */
00138     ai.play(board);
00139     EXPECT_EQ(BoardState::XWins, board.evaluateBoard());
00140 }
00141
00142 TEST(MiniMaxAgentTest, PlayerStartAIBlock)
00143 {
00144     Board board(3);
00145     MiniMaxAgent ai(9, BoardMarks::O, BoardMarks::X);
00146
00147     board.setPlayerInput(0, 0, BoardMarks::X);
00148     board.setPlayerInput(0, 1, BoardMarks::X);
00149     board.setPlayerInput(0, 2, BoardMarks::O);
00150
00151     board.setPlayerInput(1, 0, BoardMarks::O);
00152     board.setPlayerInput(1, 1, BoardMarks::X);
00153     board.setPlayerInput(1, 2, BoardMarks::Empty);
00154
00155     board.setPlayerInput(2, 0, BoardMarks::Empty);
00156     board.setPlayerInput(2, 1, BoardMarks::O);
00157     board.setPlayerInput(2, 2, BoardMarks::Empty);
00158
00159     /* X X O
00160        * - X -
00161        * - O - */
00162     ai.play(board);
00163     EXPECT_EQ(BoardMarks::O, board.at(2, 2));
00164
00165     board.reset();
00166
00167     board.setPlayerInput(0, 0, BoardMarks::X);
00168     board.setPlayerInput(0, 1, BoardMarks::X);
00169     board.setPlayerInput(0, 2, BoardMarks::Empty);
00170
00171     board.setPlayerInput(1, 0, BoardMarks::O);
00172     board.setPlayerInput(1, 1, BoardMarks::O);
00173     board.setPlayerInput(1, 2, BoardMarks::X);
00174
00175     board.setPlayerInput(2, 0, BoardMarks::Empty);
00176     board.setPlayerInput(2, 1, BoardMarks::X);
00177     board.setPlayerInput(2, 2, BoardMarks::Empty);
00178
00179     /* X X -
00180        * O O X
00181        * - X O */
00182     ai.play(board);
00183     EXPECT_EQ(BoardMarks::O, board.at(0, 2));
00184
00185     board.reset();
00186
00187     board.setPlayerInput(0, 0, BoardMarks::X);
00188     board.setPlayerInput(0, 1, BoardMarks::Empty);
00189     board.setPlayerInput(0, 2, BoardMarks::Empty);
00190
00191     board.setPlayerInput(1, 0, BoardMarks::Empty);
00192     board.setPlayerInput(1, 1, BoardMarks::O);
00193     board.setPlayerInput(1, 2, BoardMarks::Empty);
00194
00195     board.setPlayerInput(2, 0, BoardMarks::X);
00196     board.setPlayerInput(2, 1, BoardMarks::Empty);
00197     board.setPlayerInput(2, 2, BoardMarks::O);
00198
00199     /* X - -
00200        * - O -
00201        * X - O */
00202     ai.play(board);
00203     EXPECT_EQ(BoardMarks::O, board.at(1, 0));
00204 }
00205
00206 TEST(MiniMaxAgentTest, AISTartAIBlock)
00207 {
00208

```

```

00209     Board board(3);
00210     MiniMaxAgent ai(9, BoardMarks::X, BoardMarks::O);
00211
00212     board.setPlayerInput(0, 0, BoardMarks::X);
00213     board.setPlayerInput(0, 1, BoardMarks::X);
00214     board.setPlayerInput(0, 2, BoardMarks::O);
00215
00216     board.setPlayerInput(1, 0, BoardMarks::X);
00217     board.setPlayerInput(1, 1, BoardMarks::Empty);
00218     board.setPlayerInput(1, 2, BoardMarks::Empty);
00219
00220     board.setPlayerInput(2, 0, BoardMarks::O);
00221     board.setPlayerInput(2, 1, BoardMarks::O);
00222     board.setPlayerInput(2, 2, BoardMarks::Empty);
00223
00224     /* X X O
00225      * X - -
00226      * O O - */
00227     ai.play(board);
00228     EXPECT_EQ(BoardMarks::X, board.at(1, 1));
00229
00230     board.reset();
00231
00232     board.setPlayerInput(0, 0, BoardMarks::Empty);
00233     board.setPlayerInput(0, 1, BoardMarks::X);
00234     board.setPlayerInput(0, 2, BoardMarks::O);
00235
00236     board.setPlayerInput(1, 0, BoardMarks::Empty);
00237     board.setPlayerInput(1, 1, BoardMarks::Empty);
00238     board.setPlayerInput(1, 2, BoardMarks::Empty);
00239
00240     board.setPlayerInput(2, 0, BoardMarks::X);
00241     board.setPlayerInput(2, 1, BoardMarks::Empty);
00242     board.setPlayerInput(2, 2, BoardMarks::O);
00243
00244     /* - X O
00245      * - - -
00246      * X - O */
00247     ai.play(board);
00248     EXPECT_EQ(BoardMarks::X, board.at(1, 2));
00249
00250
00251     board.reset();
00252
00253     board.setPlayerInput(0, 0, BoardMarks::X);
00254     board.setPlayerInput(0, 1, BoardMarks::Empty);
00255     board.setPlayerInput(0, 2, BoardMarks::Empty);
00256
00257     board.setPlayerInput(1, 0, BoardMarks::Empty);
00258     board.setPlayerInput(1, 1, BoardMarks::O);
00259     board.setPlayerInput(1, 2, BoardMarks::O);
00260
00261     board.setPlayerInput(2, 0, BoardMarks::Empty);
00262     board.setPlayerInput(2, 1, BoardMarks::X);
00263     board.setPlayerInput(2, 2, BoardMarks::Empty);
00264
00265     /* X - -
00266      * - O O
00267      * - X - */
00268     ai.play(board);
00269     EXPECT_EQ(BoardMarks::X, board.at(1, 0));
00270 }

```

11.66 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/TTTCommonTypes.h File Reference

```

#include <cstdint>
#include <portaudio.h>
#include <pocketsphinx.h>
#include <signal.h>

```

Classes

- struct [Cell](#)

The *Cell* struct represents a board cell and all the necessary information needed about the cell.

- struct [TTOptions](#)

The *TTOptions* struct defines a tic tac toe game options.

- struct [SpeechToText](#)

Namespaces

- namespace [defaults](#)

Enumerations

- enum class [BoardMarks](#) { [Empty](#) , [X](#) , [O](#) }

The *BoardMarks* enum defines the type of mark on the board.

- enum class [BoardState](#) { [NoWinner](#) , [XWins](#) , [OWins](#) , [Tie](#) }

The *BoardState* enum defines the possible states of a board.

Variables

- constexpr short [defaults::INVALID_CELL](#) = -1
- constexpr size_t [defaults::DEFAULT_BOARD_SIZE](#) = 3
- constexpr unsigned short [defaults::DEFAULT_MINIMAX_DEPTH](#) = 3
- constexpr unsigned short [defaults::GUI_CELL_ROW_SPAN](#) = 1
- constexpr unsigned short [defaults::GUI_CELL_COLUMN_SPAN](#) = 1
- constexpr char [defaults::X_COLOR](#) [] = "#455A64"
- constexpr char [defaults::O_COLOR](#) [] = "#FF5722"
- constexpr char [defaults::DEFAULT_COLOR](#) [] = "#FFF"
- static int [global_done](#) = 0

11.66.1 Enumeration Type Documentation

11.66.1.1 BoardMarks

```
enum class BoardMarks [strong]
```

The [BoardMarks](#) enum defines the type of mark on the board.

Enumerator

Empty	
X	
O	

11.66.1.2 BoardState

```
enum class BoardState [strong]
```

The [BoardState](#) enum defines the possible states of a board.

Enumerator

NoWinner	
XWins	
OWins	
Tie	

11.66.2 Variable Documentation

11.66.2.1 global_done

```
int global_done = 0 [static]
```

Referenced by [SpeechToText::catch_sig\(\)](#).

11.67 TTCommonTypes.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TTCOMMONTYPES_H
00002 #define TTCOMMONTYPES_H
00003 #include <cstdint>
00004 #include <portaudio.h>
00005 #include <pocketsphinx.h>
00006 #include <signal.h>
00007
00008 class QPushButton;
00009 namespace defaults
00010 {
00011     constexpr short INVALID_CELL = -1;
00012     constexpr size_t DEFAULT_BOARD_SIZE = 3;
00013     constexpr unsigned short DEFAULT_MINIMAX_DEPTH = 3;
00014     constexpr unsigned short GUI_CELL_ROW_SPAN = 1;
00015     constexpr unsigned short GUI_CELL_COLUMN_SPAN = 1;
00016     constexpr char X_COLOR[] = "#455A64";
00017     constexpr char O_COLOR[] = "#FF5722";
00018     constexpr char DEFAULT_COLOR[] = "#FFF";
00019 }
00020 static int global_done = 0;
00021 struct Cell
00022 {
00023     QPushButton *cellBtn = nullptr;
00024     int row = defaults::INVALID_CELL;
00025     int col = defaults::INVALID_CELL;
00026     Cell(QPushButton *cellBtn, int row, int col)
00027         : cellBtn(cellBtn), row(row), col(col) {}
00028 };
00029
00030 struct TTOptions
00031 {
00032     size_t boardSize = defaults::DEFAULT_BOARD_SIZE;
00033     unsigned short miniMaxDepth = defaults::DEFAULT_MINIMAX_DEPTH;
00034     bool AIopponent = true;
00035     bool Aistarts = false;
00036 };
00037
00038 struct SpeechToText
00039 {
00040     PaStream *stream;
00041     PaError err;
00042     ps_decoder_t *decoder;
00043     ps_config_t *config;
00044     ps_endpoint_t *ep;
00045     short *frame;
00046     size_t frame_size;
00047     const char *hyp;
00048
00049     static void catch_sig(int signum)
00050     {
00051         (void)signum;
00052     }
00053 }
```

```

00059     global_done = 1;
00060 }
00061
00062 void configuration(){
00063     config = ps_config_init(NULL);
00064     ps_default_search_args(config);
00065     ps_config_set_str(config, "hmm",
"/home/ratty/cmuspinx/share/pocketsphinx/model/custom/custom");
00066     ps_config_set_str(config, "lm",
"/home/ratty/cmuspinx/share/pocketsphinx/model/es-es/es-20k.lm");
00067     ps_config_set_str(config, "dict",
"/home/ratty/cmuspinx/share/pocketsphinx/model/custom/es.dict");
00068 }
00069
00070 void init(){
00071     if ((err = Pa_Initialize()) != paNoError)
00072         E_FATAL("Failed to initialize PortAudio: %s\n",
Pa_GetErrorText(err));
00073     if ((decoder = ps_init(config)) == NULL)
00074         E_FATAL("PocketSphinx decoder init failed\n");
00075     if ((ep = ps_endpoint_init(0, 0.0, PS_VAD_LOOSE, 0, 0)) == NULL)
00076         E_FATAL("PocketSphinx endpointer init failed\n");
00077     frame_size = ps_endpoint_frame_size(ep);
00078     if ((frame = (short int*)malloc(frame_size * sizeof(frame[0]))) == NULL)
00079         E_FATAL_SYSTEM("Failed to allocate frame");
00080     if ((err = Pa_OpenDefaultStream(&stream, 1, 0, paInt16,
ps_config_int(config, "samprate"),
frame_size, NULL, NULL)) != paNoError)
00081         E_FATAL("Failed to open PortAudio stream: %s\n",
Pa_GetErrorText(err));
00082     if ((err = Pa_StartStream(stream)) != paNoError)
00083         E_FATAL("Failed to start PortAudio stream: %s\n",
Pa_GetErrorText(err));
00084     if (signal(SIGINT, catch_sig) == SIG_ERR)
00085         E_FATAL_SYSTEM("Failed to set SIGINT handler");
00086 }
00087
00088 bool hypIsValid(){
00089     if (hyp != NULL)
00090         return true;
00091     return false;
00092 }
00093
00094 void listen(){
00095     hyp = NULL;
00096     while (!hypIsValid()) {
00097         const int16 *speech;
00098         int prev_in_speech = ps_endpoint_in_speech(ep);
00099         if ((err = Pa_ReadStream(stream, frame, frame_size)) != paNoError) {
00100             E_ERROR("Error in PortAudio read: %s\n",
Pa_GetErrorText(err));
00101             break;
00102         }
00103         speech = ps_endpoint_process(ep, frame);
00104         if (speech != NULL) {
00105             if (!prev_in_speech) {
00106                 fprintf(stderr, "Speech start at %.2f\n",
ps_endpoint_speech_start(ep));
00107                 fflush(stderr); /* For broken MSYS2 terminal */
00108                 ps_start_utt(decoder);
00109             }
00110             if (ps_process_raw(decoder, speech, frame_size, FALSE, FALSE) < 0)
00111                 E_FATAL("ps_process_raw() failed\n");
00112             if (!ps_endpoint_in_speech(ep)) {
00113                 fprintf(stderr, "Speech end at %.2f\n",
ps_endpoint_speech_end(ep));
00114                 fflush(stderr);
00115                 ps_end_utt(decoder);
00116                 if ((hyp = ps_get_hyp(decoder, NULL)) != NULL) {
00117                     printf("%s\n", hyp);
00118                     fflush(stdout);
00119                 }
00120             }
00121         }
00122     }
00123     if ((err = Pa_StopStream(stream)) != paNoError)
00124         E_FATAL("Failed to stop PortAudio stream: %s\n",
Pa_GetErrorText(err));
00125     if ((err = Pa_Terminate()) != paNoError)
00126         E_FATAL("Failed to terminate PortAudio: %s\n",
Pa_GetErrorText(err));
00127 }
00128
00129 void free(){
00130     ps_endpoint_free(ep);
00131     ps_free(decoder);
00132     ps_config_free(config);
00133 }
00134 };

```



```

00143
00147 enum class BoardMarks { Empty, X, O };
00148
00152 enum class BoardState { NoWinner, XWins, OWins, Tie };
00153
00154 #endif // TTTCOMMONTYPES_H

```

11.68 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TicTacToeGame.cpp File Reference

```

#include "TicTacToeGame.h"
#include <QMessageBox>
#include <QPushButton>

```

11.69 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TicTacToeGame.h File Reference

```

#include "TTTCommonTypes.h"
#include "ui_TicTacToeGame.h"
#include <QDialog>
#include <vector>
#include <memory>

```

Classes

- class [TicTacToeGame](#)

The [TicTacToeGame](#) class is responsible for the GUI of a game, it is responsible for building the cells of a board, which can be of variable size, and a navigation button to take the user back to the title screen, and a button for resetting the game and playing a new one.

11.70 TicTacToeGame.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MAINWINDOW_H
00002 #define MAINWINDOW_H
00003
00004 #include "TTTCommonTypes.h"
00005 #include "ui_TicTacToeGame.h"
00006 #include <QDialog>
00007 #include <vector>
00008 #include <memory>
00009
00010 using std::vector;
00011
00012
00020
00021 class TicTacToeGame final : public QDialog
00022 {
00023     Q_OBJECT
00024
00025 public:
00030     explicit TicTacToeGame(QWidget *parent = nullptr);
00036     void updateCell(Cell &cell, BoardMarks currentPlayer);
00041     void declareGameState(BoardState boardState);
00048     vector<Cell> buildCellButtons(size_t boardSize);

```

```

00054     void reset(vector<Cell> &cells);
00055
00056 private:
00060     std::unique_ptr<Ui::TicTacToeGame> ui;
00065     void setConnections();
00071     QString getPlayerText(BoardMarks currentPlayer);
00077     QString getPlayerStyleSheet(BoardMarks currentPlayer);
00085     QString getBoardFinalStateText(BoardState boardState);
00086
00087 signals:
00092     void newGame();
00093 };
00094
00095 #endif // MAINWINDOW_H

```

11.71 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TitleScreen.cpp File Reference

```

#include "TitleScreen.h"
#include "TTTController.h"
#include "NetworkManager.h"

```

11.72 C:/Users/ferna/Voice_TicTacToe/Qt-Tic-Tac-Toe-master/View/TitleScreen.h File Reference

```

#include "TTTCommonTypes.h"
#include "ui_titlescreen.h"
#include <QMainWindow>
#include <memory>

```

Classes

- class [TitleScreen](#)

The [TitleScreen](#) class is responsible for the GUI of the title screen, its main functionality is to gather game options set by the user, then starting a game by creating a Tic Tac Toe controller with the user options.

11.73 TitleScreen.h

[Go to the documentation of this file.](#)

```

00001 #ifndef TITLES_SCREEN_H
00002 #define TITLES_SCREEN_H
00003 #include "TTTCommonTypes.h"
00004 #include "ui_titlescreen.h"
00005 #include <QMainWindow>
00006 #include <memory>
00007
00013
00014 class TitleScreen final : public QMainWindow
00015 {
00016     Q_OBJECT
00017
00018 public:
00023     explicit TitleScreen(QWidget *parent = nullptr);
00024
00025 private:
00029     std::unique_ptr<Ui::TitleScreen> ui;

```

```
00033     void setConnections();
00037     TTOptions options_;
00042     void closeEvent(QCloseEvent *event) override;
00043
00044 public slots:
00049     void updateBoardSize(int size);
00054     void updateSinglePlayer(bool checked);
00059     void updateTwoPlayers(bool checked);
00064     void updateAIStartsGame(bool checked);
00069     void updateMiniMaxDepth(int depth);
00073     void startGame();
00074
00075
00076 signals:
00080     void exited();
00081 };
00082
00083 #endif // TITLES_SCREEN_H
```

